



Object Oriented Programming Design



BITS Pilani
Pilani Campus

Dr RAMYA DEVI M
Computer Science and Information Systems
CS - 01 : 28th Jun 2025

Course Outcomes



On completion of this course we will be able to:

- 1. Demonstrate sound understanding of Object oriented concepts and able to design applications having to apply OOP paradigms.**
- 2. Express the OOP concepts in form of code snippet, small applications having to recognize the differences between procedural and OOP concepts.**
- 3. Design applications using the various constructs and features available in the programming language and assess them.**



Books

- The Java Programming Language, Ken Arnold, David Holmes, James Gosling, Prakash Goteti, 3rd Edition, Pearson.
 - Java: The Complete Reference by Herbert Schildt, 9th Edition, 2017.
-



JAVA Basics



Why **Java** is Important

- **Two reasons** :
 - Trouble with **C/C++** language is that they are not portable and are not platform independent languages.
 - Emergence of World Wide Web, which demanded portable programs
 - **Portability** and **security** necessitated the invention of Java
-



History

- **James Gosling** - Sun Microsystems
 - **Co founder** – Vinod Khosla
 - Oak - Java, May 20, 1995, Sun World
 - **JDK Evolutions**
 - JDK 1.0 (January 23, 1996)
 - JDK 1.1 (February 19, 1997)
 - J2SE 1.2 (December 8, 1998)
 - J2SE 1.3 (May 8, 2000)
 - J2SE 1.4 (February 6, 2002)
 - J2SE 5.0 (September 30, 2004)
 - Java SE 6 (December 11, 2006)
 - Java SE 7 (July 28, 2011)
-

Cont..

- Java Editions.

- **J2SE**(Java 2 Standard Edition) - to develop client-side standalone applications or applets.
 - **J2ME**(Java 2 Micro Edition) - to develop applications for mobile devices such as cell phones.
 - **J2EE**(Java 2 Enterprise Edition) - to develop server-side applications such as Java servlets and Java ServerPages.
-



What is **java**?

- A general-purpose **object-oriented** language.
 - **W**rite **O**nce **R**un **A**n anywhere (WORA).
 - Designed for easy **Web/Internet** applications.
 - **Widespread** acceptance.
-



How is **Java** different from **C...**

- **C Language:**

- Major difference is that C is a **structure oriented language** and Java is an **object oriented language** and has mechanism to define classes and objects.
- Java does not support an explicit **pointer** type
- Java does not have **preprocessor**, so we cant use #define, #include and #ifdef statements.
- Java does not include **structures, unions** and **enum data types**.
- Java does not include keywords like **goto, sizeof** and **typedef**.
- Java adds labeled **break** and **continue** statements.
- Java adds many features required for object oriented programming.



How is **Java** different from **C++**...

- C++ language

Features removed in java:

- Java doesn't support **pointers** to avoid **unauthorized** access of **memory locations**.
- Java does not include structures, unions and enum data types.
- Java does not support **operator over loading**.
- Preprocessor plays less important role in C++ and so **eliminated** entirely in java.
- Java does not perform **automatic** type conversions that result in loss of **precision**.



Cont..

- Java does not support **global variables**. Every method and variable is declared within a **class** and forms part of that class.
 - Java does not allow **default arguments**.
 - Java does not support inheritance of **multiple** super classes by a sub class (i.e., **multiple inheritance**). This is accomplished by using '**interface**' concept.
 - It is not possible to declare **unsigned integers** in java.
 - In java objects are passed by **reference** only. In C++ objects may be passed by **value** or **reference**.
-



Cont ...

New features added in Java:

- **Multithreading**, that allows two or more pieces of the same program to execute concurrently.
 - C++ has a set of library functions that use a common header file. But java replaces it with its own set of **API classes**.
 - It adds **packages** and **interfaces**.
 - Java supports automatic **garbage collection**.
 - **break** and **continue** statements have been enhanced in java to accept labels as targets.
 - The use of **unicode** characters ensures portability.
-



Advantage of Java

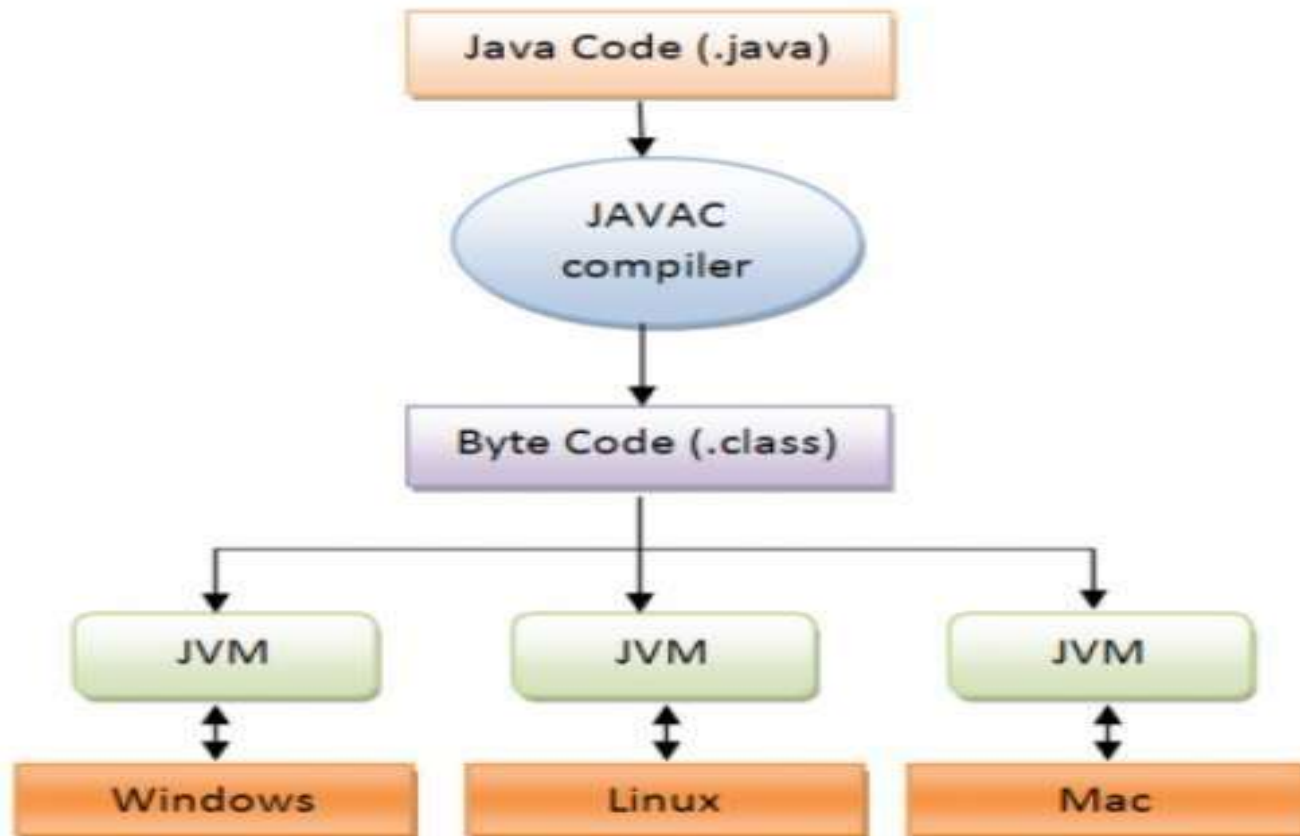
- Java is simple
 - Java is object-oriented
 - Java is distributed
 - Java is interpreted
 - Java is robust
 - Java is architecture-neutral
 - Java is portable
 - Java's performance
 - Java is multithreaded
 - Java is dynamic
 - Java is secure
-



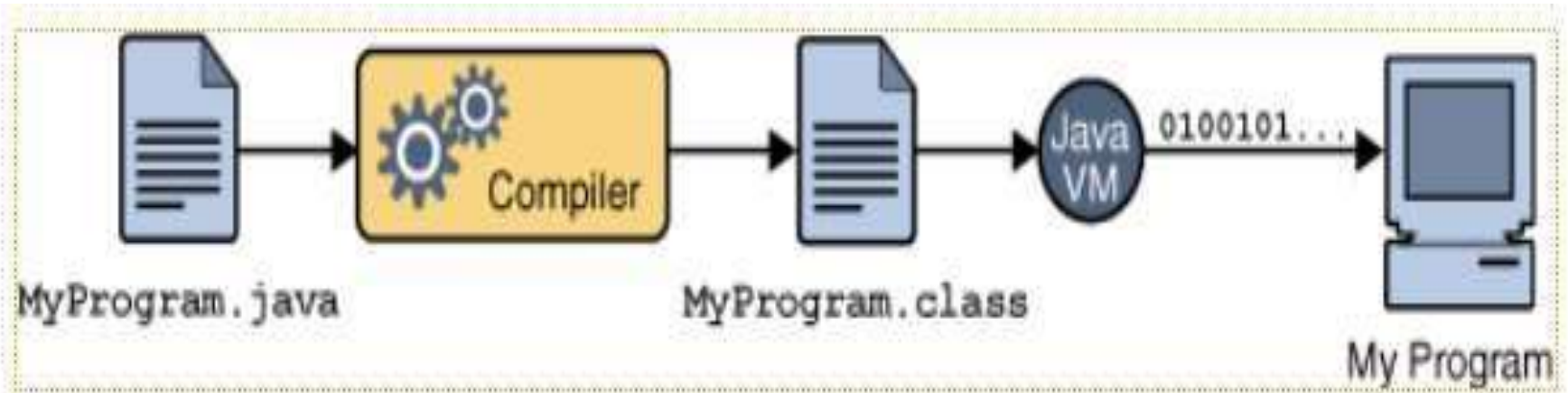
Java Environment

- Java includes many development tools, classes and methods
 - Development tools are part of Java Development Kit (JDK) and
 - The classes and methods are part of **Java Standard Library** (JSL), also known as **Application Programming Interface (API)**.
 - JDK constitutes of tools like **java compiler**, **java interpreter** and many.
 - **API** includes hundreds of **classes** and **methods** grouped into several **packages** according to their functionality.
-

Java is architecture-neutral



WORA(Write Once Run Anywhere)



Java programs are compiled to Java byte-codes, a kind of machine independent representation.

The program is then executed by an interpreter called the Java Virtual Machine (JVM).



Module 1 – Lecture 1

BASICS IN JAVA

JAVA BASICS



JAVA was developed by Sun Microsystems Inc in 1991, later acquired by Oracle Corporation. It was developed by James Gosling and Patrick Naughton.

It is a simple programming language.

Writing, compiling and debugging a program is easy in java.

It enables creating modular programs and reusable code.

The following are common java terms

Java Virtual Machine (JVM)

This is generally referred as JVM.

Before, we discuss about JVM lets see the phases of program execution which are as follows: we **write** the program, then we **compile** the program and at last we **run** the program.

JAVA BASICS



- 1) **Writing** a program in Java – requires knowledge of character set, the tokens, the syntax and other programming constructs.
- 2) **Compilation** of program is done by **javac** compiler.
javac is the primary java compiler included in java development kit (JDK).

It takes java program as input and generates java bytecode as output.

- 3) In third phase, JVM **executes** the bytecode generated by compiler.

This is called program run or execution phase.

The primary function of JVM is to execute the bytecode produced by compiler.

JAVA BASICS



Each operating system has different JVM, however the output they produce after execution of bytecode is same across all operating systems.

That is why we call java as platform independent language.

Bytecode

As discussed above, javac compiler of JDK compiles the java source code into bytecode so that it can be executed by JVM.

The bytecode is saved in a **.class** file by compiler.

JAVA BASICS



Java Development Kit(JDK)

As the name suggests this is complete java development kit that includes JRE (Java Runtime Environment), compilers and various tools like JavaDoc, Java debugger etc.

In order to create, compile and run Java program we would need JDK installed on our computer.

Java Runtime Environment(JRE)

JRE is a part of JDK which means that JDK includes JRE.

JRE includes JVM, browser plugins and applets support.

Features of Java



Main features of Java

The following are the features of Java language

1. Simple
2. Object oriented language
3. Architecture Neutral
4. Secured
5. High performance
6. Multithreaded
7. Dynamic
8. Distributed
9. Database connectivity

Features of Java



1.Simple

Java is a simple language.

We can easily learn it when we have better knowledge in Object oriented methodology.

Most of the syntax in Java are inherited from C and C++.

2.Object oriented language

It supports the features of object oriented programming concept.

The following are the principles of object oriented systems.

- a. Encapsulation
- b. Polymorphism
- c. Inheritance
- d. Abstraction

Features of Java



a.Encapsulation - Encapsulation is defined as the wrapping up of data under a single unit.

It is the mechanism that binds together code and the data it manipulates.

Other way to think about encapsulation is, it is a protective shield that prevents the data from being accessed by the code outside this shield.

b.Polymorphism - The word polymorphism means having many forms.

In simple words, we can define polymorphism as the ability of a method behaving differently in different situations.

Real life example of polymorphism: A person at the same time can have different characteristic and roles. Like a man at the same time is a father, a husband, an employee.

Features of Java



c. Inheritance - The process by which one class acquires the properties (data members) and functionalities (methods) of another class is called **inheritance**.

The aim of inheritance is to provide the reusability of code so that a class has to write only the unique features and rest of the common properties and functionalities can be extended from the another class.

Child Class:

The class that extends the features of another class is known as child class, sub class or derived class.

Parent Class:

The class whose properties and functionalities are used (inherited) by another class is known as parent class, super class or Base class.

Features of Java



d. Abstraction – Hiding the complex parts of the program from the end user.

3. Robust

Robust means reliable. Main reason for programming failures are

1. Memory management mistakes
2. Mishandled exceptional conditions (Run time)

Java programming language is developed in a way that puts a lot of emphasis on early checking for possible errors, that's why java compiler is able to detect errors that are not easy to detect in other programming languages.

The main features of java that makes it robust are garbage collection, Exception Handling and memory allocation.

Features of Java



4. Architecture Neutral

A java program is compiled to create the byte code.

Byte code is the collection of instructions that can be executed on a special machine called as Java Virtual Machine (JVM).

But any machine can be converted into JVM by implementing java interpreter in it.

5. Secured

Java is a secured language. Security is achieved in many ways.

Two important constraints are

1. Using Encapsulation
2. No support for pointers

Features of Java



6. High performance

Performance is the prime characteristic of any system which is mostly measured on the time required to get the solution.

A system's performance is reduced if it has lots of unwanted information (Garbage).

Java performs the garbage collection automatically.

So the garbage is cleaned automatically over a period of time.

Java also has a special built-in class called as “**gc**” to do garbage collection explicitly.

Features of Java



7. Distributed

In Java, we can split a program into many parts and store these parts on different computers.

A Java programmer sitting on a machine can access another program running on the other machine.

This feature in Java gives the advantage of distributed programming, which is very helpful when we develop large projects.]

Java helps us to achieve this by providing the concept of **RMI (Remote Method Invocation)** and **EJB (Enterprise JavaBeans)**.

Features of Java



8. Multi-threaded and Interactive

A thread is an independent path of execution within a program, executing concurrently.

Multithreaded means handling multiple tasks simultaneously or executing multiple processes of the same program in parallel.

Features of Java



9. Dynamic and Extensible

We can add classes and add new methods to classes, creating new classes through the property of inheritance.

This makes it easier for us to **expand** our own classes and even **modify** them.

Java gives the facility of dynamically linking new class libraries, methods, and objects. I

It is highly dynamic as it can adapt to its evolving environment.

Java even supports functions written in other languages such as C and C++ to be included in Java programs.

These functions are called “native methods”.

These methods are dynamically linked at runtime.

Features of Java



10. Database Connectivity

Java language is used to create a database environment.

For example, a Relational Database can be accessed with Java language by implementing JDBC and ODBC connectivity.

11. Network programming

Java supports network programming.

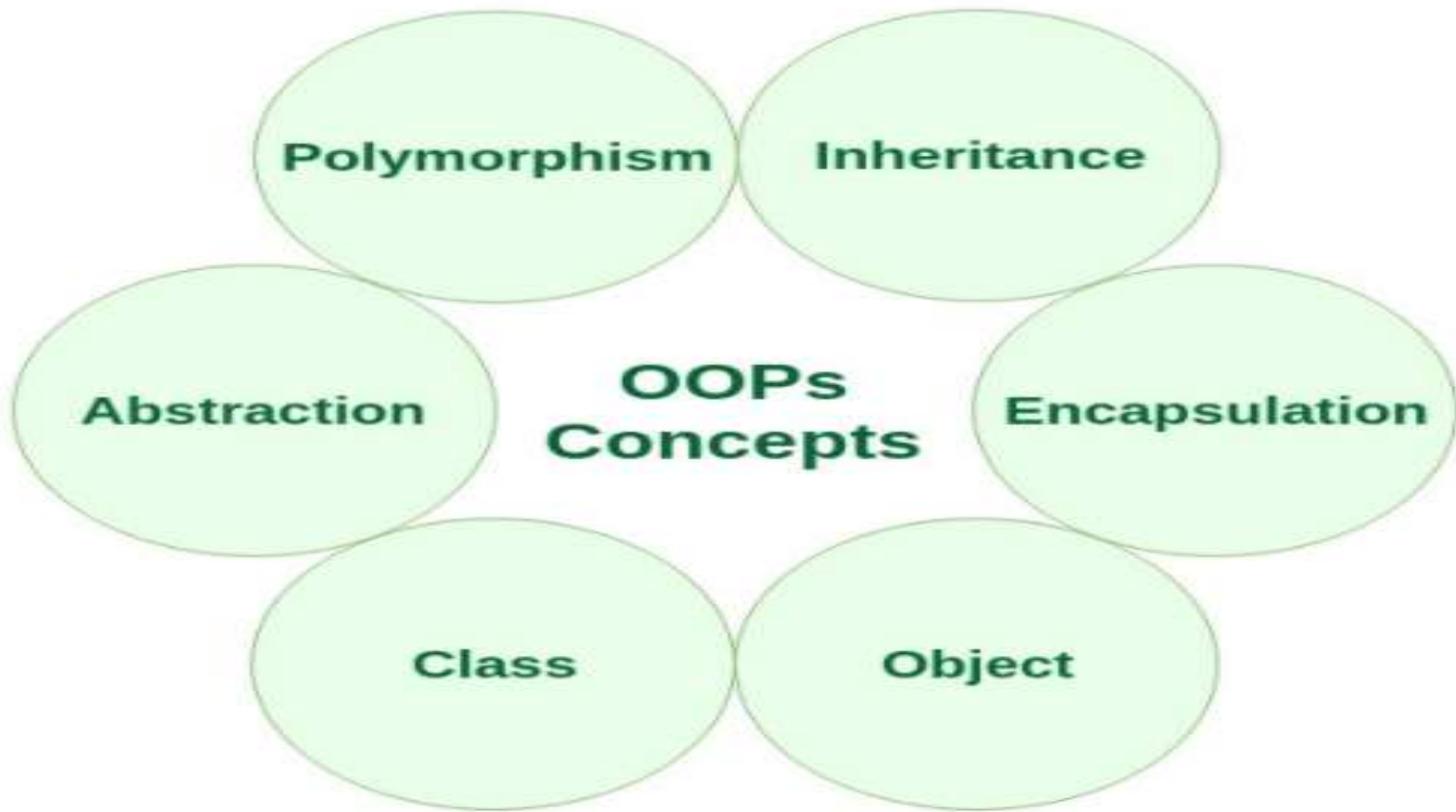
This is the prime feature of this language.

By writing program in Java, it is possible to send and receive messages / information between two or multiple users.

And the list goes on including Embedded systems, IOT, Mobile computing, etc.

Object Oriented Paradigm

Characteristics of OOP



Topics

- What is an Object ?
 - Graphical View of an Object
 - Object Examples
 - What is a Class ?
 - Object vs Class ?
 - Class Examples
-



What is Object?

- Object Means Combination of Data (**Attributes**) and Logic (**Algorithm, Behavior, Functions, Operations**) of some real world entity. For example Student, Box, Account, Time
 - Every real-world object has two characteristics :
 - **Data-Part/State** [Also known as attributes or properties]
 - **Behavior** [Also known as operations / Algorithmic / Logic Part]
 - **Software Object is conceptually similar to a every real-world object.**
-

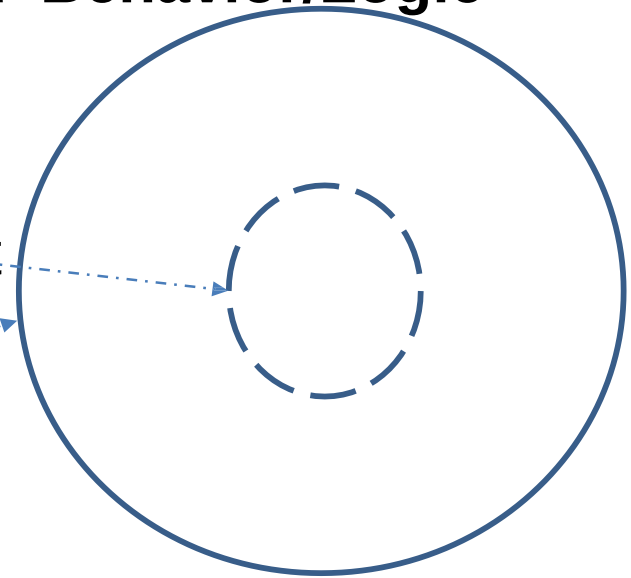
Object: Graphical View

Object Keeps State/Data Part and Behavior/Logic Part Together

State/Data Part

Operations/Logic/Algorithmic Part

A Software Object

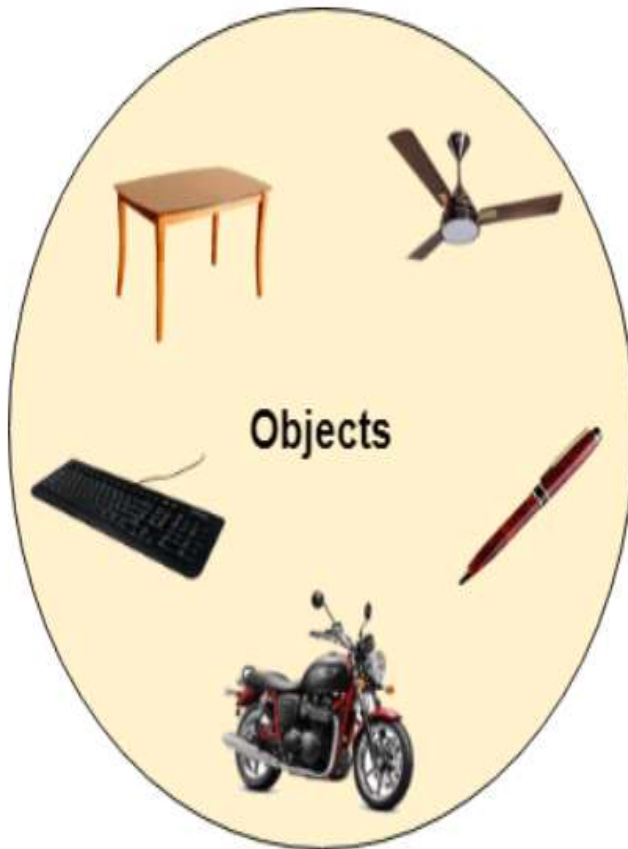




Object Examples

- **Box Object :**
 - State/Data Part : length, width, height, Color [Attributes/Instance Fields]
 - Behavior Part: computing area, computing volume [Operations, Methods]
 - **Dog Object :**
 - State/Data Part : name, breed, color [Attributes / Instance Fields]
 - Behavior Part : barking, fetching, wagging [Operations, Methods]
 - **Account Object :**
 - State/Data Part : account number, account holder name, balance, type of account [Attributes/ Instance Fields]
 - withdrawing an amount, depositing an amount, checking balance of a account [Behavior, Operations, Methods]
-

Object Examples



A

State

Represents the data of an object.

Behavior

represents the behavior of an object such as deposit, withdraw, etc.

B

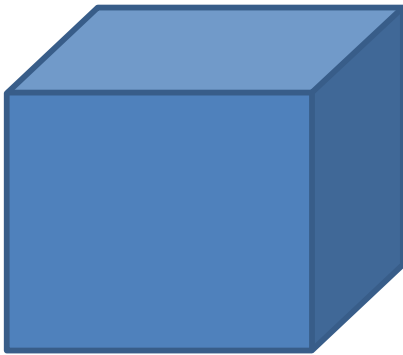
C

Identity

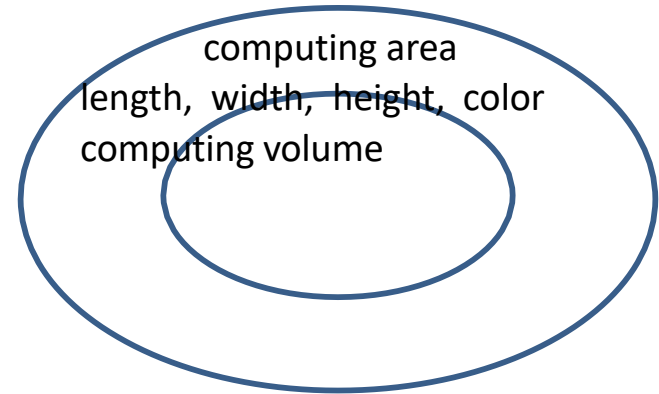
It is used internally by the JVM to identify each object uniquely.

Object Examples

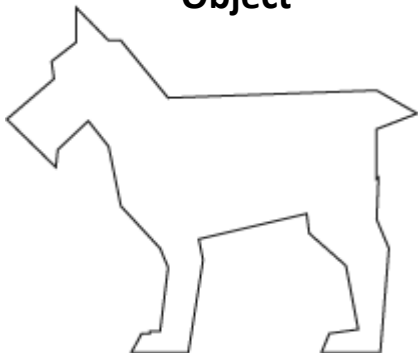
Box as a Real World Object



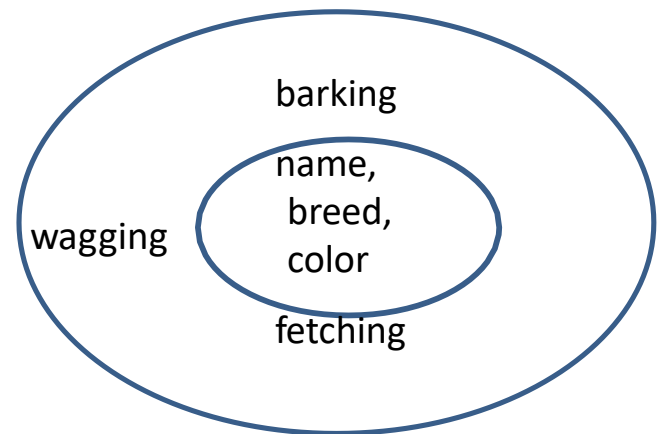
Box as a Software Object



Dog as a Real World Object



Dog as a Software Object



Object



➤ Syntax:

```
classname objectname;  
objectname = new classname();
```

➤ Example:

Class Name Keyword

↓ ↓

```
Student student1 = new Student();
```

Object Name Constructor

↑ ↑





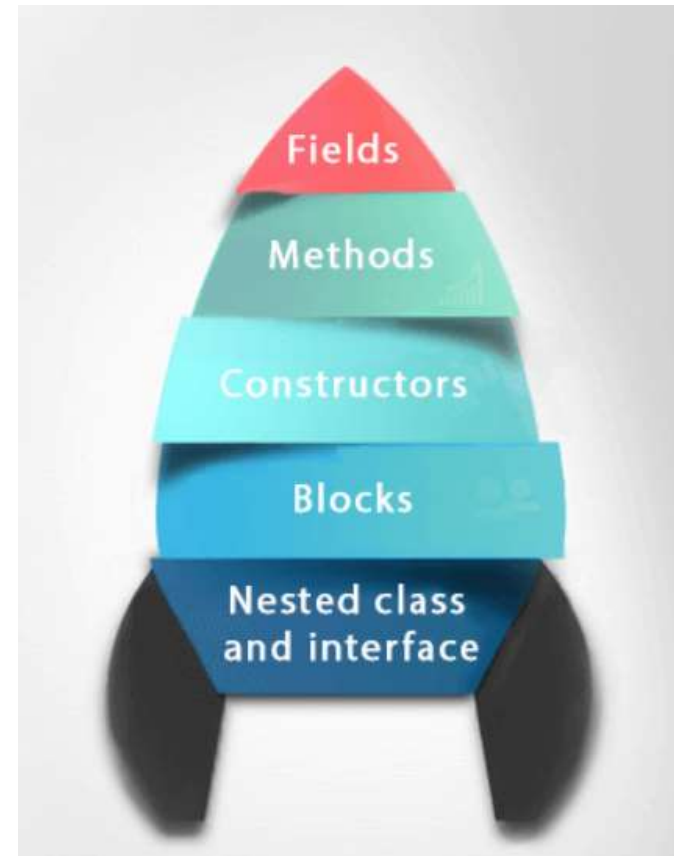
What is Class?

- Objects are grouped in classes
 - A class is a collections of objects having similar behavior and attributes
 - An object is simply a single instance of class.
 - Objects can not be instantiated (or created) without defining a class
 - Classes are defined whereas objects are created.
 - In order to create an object, you have to first define the class of that object
-

Class

Syntax:

```
modifier class ClassName  
{  
    Data_members;  
    Constructors();  
    Methods();  
}
```



Class Example : Box Class

class Box

{

private	double	length;
private	double	width; height;
private	double	

Instance Fields

public	double	area()
{		
return 2* (length * width + width * height + height * length);		
}		
public	double	volume()
{		
return length * width * height;		
}		

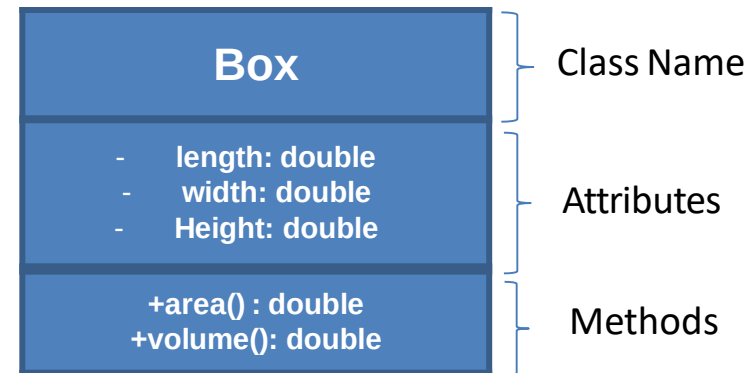
Methods

}// End of class

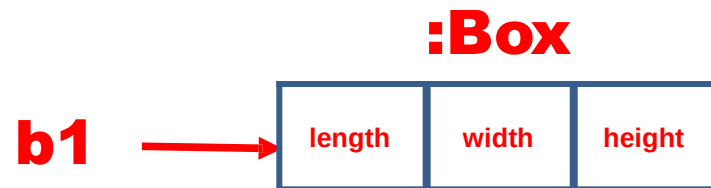


Class Example : Box Class

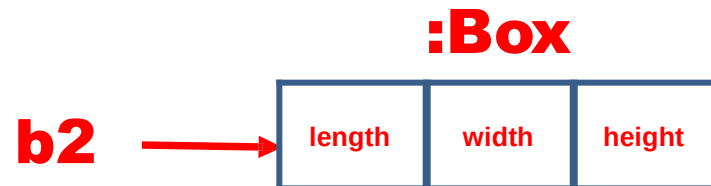
```
class Box
{
    private double    length;
    private    double width;
    private    double height;
    public double    area()
    {
        return 2* (length * width + width * height + height * length);
    }
    public double    volume()
    {
        return length * width * height;
    }
} // End of class
```



Box b1 = new Box();

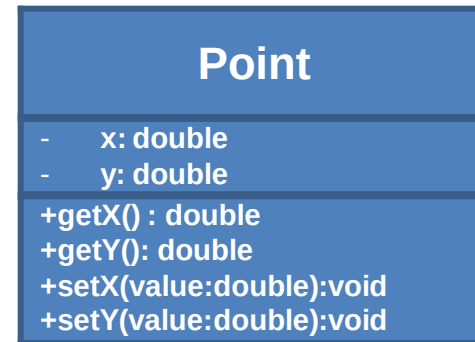


Box b2 = new Box();



Class Example : Point

```
class Point
{
    private double x;
    private double y; getX()
    public double
    {
        return x;
    }
    public double getY()
    {
        return y;
    }
    public void setX(double value)
    {
        x = value;
    }
    public void setY(double value)
    {
        y = value;
    }
}
// End of class
```



Point p1 = new Point();
:Point



Point p2 = new Point();
:Point



Abstraction



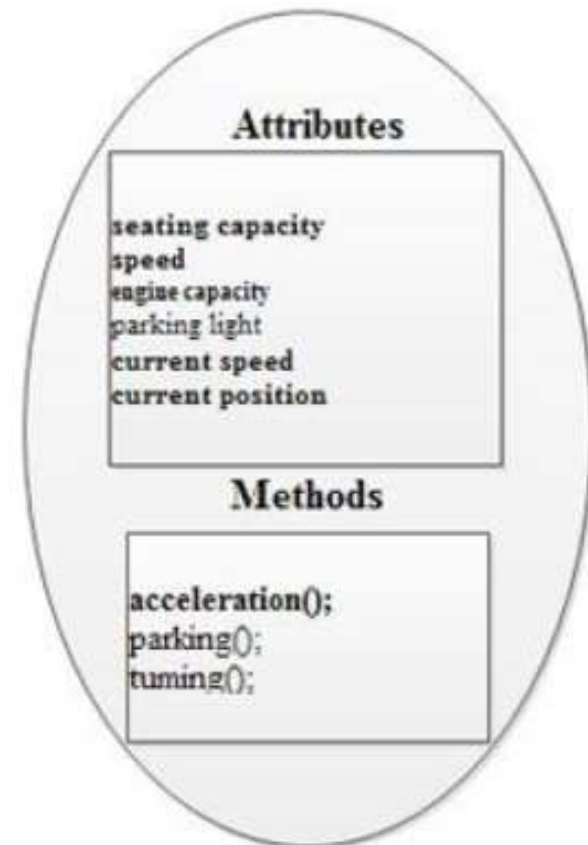
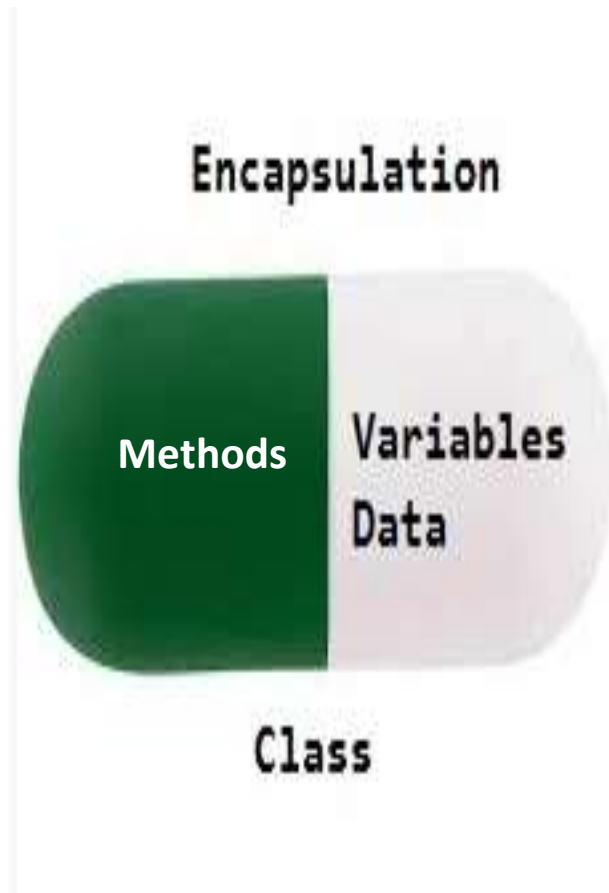
- Data Abstraction is the property by virtue of which only the essential details are displayed to the user.
- The trivial or the non-essentials units are not displayed to the user. Ex: A car is viewed as a car rather than its individual components.
- Data Abstraction may also be defined as the process of identifying only the required characteristics of an object ignoring the irrelevant details.
- The properties and behaviours of an object differentiate it from other objects of similar type and also help in classifying/grouping the objects.

Encapsulation



- Encapsulation is defined as the wrapping up of data under a single unit. It is the mechanism that binds together code and the data it manipulates.
- Technically in encapsulation, the variables or data of a class is hidden from any other class and can be accessed only through any member function of own class in which they are declared. As in encapsulation, the data in a class is hidden from other classes, so it is also known as data-hiding.
- Encapsulation can be achieved by Declaring all the variables in the class as private and writing public methods in the class to set and get the values of variables.

Encapsulation



Polymorphism



- Polymorphism is the ability of an object to take on many forms. The most common use of polymorphism in OOP occurs when a parent class reference is used to refer to a child class object.
- Any Java object that can pass more than one IS-A test is considered to be polymorphic. In Java, all Java objects are polymorphic since any object will pass the IS-A test for their own type and for the class Object.
- It is important to know that the only possible way to access an object is through a reference variable. A reference variable can be of only one type. Once declared, the type of a reference variable cannot be changed.

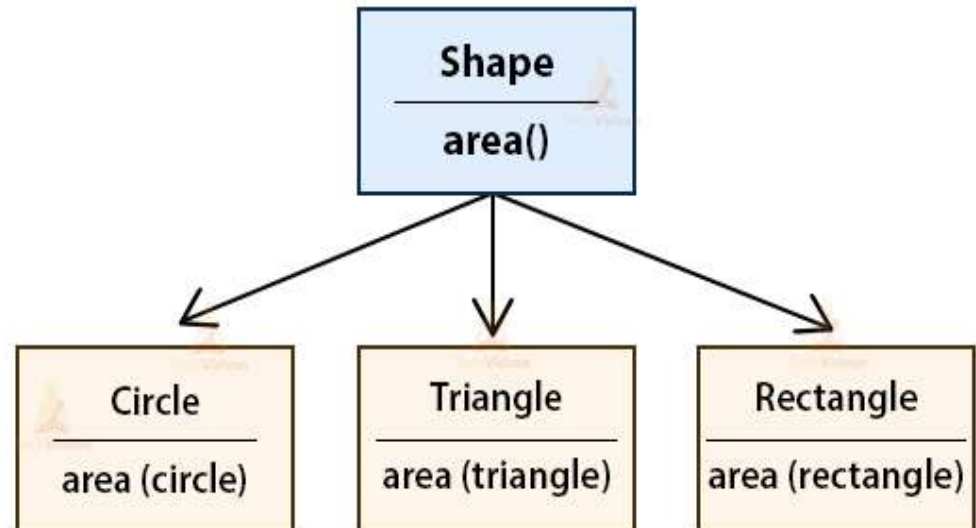
Polymorphism



➤ Polymorphism in Java are mainly of 2 types:

Overloading in Java

Overriding in Java



Inheritance



➤ Inheritance can be defined as the process where one class acquires the properties (methods and fields) of another. With the use of inheritance the information is made manageable in a hierarchical order.

➤ The class which inherits the properties of other is known as subclass (derived class, child class) and the class whose properties are inherited is known as superclass (base class, parent class).

➤ **extends** Keyword

➤ **extends** is the keyword used to inherit the properties of a class. Following is the syntax of extends keyword.

Inheritance



➤ Important terminology:

Super Class: The class whose features are inherited is known as superclass(or a base class or a parent class).

➤ **Sub Class:** The class that inherits the other class is known as subclass(or a derived class, extended class, or child class). The subclass can add its own fields and methods in addition to the superclass fields and methods.

➤ **Reusability:** Inheritance supports the concept of “reusability”, i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of the existing class.

Inheritance



➤ Syntax:

```
class derived_class extends base_class
{
    //methods and fields
}
```

Types of Inheritance





Exercise

- Think about the class named 'Student'. Define its state and operations
 - Define the instance fields (attributes) and methods of class named 'Line'.
-

JAVA BASICS – Character Set



CHARACTER SET

The character set is a set of alphabets, letters and some special characters that are valid in Java language.

The smallest unit of Java language is the characters needed to write java tokens.

These characters are defined by Unicode character set.

Whenever we write any Java program then it consists of different statements.

Each Java Program is set of statements and each statement is set of different Java programming lexemes.

In Java Programming each and every character is considered as a single lexeme. i.e Basic Lexical Element.

JAVA BASICS – Java tokens



The Java compiler breaks the line of code into text (words) is called **Java tokens**.

These are the smallest element of the Java program.

The Java compiler identifies these words as tokens.

These tokens are separated by the delimiters.

It is useful for compilers to detect errors.

Remember that the delimiters are not part of the Java tokens.

JAVA BASICS – Java tokens



JAVA BASICS – Character Set



Alphabets

Java accepts both lowercase and uppercase alphabets as variables and functions.

a..z and A..Z

Digits

0,1,....9

Special Characters

~ , ! , # , \$, % , + , - , * , / , % , (,) , [,] , { , } , > , < , ; , : , “ , ‘ , . , etc

White spaces

Tab, new line or line space

JAVA BASICS – Constants (Literals)



There are several values in the real world which will never change.

A square will always have four sides, PI to three decimal places will always be 3.142, and a day will always have 24 hours.

These values remain constant.

When writing a program it makes sense to represent them in the same way - as values that will not be modified once they have been assigned to a value.

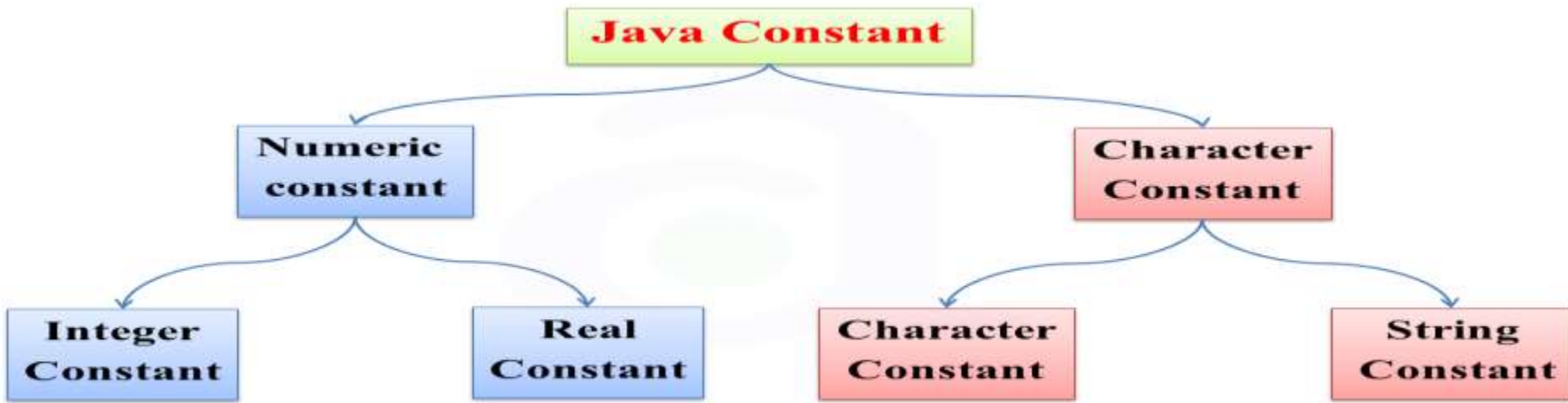
These values are known as constants.

Constants in java are fixed values those that do not change during the Execution of program.

Constants / Literals in Java



1. Integer Constants
2. Real Constants
3. Single Character Constants
4. String Constants
5. Backslash Character Constants



JAVA BASICS - Constants

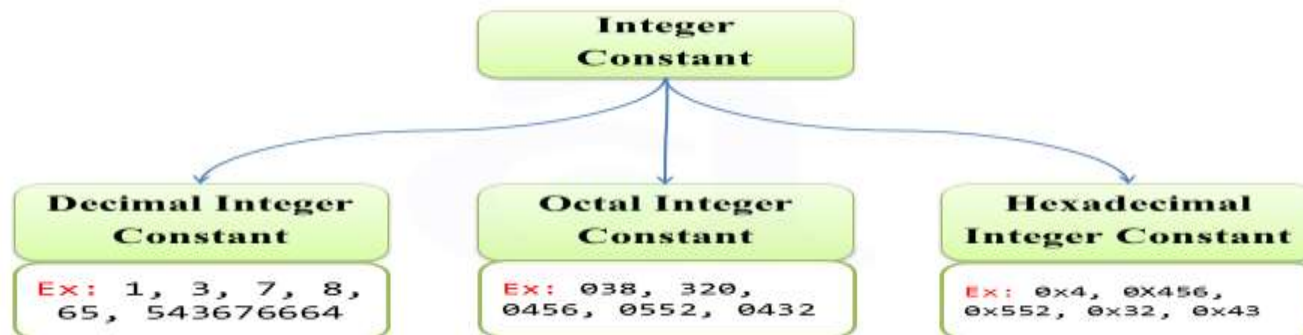


Integer Constants

Integer Constants refers to a Sequence of digits which Includes only negative or positive Values.

Example

1. Decimal Integer Constants
2. Octal Integer Constants
3. Hexadecimal Integer Constants



Object Oriented Programming , Lecture 1

JAVA BASICS - Constants



An Integer Constant must have at Least one Digit

- it must not have a Decimal value
- it could be either positive or Negative
- if no sign is Specified then it should be treated as Positive
- No Spaces and Commas are allowed

Decimal Integer Constants

Decimal constant consists of a set of digits 0 through 9, preceded by an optional –or + sign.

Examples : 234, 134, -1, -14567

Octal Integer Constants

An octal integer constant consists of any combination of digits from the set 0 through 7, with a leading 0.

JAVA BASICS - Constants



Examples : `012`, `0234`, `045`

Hexadecimal Integer Constants

An octal integer constant consists of any combination of digits from the set 0 through 7, with a leading `0x` or `0X`.

Examples : `0x12`, `0x4`, `0x45`

Real Constants

Integer numbers are unable to represent distance, height, temperature, price, and so on. These information consist of fractional parts or real parts like 56.890.

Such numbers are called Real or Floating Point Constants

JAVA BASICS - Constants



Single Character Constants

A Character is a Single Alphabet, a single digit or a Single Symbol that is enclosed within Single quotation marks.

1. Character Constant **Can hold Single character at a time.**
2. Contains Single Character Closed within a pair of **Single Quote**
3. Single Character is smallest **Character Data Type** in C.
4. It is Possible to Perform Arithmetic Operations on Character Constants

Examples are :

'A', 'c', 'l', 'l', '&', ':'

JAVA BASICS - Constants



String Constants

String is a Sequence of Characters Enclosed between double Quotes

These Characters may be digits, Alphabets Like "Hello", "1234" etc.

1. The string is a "**Sequence of Characters**".
2. String **Constant is written within a Pair of Double Quotes**.
3. The string is considered as an **Array of Characters**.
4. In C, **String data type** is not available.

Backslash Character Constants

Java Also Supports Backslash Constants (Escape sequence) those are used in output methods For Example \n is used for new line Character

67

Object Oriented Programming , Lecture 1

JAVA BASICS - Constants



Some of the backslash characters are

\a – Produce bell sound

\b – backspace

\f – form feed

\t – Horizontal tab

\v – Vertical tab

\n – new line

\? – Question mark

\' – Single quote

\r – Carriage return

JAVA BASICS - Variables



Java Variables

A variable is a location in memory (storage area) to hold data. To indicate the storage area, each variable should be given a unique name (identifier).

Declaration of a variable

A variable is declared as

```
int speedLimit=80;
```

Here, speedLimit is a variable of int data type and is assigned value 80.

Meaning, the speedLimit variable can store integer values.

In the example, we have assigned value to the variable during declaration.

However, it's not mandatory.

JAVA BASICS - Variables



We can declare variables without assigning the value, and later we can store the value as you wish.

For example,

```
int speedLimit;  
speedLimit =80;
```

The value of a variable can be changed in the program, hence the name 'variable'.

For example,

```
int speedLimit = 50;  
speedLimit =80;
```

JAVA BASICS - Naming conventions for Identifiers / Variables-



Java Naming conventions for Identifiers / Variables-

Java programming language has its own set of rules and conventions for naming variables.

Variables in Java are case-sensitive.

A variable's name is a sequence of Unicode letters and digits.

It can begin with a letter, \$ or _.

However, it's a convention to begin a variable name with a letter.

Also, variable names cannot use whitespace in Java

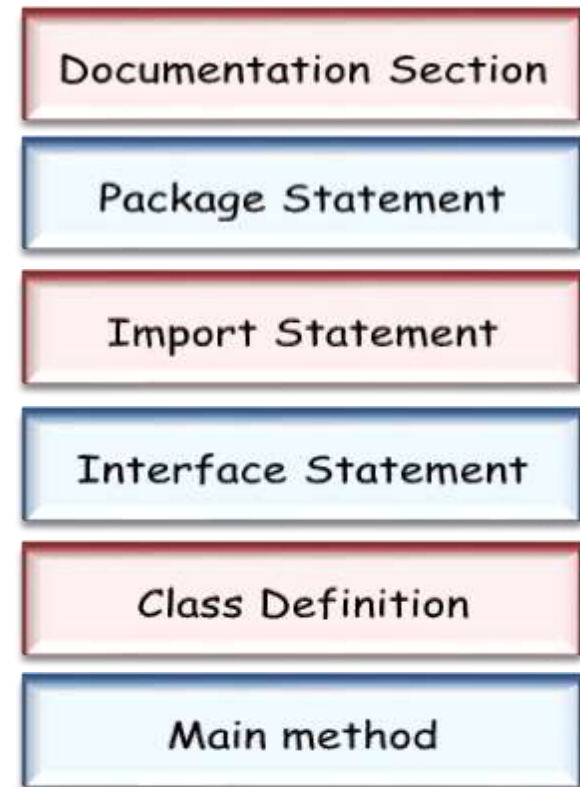
Examples

`var1`, `simint`, `Compound_Interest`, `output_1_2`

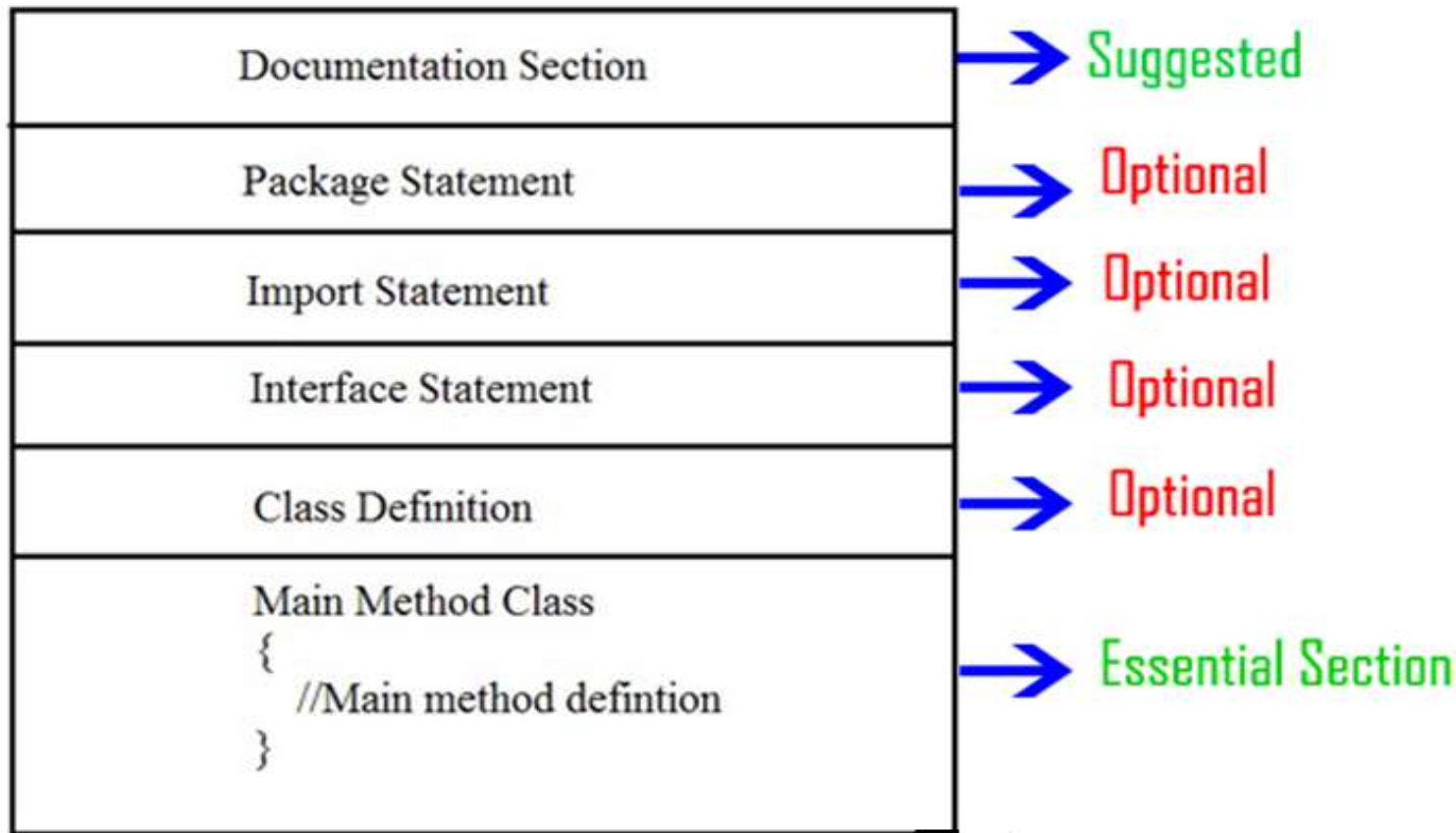
Java program structure

Basic Structure of a Java program

- Java programming language is platform-independent and a secure programming language.
- A Java program consists of different sections. Some of them are mandatory but some are optional.
- The optional section can be excluded from the program depending upon the requirements of the programmer.



Basic Structure of a Java program



Basic Structure of a Java program

```
package com.mm;
import java.util.Date;
5. /**
   * @author tutorialkart.com
   */
   public class ProgramStructure {
10.     int repetetions = 3;

       public static void main(String[] args){
           ProgramStructure programStructure = new ProgramStructure();
           programStructure.printMessage("Hello World. I started learning Java.");
15.     }

       public void printMessage(String message){
           Date date = new Date();
           for(int index=0; index < repetetions; index++){
20.               System.out.println(message+" From "+date.toGMTString());
           }
       }
   }
```

Annotations in the code:

- `package com.mm;` is labeled **package declaration**.
- `import java.util.Date;` is labeled **import statements**.
- The block `/** @author tutorialkart.com */` is labeled **comments**.
- `public class ProgramStructure {` is labeled **class name**.
- `int repetetions = 3;` is labeled **global variable**.
- The block `public static void main(String[] args){ ... }` is labeled **main method**.
- The block `public void printMessage(String message){ ... }` is labeled **method**.
- Inside the `printMessage` method, `Date date = new Date();` is labeled **variable local to the method**.
- Inside the `printMessage` method, `for(int index=0; index < repetetions; index++){` is labeled **variable local to the for loop**.

Writing a simple Java program



A first example

Program:

```
class HelloWorld
{
    public static void main (String args[])
    {
        System.out.println ("Hello World");
    }
}
```

Output:

Hello World

A first example

```
/**
 *Hello World Application
 *Our first example
 */
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World!"); // display output
    }
}
```

```
$ javac HelloWorld.java
```

```
$ ls HelloWorld.class HelloWorld.java
```

```
$ java HelloWorld Hello World
```

Structure of a Java program



A Java program is a collection of Java statements.

Each programming language follows a particular syntax for writing a program.

A Java program has the following sections

1. Documentation Section
 2. Package Statement
 3. Import Statement
 4. Class definition
 5. Abstract Class Definition
 6. Interface Definition
 7. Class containing the main() method definition
- Optional Sections

Mandatory for all Java Application programs to contain at least one class having the main method.



Thanks



Object Oriented Programming Design

BITS Pilani
Pilani Campus

Presented by
Dr Ramya Devi M
mramyadevi@wilp.bits-pilani.ac.in
CS - 02 : 05th Jul 2025



BITS Pilani
Pilani Campus

Module 1 – Lecture 2

BASICS IN JAVA



DATATYPES

A **data type** informs the compiler or interpreter what type of data will be stored in the variable.

It defines the operations that can be done on the data and what type of values can be stored.

In Java, According to the properties they possess, data types are divided into two groups:

1. Primitive Data types
2. Non – Primitive Data types

Primitive Data Types: A primitive data type is pre-defined by the programming language. The size and type of variable values are specified, and it has no additional methods.

Data Types



Non-Primitive Data Types: These data types are not actually defined by the programming language but are created by the programmer.

They are also known as “reference variables” or “object references” since they refer to a memory location which stores the data.

Primitive Data Types

Data types in Java are classified into 4 categories, namely **int**, **float**, **character** and **boolean**.

The int data type is further sub-divided into four categories as follows:

- byte data type
- short data type
- int data type
- long data type

Data Types



The decimal or floating point type has two categories namely

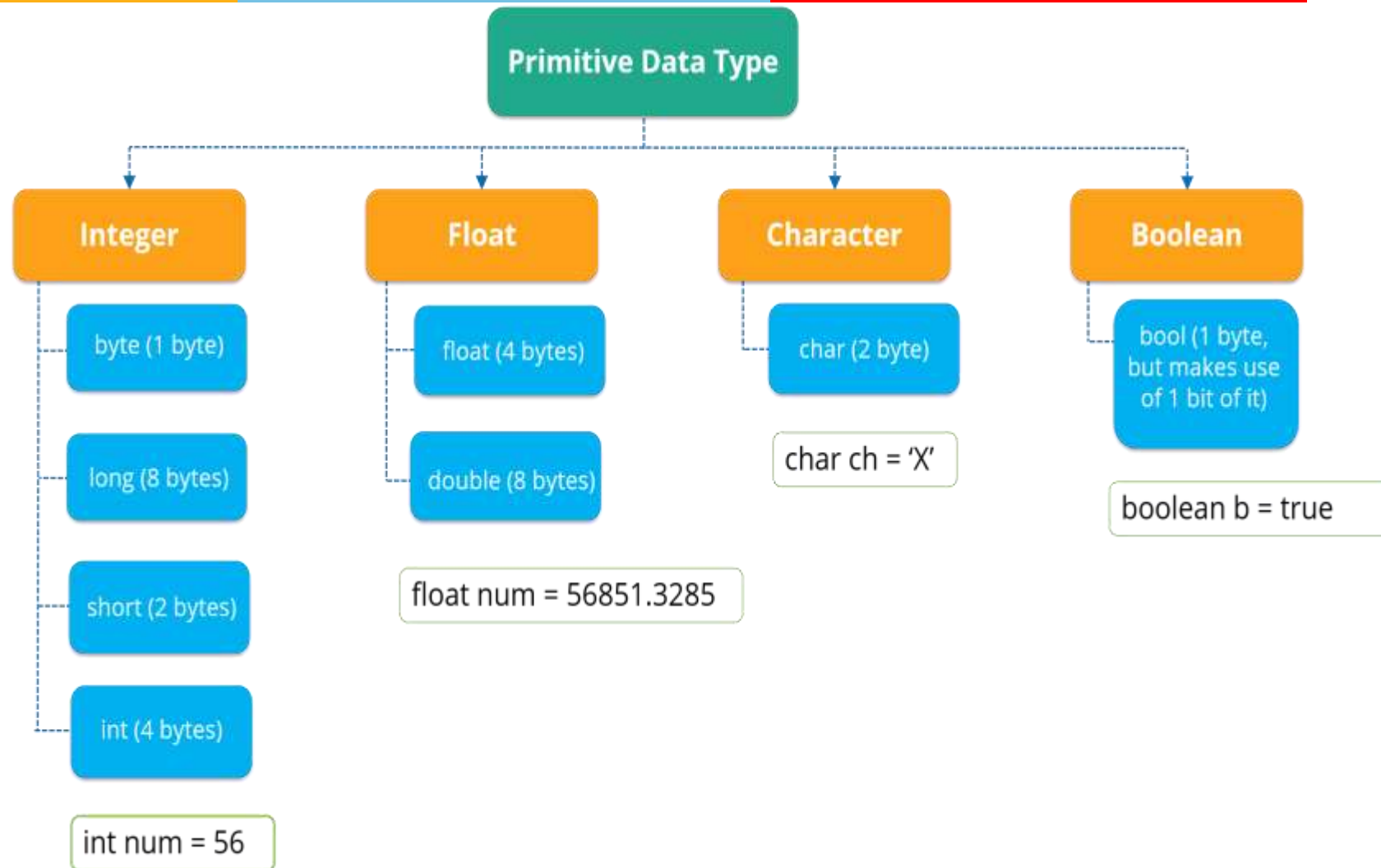
- float data type
- double data type

Apart from the two numerical categories, there are two datatypes to represent characters and Boolean value namely

- boolean data type
- char data type

We can refer to the below figure to understand the different data types with respect to the memory allocated to them.

Data Types



Data Types



boolean data type

A boolean data type comprises of a bit of information and can store only **true** or **false** values.

This data type is used to control **true/false conditions**.

byte data type

This is an example of a primitive data type.

It is an 8-bit signed two's complement integer.

It stores whole numbers that lie between -128 to 127.

A byte data type is helpful for saving memory in large amounts.

char data type

This data type is used to store a **single** character.

The character must be enclosed within single quotes, like 'E' or 'e'.

Data Types



short data type

A short data type is greater than byte in terms of size and less than a integer. It stores the value that ranges from -32,768 to 32767.

int data type

This data type can store whole numbers from -2147483648 to 2147483647. Generally, int is the preferred data type when we create variables with a numeric value.

long data type

This data type is a 64-bit two's complement integer. By default, the size of a long data type is 64 bit and its value ranges from -2^{63} to $2^{63}-1$

Data Types



Floating Point Datatypes

We should use a floating point type whenever we need a number with a decimal, such as 8.88 or 3.14515.

float data type

This data type can store fractional numbers from $3.4e-038$ to $3.4e+038$.

double data type

The double data type can store fractional numbers from $1.7e-308$ to $1.7e+308$.

Note that you should end the value with a d for double and f for floating point values.

Example : `double x=1.3452109d;`
`float y=1.3452109f;`

Data Types

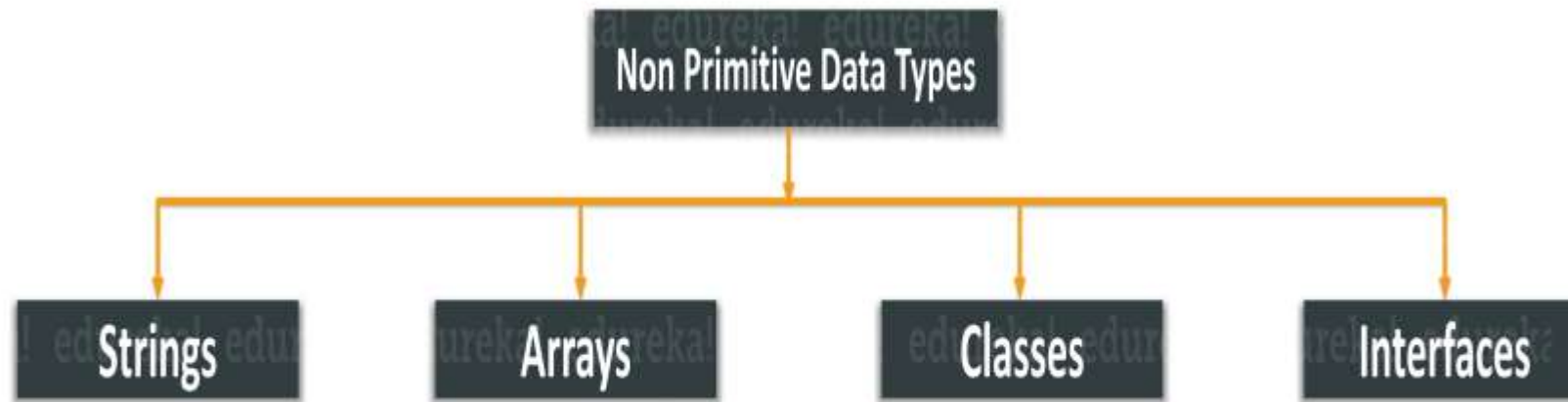


Non-Primitive Datatypes

Non-Primitive data types refer to objects and hence they are called **reference types**.

Examples of non-primitive types include

Strings, Arrays, Classes, Interface, etc. Below image depicts various non-primitive data types.



Data Types



Strings: String is a sequence of characters.

But in Java, a string is an object that represents a sequence of characters.

The *java.lang.String* class is used to create a string object.

Arrays: Arrays in Java are homogeneous data structures implemented in Java as objects.

Arrays store one or more values of a specific data type and provide indexed access to store the same.

A specific element in an array is accessed by its index.

Classes: A class in Java is a blueprint which includes all our data.

A class contains fields(variables) and methods to describe the behavior of an object.

Interface: Like a class, an *interface* can have methods and variables, but the methods declared in interface are by default abstract (only method signature, no body).



BITS Pilani
Pilani Campus

Standard I / O, Operators and Control Statements IN JAVA

Learning Objectives



- Reading the inputs using Scanner class
- Reading the inputs through command line
- Displaying output on standard output device
- Definition for an operator
- Various types of operators
- Control statements

Standard I/ O for a PROGRAM



There are various methods for getting the inputs to the program through the standard input device, Keyboard.

The following are the methods of getting user inputs to the program through the keyboard.

1. Input in the command line argument
2. Input using the Scanner class
3. Using Streams for input from different streams. (Console, File, etc.)

Standard I/ O for a PROGRAM



1. Getting the inputs using Command Line Argument

It is the information that is passed to the program at the time of execution.

The information passed is stored in the String array argument

In the main() method.

It is the information that directly follows the program's name on the command line when it is running.

Getting the INPUT in Command Line



Important Points in using Command Line Arguments are:

- There is no restriction on the number of java command line arguments. We can specify any number of arguments
- The input Information is passed to the main() method as Strings.
- They are captured into the String array of the main method
- Inside the program, there should be some type conversion to convert the string into the type what we require in the program to process.

Example While running a class **Demo**, we can specify the inputs as command line arguments as

C:\Program\Java>java Demo arg1 arg2 arg3

Getting the INPUT in Command Line



Example : Input through Command Line arguments

Step 1) Type the code into an editor.

Class ComLineDemo

```
{  
    public static void main(String aa[])  
    {  
        String name;  
        int num;  
        float salary, gross, ded;  
        name=aa[0];  
        num=Integer.parseInt(aa[1]);  
        gross=Float.parseFloat(aa[2]);  
    }  
}
```

Getting the INPUT in Command Line



```
        ded=Float.parseFloat(aa[3]);
        salary=gross-ded;
        System.out.println(" Name "+name);
        System.out.println(" Number "+num);
        System.out.println(" Net Salary "+salary);
    }
}
```

Step 2) Save & Compile the code >javac ComLineDemo.java

Step 3) Run the code as

>java ComLineDemo Virat 1995 789045.75 3456.30

aa[0] aa[1] aa[2] aa[3]

Name Num Gross Ded

Getting INPUT - Scanner



Scanner is a class in `java.util` package used for obtaining the input of the primitive types like `int`, `double`, etc. and `Strings`.

To create an object of `Scanner` class, we usually pass the predefined object `System.in`, which represents the standard input stream.

We may pass an object of class `File` if we want to read input from a file.

To read numerical values of a certain data type `D`, the function to use is `nextD`.

Examples:

To read a value of type `short`, we can use `nextShort()`

To read a value of type `int`, we can use `nextInt()`

To read a string, we use the function `nextLine()`.

Getting the INPUT - Scanner



To read a single character, we use `next().charAt(0)`. `next()` function returns the next token/word in the input as a string and `charAt(0)` function returns the first character in that string.

Example program for getting Input using SCANNER class

```
import java.util.*;
class ScannerDemo
{
    public static void main(String aa[])
    {
        String name;
        int num;
```


Getting the INPUT - Scanner



```
char gender;  
double salary;  
Scanner S=new Scanner(System.in);  
System.out.println("Enter Name and Number of EMPLOYEE");  
name=S.nextLine();  
num=S.nextInt();  
System.out.println("\n Enter Gender of the EMPLOYEE");  
gender=S.next.charAt(0);  
System.out.println("\n Enter Salary of the EMPLOYEE");  
salary=S.nextDouble();  
System.out.println("\n Name of the EMPLOYEE"+name);
```

Getting the INPUT - Scanner



```
System.out.println("\n Number of the EMPLOYEE"+num);  
System.out.println("\n Gender of the EMPLOYEE"+gender);  
System.out.println("\n Salary of the EMPLOYEE"+salary);  
}  
}
```

Note: Input from streams will be dealt in I/O streams session

Standard Output



To display the output on the screen, we use the format

`System.out.println(Data to be displayed);`

Java **`System.out.println()`** is used to print an argument that is passed to it. The statement can be broken into 3 parts which can be understood separately as:

- 1. System** - It is a final class defined in the java.lang package.
- 2. out:** This is an instance of PrintStream type, which is a public and static member field of the System class
- 3. println() :** PrintStream class has a public method println(), which can be invoked.

This is an upgraded version of print().

It prints any argument passed to it and adds a new line to the output. System.out represents the Standard Output Stream.

Standard Output



Example: `int x=1,y=2;`

`System.out.println("Sum of 10 and 20 is "+(10+20));`

`System.out.println(" X = "+x+" Y = "+y);`

The output will be

Sum of 10 and 20 is 30

X = 1 Y = 2

Difference between `System.out.print()` and `System.out.println()`

`System.out.print()`: This method prints the text on the console and the cursor remains at the end of the text at the console.

The next printing takes place from just here.

`System.out.println()`: This method prints the text on the console and the cursor remains at the start of the next line at the console.

OPERATORS in JAVA



An operator is a symbol used for doing an operation. The operators are used for mathematical or logical manipulations.

There are six types of C Operators. They are

- (i) Arithmetic Operators
- (ii) Relational Operators
- (iii) Logical Operators
- (iv) Assignment Operators
- (v) Increment and Decrement operators
- (vi) Conditional Operators
- (vii) Special operators

OPERATORS in JAVA



i. Arithmetic Operators: These operators are used in arithmetic expressions.

An arithmetic expression is an expression in which the variables and constants are combined with the arithmetic operators. The arithmetic operators are

* - Multiplication

/ - Division

+ - Addition

- - Subtraction

% - Modulo division

Examples for arithmetic expressions

$a + b \% 35$

$11 - a / b * (c / 2)$

OPERATORS in JAVA



ii. Relational Operators: These operators are used for defining relational expressions.

A relational expression is an expression in which the variables and constants are combined with the relational operators. The resultant value of a relational expression is either true or false. The relational operators are

<	-	Lesser than
>	-	Greater than
< =	-	Less than or equal to
> =	-	Greater than or equal to
= =	-	Equal to
! =	-	Not Equal to

OPERATORS in JAVA



Examples for relational expression :

(a) $a \geq b$ [true if the value of a is greater than or equal to the value of b. otherwise it is false]

(b) $b == c$ [true if the value of b is equal to the value of c, otherwise the value is false]

(c) $a < (b + c)$ [true if the value of a is lesser than the value of $b + c$, otherwise the value of the expression is false]

OPERATORS in JAVA



iii. Logical Operators - These operators are used for combining more than one relational expressions.

A Logical expression is an expression in which the variables and constants are combined with the Logical operators.

The logical operators are

&&	-	AND
	-	OR
!	-	NOT

The general format is

Relational expression 1 logical operator relational expression 2

Examples for logical expressions:

a. ((a > b) && (a > c))

b. ((Mark1 > 50) || (Mark2 > 50))

OPERATORS in JAVA



iv. Assignment Operators: The general format of this operator is

variable = expression;

A special kind of assignment operator is the short-hand assignment operator given by

Variable operator = expression;

The above assignment statement is equivalent to

variable = variable operator expression;

For example:

The statement `x = x * 100;`

can be written as `x *= 100 ;` by using the assignment operator.

By using these operators , the variable on the L.H.S need not be repeated again.

OPERATORS in JAVA



v. Increment and Decrement Operators: These operators are used for incrementing and decrementing the value of variables.

There are two formats for writing the Increment operators.

They are

1. Post increment operator which has the form **Variable ++**;
2. Pre increment operator which has the form **++ Variable** ;

Both are same till they are not assigned to variables.

Example:

Let `x = 1`;

Then after executing the statement `x ++` ; The value of `x` is 2.

Instead of the post increment operator , if we use the preincrement operator `++ x`, the value is also 2.

OPERATORS in JAVA



Otherwise if they are assigned to another variable, the post increment operator assigns the initial value first and then it is incremented whereas the pre increment operator assigns the value after the variable is incremented.

```
x = 1;
```

```
y = x ++;
```

Code 1

```
x = 1;
```

```
y = ++ x;
```

Code 2

For code 1 , the value of y is 1 and for the code 2 the value of y is 2. In both the cases , the value of x is 2.

OPERATORS in JAVA



Like the increment operators , there are two formats for writing the Decrement operators.They are

1. Post Decrement operator which has the form **Variable - -;**
2. Pre Decrement operator which has the form **- - Variable ;**

Both are same till they are not assigned to variables.

Example:

Let x = 1;

Then after executing the statement x - - ; The value of x is 0.

Instead of the post decrement operator , if we use the pre decrement operator - - x, the value is also 0.

OPERATORS in JAVA



decremented where as the pre decrement operator assigns the value after the variable is decremented.

```
x = 1;  
y = x - -;  
Code 1
```



```
x = 1;  
y = - - x;  
Code 2
```

For the code 1 , the value of y is 1 and for the code 2 the value of y is 0. In both the cases , the value of x is 2.

vi. Conditional Operator : This operator is also known as the “? :” operator or ternary operator.

The general format of the operator is

Variable = Condition ? a.e 1 : a.e 2;

OPERATORS in JAVA



Where Condition is either a relational Expression or Logical Expression

a.e 1 and a.e 2 are arithmetic expressions.

If the condition is true, the value of the a.e 1 is assigned to the variable on the L.H.S. Otherwise, the value of a.e 2 is assigned to variable.

Example:

`big = (x > y) x : y;`

for the above statement if the values of x and y are 10 and 12, then `big = 12` because the condition is false, so the value of the expression 2 (that is the value of y) is assigned to big. If the values of x and y are 100 and 10, then `big = 100`.

Homework: Find the biggest of 3 numbers using “?:” operator

OPERATORS in JAVA



vii. Special Operators : There are some special operators available in Java. They are

a. Comma operator :- It is an operator which is used to combine more than one statements into a single statement. The statements

```
x = 10;
```

```
y = 20;
```

```
z = x + y;
```

can be written in a single statement by using the comma operator as

```
z = ( x + y , x = 10 , y = 20 );
```


OPERATORS in JAVA



b. Bit wise operators:- Some operators can be used to do the bit wise operations. A bit is a binary digit whose value either 0 or 1.

The Bit wise operators are

- & - Bit wise AND
- | - Bit wise OR
- ! - Complement
- >> - Shift right
- << - Shift Left
- - Exclusive OR

OPERATORS in JAVA



c. Instanceof operator

instanceof is a binary operator used to test if an object is of a given type.

The result of the operation is either *true* or *false*.

It is also known as type comparison operator.

It helps in avoiding *ClassCastException* at runtime.

The *instanceof* operator's basic syntax is:

object instanceof (Type)

The *instanceof* operator works on the principle of the is-a relationship.

The concept of an is-a relationship is based on Class inheritance or interface implementation.

OPERATORS in JAVA



Example Program

```
class InstanceOfDemo
```

```
{
```

```
    public static void main(String aa[])
```

```
    {
```

```
        String s="Example program";
```

```
        boolean b= s instanceof String;
```

```
        System.out.println("\n Is it an Instance? "+b);
```

```
    }
```

```
}
```

The output is

Is it an Instance? true

Java Control Statements | Control Flow in Java



Java compiler executes the code from top to bottom.

The statements in the code are executed according to the order in which they appear.

However, **Java** provides statements that can be used to control the flow of Java code.

Such statements are called control flow statements.

It is one of the fundamental features of Java, which provides a smooth flow of program.

Java Control Statements



Java provides three types of control flow statements.

1. Decision Making statements

1. if statements
2. switch statement

2. Loop statements

1. do while loop
2. while loop
3. for loop
4. for-each loop

3. Jump statements

1. break statement
2. continue statement

CONTROL STRUCTURE



Control statements :

The flow is altered according to a condition. where the condition is a relational or logical expression whose value will be either TRUE or FALSE.

It is also called as the Selective Structure.

The conditional control statements are

1. If structure
2. Switch ... case structure

Decision-Making statements:



1) If Statement:

In Java, the "if" statement is used to evaluate a condition. The control of the program is diverted depending upon the specific condition. The condition of the If statement gives a Boolean value, either true or false. In Java, there are four types of if-statements given below.

1. Simple if statement
2. if-else statement
3. if-else-if ladder
4. Nested if-statement
5. Multi alternative if

CONTROL STRUCTURE



The **Common if** has the following format

```
if ( condition ) block ;
```

Where the condition may be a relational or logical expression and the block may have a single statement or more than one statements enclosed between { and }.

Example:

```
if ( marks < =50 )
```

```
System.out.println("Performance Bad , Prepare Well for the  
next exam");
```


CONTROL STRUCTURE



The **if.. else** has the following format

```
if ( condition ) block 1;
```

```
else block 2;
```

Where the condition may be a relational or logical expression and blocks may have a single or more than one statements.

Example:

```
if ( marks < =50 ) System.out.println("Performance not Good ,  
Prepare Well for the next exam");
```

```
else System.out.println(" Performance Good , Keep it UP");
```

CONTROL STRUCTURE



The **Nested if** is an if structure in which more than one if structures are overlapped.

That is , if an **if** lies within another **if** , the **if** structure is called as the **nested if**. The general format of nested if is

```
if ( condition 1 )  
{  
    if (condition 2 ) block 1  
    else block 2  
}  
else  
{  
    if (condition 3 ) block 3  
    else block 4  
}
```

CONTROL STRUCTURE



Example

```
if ( cust_cat=='A')  
    if ( salary < 2500 ) bonus = 1000;  
    else bonus= 750;  
else  
    if (salary<5000)bonus=1500;  
    else bonus=0;
```

In the example, bonus=1000 if customer category is A and getting salary less than 2500. Or bonus=750 if customer category is A and getting salary greater than or equal to 2500. Or bonus=1000 if customer category is not A and getting salary less than 5000, no bonus is given if customer category is not A and getting salary greater than or equal to 2500.

CONTROL STRUCTURE



The **Multi alternative if** structure has the following format

```
if ( condition 1 ) block1;  
else if (condition 2) block2;  
else if (condition 3) block 3;  
.....  
.....  
else block n;  
statement – X;
```

In the above format, block 1 is executed only if the condition 1 is true. After the execution , the flow goes to the statement – X.

CONTROL STRUCTURE



If the condition 1 is false, it checks whether the condition 2 is true. If it is true, block 2 is executed. After the execution, the flow goes to the statement – X and so on.

If no condition is true, block n is executed.

Note : The **multi alternative if** structure is also called as the else if ladder. **IfElseIfDemo.java**

Selective Structure with switch ... Case statement

This structure is used for doing multi way decisions such as else if ladder.

It is an alternate structure for else if ladder.

The general format for the switch .. case structure is

CONTROL STRUCTURE



```
switch ( e )
{
case op 1: block 1;
case op 2 : block 2;
case op 3: block 3;
.....
.....
.....
default: block n;
}
```

Statement – x;

CONTROL STRUCTURE



In the above format , e may be an expression or may be character or integer variable.

The options op1 , op2 , are the constant values for e.

Each block except block n has a break statement.

The block 1 is executed if the value of e is equal to op1.

After the block is executed , the flow moves to the statement – x by means of the break statement in the block.

The block 2 is executed if the value of e is equal to op2.

After the block is executed , the flow moves to the statement – x by means of break statement in the block.

The block n is executed if the value of e does not match with any optional values op1 , op2 , op n-1 .

After the block is executed , the flow moves to the statement – x automatically.

CONTROL STRUCTURE



Example for switch .. case

```
switch (cust_cat )
{
case 'A': discount = price * 10.0 / 100;
        break;
case 'B': discount = price * 7.5 / 100;
        break;
case 'C': discount = 100;
        break;
default: printf("\n Invalid customer Category");
        discount=0
}
```

SwitchDemo and SwitchDemo2



Thanks



BITS Pilani

Pilani Campus

Object Oriented Programming Design

Presented by
Dr Ramya Devi M
mramyadevi@wilp.bits-pilani.ac.in
CS - 03 : 12th Jul 2025



Looping Structure and Unconditional branching

Learning Objectives



- Various types of looping structure
- Unconditional branching

Control structures

switch Statement

A switch statement is used to test many conditions

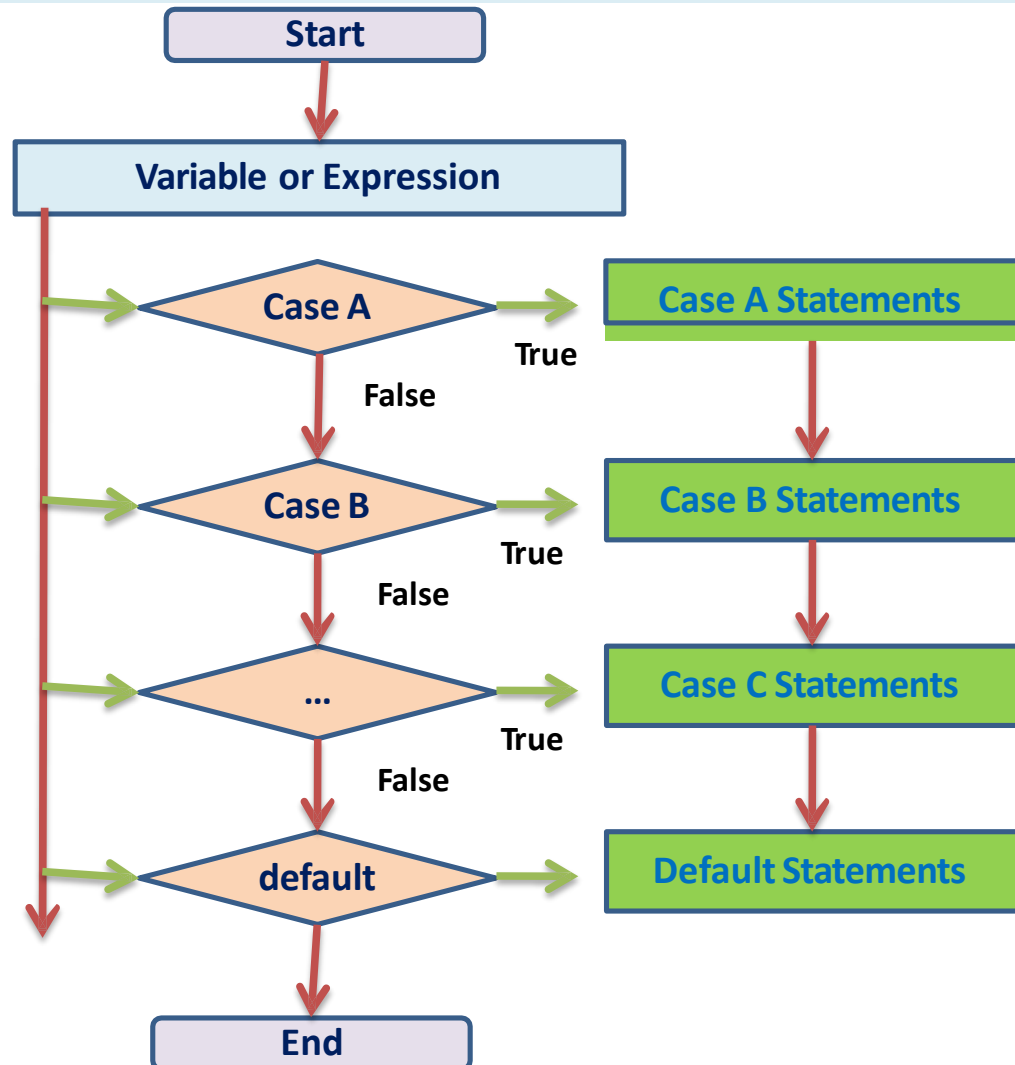
Syntax:

```
switch(expression/variable)
{
    Case value:
        case statements
        break // optional
    Case value:
        case statements
        break// optional
        .
        .
    default://optional
        default statements
}
```

Control structures

Flow Chart:

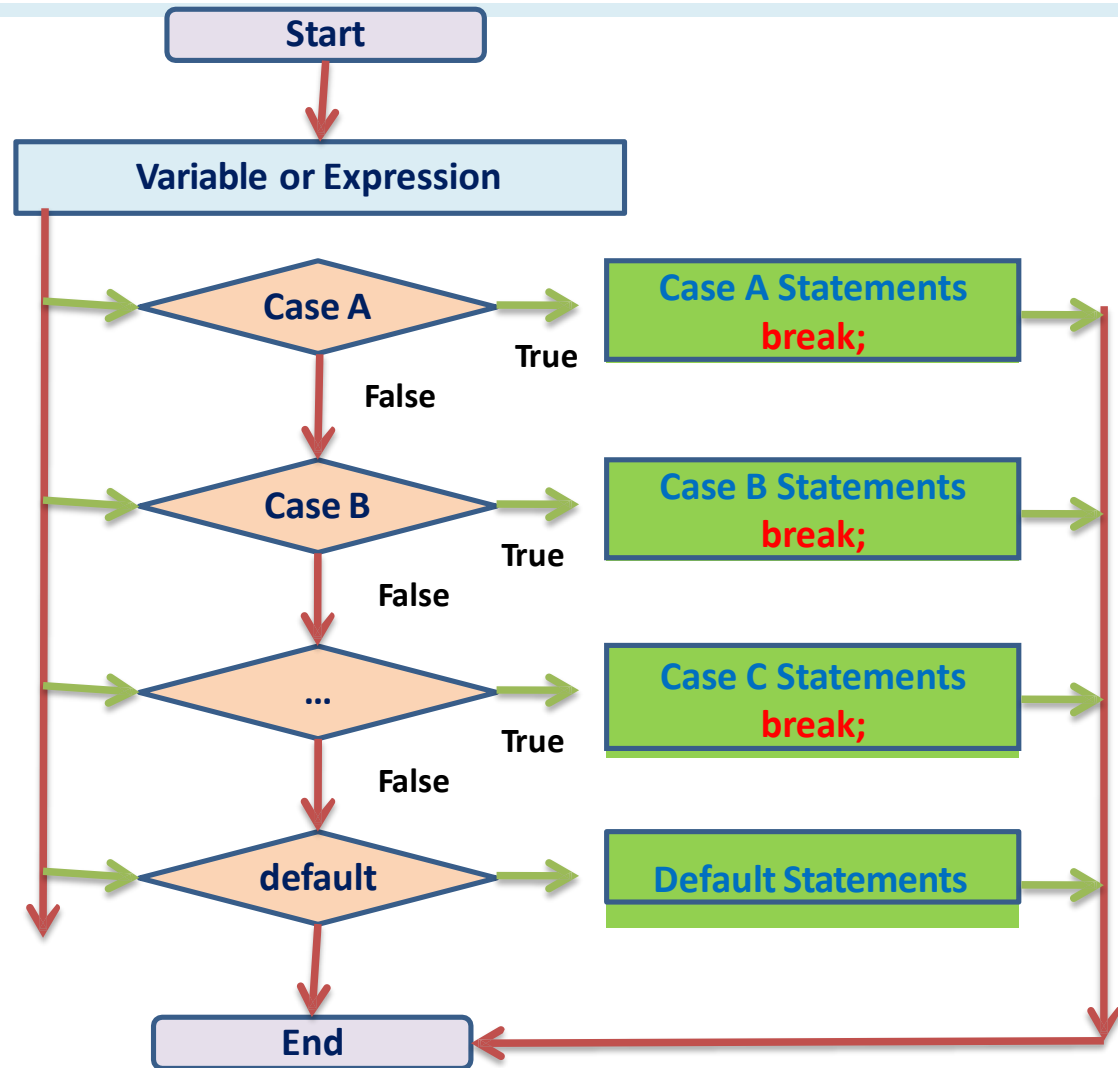
Switch
Statement
without
“break”



Control structures

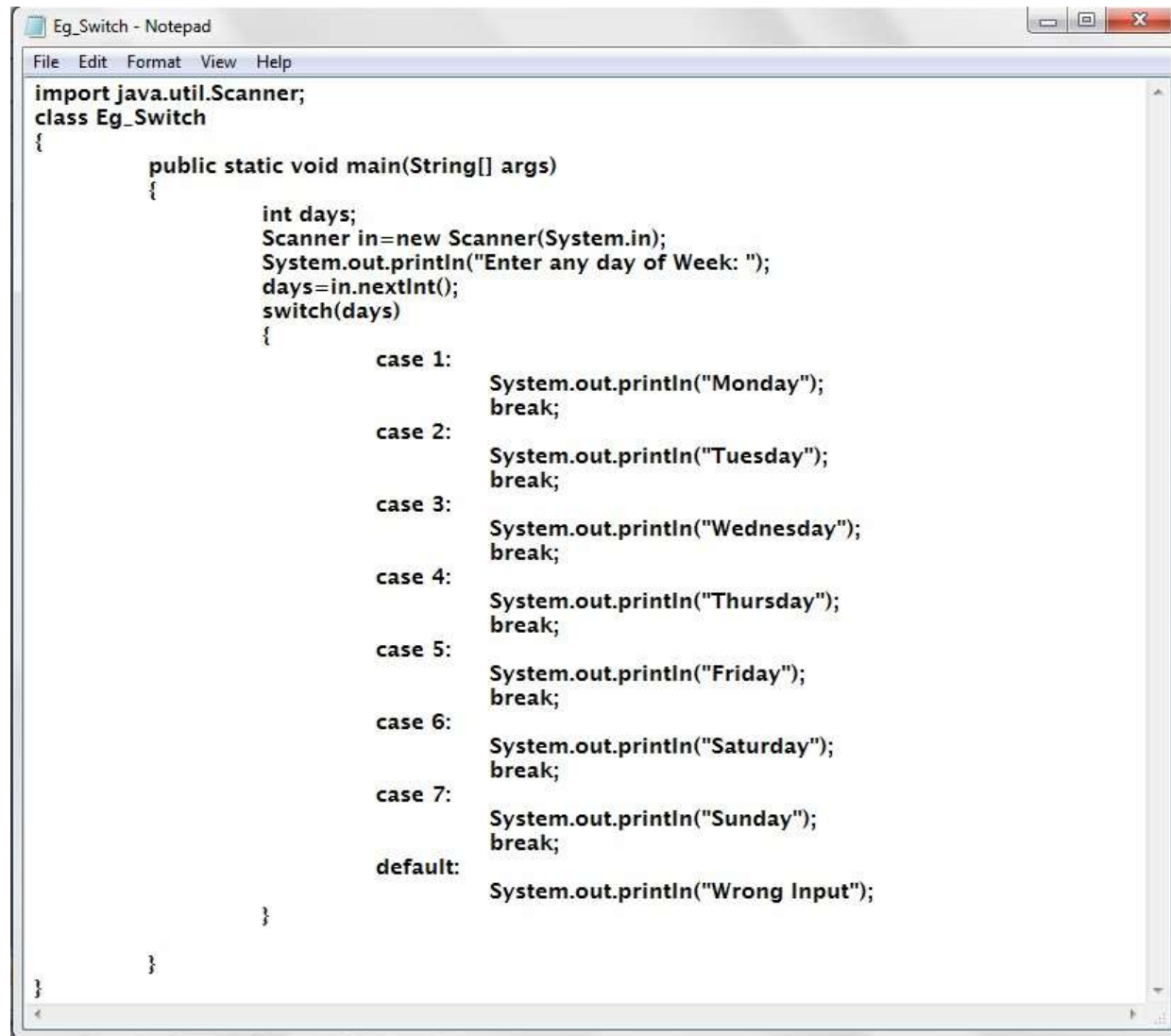
Flow Chart:

Switch
Statement
with
“break”



Control structures

Example:



```
import java.util.Scanner;
class Eg_Switch
{
    public static void main(String[] args)
    {
        int days;
        Scanner in=new Scanner(System.in);
        System.out.println("Enter any day of Week: ");
        days=in.nextInt();
        switch(days)
        {
            case 1:
                System.out.println("Monday");
                break;
            case 2:
                System.out.println("Tuesday");
                break;
            case 3:
                System.out.println("Wednesday");
                break;
            case 4:
                System.out.println("Thursday");
                break;
            case 5:
                System.out.println("Friday");
                break;
            case 6:
                System.out.println("Saturday");
                break;
            case 7:
                System.out.println("Sunday");
                break;
            default:
                System.out.println("Wrong Input");
        }
    }
}
```


Control structures

The iterative statements/ loop executes a block of statements when a particular condition is true

Each loop has four types of statements :

- Initialization
 - Condition checking
 - Execution
 - Increment // Decrement
-

Control structures

Difference Between Entry Controlled and Exit Controlled Loops

Entry Controlled	Exit Controlled
Condition is checked at the entry of the loop	Condition is checked at the exit of the loop
If condition is initially false, the loop never executes	If condition is initially false, then also the loop executes at least once
<pre> i=1; while(i==0) { System.out.println("In While loop"); } System.out.println("out of the loop"); </pre> Output: Out of the loop	<pre> i=1; do { System.out.println("In While loop"); } while(i==0); System.out.println("out of the loop"); </pre> Output: In while loop Out of the loop
Example- for, while	Example – do-while

Looping Structure - while



It is a structure in which a block is to be executed more than once.

This structure is also called as the Looping structure or Iterative structure .

The block may be a single statement or a collection of more than one statements begins with { and ends with }.

There are three types of repetitive structures. They are

(i) While Loop (ii) Do .. while Loop (iii) For

(i) The While Loop : In this structure the block specified for the while loop is executed only if the condition is false. The general format of the while Loop is

while (condition) block;

where the condition may be a relational or logical expression.

When the while loop is executed , it first checks the condition.

While loop



Case (i) : if the condition is true, the block is executed. After the block is executed , the control returns to the condition again and checks whether the condition is true now. If so, the block is executed again. The process is repeated as long as the condition is true.

Case (ii): if the condition is false, the control is transferred to the next statement of while loop.

The minimum number of iteration (ie.,, when the condition is false) of executing the block for while loop is 0.

While loop



Examples:

```
1.      i=1;
        while ( i <= 10 )
        {
            System.out.println(" "+i);
            i = i + 2;
        }
        System.out.println(" OK..");
```

the output of the above segment is

```
1
3
5
7
9
OK..
```

While loop



```
2.      i=11;
        while ( i <= 10 )
        {
            System.out.println(" "+i);
            i = i + 2;
        }
        System.out.println(" OK..");
```

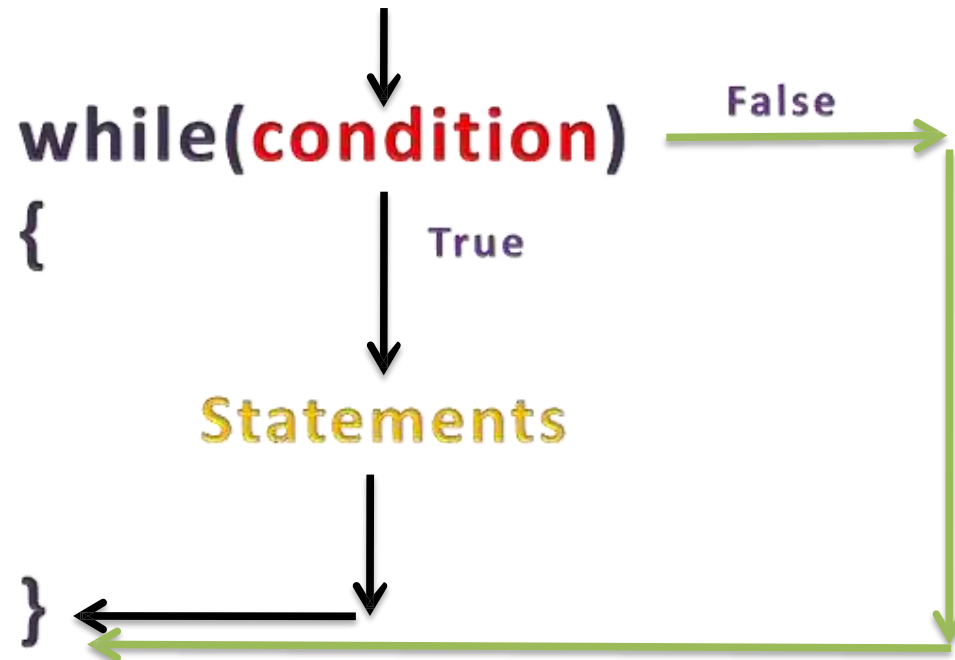
the output of the above segment is
OK..

In the second example, the block is not executed because the condition is false and the control does not enter into the block.

Control structures

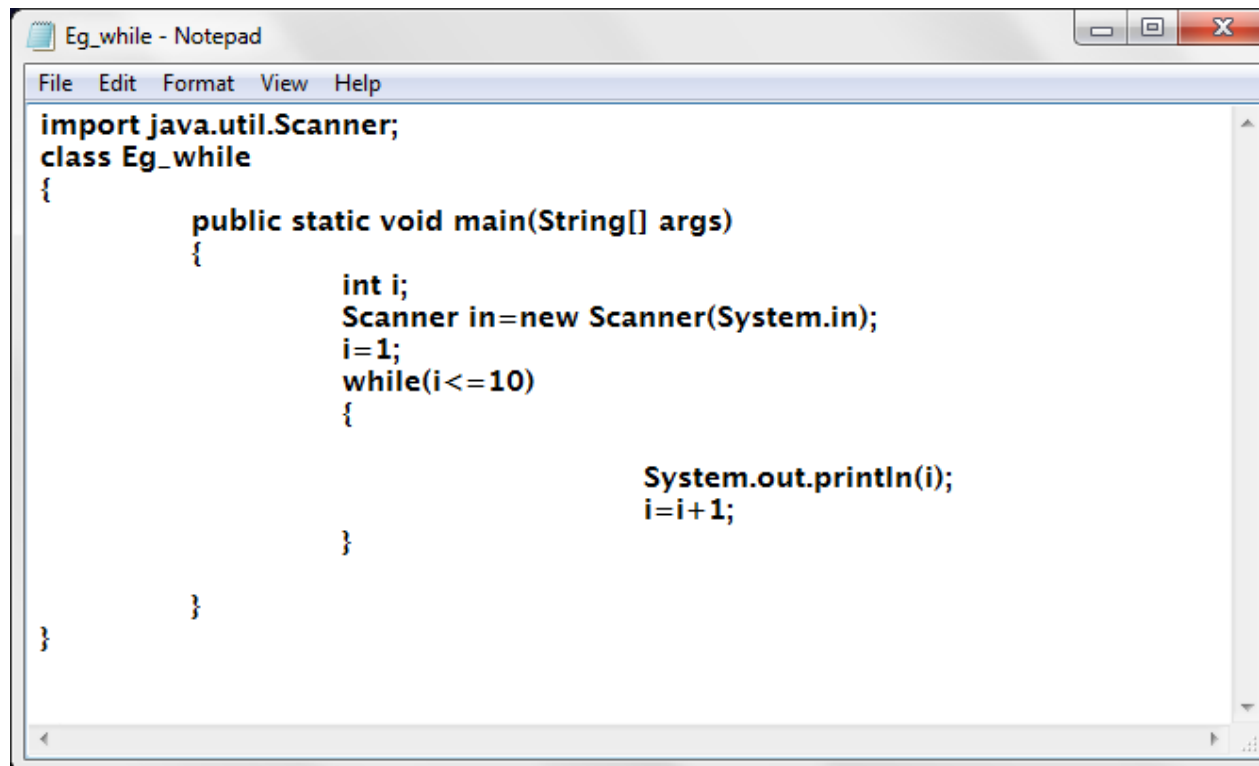
while Loop

Syntax



Control structures

Example: print values from 1 to ten



```
File Edit Format View Help
import java.util.Scanner;
class Eg_while
{
    public static void main(String[] args)
    {
        int i;
        Scanner in=new Scanner(System.in);
        i=1;
        while(i<=10)
        {
            System.out.println(i);
            i=i+1;
        }
    }
}
```




Control structures

Example: print values from 1 to ten

```
Command Prompt
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\Jyoti>D:
D:\>cd java programs
D:\Java Programs>javac Eg_while.java
D:\Java Programs>java Eg_while
1
2
3
4
5
6
7
8
9
10
D:\Java Programs>
```

Control structures

Functionality

Initialization Statement is used to initialize a variable/ counter.

`i=1;`

Initialization
statement

```
while(i<=10)  
{
```

```
    System.out.println(i);  
    i=i+1;
```

```
}
```

Control structures

Functionality

The condition statement controls the execution of loop

The loop executes till the condition statement is true

```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);  
    i=i+1;
```

```
}
```



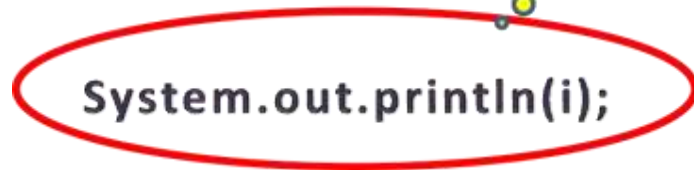
Control structures

Functionality

The execution statements are
the main body of a loop

All action statements of loop are
written here

```
i=1;  
while(i<=10)  
{  
  
    System.out.println(i);  
  
    i=i+1;  
}
```



Control structures

Functionality

This section is used to increment or decrement the variable value

```
i=1;  
  
while(i<=10)  
{  
    System.out.  
    i=i+1;  
}
```



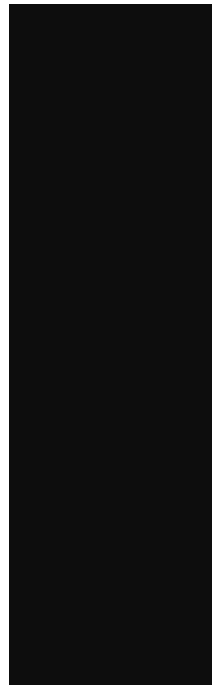
Increment/
Decrement

Control structures

Functionality



Output



`i=1;`

```
while(i<=10)
{
```

```
    System.out.println(i);
```

```
    i=i+1;
```

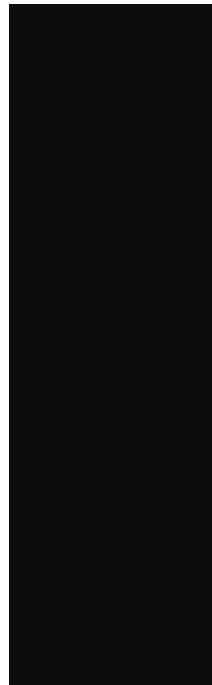
```
}
```

Control structures

Functionality



Output



```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

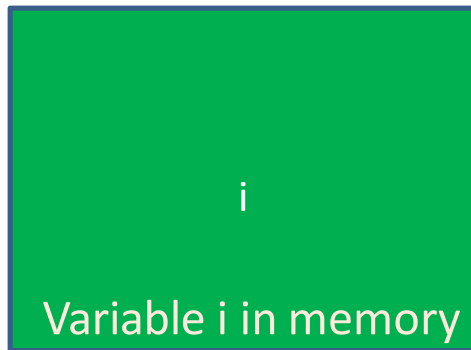
```
    i=i+1;
```

```
}
```

Condition is
True

Control structures

Functionality



Output

1

Check it
out here

```
i=1;
```

```
while(i<=10)  
{
```

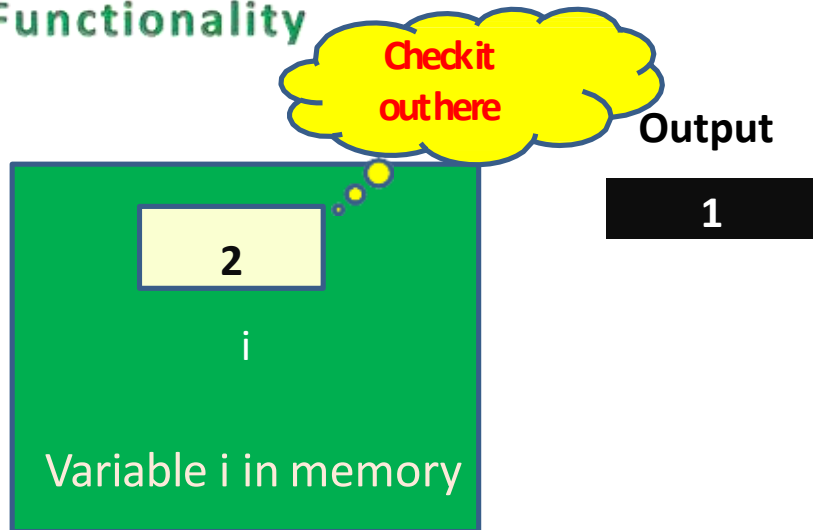
```
    System.out.println(i);
```

```
    i=i+1;
```

```
}
```


Control structures

Functionality



```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

```
    i=i+1;
```

```
}
```

Control structures

Functionality



Output

1

```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

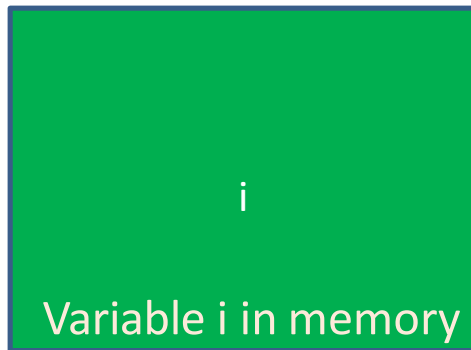
```
    i=i+1;
```

```
}
```

Condition is checked again

Control structures

Functionality



Output

1

```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

```
    i=i+1;
```

```
}
```



Condition is
True

Control structures

Functionality



Output

1
2

Check it
out here

i=1;

```
while(i<=10)
{
```

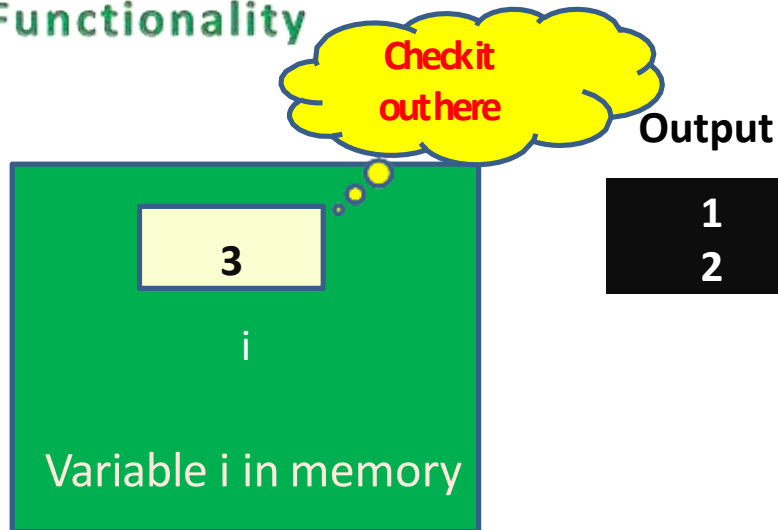
System.out.println(i);

i=i+1;

```
}
```

Control structures

Functionality



```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

```
    i=i+1;
```

```
}
```

Control structures

Functionality



Output

1
2

```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

```
    i=i+1;
```

```
}
```

Condition is checked again

Control structures

Functionality



Output



A black rectangular box containing the numbers '1' and '2' stacked vertically.

1
2

```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

```
    i=i+1;
```

```
}
```



A yellow thought bubble with a blue outline. Inside, the text 'Condition is True' is written in red. A red oval highlights the condition 'i<=10' in the code above.

Condition is
True

Control structures

Functionality



Output

1
2
3

Check it
out here

i=1;

```
while(i<=10)
{
```

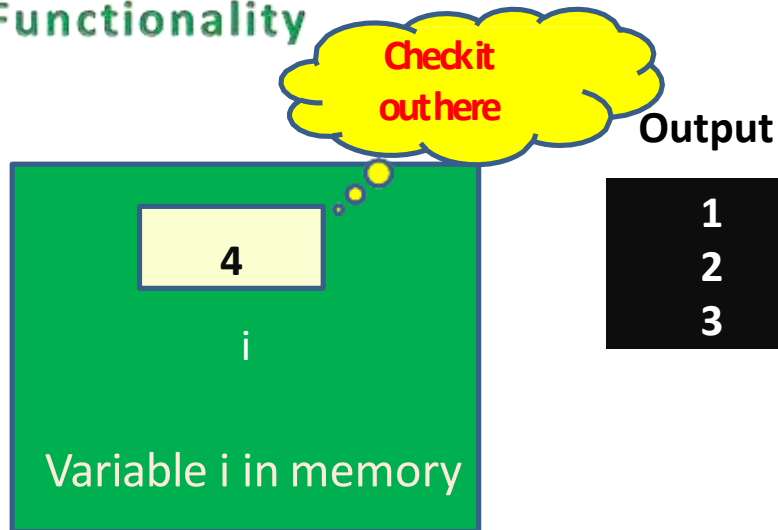
System.out.println(i);

i=i+1;

```
}
```


Control structures

Functionality



```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

```
    i=i+1;
```

```
}
```

Control structures

Functionality



Output



A black square containing the numbers 1, 2, and 3 stacked vertically.

```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

```
    i=i+1;
```

```
}
```



Condition is checked again

Control structures

Functionality



Output

1
2
3

```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

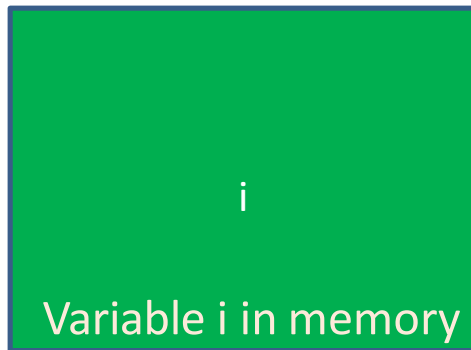
```
    i=i+1;
```

```
}
```

Condition is
True

Control structures

Functionality



Output

```
1
2
3
4
```

Check it
out here

```
i=1;
```

```
while(i<=10)
{
```

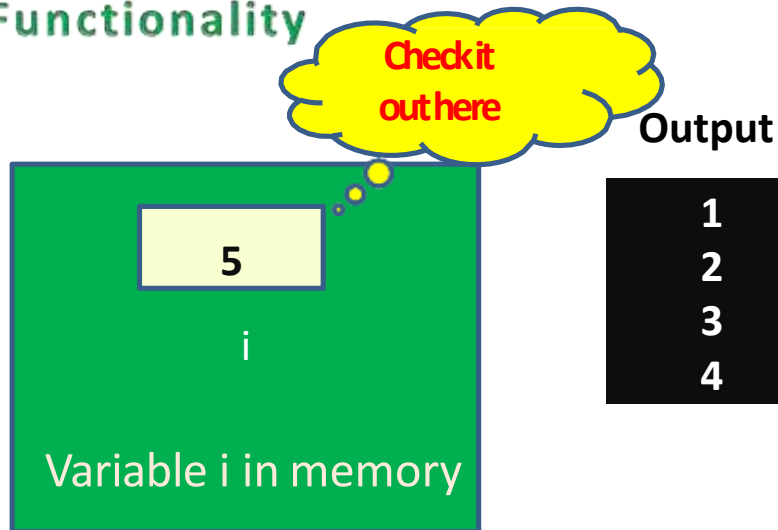
```
    System.out.println(i);
```

```
    i=i+1;
```

```
}
```

Control structures

Functionality



```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

```
    i=i+1;
```

```
}
```

Control structures

Functionality



Output



A black square containing the numbers 1, 2, 3, and 4 stacked vertically.

1
2
3
4

i=1;

while(i<=10)
{

System.out.println(i);

i=i+1;

}



A yellow thought bubble with a blue outline. Inside, the text 'Condition is checked again' is written in red. A red oval highlights the condition 'i<=10' in the code above, with a small dot connecting it to the thought bubble.

Condition is checked again



Control structures

This process will continue till the condition become false

Suppose value of i is 9 now



Control structures

Functionality



Output

1
2
3
4
5
6
7
8

```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

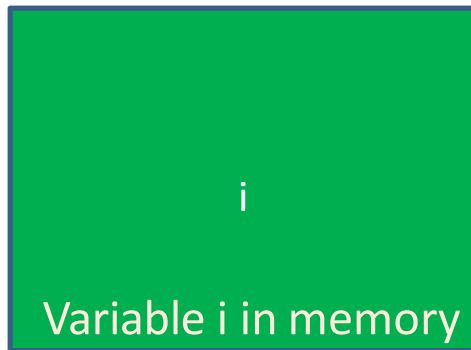
```
    i=i+1;
```

```
}
```

Condition
is True

Control structures

Functionality



Output

1
2
3
4
5
6
7
8
9

Check it
out here

i=1;

```
while(i<=10)
{
```

System.out.println(i);

i=i+1;

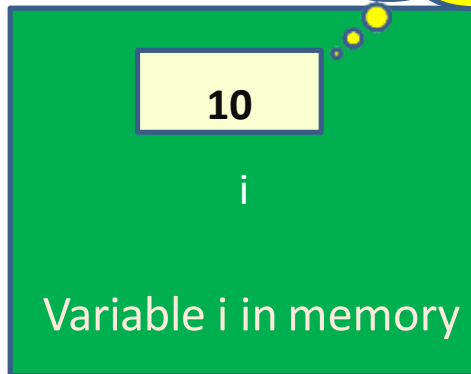
```
}
```

Control structures

Functionality

Check it
out here

Output



1
2
3
4
5
6
7
8
9

```
i=1;
```

```
while(i<=10)  
{
```

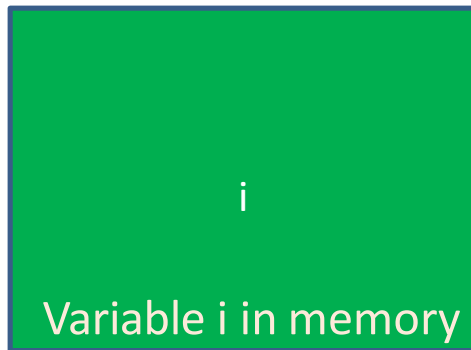
```
    System.out.println(i);
```

```
    i=i+1;
```

```
}
```

Control structures

Functionality



Output

1
2
3
4
5
6
7
8
9

```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

```
    i=i+1;
```

```
}
```

Condition is checked again

Control structures

Functionality



Output

1
2
3
4
5
6
7
8
9

```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

```
    i=i+1;
```

```
}
```

Condition
is Still True

Control structures

Functionality



Output



A black vertical rectangle containing a list of numbers from 1 to 10, representing the output of the program.

1
2
3
4
5
6
7
8
9
10



A yellow thought bubble with a blue outline and three small circles leading to it. It contains the text 'Check it out here' in red.

Check it
out here

```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

```
    i=i+1;
```

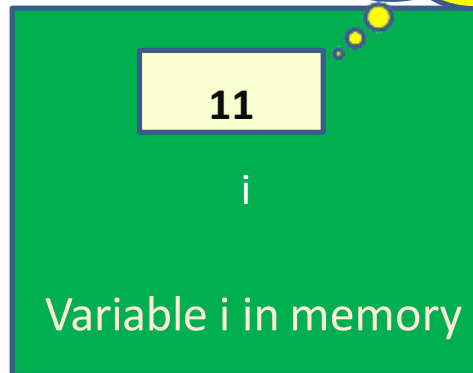
```
}
```

Control structures

Functionality

Check it
out here

Output



1
2
3
4
5
6
7
8
9
10

```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

```
    i=i+1;
```

```
}
```

Control structures

Functionality



Output

1
2
3
4
5
6
7
8
9
10

```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

```
    i=i+1;
```

```
}
```

Condition is checked again

Control structures

Functionality



Output

1
2
3
4
5
6
7
8
9
10

```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

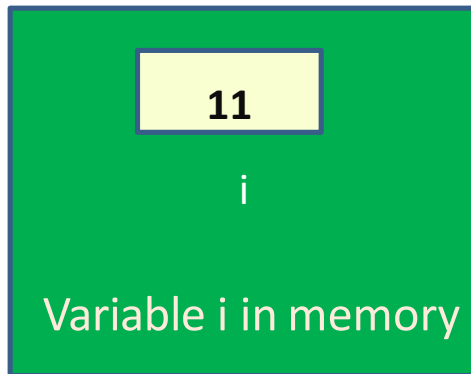
```
    i=i+1;
```

```
}
```

Condition
is FALSE

Control structures

Functionality



Output



A vertical list of numbers from 1 to 10. The number 10 is highlighted with a yellow circle. A small yellow circle is also present to the right of the number 10.

```
i=1;
```

```
while(i<=10)  
{
```

```
    System.out.println(i);
```

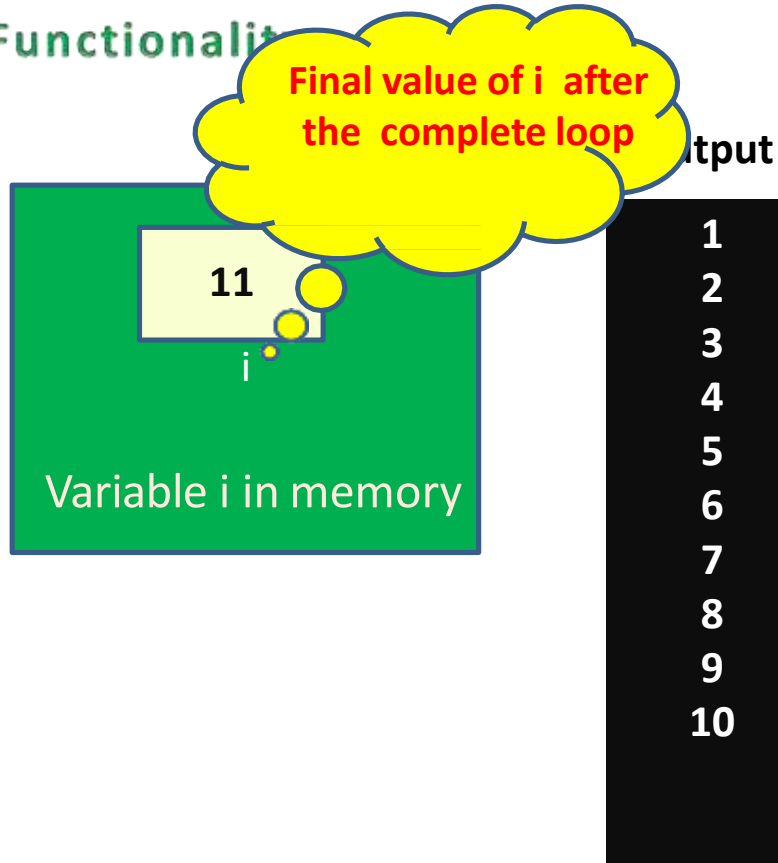
```
    i=i+1;
```

```
}
```

Skip the body of the loop and
executes the statement just
after the loop

Control structures

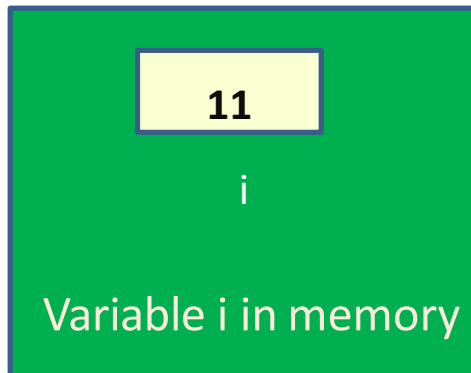
Functionality



```
i=1;  
  
while(i<=10)  
{  
  
    System.out.println(i);  
  
    i=i+1;  
  
}
```

Control structures

Functionality



Output

1
2
3
4
5
6
7
8
9
10

Final output of the loop

while(i<=10)

{

System.out.println(i);

i=i+1;

}

The for loop



```
} while ( i <= 10 );  
    System.out.println(" OK..");
```

The output of the above segment is

```
11  
OK..
```

In the second example, the block is executed once because The condition is false and the control does not enter into the block.

(iii)The for loop: When this loop is executed , the condition is checked for each iteration. The for loop has the following format

for (initialization part ; conditional part ; incremental / decremental part) block;

The for loop



When the FOR loop is executed , the following steps are done.

Step 1: Initialization part is performed first

Step 2: The condition is verified.

Step 3: If the condition is **True** ,

- (a)Block is executed

- (b)The incremental or decremental part is done

- (c)Again , the control goes to step (2)

If the condition is **False** ,

The control goes to the statement which lies after the For loop without executing the block.

The for loop



Examples:

```
for ( i =1 ; i<=8 ; i++)System.out.println("  "+i);  
System.out.println("\n OK..");
```

The output of the above segment is

```
1  
2  
3  
4  
5  
6  
7  
8  
OK..
```

The for loop



```
for ( i =10 ; i<=8 ; i++)System.out.println(“%d\n”,i);  
    System.out.println(“\n OK..”);
```

The output of the above segment is
OK..

Control structures

for Loop

Syntax

for(initialization ; condition ; increment)

{

True

Statements

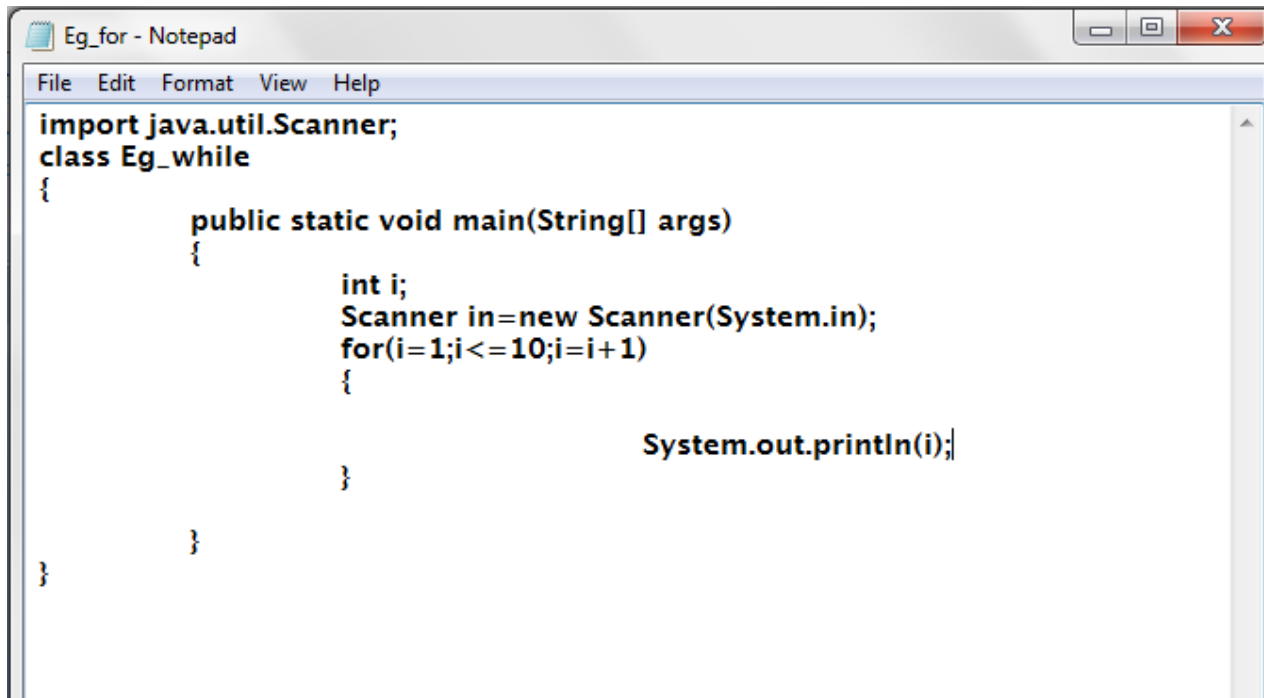
}

False



Control structures

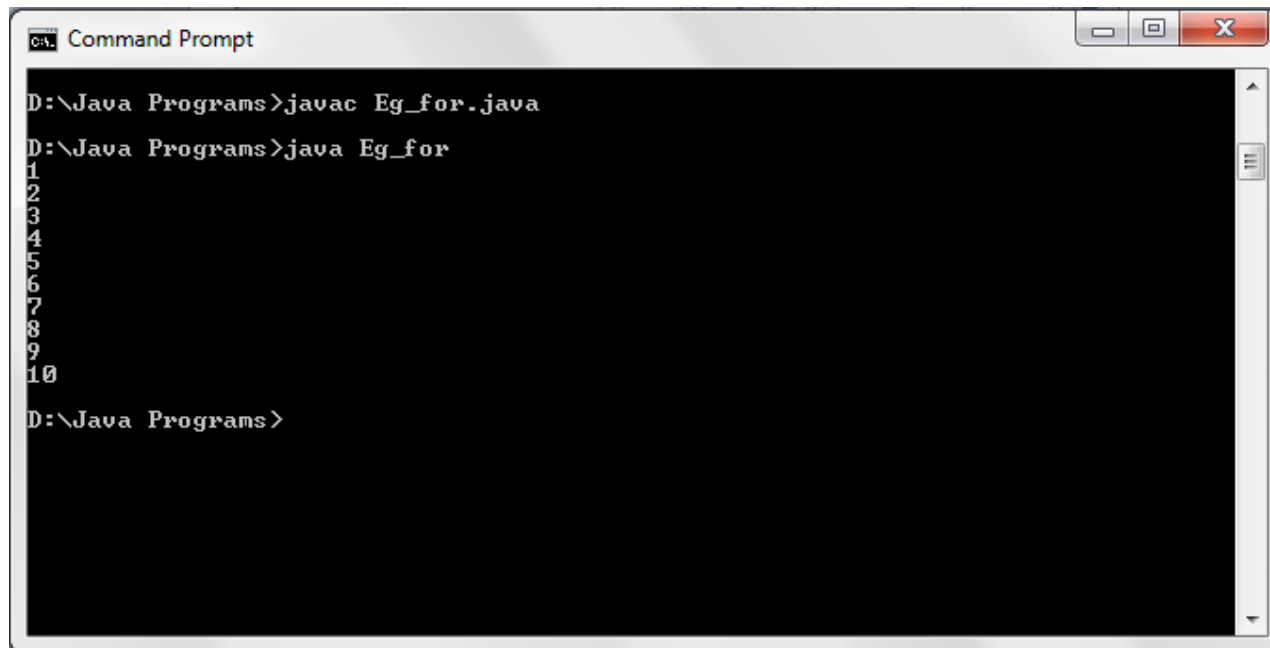
Example: print values from 1 to ten



```
import java.util.Scanner;
class Eg_while
{
    public static void main(String[] args)
    {
        int i;
        Scanner in=new Scanner(System.in);
        for(i=1;i<=10;i=i+1)
        {
            System.out.println(i);
        }
    }
}
```

Control structures

Example: print values from 1 to ten



```
Command Prompt
D:\Java Programs>javac Eg_for.java
D:\Java Programs>java Eg_for
1
2
3
4
5
6
7
8
9
10
D:\Java Programs>
```

Do-While loop

(ii)The Do ..While loop: When this loop is executed , the condition is checked after the block is executed once. The general format of the do .. while Loop is

```
do
{
    block;
}while ( condition ) ;
```

where the condition may be a relational or logical expression. When the do .. while loop is executed , it executes the block once and then the condition is checked.

Case (i) : if the condition is true, the block is executed again. After the block is executed , it again checks the condition whether it is true. The process is repeated as long as the



Do-While loop

condition is true.

Case (ii): if the condition is false, after the execution of block once, the control is transferred to the next statement of while loop.

The minimum number of iteration (ie., though the condition is false) of executing the block for while loop is 1.

Examples:

```
1.      i=1;
        do
        {
            System.out.println("  "+i);
            i = i + 2;
        } while ( i <= 10 );
```

Do-While loop

```
System.out.println("OK..");
```

The output of the above segment is

1

3

5

7

9

OK..

2.

```
i=11;
```

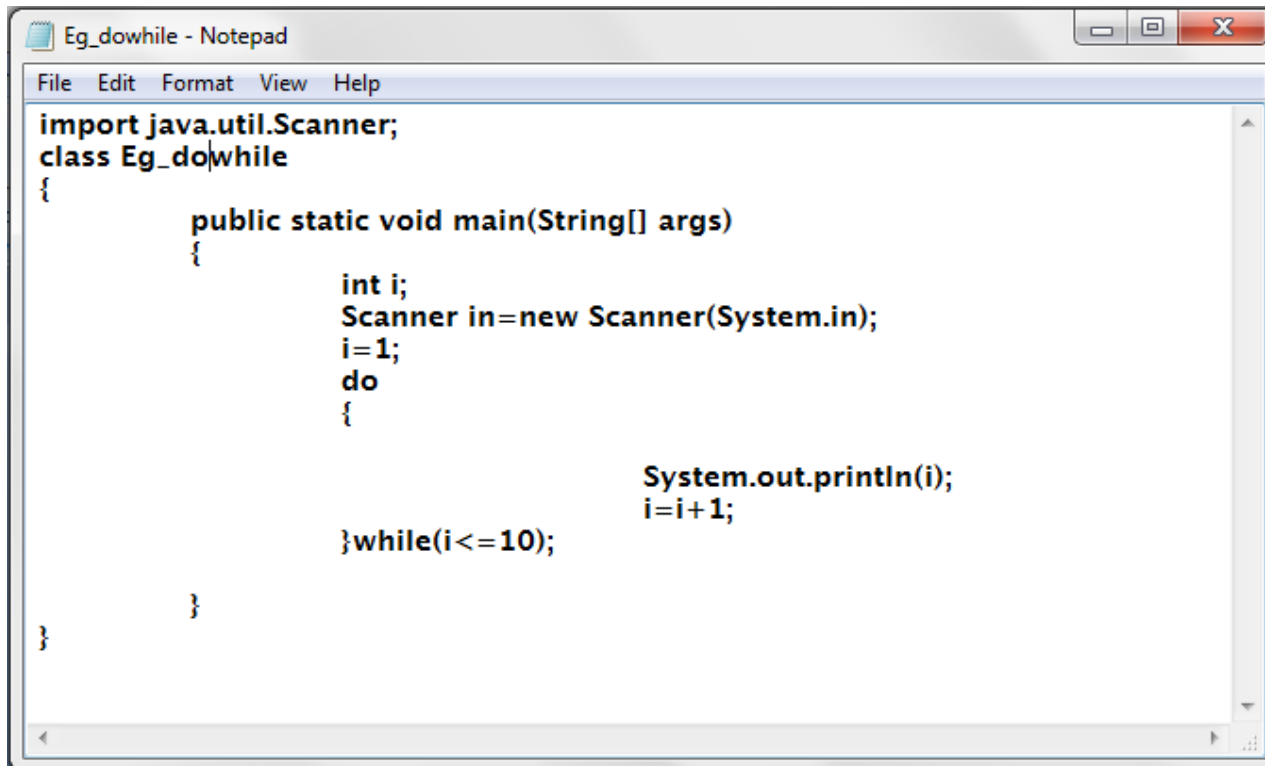
```
do {
```

```
    System.out.println("  "+i);
```

```
    i = i + 2;
```

Control structures

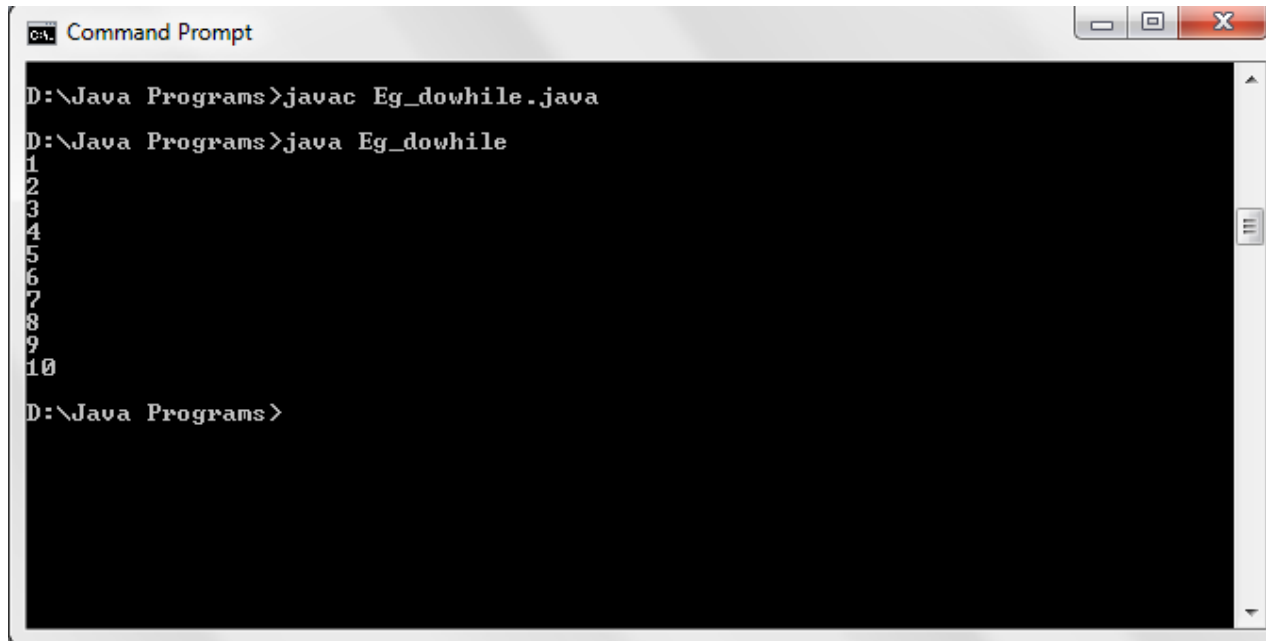
Example: print values from 1 to ten



```
import java.util.Scanner;
class Eg_dowhile
{
    public static void main(String[] args)
    {
        int i;
        Scanner in=new Scanner(System.in);
        i=1;
        do
        {
            System.out.println(i);
            i=i+1;
        }while(i<=10);
    }
}
```

Control structures

Example: print values from 1 to ten



```
Command Prompt
D:\Java Programs>javac Eg_dowhile.java
D:\Java Programs>java Eg_dowhile
1
2
3
4
5
6
7
8
9
10
D:\Java Programs>
```

Control structures

Functionality

Initialization Statement is used to initialize a variable/ counter.



```
i=1;  
do  
{  
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```


Control structures

Functionality

The condition statement controls the execution of loop

The loop executes till the condition statement is true

```
i=1;  
do  
{  
    System.out.println("i=" + i);  
    i=i+1;  
}while(i<=10);
```



Control structures

Functionality

The execution statements are
the main body of a loop

All action statements of loop are
written here

```
i=1;  
do  
{  
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```



System.out.println(i);

i=i+1;

}while(i<=10);

Control structures

Functionality

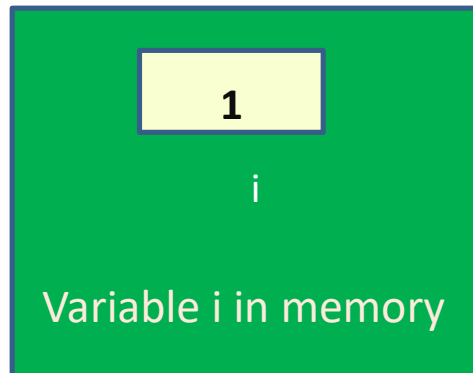
This section is used to increment or decrement the variable value

```
i=1;  
do  
{  
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```

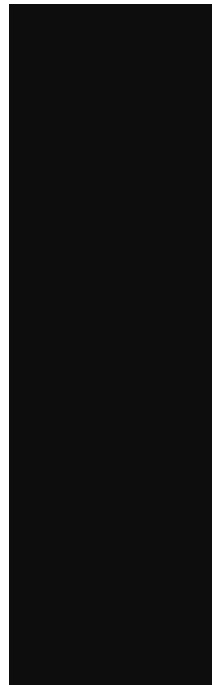


Control structures

Functionality



Output



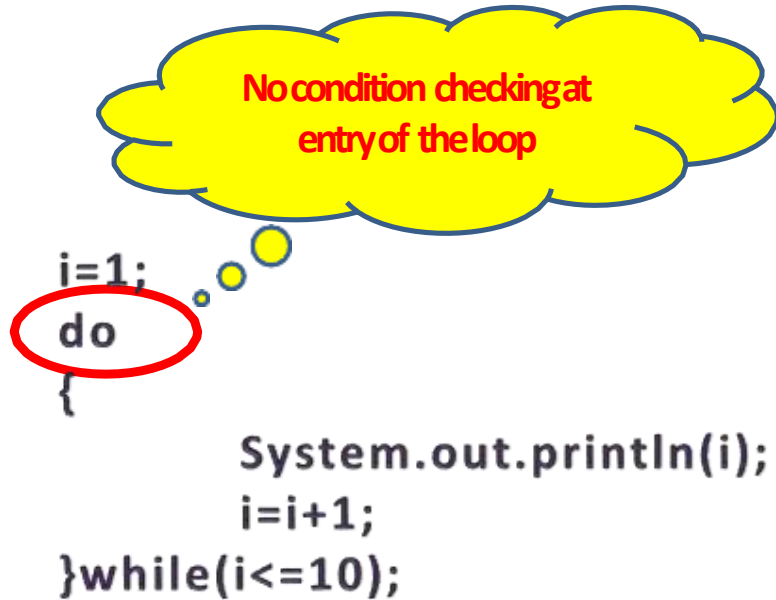
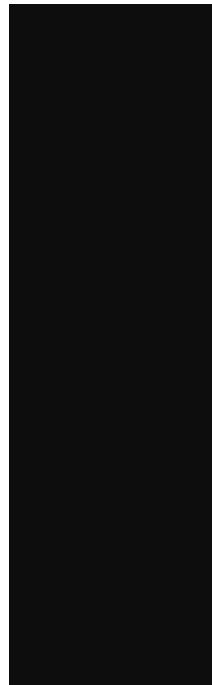
```
i=1;  
do  
{  
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```

Control structures

Functionality



Output



Control structures

Functionality



Output

1

Check it
out here

```
i=1;  
do  
{  
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```

Control structures

Functionality

Check it
out here

Output

1

2

i

Variable i in memory

```
i=1;  
do  
{
```

```
    System.out.println(i);
```

```
    i=i+1;
```

```
}while(i<=10);
```

Control structures

Functionality



Output

1

```
i=1;  
do  
{
```

```
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```

Condition is
checked again

Control structures

Functionality



Output

1

```
i=1;  
do  
{  
    system.out.println(i);  
    i=i+1;  
}while(i<=10);
```

Condition is True

Control structures

Functionality



Output

1
2

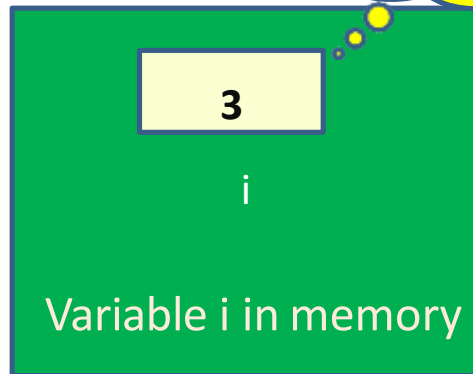
Check it
out here

```
i=1;  
do  
{
```

```
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```

Control structures

Functionality



Output

1
2

```
i=1;  
do  
{  
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```

Control structures

Functionality



Output

1
2

```
i=1;  
do  
{  
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```

Condition is checked again

Control structures

Functionality



Output

1
2

```
i=1;  
do  
{  
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```

A yellow thought bubble with the text "Condition is True" is positioned above the closing brace of the while loop. A red oval highlights the while loop condition `while(i<=10);`.

Control structures

Functionality



Output

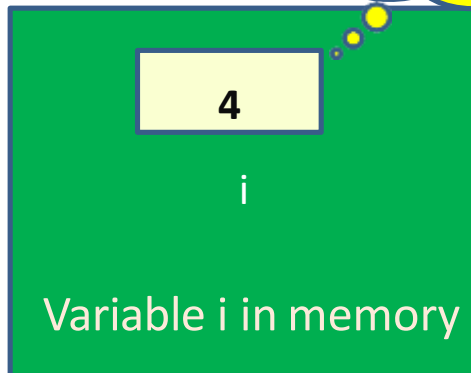
1
2
3

Check it
out here

```
i=1;  
do  
{  
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```

Control structures

Functionality



Output

1
2
3

```
i=1;  
do  
{  
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```

Control structures

Functionality



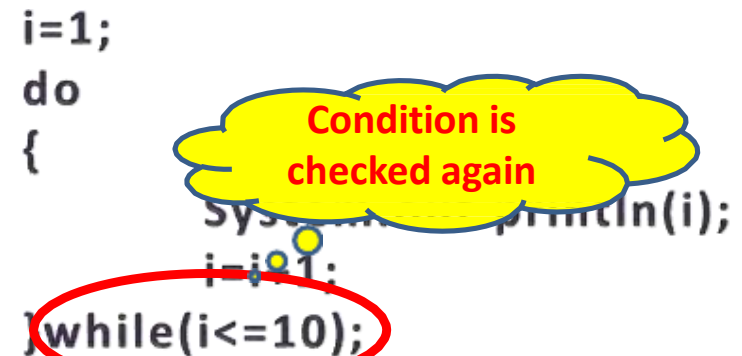
Output



A black rectangular box containing the numbers 1, 2, and 3 stacked vertically.

1
2
3

```
i=1;  
do  
{  
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```



The code is annotated with a yellow thought bubble and a red circle. The thought bubble, containing the text 'Condition is checked again', points to the 'i=i+1;' line. The red circle highlights the 'while(i<=10);' line.

Condition is checked again

Control structures

Functionality



Output



A black rectangular box containing the numbers 1, 2, and 3 stacked vertically.

1
2
3

```
i=1;  
do  
{
```

```
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```



Control structures

Functionality



Output

1
2
3
4

Check it
out here

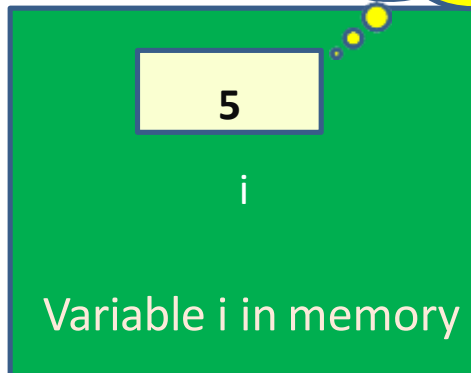
```
i=1;  
do  
{  
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```

Control structures

Functionality



Output



1
2
3
4

```
i=1;  
do  
{
```

```
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```

Control structures

Functionality



Output



A black rectangular box with white text. The numbers 1, 2, 3, and 4 are listed vertically.

1
2
3
4

```
i=1;  
do  
{
```



A yellow cloud-shaped annotation with a blue outline. It contains the text 'Condition is checked again' in red.

Condition is checked again

```
System.out.println(i);  
i=i+1;  
}while(i<=10);
```



Control structures

This process will continue till the condition become false

Suppose value of i is 9 now

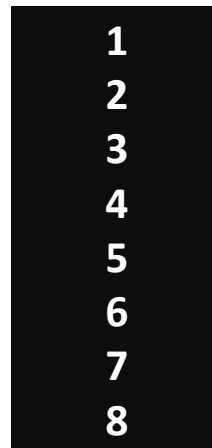


Control structures

Functionality



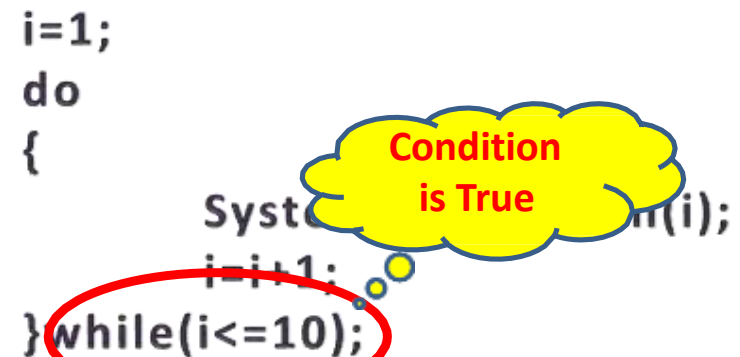
Output



A black vertical rectangle containing a list of numbers from 1 to 8, arranged vertically from top to bottom.

1
2
3
4
5
6
7
8

```
i=1;  
do  
{  
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```



A diagram illustrating a while loop. The code is shown in a monospaced font. The condition 'i<=10' in the while statement is circled in red. A yellow thought bubble with a blue outline and a small tail pointing to the red circle contains the text 'Condition is True' in red. The word 'System' in the print statement is partially visible as 'Syste'.

Condition is True

Control structures

Functionality



Output

1
2
3
4
5
6
7
8
9

Check it
out here

```
i=1;  
do  
{
```

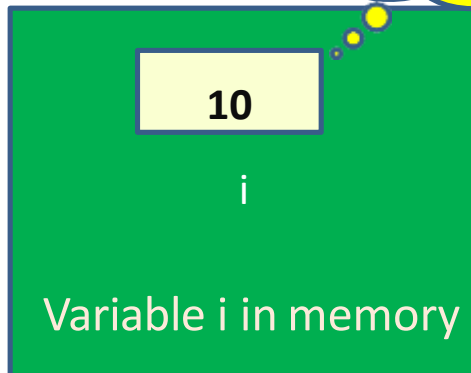
```
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```

Control structures

Functionality

Check it
out here

Output



1
2
3
4
5
6
7
8
9

```
i=1;  
do  
{
```

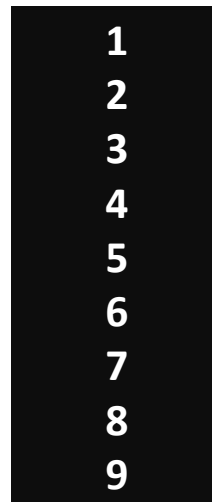
```
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```


Control structures

Functionality



Output



A vertical black bar containing a list of numbers from 1 to 9, representing the output of a loop.

1
2
3
4
5
6
7
8
9

```
i=1;  
do  
{
```

```
System.out.println(i);  
i=i+1;  
}while(i<=10);
```



A yellow thought bubble with a blue outline. Inside, the text 'Condition is checked again' is written in red. The bubble is connected to the 'while' loop in the code by a series of small circles.

Condition is checked again

Control structures

Functionality



Output

1
2
3
4
5
6
7
8
9

```
i=1;  
do  
{  
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```

Condition is Still True

Control structures

Functionality



Output

1
2
3
4
5
6
7
8
9
10

Check it
out here

```
i=1;  
do  
{
```

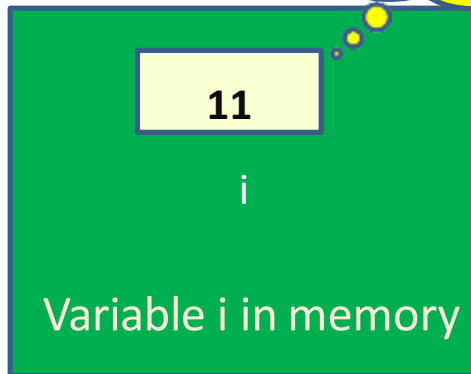
```
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```

Control structures

Functionality

Check it
out here

Output



1
2
3
4
5
6
7
8
9
10

```
i=1;  
do  
{  
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```

Control structures

Functionality



Output

1
2
3
4
5
6
7
8
9
10

```
i=1;  
do  
{
```

```
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```

Condition is
checked again

Control structures

Functionality



Output

```
1
2
3
4
5
6
7
8
9
10
```

```
i=1;
do
{
    System.out.println(i);
    i=i+1;
}while(i<=10);
```

A yellow thought bubble with a blue outline contains the text "Condition is FALSE" in red. A red oval highlights the condition `i<=10` in the code. A small yellow dot connects the bubble to the code line `i=i+1;`.

Control structures

Functionality



Output

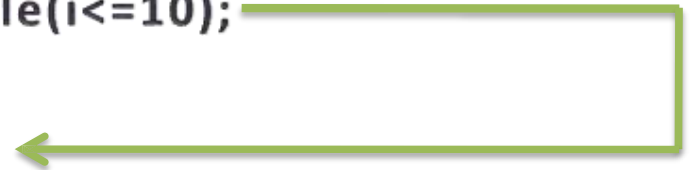
1
2
3
4
5
6
7
8

```
i=1;  
do  
{
```

```
    System.out.println(i);  
    i=i+1;
```

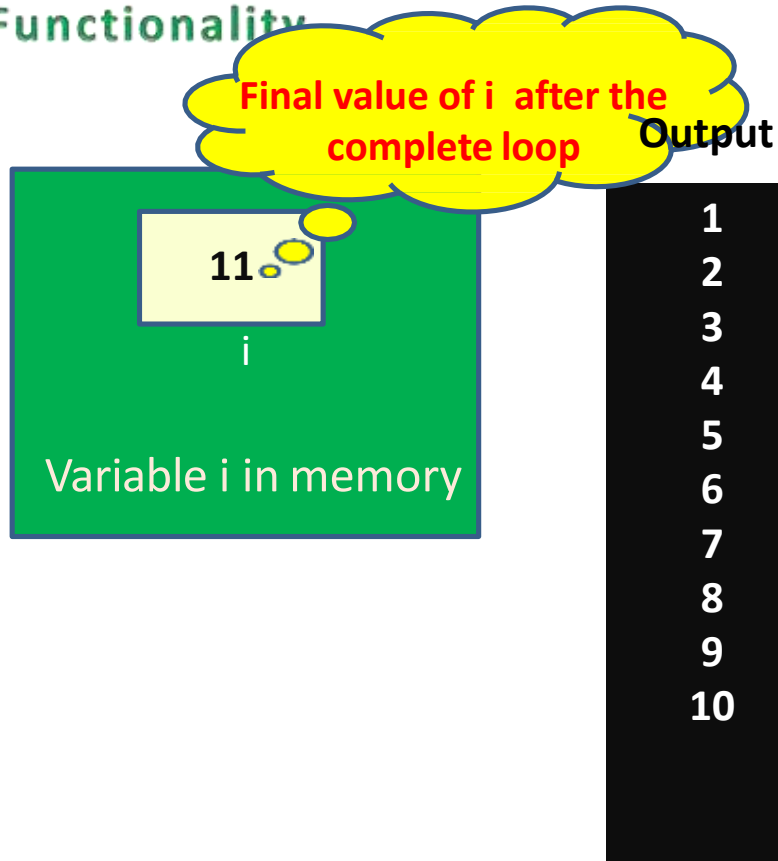
```
}while(i<=10);
```

Stop the
execution of the
loop and
executes the
statement just
after the loop



Control structures

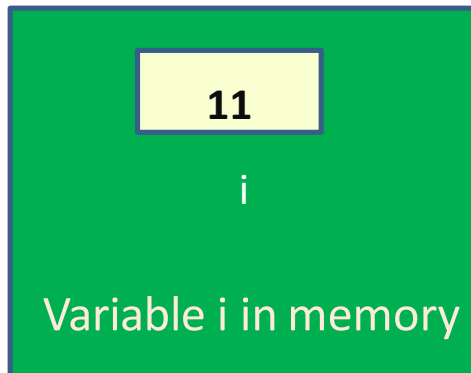
Functionality



```
i=1;  
do  
{  
    System.out.println(i);  
    i=i+1;  
}while(i<=10);
```


Control structures

Functionality



Output

1
2
3
4
5
6
7
8
9
10

Final output of
the loop

```
System.out.println(i);  
i=i+1;  
}while(i<=10);
```

JUMP statements in Looping Structure



Jump statement is used to alter the sequential flow of execution inside the loop.

There are four types of jump statements in java

- Break
- Labelled break
- Continue
- Labelled Continue
- Return

1. Break

In java the break statement has 3 uses

- Terminates a statement sequence in a switch statement
- It can be used to exit loop
- It is the more civilized form of goto statement



The break statement

Using Break to exit the loop

By using break, we can force immediate termination of a loop, when break statement is encountered inside a loop the loop is terminated and program control resumes at the next statement following the loop

```
public static void main(String[] y)
{
    for(int i=0;i<10;i++)
    {
        if(i==5)
        {
            System.out.println("condition break");
            break;
        }
    }
}
```

The break statement



```
System.out.println("the value of i is::"+i);  
}  
}
```

The output for the above program is

the value of i is::0

the value of i is::1

the value of i is::2

the value of i is::3

the value of i is::4

ii. Labelled Break

Using the break with the labels which is equivalent to goto the syntax is shown.

Syntax

```
break label_name;
```

```
public class LabelledBreakExample {  
    public static void main(String[] args) {  
        System.out.println("Example of 'labelled break'  
statement:");  
        outerLoop:  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 3; j++) {  
                if (i == 2 && j == 2) {  
                    break outerLoop; // Breaks out of both loops  
when i == 2 and j == 2  
                }  
                System.out.println("i = " + i + ", j = " + j);  
            }  
        }  
    }  
}
```

The break statement



```
public static void main(String[] aa)
{
first:{
second:{
third:{
System.out.println("third block");
if(true)
break second;
}
System.out.println("second block");
}
System.out.println("first block");
}
}
```

The continue statement



The output for the above code is

third block

first block

iii. Continue

A continue statement causes control to be transferred directly to the conditional expression that controls the loop

```
public static void main(String[] y)
{
    for(int i=0;i<5;i++)
    {
        if(true)
        {
            System.out.println("the if block");
            continue;
        }
    }
}
```

The continue statement



```
System.out.println("the for loop");  
    }  
}
```

The output for the above code is

- the if block
- the if block
- the if block
- the if block
- the if block

The continue statement



iv. Using continue with labels

```
public static void main(String[] y)
{
outer:for(int i=0;i<=2;i++){
    for(int j=0;j<=2;j++)
    {
        System.out.println("value of J is::"+j);
        if(j==1)continue outer;
    }
    System.out.println("the value of i is::"+i);
}
}
```

The output for the above program is

```
value of J is::0
value of J is::0
value of J is::0
```

The return statement



v. Return

The return statement is used to explicitly return from a method.

That is, it causes program control to transfer back to the caller of the method.

At any time in a method the return statement can be used to cause execution to branch back to the caller of the method.

Thus, the return statement can be used to cause execution to branch back to the caller method.

Thus, the return statement immediately terminates the method in which it is executed.

The return statement



```
public static void main(String[] aa)
{
    System.out.println("before return statement");
    if(true)
    {
        return;
    }
    System.out.println("after return statement");
}
```

The output of the above program is
before return statement

```
public class ReturnExample {  
    public static void main(String[] args) {  
        System.out.println("Example of 'return' statement:");  
        for (int i = 1; i <= 5; i++) {  
            if (i == 3) {  
                return; // Exits the method when i equals 3  
            }  
            System.out.println("i = " + i);  
        }  
  
        System.out.println("This line will not be executed.");  
    }  
}
```

Thanks



BITS Pilani
Pilani Campus

Object Oriented Programming Design

Presented by
Dr Ramya Devi M
mramyadevi@wilp.bits-pilani.ac.in
CS - 04 : 19th Jul 2025



BITS Pilani
Pilani Campus



Classes & Objects

Learning Objectives



- **Class definition**
- **Object**
- **Exercise in class and object**

CLASS and OBJECTS



In Java everything is **encapsulated under classes**.

Class is the core of Java language.

It can be defined as **a template** that describe the behaviors and states of a particular entity.

A class defines new data type.

Once defined this new type can be used to create object of that type.

Object is **an instance of class**.

We call it as physical existence of a logical template class.

In Java, to declare a class the “**class**” keyword is used.

A class contains both **data and methods** that operate on that data.

CLASS and OBJECTS



The data or variables defined within a class are called **instance variables** and the code that operates on this data is known as **methods**.

Thus, the instance variables and methods are known as **class members**.

Rules for Java Class

- A class can have only **public or default** access specifier.
- It can be either **abstract, final or concrete (normal class)**.
- It must have the **class keyword**, and class must be followed by a legal identifier.
- It may optionally extend only one parent class. By default, it extends **Object** class.
- The variables and methods are declared within a set of curly braces.

CLASS and OBJECTS



The syntax for declaring a class is

Access Specifier class className

{

AS data member 1;

AS data member 2;

AS return type Method name1 (Parameter)

{

Body of the Method

}

AS return type Method name2 (Parameter)

{

Body of the Method

CLASS and OBJECTS



```
}
```

```
-----
```

```
-----
```

AS return type Method name n(Parameter)

```
{
```

Body of the Method

```
}
```

```
}
```

Where AS is the access specifier.

If the access specifier is not specified, the corresponding member is considered as default.

Access Specifier



Types of Access Specifier /Modifier

There are four access modifiers in Java and they are:

Modifier	Description
Default	Declarations are visible only within the package i.e. package private (equivalent to friend in C++)
Private	Declarations are visible within the class only
Protected	Declarations are visible within the package or all subclasses
Public	Declarations are visible everywhere

CLASS and OBJECTS



Example for a class

The following is the class for Student

```
import java.util.*;
```

```
class Student
```

```
{
```

```
    String Sname;
```

```
    int Regno;
```

```
    String Dept;
```

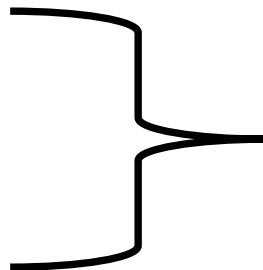
```
    float Percent;
```

```
    void getInput()
```

```
{
```

```
        Scanner sc=new Scanner(System.in));
```

```
        Sname=sc.nextLine();
```



Data Members

CLASS and OBJECTS



```
Regno=sc.nextInt();
Dept=sc.nextLine();
Percent=sc.nextFloat();
}
void printOutput()
{
System.out.println(" Name   is "+Sname);
System.out.println(" Regno   is "+Regno);
System.out.println(" Dept    is "+Dept);
System.out.println(" Percentage is "+Percent);
}
}
```

Why string inputs after integer input gets skipped in Java?



This is a common problem, and it happens because the [nextInt](#) method doesn't read the newline character of your input.

So when you issue the command [nextLine](#), the Scanner finds the newline character and gives you that as a line.

```
void getInput() {  
    Scanner sc=new Scanner(System.in);  
    System.out.println("Enter name");  
    Sname=sc.nextLine();  
    System.out.println("Enter Regno");  
    Regno=sc.nextInt();  
    sc.nextLine(); //This is for reading the newline after integer  
    System.out.println("Enter Department");  
    Dept=sc.nextLine();  
    System.out.println("Enter Percentage");  
    Percent=sc.nextFloat();  
}
```


CLASS and OBJECTS



In the above class ,

getInput() and printOutput() are the methods.

Since no access specifier is given for all the data members and methods , all have the default access modifier automatically.

This is how a class looks structurally.

It is a blueprint for an object.

We can call its fields and methods by using the object.

The fields declared inside the class are known as **instance variables**.

Memory is assigned when an object is created at runtime.

Methods in the class are similar to the functions that are used to perform operations and represent behavior of an object.

Java OBJECTS



Java Object

Object is an instance of a class while class is a blueprint of an object.

An object represents the class and consists of properties and behavior.

Properties refer to the fields declared with in class and behavior represents the methods available in the class.

In real world, we can understand object as a Computer that has its properties like: name, cost, speed etc and behavior like reading, processing etc.

So we can say that object is a real world entity.

Some real world objects are: Man, Mouse, Bike, Car etc.

Creating OBJECTS



The syntax for creating an object is

className variableName = new className (P);

Here,

- **className** is the name of class that can be anything like: Student that we declared in the above example.
- **variableName** is name of reference variable that is used to **hold the reference** of created object.
- **new** is a keyword which is used to allocate memory for the object.
- P is the list of parameter to be sent to the constructor .
- It is optional.

Lets see an example to create an object of class Student that we created in the above class section.

OBJECTS – Memory allocation



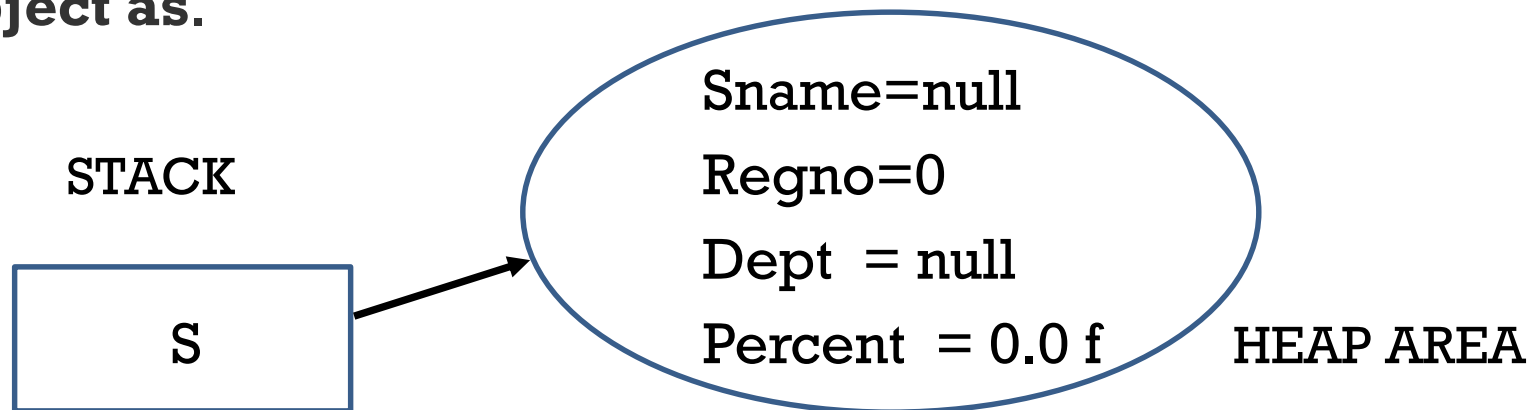
Student S = new Student();

Here, S is an object that represents the class Student during runtime.

The **new** keyword creates an actual physical copy of the object and assigns it to the variable S.

It will have physical existence and get memory in heap area.

The new operator dynamically allocates memory for an object as.





Method Overloading , Constructors, and Static members

Learning Objectives

- **Method Overloading**
 - Rules for overloading methods
 - Example for method overloading
- **Constructors**
 - Different types of Constructors
 - Constructor overloading?
- **Static members?**
 - Static data members
 - Static methods

METHOD OVERLOADING



Method overloading in Java means multiple methods having the same name but different parameters.

A class can hold several methods having the same name, but different types/order/number of parameters.

Suppose we want to perform the addition of the given numbers. But user may enter any number of values to add.

The problem without overloading: If we write the method **addTwo(int, int)** for two parameters, and **addThree(int,int,int)** for three parameters then it may be difficult for us as well as other programmers to remember the names of the methods.

It may be difficult to track how many methods in class that are performing addition because its name differs.

METHOD OVERLOADING - Types



In Java, we can create both the methods with the same name but with different numbers of parameters.

There are three ways to overload the methods:

1. We can overload by changing the number of arguments/parameters.
2. We can overload by changing the data type of arguments.
3. We can overload by changing the Order of the parameters of methods.

1. By changing the number of arguments

We can create multiple methods with the same name, but the number of arguments/parameters should be different.

METHOD OVERLOADING - Types



For Eg.

```
class MethodOverloading
```

```
{
```

```
public void show(String name)
```

```
{
```

```
System.out.println("Name of person = "+name);
```

```
}
```

```
public void show(String name, String address)
```

```
{
```

```
System.out.println("Name of person = "+name+ " and Place is = "+  
                    address);
```

```
}
```

METHOD OVERLOADING - Types



```
public static void main (String [] args)
{
    MethodOverloading example = new MethodOverloading();
    // If user provides one parameter then the first method is called
    example.show("Virat Kohli");
    // If user provides two parameters then the second method is called
    example.show("Rohit Sharma", "Mumbai");
}
```

Output:

Name of person = Virat Kohli

Name of person = Rohit Sharma and Place is = Mumbai

METHOD OVERLOADING - Types



2. By changing the type of the argument

We can create multiple methods with the same name but the data type of the parameter will be different.

Let's create two methods with the same name but the parameter type will be different.

```
class MethodOverloading
```

```
{
```

```
public void show(String name)
```

```
{
```

```
System.out.println("Name of person = "+name);
```

```
}
```

```
public void show(int age)
```

```
{
```

```
System.out.println("Person age is = "+ age);
```

METHOD OVERLOADING - Types



```
}  
public static void main (String [] args)  
{  
MethodOverloading example = new MethodOverloading();  
// If user providing parameter of String type then first method is called  
    example.show("Sachin");  
// If user providing parameter of int type then second method is called  
    example.show(38);  
}  
}
```

The output of the above program:

Name of person = Sachin

Person age is = 38

METHOD OVERLOADING - Types



3. By changing the order of the parameter of methods

We can overload the method by changing the order of parameter. But it is only possible if any method has different types of parameters.

Example:

```
class MethodOverloading
{
    public void show(String name, int age)
    {
        System.out.println("Name = "+name+ " and age = "+ age);
    }
}
```

METHOD OVERLOADING - Types



```
public void show(int age, String name)
{
    System.out.println("Name = "+name+ " and age = "+ age);
}

public static void main (String [] args)
{
    MethodOverloading example = new
                                MethodOverloading();

    // If user providing parameter of String and int type then first
    // method called
    example.show("Laxman", 45);

    // If user providing parameter of int and String type then
    // second method called
    example.show(25, "Ram");
```

METHOD OVERLOADING - Types



```
}  
}
```

Output:

Name = Laxman and age = 45

Name= Ram and age = 25

NOTE: We can't overload by changing the return type of method

CONSTRUCTORS



CONSTRUCTOR is a special method that is used to initialize newly created object and is called just after the memory is allocated for the object.

It can be used to initialize the objects to desired values or default values at the time of creating the object.

It is not mandatory for the coder to write a constructor for a class.

If no user-defined constructor is provided for a class, compiler initializes member variables to their default values.

- numeric data types are set to 0
- char data types are set to null character('\0')
- reference variables are set to null

CONSTRUCTORS



Rules for creating a Java Constructor

1. A constructor must have the **same name** as the class
2. It does not return a value, return type should not be given
3. A constructor may have the parameter(s). If it has at least one parameter, it is called as parameterized constructor
4. If constructor has no parameter, it is called as non parameterized constructor
5. A class may have multiple constructors. If so, it is known as constructor overloading

CONSTRUCTORS - Syntax



The syntax for including a constructor in a class is

```
class className
```

```
{
```

```
  className ( [parameter(s)])
```

```
  {
```

```
    Body of the Constructor
```

```
  }
```

```
}
```

Constructor

The parameter(s) is optional

CONSTRUCTORS - Example



Example for a Constructor

Class Student

```
{  
    String name;  
    int age;  
    float avg;  
    Student( )  
    {  
        name="*****";  
        age=20;  
        avg=50.0f;  
    }  
}
```

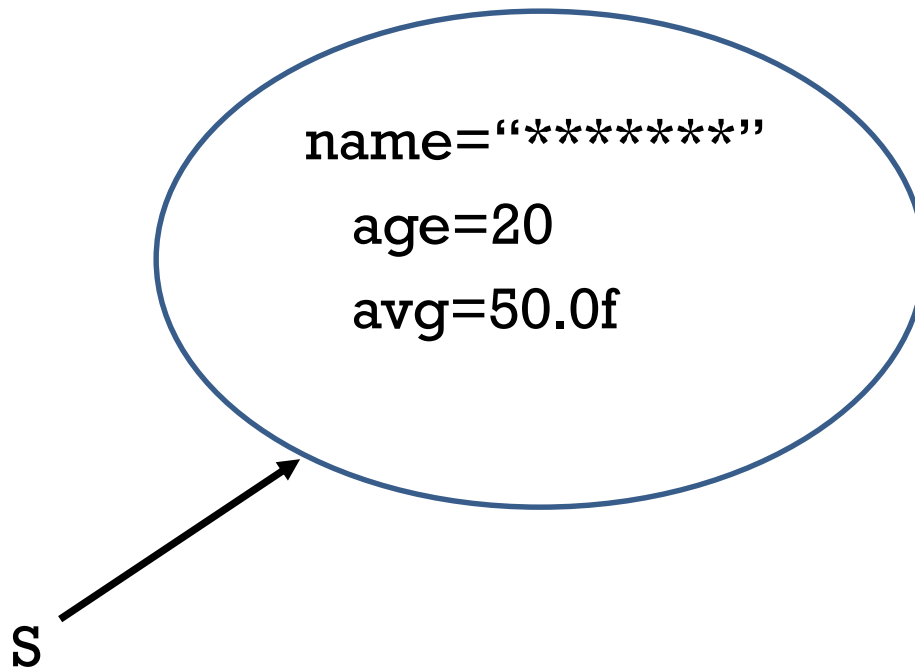
CONSTRUCTORS



When an object is created as,

Student S = new Student();

the data members are initialized with the following values



class Student6

{

int id;

String name;

Student6(int i,String n)

{

id = i;

name = n;

}

Student6(Student6 s)

{

id = s.id;

name =s.name;

}

void display()

{

System.out.println(id+" "+name);

}

public static void main(String args[])

{

Student6 s1 = new Student6(111,"Karan");

Student6 s2 = new Student6(s1);

s1.display();

s2.display();

}

}



Types of Constructor



Types of Constructor

In Java, constructors can be divided into 3 types:

1. No-Arg Constructor/ non parameterized Constructor
2. Parameterized Constructor
3. Default Constructor

1. Java No-Arg Constructors

Similar to methods, a Java constructor may or may not have any parameters (arguments).

If a constructor does not accept any parameters, it is known as a no-argument constructor. For example,

Types of Constructor



Class Student

```
{
```

```
    String name;
```

```
    int age;
```

```
    float avg;
```

```
    Student( )
```

```
{
```

```
        name="*****";
```

```
        age=20;
```

```
        avg=50.0f;
```

```
}
```

Constructor having no parameter



Types of Constructor



```
void display()
{
    System.out.println(" Name "+name+" Age "+age+" AVG "+avg);
}

class Cons_Demo
{
    public static void main(String aa[])
    {
        Student S = new Student();
        S.display();
    }
}
```


Types of Constructor



The output for the above program is

Name ***** Age 20 AVG 50.0

2. Java Parameterized Constructor

Java constructors can also accept one or more parameters.

Such constructors are known as parameterized constructors (constructor with parameters). For example,

```
class Student
{
    String name;
    int age;
    float avg;
```

Types of Constructor



```
Student(String sn,float av )
```

```
{
```

```
name=sn;
```

```
age=20;
```

```
avg=av;
```

```
}
```

**Constructor having
two parameters**

```
void display()
```

```
{
```

```
System.out.println(" Name "+name+" Age "+age+"
```

```
AVG " "+avg);
```

```
}
```

```
}
```

Types of Constructor



```
class Cons_Demo
{
    public static void main(String aa[])
    {
        Student S = new Student("NAVEEN",92.55);
        S.display();
    }
}
```

The output for the above program is
Name NAVEEN Age 20 AVG 92.55

Types of Constructor



3. Java Default Constructor

If we do not include any constructor, the Java compiler automatically creates a no-arg constructor during the execution of the program.

This constructor is called default constructor.

```
class Student
{
    String name;
    int age;
    float avg;           // No Constructor
```

Types of Constructor



```
void display()
{
    System.out.println(" Name "+name+" Age "+age+"
    AVG "+avg);
}

class Cons_Demo
{
    public static void main(String aa[])
    {
        Student S = new Student();
        S.display();
    }
}
```

Types of Constructor



```
}
```

```
}
```

The output for the above program is

Name NULL Age 0 AVG 0.00f [Default values are assigned to the data members]

Constructors Overloading in Java

Similar to JAVA method overloading, we can also create two or more constructors with different parameters.

Using different types of constructors is called constructors overloading.

Types of Constructor



Class Student

{

String name;

int age;

float avg;

Student()

{

name="*****";

age=20;

avg=50.0f;

}

**Constructor having
no parameter**

Types of Constructor



```
Student(String sn )  
{  
    name=sn;  
    age=30;  
    avg=60.25f;  
}
```

**Constructor having
one parameter**

```
Student(String sn,float av )  
{  
    name=sn;  
    age=21;  
    avg=av;  
}
```

**Constructor having
two parameters**

Types of Constructor



```
void display()
{
    System.out.println(" Name "+name+" Age "+age+"
    AVG "+avg);
}

class Cons_Demo
{
    public static void main(String aa[])
    {
        Student S1 = new Student( ); // Execution of non parameterized
        Student S2 = new Student("RAHUL");
        // Execution of parameterized having a single parameter
```

Types of Constructor



```
S1.display();  
    S2.display();  
}  
}
```

The output for the above program is

Name ***** Age 20 AVG 50.0

Name RAHUL Age 30 AVG 60.25

Types of Constructor



Copy constructor

```
Student(Student sn )
```

```
{  
    this.name=sn.name;  
    this.age=sn.age;  
    this.avg=sn.avg;  
}
```

**Constructor having
Object as parameter**

An object is passed as parameter to the constructor.

STATIC MEMBERS



Static data member in Java

Static data member in Java is a variable which belongs to the Class and initialized only once at the start of the execution.

It is a variable which belongs to the class and not to any objects (instance) created for the class.

Static variables are initialized only once, at the start of the execution.

These variables will be initialized first, before the initialization of any instance variables.

- A single copy will be shared by all instances of the class
- A static variable can be accessed directly by the class name and doesn't need any object

STATIC MEMBERS



To make any data member as static data member, the declaration statement must begin with the keyword “static”.

There will be a single copy of static data member irrespective of number of objects created for the class.

A static data member is also known as non instance data member or class member.

Example:

A class with a static data member is created as follows:

```
class C
{
    int a;
    float b;
    static int c; // static data member
}
```

non static data members

STATIC MEMBERS

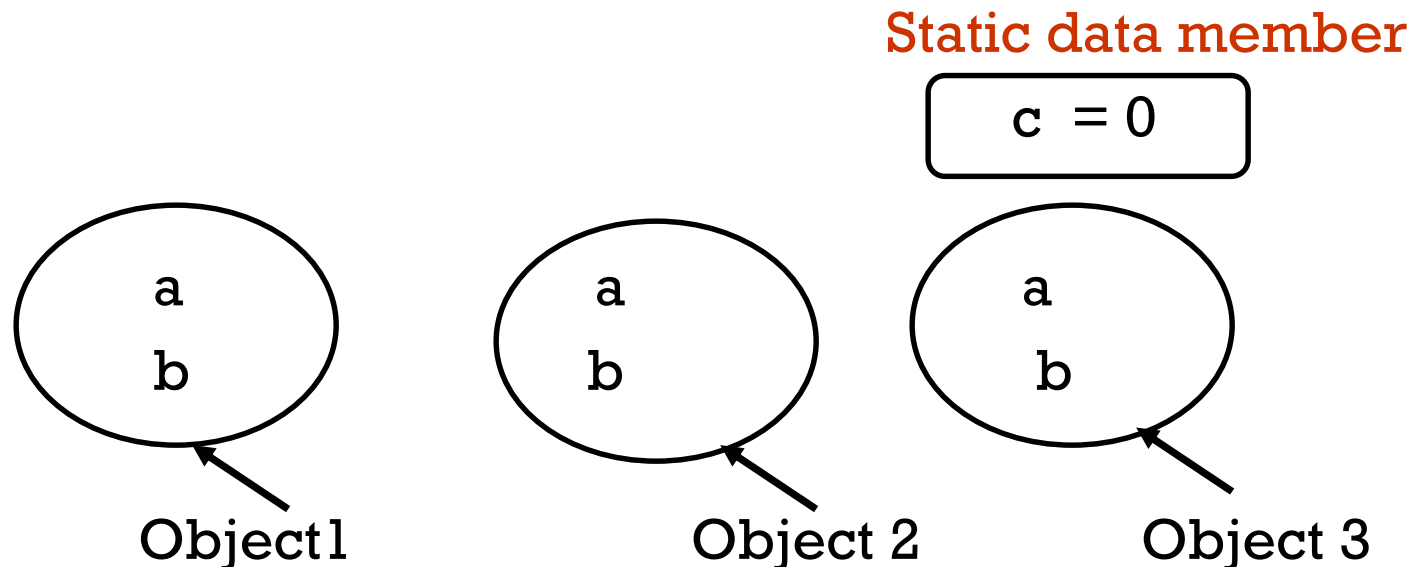


Assume that three objects (instances) are created for the class.

The non static data members (Instance data members / Object data members), a and b are stored in each object created.

So there are three copies of the data members a and b.

But the static data member (Non Instance data member / Class data member) is stored only once outside of all the objects.



STATIC MEMBERS



To access the non static data members, we use the syntax

Object reference.data member

To access the static data members, we use the syntax

class name.data member

Static Method in Java

Static method in Java is a method which belongs to the class and not to the object.

A static method can access only static data.

It is a method which belongs to the class and not to the object(instance).

A static method can access only static data.

It cannot access non-static data (instance variables).

STATIC MEMBERS



The rules for using static methods are

1. A static method can call only other static methods and can not call a non-static method.
2. A static method can be accessed directly by the class name and doesn't need any object
3. A static method cannot refer to "this" or "super" keywords in anyway

To call the static method, we use the syntax

class name.method name (parameter)

STATIC MEMBERS



Example program

```
class Student {  
    int a;  
  
    static int c=0; //initialized only once to zero  
  
    Student()  
    {  
        a=10;  
        c++;  
    }  
}
```

STATIC MEMBERS



```
public void showData()
{
    System.out.println("Value of a = "+a);
    System.out.println("Value of c = "+c);
}
}
```

STATIC MEMBERS



```
public class Demo
{
    public static void main(String args[])
    {
        Student s1 = new Student();
        s1.showData();
        Student s2 = new Student();
        s2.showData();
        Student s3 = new Student();
        s3.showData();
    }
}
```

STATIC MEMBERS



```
}
```

```
}
```

Expected output is shown below

Value of a = 10

Value of c = 1

Value of a = 10

Value of c = 2

Value of a = 10

Value of c = 3

STATIC MEMBERS



Note 1: If we want to show only the value of c, we can include a static method, showCount() as

```
static void showCount()
```

```
{
```

```
    System.out.println(" No: - of Objects "+c);
```

```
}
```

To call this method in the main() method, we don't require the object reference. It can be called with the class name as

```
Student.showCount();
```

Note 2: In the above method, we cannot display the value of 'a' because a static method can not access the non static data member.

STATIC BLOCK



Static Block in Java

A **static block** is a block of statement inside a Java class that will be executed when a class is first loaded into the JVM.

A **static block helps to initialize the static data members**, just like constructors help to initialize instance members.

Its syntax is

```
class class name
{
    static
    {
        Statements for the block
    }
}
```

STATIC BLOCK



Following program is the example of java static block.

```
public class Demo
{
    static int a;
    static int b;
    static {

        a = 10;
        b = 20;

    }
    public static void main(String args[])
    {
```

STATIC BLOCK



```
System.out.println("Value of a = " + a);  
System.out.println("Value of b = " + b);  
}  
}
```

We will get following output of the program.

Value of a = 10

Value of b = 20



Arrays in Java

Arrays Introduction

- Array → Collection of Homogeneous Data Elements
- Arrays are objects and implements only static (fixed-length) arrays
- Java follows strict bound checking for referencing array elements. If an attempt is made to reference the elements outside the bounds then “**ArrayIndexOutOfBoundsException**” will be thrown at run time.
- Arrays can be 1-dimensional (1-D), 2-dimensional (2-D) or multidimensional.
 - Maximum Dimensions an Array can have = 255
 - Arrays are Objects in Java. So, every element at each index is automatically initialized to some default value depending upon the type of array. [byte, short, int and long → 0, float, double → 0.0, boolean → false, char → “ (whitespace) and any class type array → null]
 - **<<length>>** is the attribute of each array which can be referenced by **<array-reference-name> . <length>**

1-dimensional Arrays

Syntax : One-Dimensional Arrays :

type[] arrayname = new type[Size];

or

type arrayname[] = new type[Size];

Examples :

1.int[] marks = new int[10];

// Each element is initialized to 0

marks is an int type array, **marks.length = 10**, LB index =0 , UB index = 9

2.float[] values = new float[20];

// Each element is initialized to 0.0

values is an float type array, **values.length = 20**, LB index =0 , UB index =19

3.double cgpa[] = new double[5];

// Each element is initialized to 0.0

cgpa is double type array, **cgpa.length = 5**, LB index =0 , UB index = 4

4.Box[] boxes = new Box[20];

// Each element is initialized to 'null' value

boxes is a Box type array, **boxes.length = 20**, LB index =0 , UB index = 19

5.Point points[] = new Point[20];

// Each element is initialized to 'null' value

points is a Point type array, **points.length = 20**, LB index =0 , UB index =19

6. int[] marks = {10, 8, 6, 34, 0, 34};

marks is a int type array, **marks.length = 6**, LB index = 0, UB index = 5

2-dimensional Arrays

Syntax : Two-Dimensional Arrays :

type[][] arrayname = new type[Row_Size][Col_Size];

or

type arrayname[][] = new type[Row_Size][Col_Size]; Row index varies from 0 to Row_Size – 1

Column index varies from 0 to Col_Size – 1

Examples :

1.int[][] data = new int[3][3]; **// Each element is initialized to 0**

data is 2-D int array, capacity = 9, row index 0 to 2 col index 0 to 2

2.float values[][] = new float[10][4]; **// Each element is initialized to 0.0**

values is 2-D float array, capacity = 40, row index 0 to 9 col index 0 to 3

3.int table[][] = {{ 0,0,0},{1,1,1}}; *// initializes first row to 0 & second to 1*

2-dimensional Arrays

- Java treats a 2-D array as a collection of 1-D arrays.
- In two-dimensional arrays, individual rows can have variable number of elements **<Variable Size Array>**
- Example

`int[][] data = new int[4][];` // Variable-Size Array, column-size not mentioned data is a 2-D array
having 4 rows (row-index varies from 0 to 3)

`data[0] = new int[10];` // Row-0 has 10 elements, column-index varies from 0 to 9
`data[1] = new int[5];` // Row-1 has 5 elements, column-index varies from 0 to 4
`data[2] = new int[6];` // Row-2 has 6 elements, column-index varies from 0 to 5
`data[3] = new int[10];` // Row-3 has 10 elements, column-index varies from 0 to 9

`data[1][6] →` Results in `ArrayIndexOutOfBoundsException`

`data[2][6] →` Results in `ArrayIndexOutOfBoundsException`

Referring Array Elements

- 1-D Arrays, **Syntax** → **array-name[<index>]**, where <index> varies from **0 (LB)** to **array-name.length-1(UB)**
- 2-D Arrays, **Syntax** → **array-name[<row-index>][<col-index>]**, where <row-index> varies from **0** to **no-of-rows -1**, and <col-index> varies from **0** to **no-of-cols -1**.
- Example
 1. `int[] data = new int[5];` `data[0] = 0; ....., data[4] = 0; flag[0]`
 2. `boolean[] flags = new boolean[10];` `= false, ....., flags[9] = false`
 3. `String[] names = new String[5];` `names[0]=null, ....., names[4] = null`
 4. `double[] values = {10.5, 5.6, 7.5, 4.5};`
 `values[0] = 10.5, values[1] = 5.6, values[2] = 7.5, values[3] = 4.5`
 5. `int [][] marks = {{10,40,20},{25,56,57},{10,89,94}}`
 `marks[0][0] = 10, marks[0][1] = 40, marks[0][2] = 20,`
 `marks[1][0] = 25, marks[1][1] = 56, marks[1][2] = 57,`
 `marks[2][0] = 10, marks[2][1] = 89, marks[2][2] = 94`

Thanks

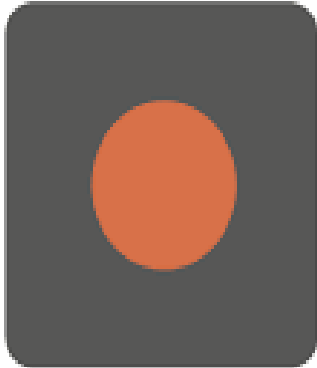


Object Oriented Programming and Design



BITS Pilani
Pilani Campus

Presented by
Dr Ramya Devi M
mramyadevi@wilp.bits-pilani.ac.in
CS - 05 : 26th Jul 2025



Start Recording



IMP Note to Students

- It is important to know that just login to the session does not guarantee the attendance.
- Once you join the session, continue till the end to consider you as present in the class.
- IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
- **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT**



BITS Pilani
Pilani Campus



Object-Oriented Design Process

Objectives



- To explain how a software design may be represented as a set of interacting objects that manage their own state and operations
- To describe the activities in the object-oriented design process
- To introduce various models that describe an object-oriented design
- To show how UML may be used to represent these models



Topics covered

- Qualities of a good software
- Object Oriented Software Life Cycle
- Objects and object classes
- An object-oriented design process
- Case Study

Qualities of Good Software



Software is treated as a good software by means of different factors.

A software product is concluded as a good software by what it offers and how well it can be used.

The factors that decide the software properties are divided into three categories:

- ✓ Operational,
- ✓ Transitional, and
- ✓ Maintenance

Qualities of Good Software



Operational:

In operational categories, the factors that decide the software performance in operations. It can be measured on:

- Budget
- Usability
- Efficiency
- Correctness
- Functionality
- Dependability
- Security
- Safety

Qualities of Good Software



Transitional:

When the software is moved from one platform to another, the factors deciding the software quality:

- Portability
- Interoperability
- Reusability
- Adaptability

Qualities of Good Software



Maintenance:

In this categories all factors are included that describes about how well a software has the capabilities to maintain itself in the ever changing environment:

- Modularity
- Maintainability
- Flexibility
- Scalability

Object-oriented Life Cycle Model in Software Engineering



The object-oriented life cycle model considers 'objects' as the basis of the software engineering process.

The development team starts by observing and analyzing the system they intend to develop before defining the requirements.

Once the process is over, they focus on identifying the objects of the system.

Now, an object could be anything; it can have a physical existence like a customer, car, etc.

An object also constitutes intangible elements like a process or a project.

Advantages of Object-Oriented Life Cycle Model



Apart from enhancing the system performance, object-oriented programming offers some advantages such as:

- It is **data-focused** and easy to work with problem domains.
- It uses **encapsulation and data hiding** process that allows a developer to build tamper-proof systems.
- It enables software **modularity**, making it easier to manage and maintain complex software.
- It allows developers to create **new modules using existing models**, saving time and development cost of organizations.

Object-Oriented Software Engineering



The primary objectives of the Object-Oriented Model

- Object-oriented Analysis
- Object-oriented Design
- Object-oriented Implementation

Object-Oriented Analysis (OOA)



Object-oriented analysis consists of the process where a development team evaluates the system and organizes the requirements as objects.

Contrary to traditional structural analysis, the OOA heavily depends on advanced data like **Use Cases** and **Object Models**.

Use case

Use Cases are written descriptions about how users will behave when they enter your website or application.

It comprises the goals of each user from the point of their entry to exit.

Object-Oriented Analysis (OOA)



Object Model

An object model allows the development team to create an architectural software or system model before implementing or programming.

It helps in defining a software/system (Application) in objects and classes.

It informs the developers about

- Interaction between different models
- Inheritance
- Encapsulation
- Other types of object-oriented interfaces



Object-oriented Design

Designing systems using self-contained objects and object classes



Object-oriented development

Object-oriented analysis, design and programming are related but distinct

OOA is concerned with developing an object model of the application domain – **Identifying the objects**

OOD is concerned with developing an object-oriented system model to implement requirements – **System architecture using the identified objects.**

OOP is concerned with realising an OOD using an OO programming language such as Java or C++ - **Coding**



Objects and object classes

Objects are entities in a software system which represent instances of real-world and system entities

Object classes are templates for objects.

Classes of real world object are defined and used to create objects in the software system.

Object classes may inherit attributes and services from other object classes.

Object = Data + Methods

Data = State of the object

Method = Behaviour of the object



The Unified Modeling Language

Several different notations for describing object-oriented designs were proposed in the 1980s and 1990s

The Unified Modeling Language is an integration of these notations

It describes notations for a number of different models that may be produced during OO analysis and design

It is now a *de facto* standard for OO modelling

Introduction to UML diagrams



There are 9 UML diagrams. They are

- 1. Use case diagram**
- 2. Class diagram**
- 3. Object diagram**
- 4. Sequence diagram**
5. Collaboration diagram
6. State chart diagram
7. Activity diagram
8. Deployment diagram
9. Component diagram

UML Diagram - Categories



UML defines various kinds of diagrams to cover most of the aspects of a system.

There are two broad categories of diagrams

1. Structural Diagrams
2. Behavioral Diagrams

These are further divided into sub-categories

Structural Diagrams



The *structural diagrams* represent the static aspect of the system.

These static aspects represent those parts of a diagram which forms the main structure and therefore stable.

These static parts represent classes, interfaces, objects, components and nodes.

The four structural diagrams are:

(i) Class diagram

(ii) Object diagram

(iii) Component diagram

(iv) Deployment diagram

Behavioral diagrams

Behavioral diagrams basically capture the dynamic aspect of a system.

Dynamic aspect can be further described as the changing/moving parts of a system.

The five behavioral diagrams are:

- (i) Use case diagram**
- (ii) Sequence diagram**
- (iii) Collaboration diagram
- (iv) State Chart diagram
- (v) Activity diagram



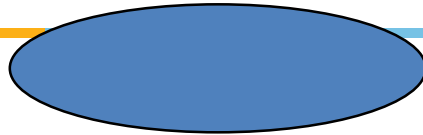
Use case and Use case diagrams

- Use case is one way of representing system functionality.
- Use case refers to
 - A system's behavior (functionality)
 - A set of activities that produce some output.
- Use verbs (Action words) for naming use cases.
- Use case diagrams represent system's behavior using use cases and actors.
- Provides a global look of a system that gives insight into functionality (use cases) and environment (actors).
- Useful for early structuring of requirements; iterative revisions

Use Case Symbols



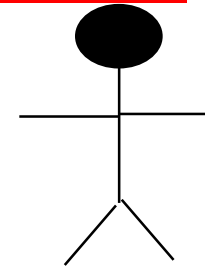
◆ Use Case



◆ Boundary



Actor



◆ Connection



◆ Include relationship

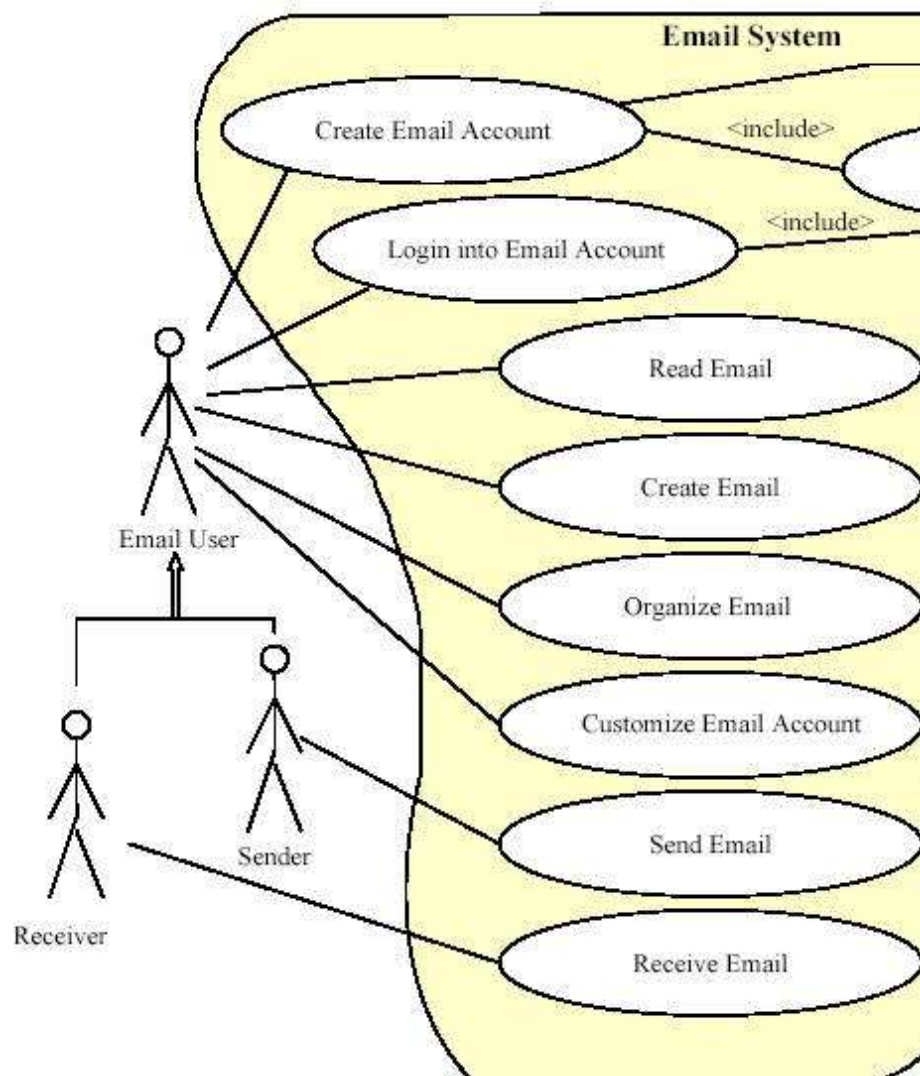
<<include>>

◆ Extend relationship

<<extend>>

Note: different names
used in different software

Use Cases of Email System – End User View (partial)



USE CASE DIAGRAM



These diagrams represent the use case view of a system.

A use case is a summary of **scenarios** of a single task.

A scenario is an example of what happens when someone communicates / interacts with the system.

Example 1 - For an order management system, we know that salesperson is the actor who does the sale to the customer.

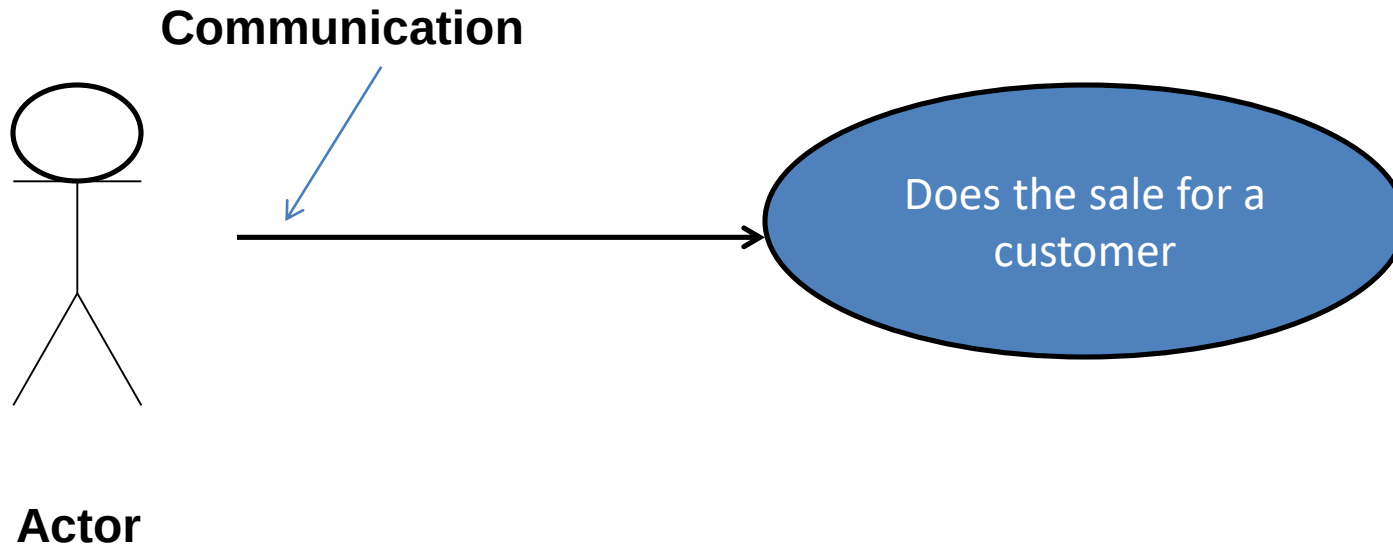
The simple use case diagram for the order management system is as follows:

USE CASE DIAGRAM



The diagram for order processing system

Actor is the Salesperson

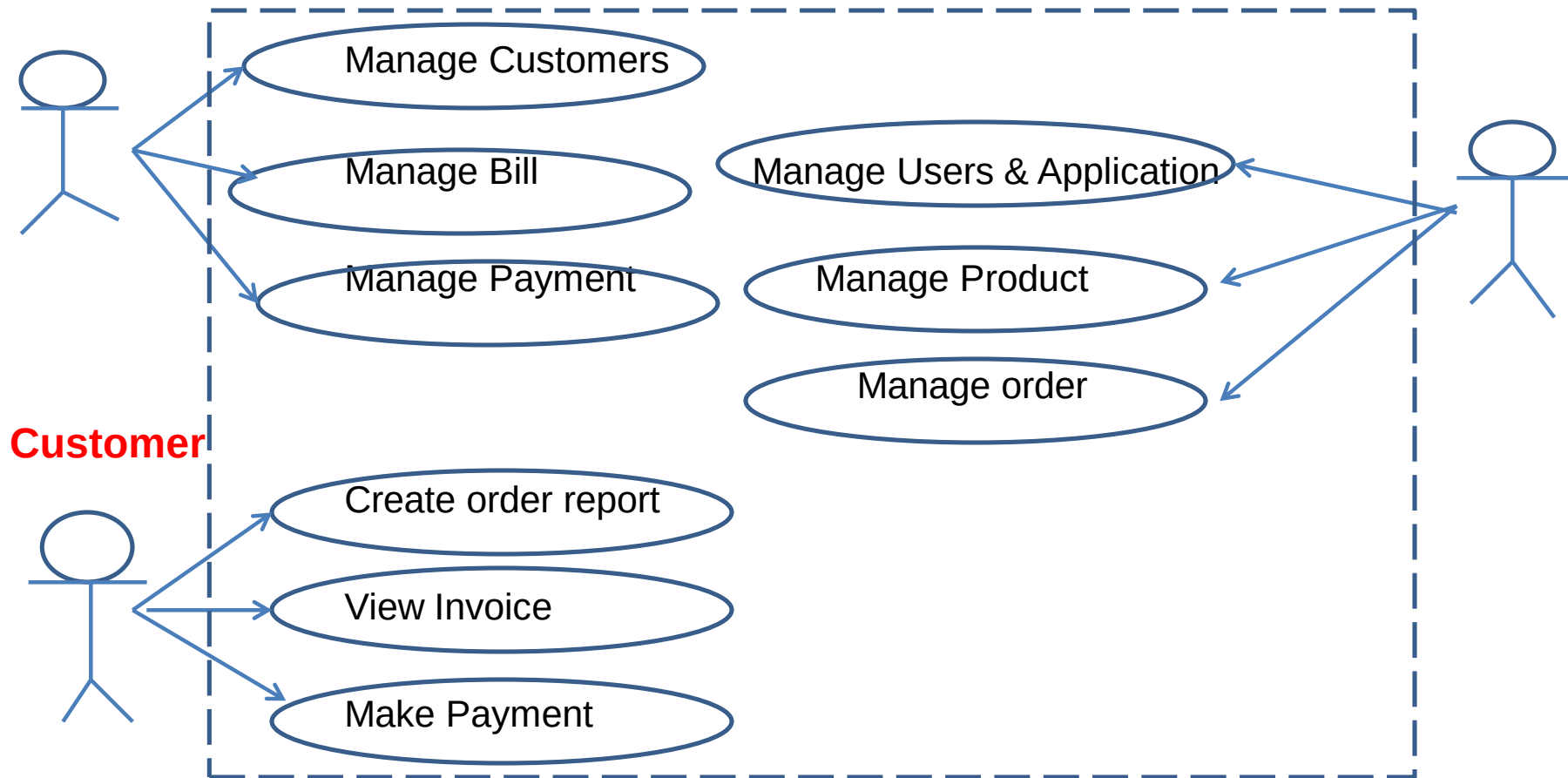


USE CASE DIAGRAM – Detailed – Order processing



Salesman

Admin



Use case diagrams - Summary



- Use cases start with a textual description.
- Some important details to show in a use case are:
 - Name of use case
 - Objective
 - Actor
 - Interactions between use case and actor

UML Class Diagrams- Basics



When it comes to system construction, a **class diagram** is the most widely used diagram.

This diagram generally consists of **interfaces, classes, associations and collaborations**.

Such a diagram would illustrate the object-oriented view of a system, which is static in nature.

The object orientation of a system is indicated by a class diagram.

Since class diagrams are used for many different purposes, such as making stakeholders aware of requirements to highlighting your detailed design, you need to apply a different style in each circumstance.

UML Class Diagrams



A class diagram specifies an overview of a system by showing the classes involved in the system and the relationships between them.

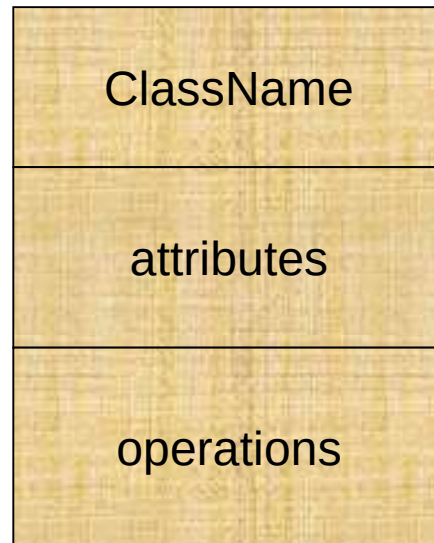
A *class* is a description of a set of objects that share the common attributes, operations, relationships, and semantics.

Graphically, a class is rendered as a **rectangle**, usually including its name, attributes, and operations in separate, designated compartments.

UML Class Diagrams - Basics



A class is represented in the following format



UML Class Diagrams - Basics



The **name of the class** is the only required tag in the graphical representation of a class.

It always appears in the top-most compartment.

An **attribute** is a named property of a class that describes the object being modeled.

In the class diagram, attributes appear in the second compartment just below the name-compartment.

Attributes are usually listed in the form:

attributeName : Type

UML Class Diagrams - Basics



A *derived* attribute is one that can be found from other attributes, but doesn't actually exist.

For example, the age of a person can be calculated from his / her birth date.

A derived attribute is designated by a Preceding '/' as in:
/ age : Date

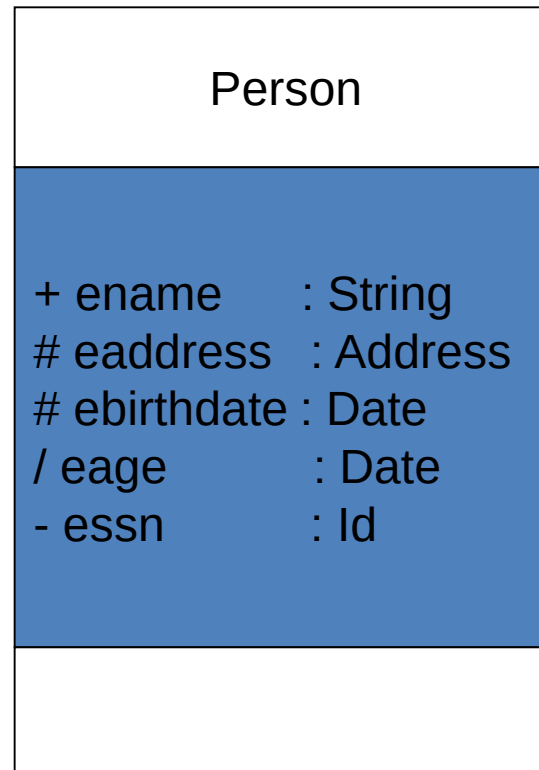
Visibility for the attributes is represented as follows:

- + - Public
- - Private
- # - Protected

UML Class Diagrams - Basics



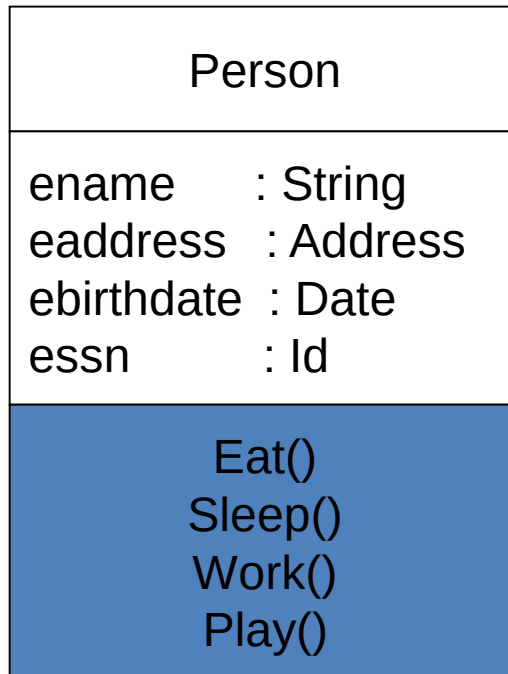
The attributes of a person can be specified as



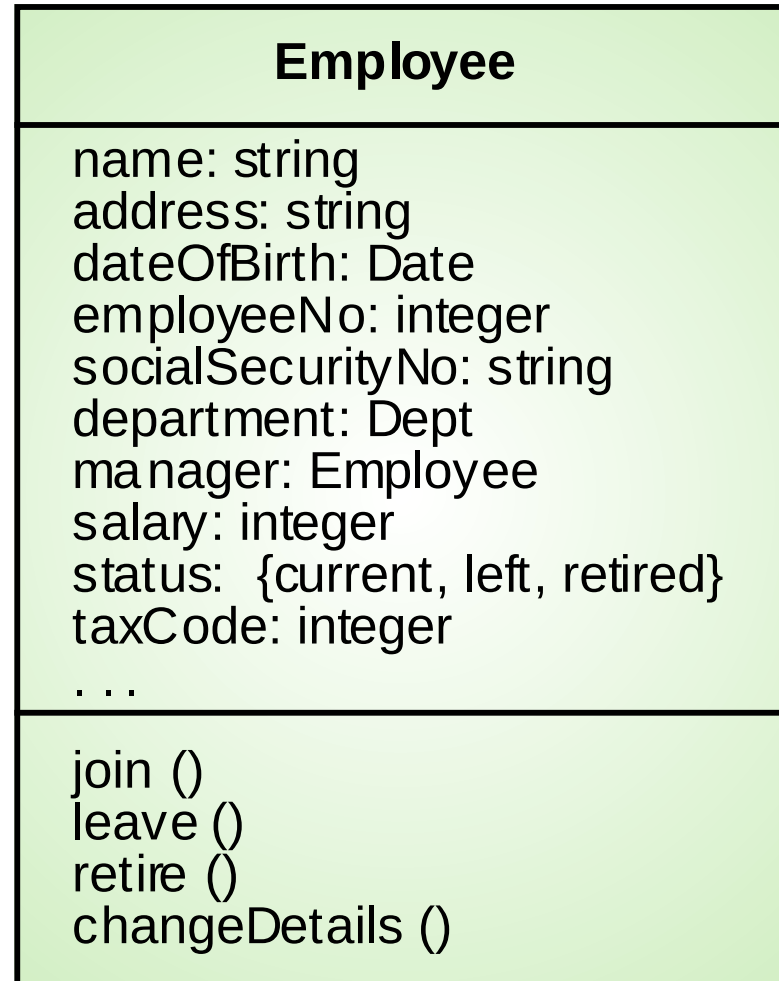
UML Class Diagrams - Basics



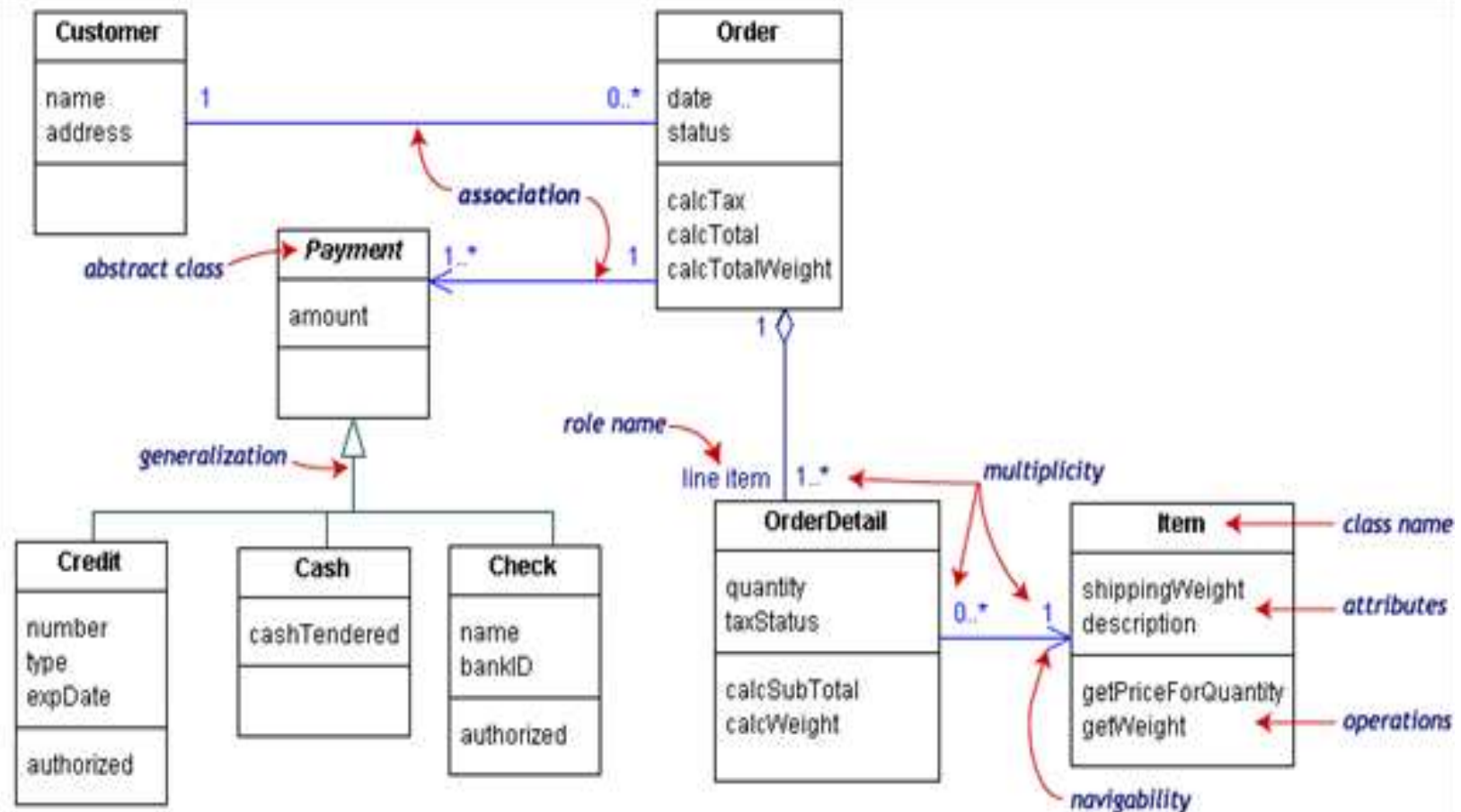
Operations specify the class behavior and appear in the third compartment.



Employee object class (UML)



Class Diagrams for Order management System



Packages and Object diagrams



Object Diagram :

An object is an instance of an existing class.

Multiple objects can be pictorially represented in a single diagram.

Package Diagram

Package is a collection of classes.

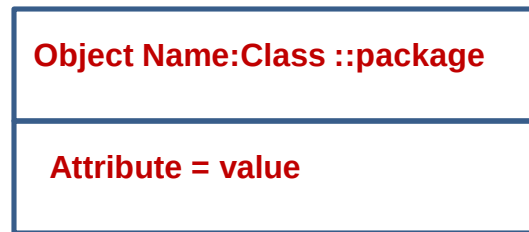
Package appears as rectangles with small tabs at the top of the rectangle.

Package name is on the tab or inside the rectangle

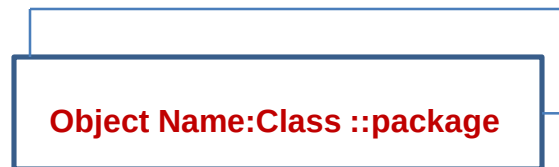
Packages and Object diagrams



Single Object



Multiple Objects



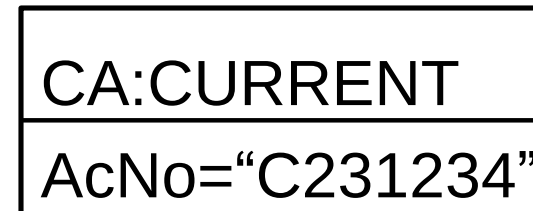
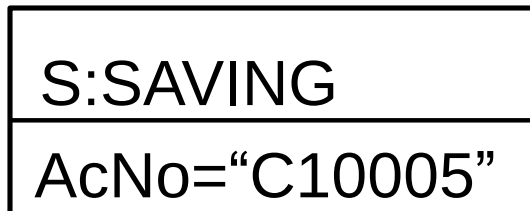
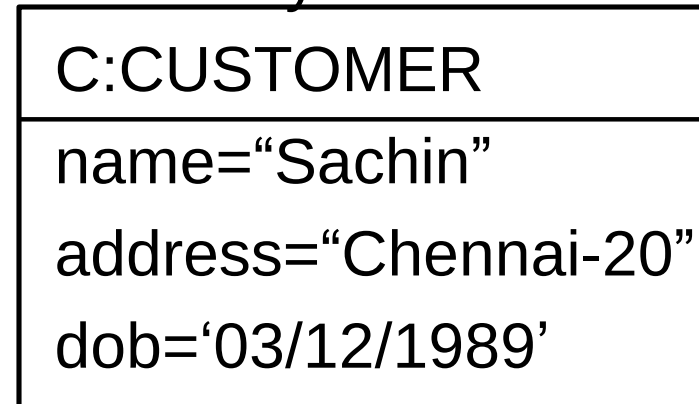
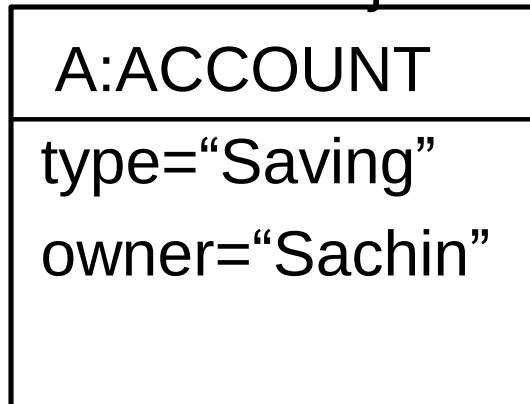
Package



Object diagrams



Some of the objects created for the ATM system are



A, C , S and CA are the objects of the classes ACCOUNT, CUSTOMER,SAVING and CURRENT respectively.



UML Sequence Diagrams

The diagrams which describe how objects collaborate are called as Interaction diagrams.

A sequence diagram is an interaction diagram that specify how operations are carried out and what messages are sent and **when**.

These diagrams are organized according to time.

The objects involved in the operation are listed from the left to right according to the sequence of the message passed.

UML Sequence Diagrams

- A sequence diagram is an interaction diagram dealing with the sequence of messages flowing from one object to another and includes time aspect.
- Describes the flow of messages, events, actions between objects
- Show concurrent processes and activations
- Show time sequences that are not easily depicted in other diagrams

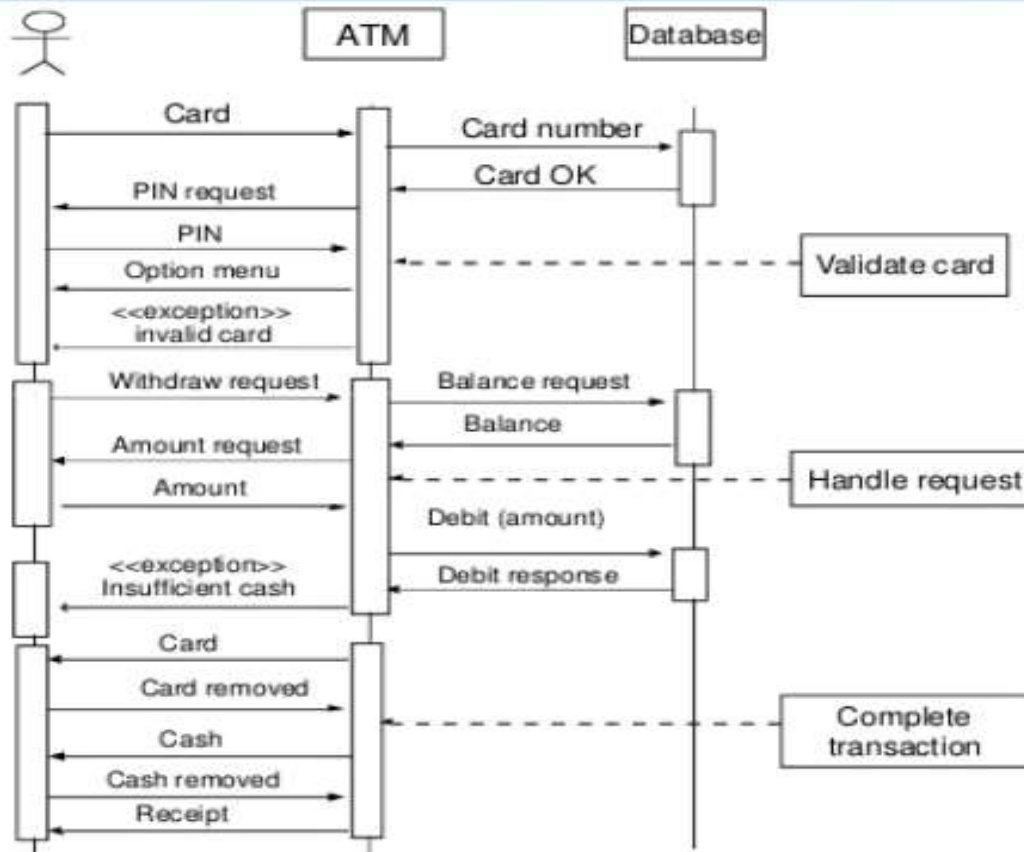
Typically used during analysis and design to document and understand the logical flow of our system.

The following diagrams represent the Use case diagram and Sequence diagram for withdrawing money from ATM machine.

UML Sequence Diagrams – Example



Sequence diagram of ATM withdrawal



An object-oriented design process – Case study



- ✓ Define the context and modes of use of the system
- ✓ Design the system architecture
- ✓ Identify the principal system objects
- ✓ Develop design models
- ✓ Specify object interfaces



Weather system description

A weather data collection system is required to generate weather maps on a regular basis using data collected from remote, unattended weather stations and other data sources such as weather observers, balloons and satellites.

Weather stations transmit their data to the area computer in response to a request from that machine.

The area computer validates the collected data and integrates it with the data from different sources.

The integrated data is archived and, using data from this archive and a digitised map database a set of local weather maps is created.

Maps may be printed for distribution on a special-purpose map printer or may be displayed in a number of different formats.



Weather station description

A weather station is a package of software controlled instruments which collects data, performs some data processing and transmits this data for further processing.

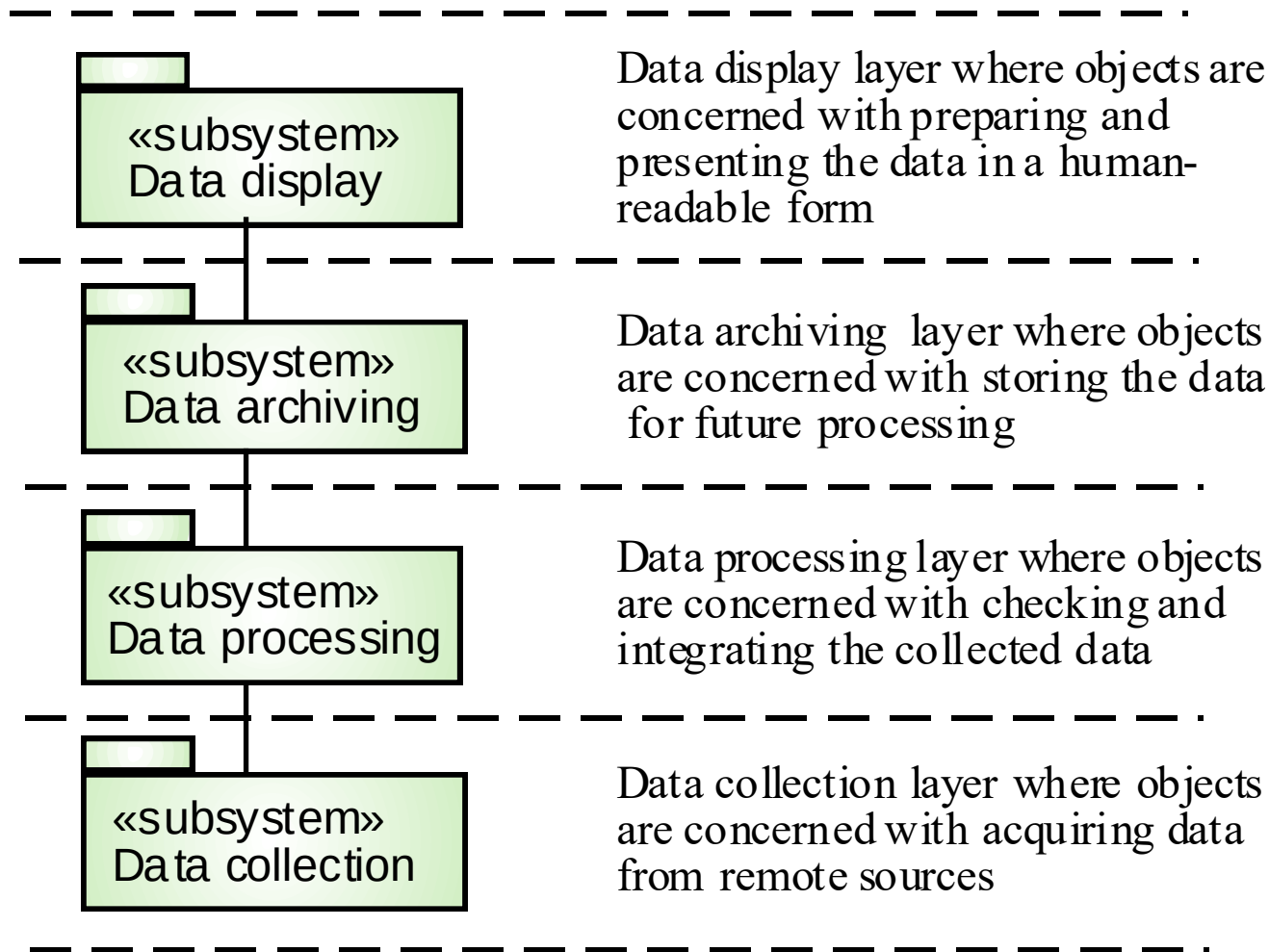
The instruments include air and ground thermometers, an anemometer, a wind vane, a barometer and a rain gauge.

Data is collected every five minutes.

When a command is issued to transmit the weather data, the weather station processes and summarises the collected data.

The summarised data is transmitted to the mapping computer when a request is received.

Layered architecture



System context and models of use



Develop an understanding of the relationships between the software being designed and its external environment

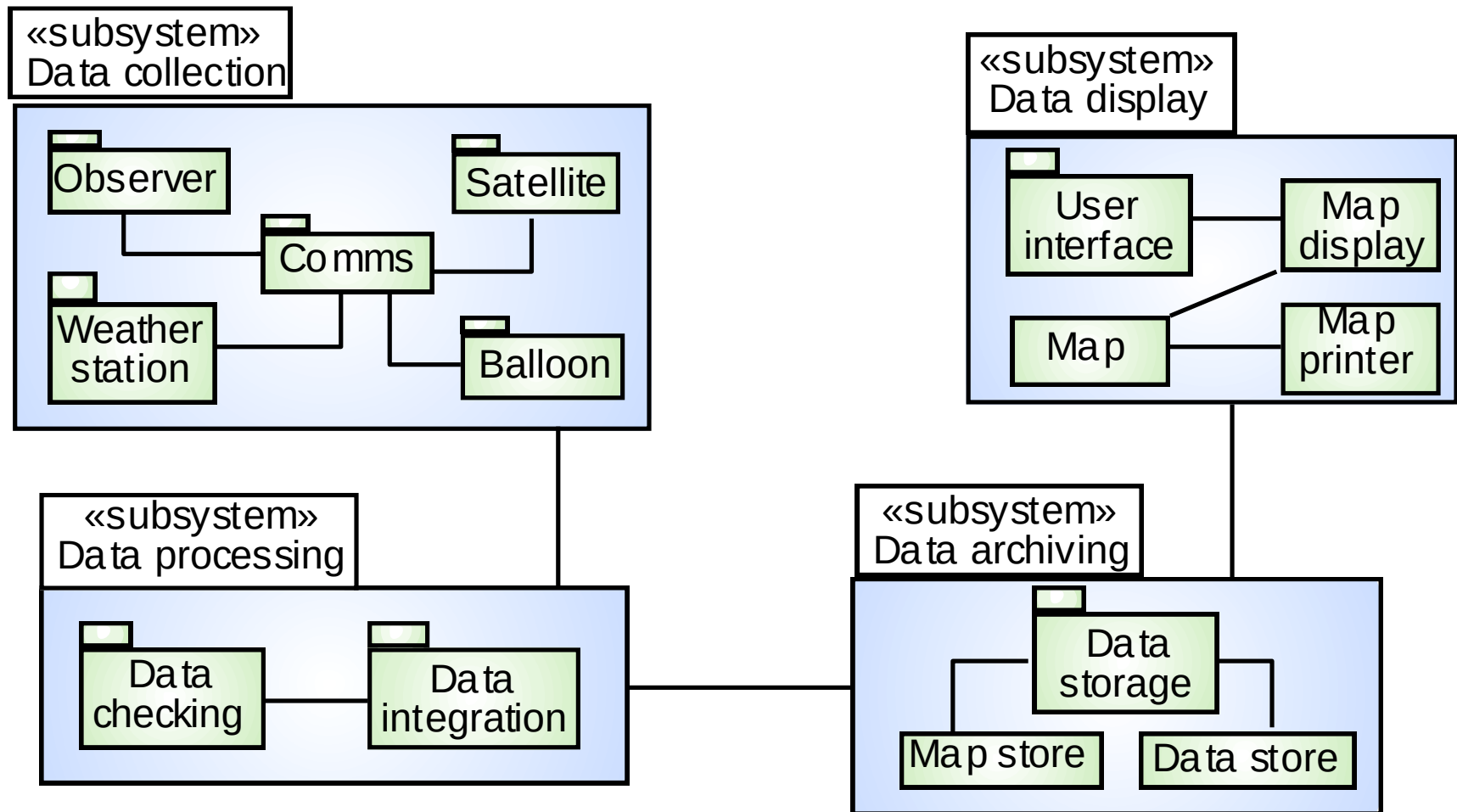
System context

- A static model that describes other systems in the environment.
- Use a subsystem model to show other systems.
- Following slide shows the systems around the weather station system.

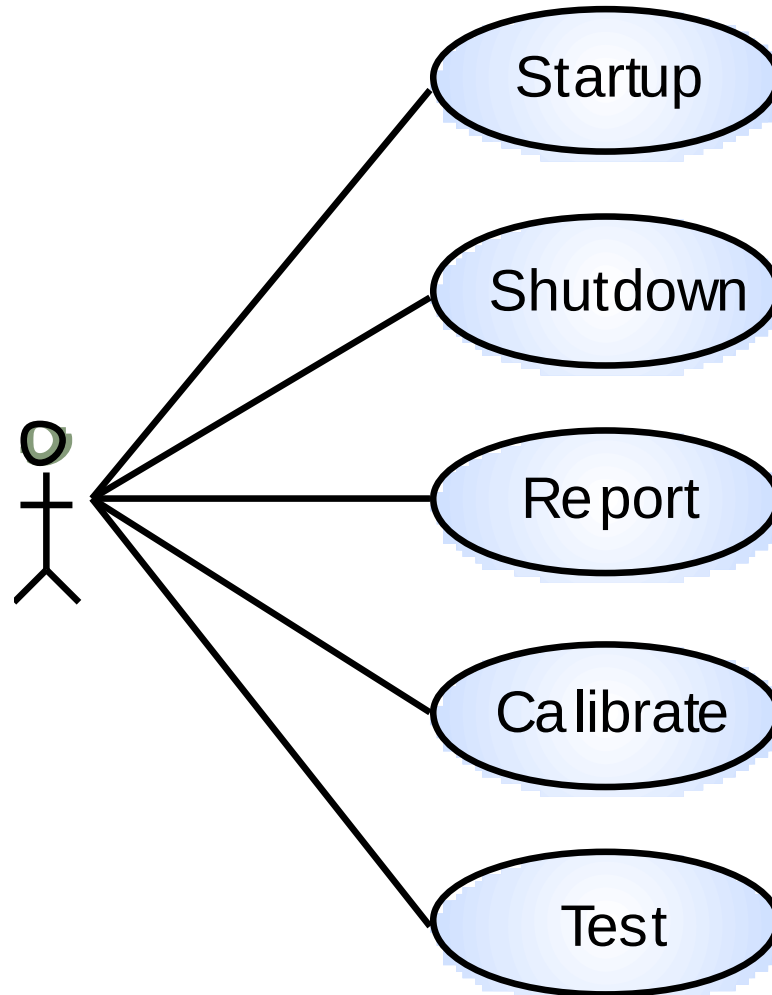
Model of system use

- A dynamic model that describes how the system interacts with its environment.
- Use use-cases to show interactions

Subsystems in the weather mapping system



Use-cases for the weather station



Use-case description

System	Weather station
Use-case	Report
Actors	Weather data collection system, Weather station
Data	The weather station sends a summary of the weather data that has been collected from the instruments in the collection period to the weather data collection system. The data sent are the maximum minimum and average ground and air temperatures, the maximum, minimum and average air pressures, the maximum, minimum and average wind speeds, the total rainfall and the wind direction as sampled at 5 minute intervals.
Stimulus	The weather data collection system establishes a modem link with the weather station and requests transmission of the data.
Response	The summarised data is sent to the weather data collection system
Comments	Weather stations are usually asked to report once per hour but this frequency may differ from one station to the other and may be modified in future.



Architectural design

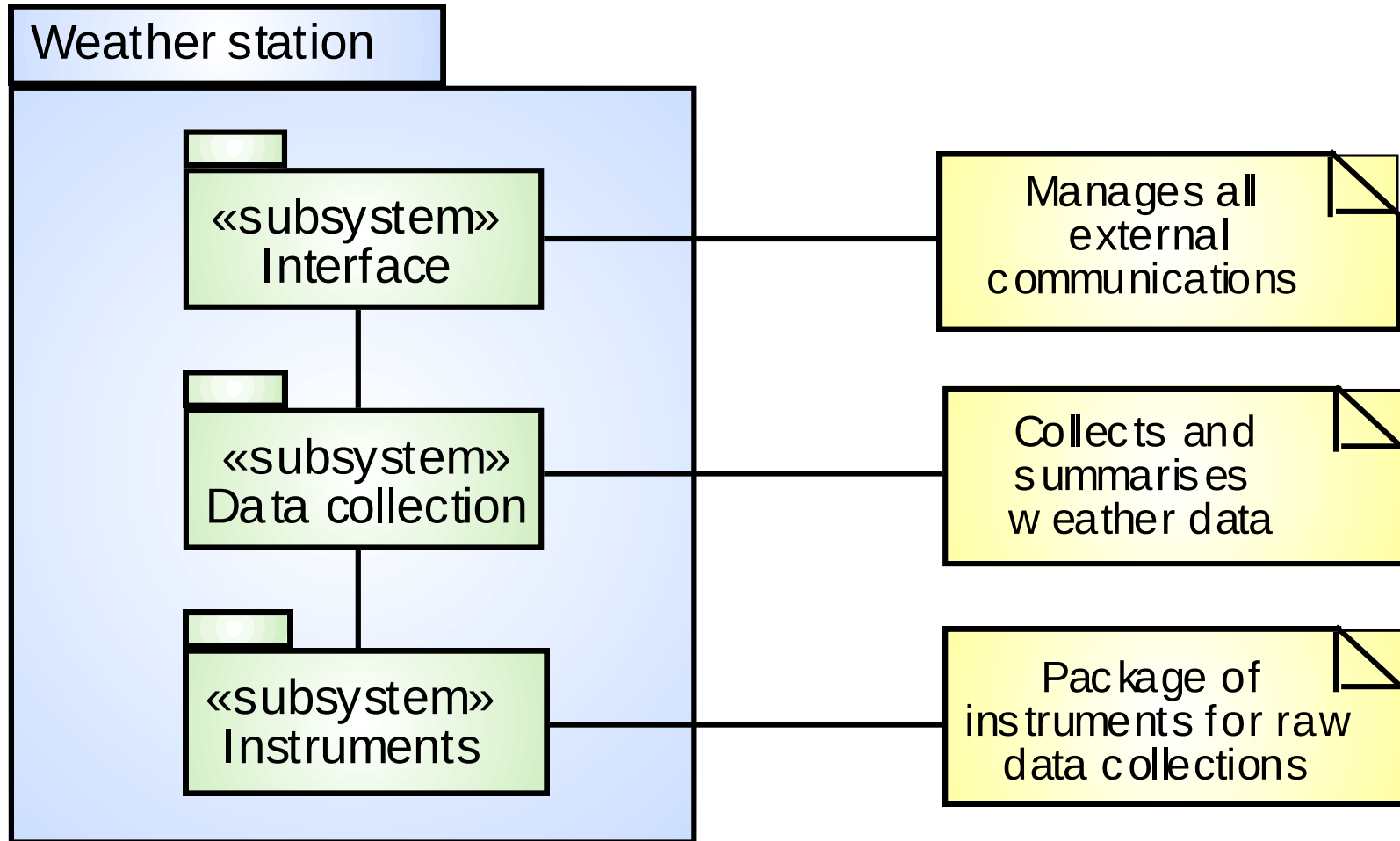
Once interactions between the system and its environment have been understood, you use this information for designing the system architecture

Layered architecture is appropriate for the weather station

- Interface layer for handling communications
- Data collection layer for managing instruments
- Instruments layer for collecting data

There should be no more than 7 entities in an architectural model

Weather station architecture





Object identification

Identifying objects (or object classes) is the most difficult part of object oriented design

There is no 'magic formula' for object identification.

It relies on the skill, experience and domain knowledge of system designers

Object identification is an iterative process.

You are unlikely to get it right first time



Approaches to identification

Use a grammatical approach based on a natural language description of the system (used in Hood method)

Base the identification on tangible things in the application domain

Use a behavioural approach and identify objects based on what participates in what behaviour

Use a scenario-based analysis.

The objects, attributes and methods in each scenario are identified

Weather station object classes



Ground thermometer, Anemometer, Barometer

- Application domain objects that are ‘hardware’ objects related to the instruments in the system

Weather station

- The basic interface of the weather station to its environment. It therefore reflects the interactions identified in the use-case model

Weather data

- Encapsulates the summarised data from the instruments

Weather station object classes



WeatherStation
identifier
reportWeather () calibrate (instruments) test () startup (instruments) shutdown (instruments)

WeatherData
airTemperatures groundTemperatures windSpeeds windDirections pressures rainfall
collect () summarise ()

Ground thermometer
temperature
test () calibrate ()

Anemometer
windSpeed windDirection
test ()

Barometer
pressure height
test () calibrate ()

Further objects and object refinement



Use domain knowledge to identify more objects and operations

- Weather stations should have a unique identifier
- Weather stations are remotely situated so instrument failures have to be reported automatically. Therefore attributes and operations for self-checking are required

Active or passive objects

- In this case, objects are passive and collect data on request rather than autonomously. This introduces flexibility at the expense of controller processing time



Design models

Design models show the objects and object classes and relationships between these entities

Static models describe the static structure of the system in terms of object classes and relationships

Dynamic models describe the dynamic interactions between objects.



Examples of design models

Sub-system models that show logical groupings of objects into coherent subsystems.

State machine models that show how individual objects change their state in response to events

Sequence models that show the sequence of object interactions

Other models include use-case models, aggregation models, generalisation models, etc.



Subsystem models

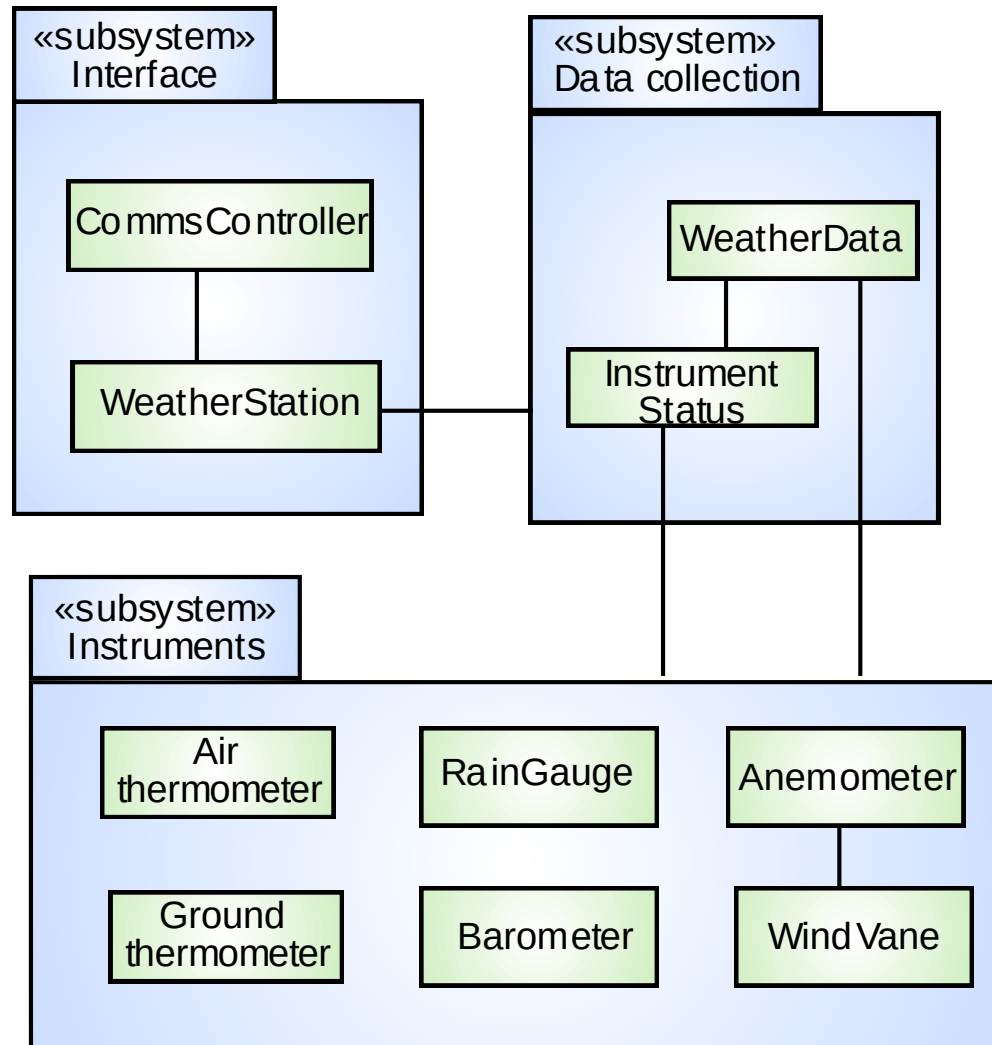
Shows how the design is organised into logically related groups of objects and / or packages.

In the UML, these are shown using packages - an encapsulation construct.

This is a **logical model**.

The actual organisation of objects in the system may be different.

Weather station subsystems



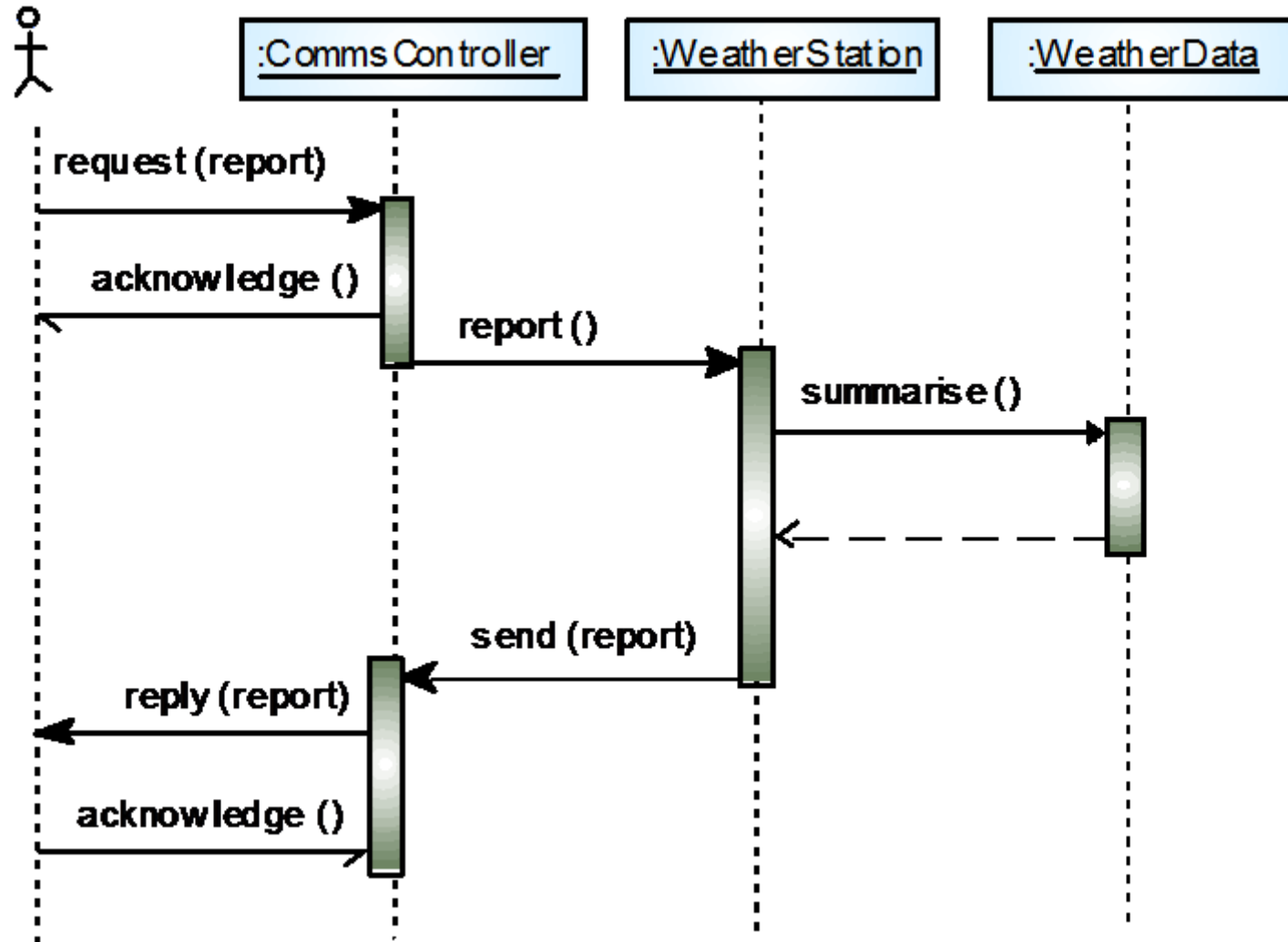


Sequence models

Sequence models show the sequence of object interactions that take place

- Objects are arranged horizontally across the top
- Time is represented vertically so models are read top to bottom
- Interactions are represented by labelled arrows, Different styles of arrow represent different types of interaction
- A thin rectangle in an object lifeline represents the time when the object is the controlling object in the system

Data collection sequence





Key points

OOD is an approach to design so that design components have their own private state and operations

Objects should have constructor and inspection operations. They provide services to other objects

Objects may be implemented sequentially or concurrently.

The Unified Modeling Language provides different notations for defining different object models



Key points

A range of different models may be produced during an object-oriented design process. These include static and dynamic system models

Object interfaces should be defined precisely using e.g. a programming language like Java

Object-oriented design simplifies system evolution

Write simple Java code to represent the Weather System.



STOP REG

Thank You



Object Oriented Programming and Design



BITS Pilani
Pilani Campus

Presented by
Dr Ramya Devi M
mramyadevi@wilp.bits-pilani.ac.in
CS - 06 : 02nd Aug 2025



Arrays class



Arrays class in Java

- Arrays class in **<<java.util>>** package is defined as follows
 - This class contains the required methods for manipulating arrays such as sorting and searching
 - All methods are **<<static>>** and hence can be called via class name.
 - Arrays class is part of Java's Collection Framework
-

Arrays class: Important Methods



binarySearch()

<i>static int</i>	<u>binarySearch</u> (byte[] a, byte key)
<i>static int</i>	<u>binarySearch</u> (char[] a, char key)
<i>static int</i>	<u>binarySearch</u> (double[] a, double key)
<i>static int</i>	<u>binarySearch</u> (float[] a, float key)
<i>static int</i>	<u>binarySearch</u> (int[] a, int key)
<i>static int</i>	<u>binarySearch</u> (long[] a, long key)
<i>static int</i>	<u>binarySearch</u> (<u>Object</u> [] a, <u>Object</u> key)
<i>static int</i>	<u>binarySearch</u> (<u>Object</u> [] a, <u>Object</u> key, <u>Comparator</u> c)
<i>static int</i>	<u>binarySearch</u> (short[] a, short key)

Arrays class: Important Methods



....

equals()

static boolean	<u>equals</u> (boolean[] a, boolean[] a2)
static boolean	<u>equals</u> (byte[] a, byte[] a2)
static boolean	<u>equals</u> (char[] a, char[] a2)
static boolean	<u>equals</u> (double[] a, double[] a2)
static boolean	<u>equals</u> (float[] a, float[] a2)
static boolean	<u>equals</u> (int[] a, int[] a2)
static boolean	<u>equals</u> (long[] a, long[] a2)
static boolean	<u>equals</u> (<u>Object</u> [] a, <u>Object</u> [] a2)
static boolean	<u>equals</u> (short[] a, short[] a2)



```
import java.util.Arrays;
public class FillRangeExample {
    public static void main(String[] args) {
        int[] array = new int[10];
        // Fill a specific range (index 3 to 7) with the value 5
        Arrays.fill(array, 3, 7, 5);
        System.out.println("Array after filling a range: " + Arrays.toString(array)); // [0, 0,
0, 5, 5, 5, 5, 0, 0, 0]
    }
}
```

```
import java.util.Arrays;
```



```
public class SortRangeExample {
```

```
    public static void main(String[] args) {
```

```
        int[] array = {5, 2, 8, 3, 1, 7, 6};
```

```
        // Sort a specific range (index 2 to 5)
```

```
        Arrays.sort(array, 2, 6);
```

```
        System.out.println("Array after range sort: " + Arrays.toString(array)); // [5, 2, 1, 3, 7, 8, 6]
```

```
    }
```

```
}
```

Arrays class: Important Methods



....

fill()

static void	<u>fill</u> (boolean[] a, boolean val)
static void	<u>fill</u> (boolean[] a, int fromIndex, int toIndex, boolean val)
static void	<u>fill</u> (byte[] a, byte val)
static void	<u>fill</u> (byte[] a, int fromIndex, int toIndex, byte val)
static void	<u>fill</u> (char[] a, char val)
static void	<u>fill</u> (char[] a, int fromIndex, int toIndex, char val)
static void	<u>fill</u> (double[] a, double val)
static void	<u>fill</u> (double[] a, int fromIndex, int toIndex, double val)
static void	<u>fill</u> (float[] a, float val)

Arrays class: Important Methods



....

fill()

static void	<u>fill</u> (float[] a, int fromIndex, int toIndex, float val)
static void	<u>fill</u> (int[] a, int val)
static void	<u>fill</u> (int[] a, int fromIndex, int toIndex, int val)
static void	<u>fill</u> (long[] a, int fromIndex, int toIndex, long val)
static void	<u>fill</u> (long[] a, long val)
static void	<u>fill</u> (<u>Object</u> [] a, int fromIndex, int toIndex, <u>Object</u> val)
static void	<u>fill</u> (<u>Object</u> [] a, <u>Object</u> val)
static void	<u>fill</u> (short[] a, int fromIndex, int toIndex, short val)
static void	<u>fill</u> (short[] a, short val)

Arrays class: Important Methods



....

sort()

static void	<u>sort</u> (byte[] a)
static void	<u>sort</u> (byte[] a, int fromIndex, int toIndex)
static void	<u>sort</u> (char[] a)
static void	<u>sort</u> (char[] a, int fromIndex, int toIndex)
static void	<u>sort</u> (double[] a)
static void	<u>sort</u> (double[] a, int fromIndex, int toIndex)
static void	<u>sort</u> (float[] a)
static void	<u>sort</u> (float[] a, int fromIndex, int toIndex)
static void	<u>sort</u> (int[] a)
static void	<u>sort</u> (int[] a, int fromIndex, int toIndex)

Arrays class: Important Methods



....

sort()....

static void	<u>sort</u> (long[] a)
static void	<u>sort</u> (long[] a, int fromIndex, int toIndex)
static void	<u>sort</u> (<u>Object</u> [] a)
static void	<u>sort</u> (<u>Object</u> [] a, <u>Comparator</u> c)
static void	<u>sort</u> (<u>Object</u> [] a, int fromIndex, int toIndex)
static void	<u>sort</u> (<u>Object</u> [] a, int fromIndex, int toIndex, <u>Comparator</u> c)
static void	<u>sort</u> (short[] a)
static void	<u>sort</u> (short[] a, int fromIndex, int toIndex)

Arrays class: Example



```
import java.util.*;  Import java.util package to use  
Arrays class  
class ArrayExample
```

```
{  
public static void main(String args[])  
{
```

```
int x[] = {10,6,8,20};      int array Size = 4 LB=0 UB =3
```

```
double data[] = { 12.5,34.6,90.56,12.34,12.56};
```

double array Size = 5 LB=0 UB =4

```
float values[] = { 10.45f,23.56f,12.67f};
```

float arrays Size = 3 LB=0 UB =2

```
double data1[] = new double[10];
```

*double array Size = 10 LB=0
UB =9*

```
boolean flags[] = new boolean[5];
```

boolean array Size = 5 LB=0 UB =4

```
int x1[] = {10,6,8,20};  int array Size = 4 LB=0 UB =3
```

Arrays class: Example



```
System.out.println(Arrays.binarySearch(x,20));
```

```
Arrays.sort(x);
```

```
for(int i=0;i<x.length;i++)
```

```
System.out.print(x[i]+" ");
```

```
System.out.println();
```

```
Arrays.fill(data1,12.56);
```

```
for(int i=0;i<data1.length;i++)
```

```
System.out.print(data1[i]+" ");
```

```
System.out.println();
```

Prints index of 20 in x

3

Sorts elements of x

Prints Elements of x

6 8 10 20

Fills a single value
12.56 in all indexes of
data1

Prints Elements of
data1

12.56 12.56 12.56 12.56 12.56 12.56 12.56 12.56 12.56 12.56

Arrays class: Example



```
Arrays.fill(flags,2,5,true);  
//Arrays.fill(flags,2,6,true);  
for(int i=0;i<flags.length;i++)  
System.out.print(flags[i]+" ");  
System.out.println();  
  
System.out.println(Arrays.equals(x,x1));  
  
Arrays.sort(values);  
  
for(int i=0;i<values.length;i++)  
System.out.print(values[i]+" ");  
System.out.println();  
  
}  
} // End of class
```

Arrays.fill(flags,2,5,true); — Fills true value in boolean array *flags* from index 2 to 4

//Arrays.fill(flags,2,6,true); — **ArrayIndexOutOfBoundsException**

for(int i=0;i<flags.length;i++) — **false false true true true**

System.out.print(flags[i]+" ");

System.out.println();

System.out.println(Arrays.equals(x,x1)); — Prints true or false whether x and x1 equals

Arrays.sort(values); — Sorts the elements of arrays

for(int i=0;i<values.length;i++)

System.out.print(values[i]+" "); — **10.45 12.67 23.56**

System.out.println();

}

} // End of class

Array : Examples



```
int a[] = { 10,8,6};  
for(int i=0;i<a.length;i++)  
System.out.println(a[i]);
```



10
8
6

```
int a[3] = { 10,8,6};  
for(int i=0;i<a.length;i++)  
System.out.println(a[i]);
```



']' expected
int a[3] = { 10,8,6};
^
1 error

```
//int table[][]={0,0,0,1,1,1};  
//int table[2][3]={0,0,0,1,1,1};
```

```
int table[ ][ ]={{0,0,0},{1,1,1}};  
for(int i=0; i<table.length;i++)  
{  
for(int j=0;j<table[i].length;j++)  
{  
System.out.print(table[i][j]);  
System.out.println();  
}  
}
```



000
111

WRONG

Thanks



Object Oriented Programming and Design



BITS Pilani
Pilani Campus

Presented by
Dr Ramya Devi M
mramyadevi@wilp.bits-pilani.ac.in
CS - 07 : 09th Aug 2025

IMP Note to Self



String Introduction



- Represents character strings such as “a”, “David”, “Mike” etc
- Every String literal (such as “123”, “Ram”, “151103” etc.) is a String type object in Java and hence any String class method can be invoked via string type literals also. For Example, “abc”.length(), “abc”.equals(“ABC”);
- String str = “abc”; , or String str = new String(“abc”); are equivalent statements
- Strings in Java represents immutable strings
- Every string has a length which can be retrieved using **length()** method. For Example “Mike”.length() returns 4
- Every character of String object is indexed and index varies from **0** to **L-1** (Where ‘L’ is length of String). For example, string “**Mike**” has character ‘**M**’ at index **0**, character ‘**i**’ at index **1** etc.
- Getting characters out of index will results in **StringIndexOutOfBoundsException**.

Important String Constructors



1. `String()` → Creates an Empty String, length = 0
2. `String(char[] str)` → Creates a string from char array
3. `String(char chars[], int start, int numChars)` → Creates a String from `chars` array starting from index '`start`' to '`start + numChars - 1`'
4. `String(String str)` → Creates a String from another String
5. `String(byte[] bytes)` → Creates a String from byte array
6. `String(byte bytes[], int start, int numChars)` → Creates a String from `bytes` starting from index '`start`' to '`start + numChars - 1`'

String Constructors: Example



// Creating Strings from String Literals

```
String str    = "abc";           // String str = new String("abc");
String str1   = new String("abc");
```

// Creating Strings from char[] Array

```
char[]      chs = {'o','b','j','e','c','t'};
String str2 = new String(chs);    // str2: "object"
```

```
String str3 = new String(chs,2,4); // start-index=1, number-of-characters=4, index range: 2 to 5 ("ject");
```

```
//String str4 = new String(chs,3,4); // start-index=3, number-of-characters=4, index range: 3 to 6
```

Results in **StringIndexOutOfBoundsException**

// Creating Strings from byte[] Array

```
byte[]      bytes = {45,65,90,78,23,89};
```

```
String str5 = new String(bytes); // str5: -AZN↑Y
```

```
String str6 = new String(bytes,2,4); // start-index=1, number-of-characters=4, index range: 2 to 5, str6: ZN↑Y
```

```
//String str7 = new String(bytes,3,4); // start-index=3, number-of-characters=4, index range: 3 to 6
```

Comparing Strings : equals() Method



- **public boolean equals(String other)** → Returns true if this string is equal to other otherwise false. Two strings are equal if they have same character at each index and their lengths are equal.
- **equals()** → Checks states of two string references
- Examples
 1. "abc".equals("abc") returns true
 2. String s1 = "xyz"; String s2 = "abc"; s1.equals(s2) or s2.equals(s1) returns false

Comparing Strings : equals() Method vs ==



- Equality operator (==) returns true if both string references are pointing to same object otherwise false.
- equals() method checks the states of the string references
- Example:

```
String    s1 = new String("abc");
```

```
String    s2 = new String("abc");
```

```
String    s3 = s1;
```

```
if(s1 == s2)
```

```
else
```

```
System.out.println("Hello");
```

```
System.out.println("Hi");
```

Hi

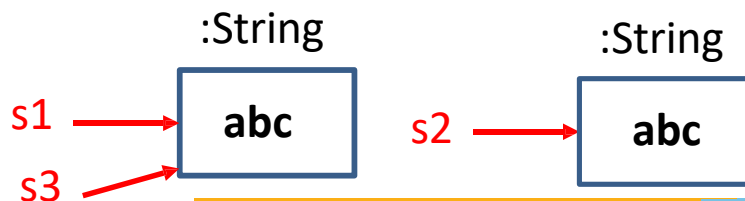
```
if(s1.equals(s2))
```

```
else
```

```
System.out.println("Hello");
```

```
System.out.println("Hi");
```

Hello



Comparing Strings : equals() Method vs ==



- Example:

```
String s1 = new String("abc");
```

```
String s2 = new String("abc");
```

```
String s3 = s1;
```

```
if(s1 == s3)
```

```
else
```

```
System.out.println("Hello");
```

```
System.out.println("Hi");
```

Hello



Predict the Output



```
//File Name: StringDemo.java
class StringDemo
{
    public static void main(String[] args)
    {
        String str = new String("Object");

        str = str + " Oriented";

        str = str + " Programming";

        System.out.println(str);

    } // End of Method
} // End of class StringDemo
```

Predict the Output



```
//File Name: StringDemo.java
```

```
class StringDemo
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        String str = new String("Object");
```

str



:String

Object

```
        str = str + " Oriented";
```

```
        str = str + " Programming";
```

```
        System.out.println(str);
```

```
    } // End of Method
```

```
} // End of class StringDemo
```

Predict the Output



```
//File Name: StringDemo.java
```

```
class StringDemo
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        String str = new String("Object");
```

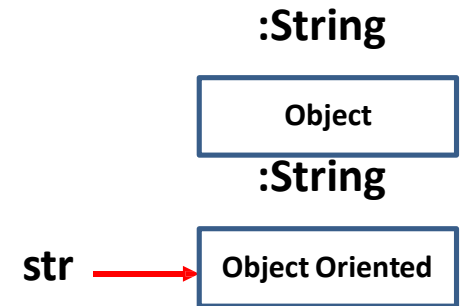
```
        str = str + " Oriented";
```

```
        str = str + " Programming";
```

```
        System.out.println(str);
```

```
    } // End of Method
```

```
} // End of class StringDemo
```



Predict the Output



```
//File Name: StringDemo.java
```

```
class StringDemo
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        String str = new String("Object");
```

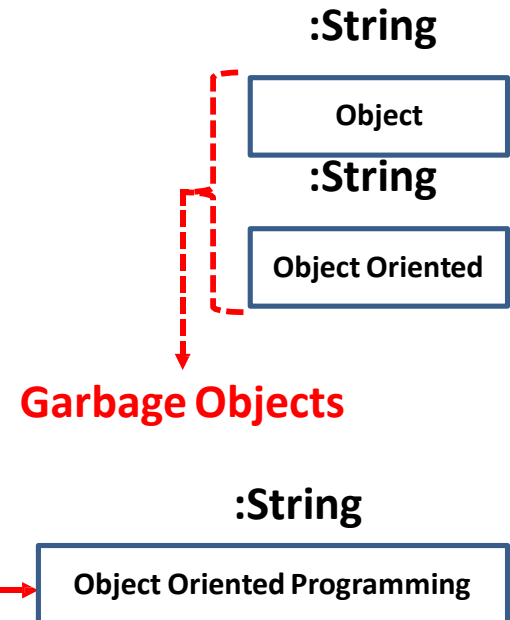
```
        str = str + " Oriented";
```

```
        str = str + " Programming";
```

```
        System.out.println(str);
```

```
    } // End of Method
```

```
} // End of class StringDemo
```



Object Oriented Programming

Each Time You Update a string reference via + operator , A new String Object is Created with updated Contents

Predict the Output



//File Name: StringDemo.java

class StringDemo

{

 public static void main(String[] args)

 {

 String str = new String("Object");

 str = new String("Object Oriented");

 str = new String("Object Oriented Programming");

 System.out.println(str);

 } // End of Method

} // End of class StringDemo

How ManyString Objects are Created?



```
//File Name: StringDemo.java
class StringDemo
{
    public static void main(String[] args)
    {
        String s1 = "abc";

        String s2 = "abc";

        String s3 = "abc";

        String s4 = "abc";
    } // End of Method
} // End of class StringDemo
```

How Many String Objects are Created?



//File Name: StringDemo.java

class StringDemo

{

public static void main(String[] args)

{

~~String s1 = "abc";~~ → String s1 = new String("abc");

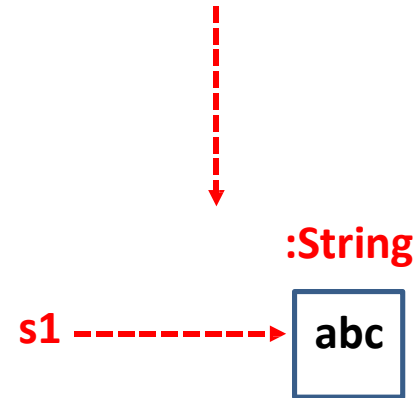
String s2 = "abc";

String s3 = "abc";

String s4 = "abc";

}// End of Method

}// End of class StringDemo



How Many String Objects are Created?



//File Name: StringDemo.java

class StringDemo

{

public static void main(String[] args)

{

String s1 = "abc"; String s1 = new String("abc");

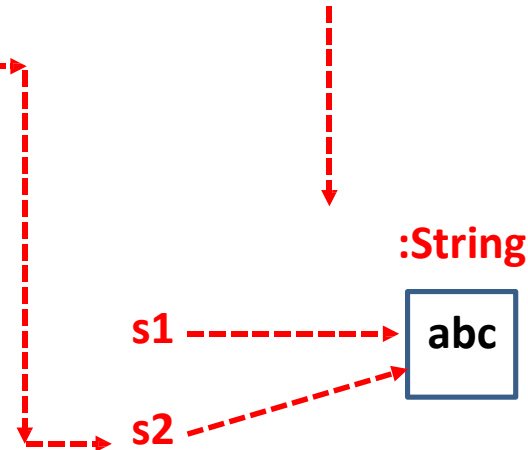
String s2 = "abc";

String s3 = "abc";

String s4 = "abc";

}// End of Method

}// End of class StringDemo



How Many String Objects are Created?



```
//File Name: StringDemo.java
class StringDemo
{
```

```
    public static void main(String[] args)
    {
```

```
        String s1 = "abc";          String s1 = new String("abc");
```

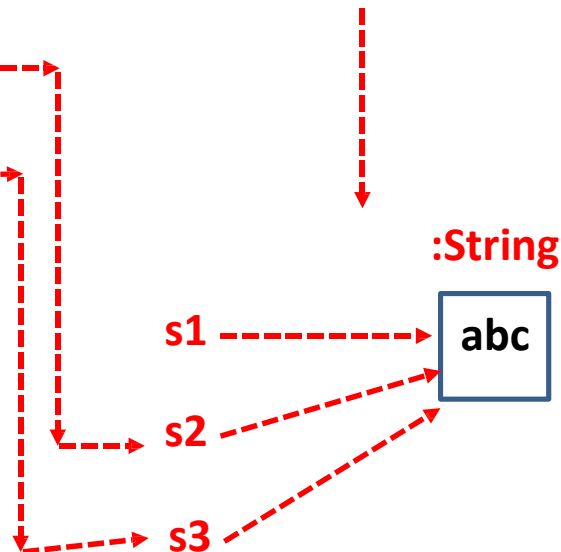
```
        String s2 = "abc";
```

```
        String s3 = "abc";
```

```
        String s4 = "abc";
```

```
    } // End of Method
```

```
} // End of class StringDemo
```



How Many String Objects are Created?



```
//File Name: StringDemo.java  
class StringDemo  
{
```

```
    public static void main(String[] args)  
    {
```

```
        String s1 = "abc";  
        String s1 = new String("abc");
```

```
        String s2 = "abc";
```

```
        String s3 = "abc";
```

```
        String s4 = "abc";
```

```
    } // End of Method
```

```
} // End of class StringDemo
```

:String

s1

abc

s2

s3

s4

**Only One Object is
Created in This Example**

Predict The Output?



```
//File Name: StringDemo.java
```

```
class StringDemo
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        String s1 = "a" + "b" + "c" + "d";
```

```
        String s2 = "ab" + "cd";
```

```
        String s3 = "abc" + "d";
```

```
        String s4 = "a" + "bc" + "d";
```

```
        if(s1 == s2)
```

```
        else
```

```
            System.out.println("Hello");
```

```
            System.out.println("Hi");
```

```
        if(s2 == s3)
```

```
        else
```

```
            System.out.println("Hello");
```

```
            System.out.println("Hi");
```

```
        if(s3 == s4)
```

```
        else
```

```
            System.out.println("Hello");
```

```
            System.out.println("Hi");
```

```
    } // End of Method
```

```
} // End of class StringDemo
```

<<OUTPUT>>

F:\>java StringDemo

Hello

Hello

Hello



- Important String Methods



Getting length of Strings



- `int length()` → Returns the length of this String reference
- Example
 1. `"abc".length()` returns 3
 2. `String s1 ="Object"; s1.length()` returns 6

String Concatenation Using '+' operator



- + operator can be used for string concatenation
- + operator can concatenate not only two strings but also a string type value with any other type
- Suppose 's' is String reference and 't' is any type other than string. Then an expression 's + t' will convert t to string type and final result will be of String type
- Examples
 1. "abc" + 10 → results in "abc10"
 2. 10 + "abc" → results in "10abc"
 3. "abc" + "def" → results in "abcdef"
 4. 4.56 + "Object" → results in "4.56Object"
 5. 10 + 20 + "Java" → results in 30Java
 6. "Java"+10+20 → Java1020

String Concatenation Using 'concat()' Method



- **String concat(String s)** → Appends 's' with this String and returns a new String with updated contents
- However, preferred way for string concatenation is via '+' operator
- Examples
 1. `String s1 = "Object";`
`String s2 = s1.concat("-Oriented Programming");`
 2. `System.out.println("Java ".concat(" World"));`

Character Extraction : charAt()



- **char charAt(int index)** → Returns the character at index. The value of index should be $0 \leq \text{index} \leq L-1$, where 'L' is length of string.
- Examples:
 1. "abc".charAt(0) → returns 'a'
 2. "abc".charAt(3) → results in StringIndexOutOfBoundsException
 3. String s1 = "Object-Oriented Programming"; s1.charAt(10) → returns 'e'

Character Extraction : getChars()



- `void getChars(int sourceStart, int sourceEnd, char target[], int targetStart)`
- Characters are extracted from invoking (this) string reference
- `sourceStart` → start index of invoking (this) string reference from where character extraction will start ($0 \leq \text{sourceStart} \leq L-1$, where L is length of invoking string)
- `sourceEnd` → end index (exclusive) of invoking (this) string reference. ($0 \leq \text{sourceEnd} \leq L-1$, where L is length of invoking string and also `sourceStart` ≤ `sourceEnd`)
- Characters are extracted from `sourceStart` to `sourceEnd-1`
- `target` represents the target character array where extracted characters will be stored
- `targetStart` → start index of `target` from where characters will be stored. Note: `target` array should be large enough to store the extracted characters.

Character Extraction : getChars() : Example 1



//File Name:

StringDemo.java class

StringDemo

{

public static void main(String[]

args) String s = "When Java converts
data";

{

char[] chars = new char[100];

s.getChars(10,30,chars

s,0);

System.out.println(chars);

}// End of Method

}// End of class

StringDemo

F:\>java StringDemo

Exception in thread "main"

java.lang.StringIndexOutOfBoundsException: String index out of
range: 30

at java.lang.String.getChars(Unknown

Source) at StringDemo.main(xyz.java:8)

Start index
=10 End
index = 29

Character Extraction : getChars() : Example 2



//File Name:

StringDemo.java class

StringDemo

{

public static void main(String[]

args)

String s

= "When Java converts

data";

{

char[] chars

= new char[100];

s.getChars(10, 5

,chars,0);

System.out.println(chars);

}// End of Method

}// End of class

StringDemo



endIndex

<

startIndex

x

F:\>java StringDemo

Exception in thread "main"

java.lang.StringIndexOutOfBoundsException: String index out of range: -5

at java.lang.String.getChars(Unknown

Source) at StringDemo.main(xyz.java:8)

Character Extraction : getChars() : Example 3



//File Name: StringDemo.java class

StringDemo

{

public static void main(String[] args)

{

String s = "When Java converts
data";

char[] chars = new

char[20]; s.getChars(4,
14 ,chars,14);

System.out.println(chars);

// End of class

StringDemo

// End of Method

F:\>java

StringDemo thread "main"

java.lang.ArrayIndexOutOfBoundsException at

java.lang.System.arraycopy(Native Method)

at java.lang.String.getChars(Unknown

Source) at StringDemo.main(xyz.java:8)

Characters will be extracted
from index 4 to 13 (10
characters)

When Java converts

store 10 chacters from
index 14 the last index
should be 23

However the last index of chars

is 19

Character Extraction : getChars() : Example4



//File Name: StringDemo.java class

StringDemo

{

public static void main(String[] args)

{

String s = "When Java converts
data";

char[] chars = new
char[20]; s.getChars(4,
14 ,chars,4);

System.out.println(chars);

// End of Method

// End of class

StringDemo

Characters will be extracted
from index 4 to 13 (10
characters)

<<OUTPUT>>

F:\>java



byte[] getBytes() : Method

- **byte[] getBytes()** → Converts each character of string to byte and returns a byte arrays

- Example

```
1. byte[] values = "Object".getBytes();  
   for(int i =0; i < values.length; i++)  
       System.out.print(values[i] + " ");
```

79 98 106 101 99 116

```
2. byte[] bytes = new byte[10];  
   String s1 = "Java";  
   bytes = s1.getBytes();  
   for(int i =0; i < bytes.length; i++)  
       System.out.print(bytes[i] + " ");
```

74 97 118 97

char[] toCharArray()



- **char[] toCharArray()** → converts a string into a char type array and returns char[]

//File Name:

StringDemo.java class

StringDemo

{

public static void main(String[] args)

{

**System.out.println("Thomas Alva
Edison".length()); char[] name = "Thomas Alva
Edison".toCharArray();**

System.out.println(name.length

// End of class

StringDemo

// End of Method

h); System.out.println(name);

StringDemo 18

18

Thomas Alva

Edison

String Comparisons: equals() and equalsIgnoreCase()



- **boolean equals(String other)** → case-sensitive comparison [upper and lower-case letters are not equal]
- Example : “abc”.equals(“Abc”) returns false, “abc”.equals(“abc”) returns true
- **boolean equalsIgnoreCase(String other)** → Comparison by ignoring the case of characters
- Example : “abc”.equalsIgnoreCase(“ABC”) or “aBc”.equalsIgnoreCase(“abC”) returns true

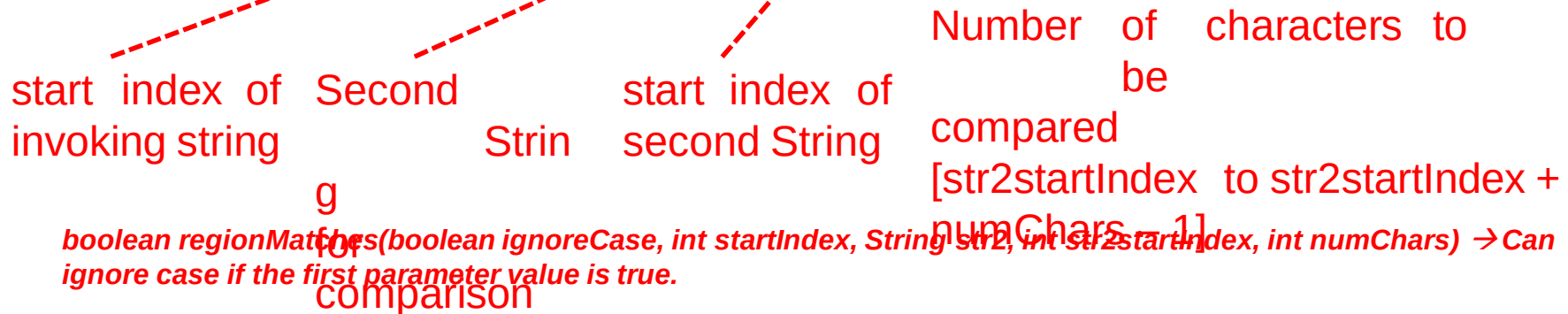
String Comparisons: regionMatches()



- **regionMatches()** → compares the regions of two string references for equality [returns true if regions matches otherwise false]

- Syntax:

1. **boolean regionMatches(int startIndex, String str2, int str2startIndex, int numChars)** → case-sensitive comparison



- **Note** : Index parameters should be within range **StringIndexOutOfBoundsException** will be thrown

String Comparisons:

regionMatches() : Example



```
String s1 = "India finally has started preparations for Rio  
Olympics";
```

```
String s2 = "David has started preparations for Final Exams";
```

```
String s3 = "David has Started PREPARATIONS FOR Final Exams";
```

System.out.println(s1.regionMatches(14,s2,6,24));	//	Case-Sensitive	true
System.out.println(s1.regionMatches(14,s3,6,24));	//	Case-Sensitive	false
System.out.println(s1.regionMatches(true,14,s3,6,24));	//	Ignore-case	false
System.out.println(s1.regionMatches(false,14,s3,6,24));	//	Case-Sensitive	true

String Comparisons: startsWith() and endsWith()



- **boolean startsWith(String str)** → Returns true if **this** string starts with **'str'** otherwise false. For Example, "Object".startsWith("Obj") returns true.
 - **boolean startsWith(String str, int startIndex)** → Overloaded Flavor. Returns true if **this** string starts with **'str'** from index **startIndex** otherwise false. For Example, "Object".startsWith("ect",3) returns true.
 - **boolean endsWith(String str)** → Returns true if **this** string ends with **'str'** otherwise false. For Example, "FooBar".endsWith("Bar"); returns true
-

String Comparisons: compareTo()



- **int compareTo(String str)** → Case-Sensitive Comparison. returns a +ive value if **this** string is > **str** , -ive value if **this** string is < **str**, otherwise a 0 (zero) value. Used For sorting an arrays of String values.
- Comparison is done character by character. If all characters of both strings matches then 0 is returned otherwise a difference of the first non-matched character is returned
- Examples
 1. `System.out.println("Ram".compareTo("RAM"));` **// Prints 32**
 2. `System.out.println("Java".compareTo("Object"));` **// Prints -5**
 3. `System.out.println("Hello".compareTo("Welcome"));` **// Prints -15**
- **int compareToIgnoreCase(String str)** → Ignore case comparison.
- Examples
 1. `System.out.println("Ram".compareToIgnoreCase("RAM"));` **// Prints 0**
 2. `System.out.println("oBjeCt".compareTo("OBJECT"));` **// Prints 0**

• Important String Methods

- Searching Strings [`indexOf()` and `lastIndexOf()`]
- Modifying a String [`substring()`, `replace()` and `trim()`]
- Data Conversion [`valueOf()`]
- Changing Case of Characters [`toUpperCase()` and `toLowerCase()`]

Searching Strings : indexOf() and lastIndexOf()



- **int indexOf(int ch)** → Searches the index of character 'ch' in **this** string and returns that index value (if found) otherwise returns -1 (if not found)
- **int lastIndexOf(int ch)** → Searches the last index of character 'ch' in **this** string and returns that index value (if found) otherwise returns -1 (if not found)
- **int indexOf(String str)** → Searches the index of string "str" in **this** string and returns that index value (if found) otherwise returns -1 (if not found)
- **int lastIndexOf(String str)** → Searches the last index of string "str" in **this** string and returns that index value (if found) otherwise returns -1 (if not found)
- **int indexOf(int ch, int startIndex)** → Searches the index of character 'ch' in **this** string starting from **startIndex** and returns that index value (if found) otherwise returns -1 (if not found)
- **int lastIndexOf(int ch, int startIndex)** → Searches the last index of character 'ch' in **this** string starting from **startIndex** and returns that index value (if found) otherwise returns -1 (if not found)
- **int indexOf(String str, int startIndex)** → Searches the index of string "str" in **this** string starting from **startIndex** and returns that index value (if found) otherwise returns -1 (if not found)
- **int lastIndexOf(String str, int startIndex)** → Searches the last index of string "str" in **this** string starting from **startIndex** and returns that index value (if found) otherwise returns -1 (if not found)

Searching Strings : indexOf() and lastIndexOf() : Example



```
String s = "Now is the Time For All Indian";  
s = s + "to come to the aid of India";
```

```
System.out.println(s.indexOf("x"));
```



-1

```
System.out.println(s.indexOf("a"));
```



28

```
System.out.println(s.lastIndexOf("a"));
```



56

```
System.out.println(s.indexOf("Ind"));
```



24

```
System.out.println(s.lastIndexOf("Ind"));
```



52

```
System.out.println(s.indexOf("Ind",30));
```



52

Modifying Strings : substring()



- `String substring(int startIndex)` → Returns a substring starting from `startIndex` (inclusive) to the last of `this` string [Note: $0 \leq \text{startIndex} \leq L$, where L is length of `this` String]
- `String substring(int startIndex, int endIndex)` → Returns a substring starting from `startIndex` (inclusive) to the `endIndex` (exclusive) of `this` string. Returned substring will have characters from `startIndex` to `endIndex-1` [Note: $0 \leq \text{startIndex}, \text{endIndex} \leq L$, where L is length of `this` String and $\text{startIndex} \leq \text{endIndex}$]
 - If values of `startIndex` and `endIndex` are equal, then no character will be extracted
 - If value of `startIndex < endIndex` OR `startIndex, endIndex > L`, then `StringIndexOutOfBoundsException` will be thrown

Modifying Strings : substring() Example



String s = "Now is the Time For All Indian";

System.out.println(s.length());

→ 30

System.out.println(s.substring(30,30));

→ <<No Character
Will be
extracted

System.out.println(s.substring(20,20));

→ <<No Character
Will be
extracted

System.out.println(s.substring(0,2));

→ No

System.out.println(s.substring(11));

→ Time For All Indian

Replacing Characters in String



- **String replace(char original, char replacement)** → Replaces all occurrences of a character **original** in **this** String with a character **replacement** and returns the modified String
- Example : `System.out.println("Object-Oriented Programming".replace('m','x'))` will result in "Object-Oriented Prograxxing" as output
- **String trim()** → Removes the leading and trailing white spaces from the invoking string and returns the modified String
- Example

`System.out.println(" My Name is David ".length());` → 20
`System.out.println(" My Name is David ".trim());` → My Name is David
`System.out.println(" My Name is David ".trim().length());` → 16

Changing the Case of Characters



- `String toUpperCase()` → Changes each character of the invoking string to upper case and returns the modified string
- `String toLowerCase()` → Changes each character of the invoking string to lower case and returns the modified string
- For Example, the statement `System.out.println("My Name is David".toUpperCase());` displays "MY NAME IS DAVID" over console
- For Example, the statement `System.out.println("My Name is David".toLowerCase());` displays "my name is david" over console

STRING CLASS



2. Using the new Keyword

We can create a String object like any other object as

```
String S = new String("Welcome");
```

Java String Methods

Here are the list of the methods available in the Java String class.

1. **char charAt(int index)**: It returns the character at the specified index.

Specified index value should be between 0 to length() -1 both inclusive.

It throws IndexOutOfBoundsException if $\text{index} < 0$ || $\text{index} \geq \text{length}$ of String.

STRING CLASS



- 2. `boolean equals(Object obj)`**: Compares the string with the specified string and returns true if both matches else false.
- 3. `boolean equalsIgnoreCase(String string)`**: It works same as equals method but it doesn't consider the case while comparing strings. It does a case insensitive comparison.
- 4. `int compareTo(String string)`**: This method compares the two strings based on the Unicode value of each character in the strings.
- 5. `int compareToIgnoreCase(String string)`**: Same as CompareTo method however it ignores the case during comparison.

STRING CLASS



- 6. boolean startsWith(String prefix, int offset)**: It checks whether the substring (starting from the specified offset index) is having the specified prefix or not.
- 7. boolean startsWith(String prefix)**: It tests whether the string is having specified prefix, if yes then it returns true else false.
- 8. boolean endsWith(String suffix)**: Checks whether the string ends with the specified suffix.
- 9. int indexOf(int ch)**: Returns the index of first occurrence of the specified character ch in the string.
- 10. int indexOf(int ch, int fromIndex)**: Same as indexOf method however it starts searching in the string from the specified fromIndex.

STRING CLASS



11. **int lastIndexOf(int ch)**: It returns the last occurrence of the character ch in the string.
12. **String substring(int beginIndex)**: It returns the substring of the string. The substring starts with the character at the specified index.
13. **String substring(int beginIndex, int endIndex)**: Returns the substring.
The substring starts with character at beginIndex and ends with the character at endIndex.
14. **String concat(String str)**: Concatenates the specified string “str” at the end of the string.
15. **String replace(char oldChar, char newChar)**: It returns the new updated string after changing all the occurrences of oldChar with the newChar.

STRING CLASS



16. String toLowerCase(): Equivalent to toLowerCase(Locale.getDefault()).

17. String trim(): Returns the substring after omitting leading and trailing white spaces from the original string.

18. char[] toCharArray(): Converts the string to a character array.

19. void getChars(int srcBegin, int srcEnd, char[] dest, int destBegin):

It copies the characters of **src** array to the **dest** array.

Only the specified range is being copied(srcBegin to srcEnd) to the dest subarray(starting fromdestBegin).

STRING BUFFER CLASS



Strings in [Java](#) are a sequence of characters which is immutable.

Java provides the StringBuffer class to create a mutable String.

Creating StringBuffer Using Constructors

(a) Empty StringBuffer

An empty string buffer with an initial capacity of 16 characters is created.

Syntax:

```
StringBuffer S=new StringBuffer();
```

STRING BUFFER CLASS



(b) **StringBuffer** with arguments

The string buffer created is of the size specified in the argument.

```
StringBuffer S=new StringBuffer(20);// int argument
```

(c) **Str StringBuffer**

The argument specified sets the initial contents of the StringBuffer object and reserves space for 16 more characters without reallocation.

```
StringBuffer S=new StringBuffer("Java");//String argument
```

STRING BUFFER CLASS



Methods in StringBuffer

The methods used in StringBuffer are specified as follows:

1. **length()**

It specifies the number of elements present.

2. **capacity():**

The capacity of the StringBuffer can be found using this method.

3. **append():**

The method is used to add elements at the end of the StringBuffer.

4. **insert():**

The method is used to insert an element at the specified index position.

STRING BUFFER CLASS



5. **delete(int startIndex, int endIndex)**

The method is used to delete the elements present in the StringBuffer.

The first character to be removed is specified by the first index.

The elements between the startIndex and endIndex-1 are deleted.

6. **deleteCharAt(int index)**

The method removes a single character within the string present in the StringBuffer.

The int index specifies the location of the character.

STRING BUFFER CLASS



7. replace()

The method is used to replace a set of elements or characters with another, inside the StringBuffer.

The arguments `startIndex` and `endIndex` are present in this method.

The substring from the `startIndex` till the `endIndex - 1` is replaced.

8.String toString()

This inbuilt method outputs a string representing the data present in the StringBuffer.

A new String object is declared and initialized to get the character sequence from the StringBuffer object.

The string is then returned by `toString()`.

Packages



- Java API packages
- User defined package
- Accessing classes in packages
- Example programs

Packages



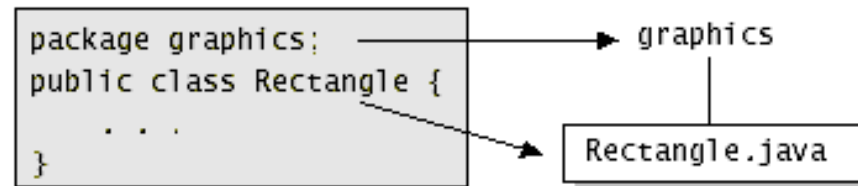
- A java package is a group of similar types of classes, interfaces and sub-packages.
- Think of it as a folder in a file directory.
- Packages are used to avoid name conflicts, and to write a better maintainable code.
- package $\leftrightarrow$ directory (folder)
- class $\leftrightarrow$ file

Packages and directories

A class named D in package a.b.c should reside in this file:

a/b/c/D.class

(relative to the root of your project)



The "root" directory of the package hierarchy is determined by your *class path* or the directory from which java was run.

Packages



- Provides a mechanism for grouping a variety of classes and / or interfaces together.
- Grouping is based on functionality.

Benefits:

- Java package is used to categorize the classes and interfaces so that they can be easily maintained.
- Java package provides access protection.
- Java package removes naming collision.

Packages



- Package in java can be categorized in two form, built-in package and user-defined package.

Two types of packages:

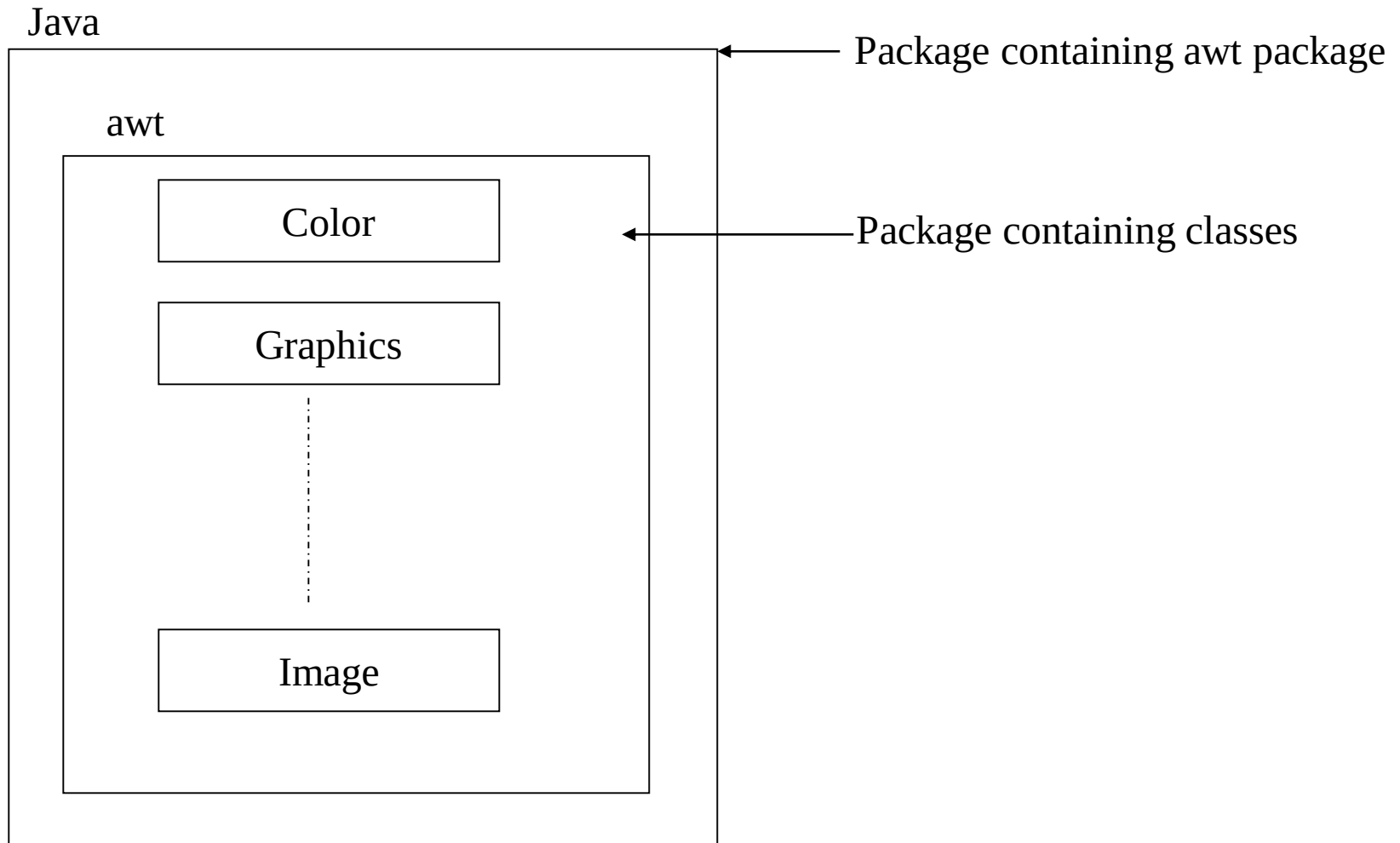
1. Built-in packages
2. User defined packages

Java API Packages:

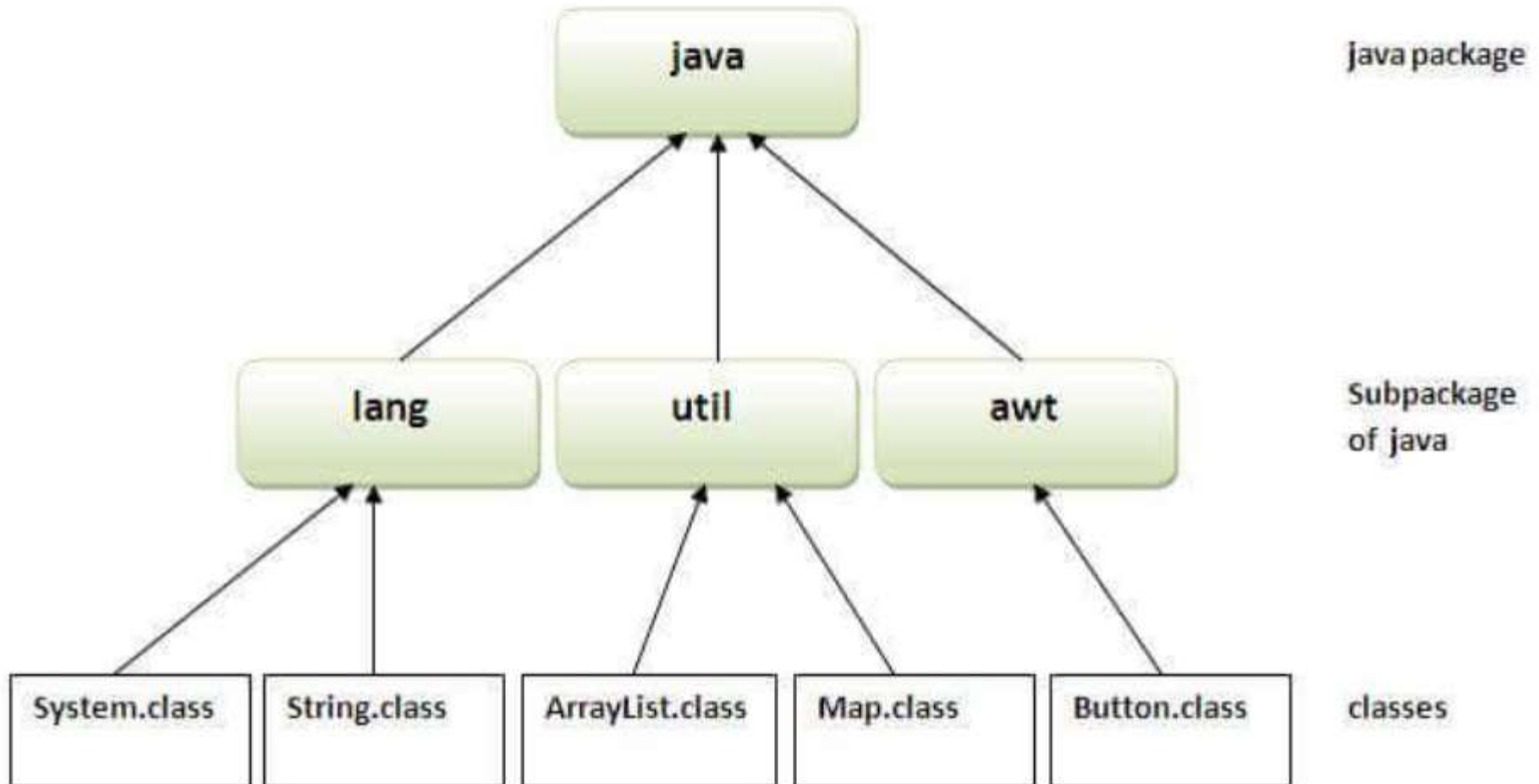
A large number of classes grouped into different packages based on functionality. Examples:

1. java.lang
2. java.util
3. java.io
4. java.awt
5. java.net
6. java. applet etc.

Package



Packages





Creating Your Own Package

1. Declare the package at the beginning of a file using the form
package *packagename*;
2. Define the class that is to be put in the package and declare it **public**(optional)
3. Create a subdirectory under the directory where the main source files are stored
(or **javac -d . File.java**)
3. Store the listing as *classname.java* in the subdirectory created.
4. Compile the file. From subdirectory.

Example:

```
package mypack;
```

```
public class Simple  
{
```

```
//Body of the class
```

```
}
```

To Compile: `javac -d . Simple.java`

To Run: `java mypack.Simple`



Accessing Classes in a Package

There are three ways to access the package from outside the package.

1. `import package.*;`
2. `import package.classname;`
3. fully qualified name



Adding a Class to a Package

- Every java source file can contain only class declared as **public**.
- The name of the source file should be same as the name of the public class with **.java** extension.

```
package p1;
```

```
public ClassA{  
.....}
```

Source file :

ClassA.java

Subdirectory: p1

```
package p1;
```

```
public  
ClassB{.....}
```

Source file: ClassB.java

Subdirectory:p1

Example1-Package



```
package p1;
public class ClassA
{
    public void displayA( )
    {
        System.out.println("Class A");
    }
}
```

Source file – ClassA.java

Subdirectory-p1

ClassA.java and ClassA.class->p1

```
import p1.ClassA;

class testclass
{
    public static void main(String s[])
    {
        ClassA obA=new ClassA();
        obA.displayA();
    }
}
```

Source file-testclass.java

testclass.java and testclass.class->in
a directory of which *p1* is
subdirectory.

Creating Packages



- Consider the following declaration:
`package firstPackage.secondPackage;`
This package is stored in subdirectory named **firstPackage.secondPackage**.
- A java package can't contain more than one class definitions that can be declared as public.
- Only one of the classes which is declared **public** and that class name with **.java** extension is the source file name.



Example2-Package

```
package p2;
public class ClassB
{
    protected int m =10;
    public void displayB()
    {
        System.out.println("Class B");
        System.out.println("m= "+m);
    }
}
```

```
import p1.ClassA;
import p2.ClassB;

class PackageTest2
{
    public static void main(String str[])
    {
        ClassA obA=new ClassA();
        Classb obB=new ClassB();
        obA.displayA();
        obB.displayB();
    }
}
```



Example 3- Package

```
import p2.ClassB;
class ClassC extends ClassB
{
    int n=20;
    void displayC()
    {
        System.out.println("Class C");
        System.out.println("m= "+m);
        System.out.println("n= "+n);
    }
}
```

```
class PackageTest3
{
    public static void main(String args[])
    {
        ClassC obC = new ClassC();
        obC.displayB();
        obC.displayC();
    }
}
```


Package



```
package p1;  
public class Teacher  
{.....}  
public class Student  
{.....}
```

In Different files

```
package p2;  
public class Courses  
{.....}  
public class Student  
{.....}..Different files  
import p1.*;  
import p2.*;  
Student student1; //Error
```

Correct Code:

```
import p1.*;  
import p2.*;
```

```
p1.Student student1;  
p2.Student student2;
```



Default Package

- If a source file does not begin with the *package* statement, the classes contained in the source file reside in the *default package*
- The java compiler automatically looks in the default package to find classes.



Finding Packages

Two ways:

1. By default, java runtime system uses current directory as starting point and search all the subdirectories for the package.
2. Specify a directory path using CLASSPATH environmental variable.



CLASSPATH Environment Variable

- The compiler and runtime interpreter know how to find standard packages such as *java.lang* and *java.util*
- The CLASSPATH environment variable is used to direct the compiler and interpreter to where programmer defined imported packages can be found
- The CLASSPATH environment variable is an ordered list of directories and files



CLASSPATH Environment Variable

- To set the CLASSPATH variable we use the following command:
`set CLASSPATH=c:\`
- Java compiler and interpreter searches the user defined packages from the above directory.
- To clear the previous setting we use:
`set CLASSPATH=`



Example1-Package[Using CLASSPATH]

```
package p1;  
public class ClassA  
{  
    public void displayA( )  
    {  
        System.out.println("Class A");  
    }  
}
```

Source file – c:\p1\ClassA.java
Compile-javac c:\p1\ClassA.java
Class file in –c:\p1\ClassA.class

```
import p1.ClassA;  
Class PackageTest1  
{  
    public static void main(String str[])  
    {  
        ClassA obA=new ClassA();  
        obA.displayA();  
    }  
}
```

Source file-
c:\java\jdk1.6.0_06\bin\PackageTest1.java
Compile-javac PackageTest1.java
Copy –PackageTest1.class -> c:\
Execute-java PackageTest1



Example2-Package[Using CLASSPATH]

```
package p2;
public class ClassB
{
    protected int m =10;
    public void displayB()
    {
        System.out.println("Class B");
        System.out.println("m= "+m);
    }
}
```

Source file – c:\p2\ClassB.java

Compile-c:\p2\ClassB.java

Class file in –c:\p2\ClassB.class

```
import p1.*;
import p2.*;
class PackageTest2
{
    public static void main(String str[])
    {
        ClassA obA=new ClassA();
        Classb obB=new ClassB();
        obA.displayA();
        obB.displayB();
    } }
```

Source file-

c:\java\jdk1.6.0_06\bin\PackageTest2.java

Compile-javac PackageTest2.java

Copy –PackageTest2.class -> c:\

Execute-java PackageTest2



Example 3- Package[Using CLASSPATH]

```
import p2.ClassB;
class ClassC extends ClassB
{
    int n=20;
    void displayC()
    {
        System.out.println("Class C");
        System.out.println("m= "+m);
        System.out.println("n= "+n);
    }
}
```

Source file – c:\ClassC.java

Compile-c:\ClassC.java

Class file in –c:\ClassC.class

```
class PackageTest3
{
    public static void main(String args[])
    {
        ClassC obC = new ClassC();
        obC.displayB();
        obC.displayC();
    }
}
```

Source file-

c:\java\jdk1.6.0_06\bin\PackageTest3.java

Compile-javac PackageTest3.java

Copy –PackageTest3.class -> c:\

Execute-java PackageTest3

Adding a Class to a Package



- Every java source file can contain only class declared as **public**.
- The name of the source file should be same as the name of the public class with **.java** extension.

package p1;

**public ClassA{
.....}**

**Source file :
ClassA.java**

Subdirectory: p1

package p1;

**public
ClassB{.....}**

Source file: ClassB.java

Subdirectory:p1

Adding a Class to a Package



1. Decide the name of the package.
2. Create the subdirectory with this name under the directory where the main source file is located.
3. Create classes to be placed in the package in separate source files and declare the package statement
package packagename;
4. Compile each source file. When completed the package will contain .class files of the source files.



public/package/private scope

- Scope is concerned with the visibility of program elements such as classes and members
- Class members (methods or instance fields) can be defined with public, package (default), private or protected scope
- A class has two levels of visibility:
 - public** scope means it is visible outside its containing package
 - default scope means it is visible only inside the package. (package scope/ friendly scope)



public/package/private scope

- A class member with **public** scope means it is visible anywhere its class is visible
- A class member with **private** scope means it is visible only within its encapsulating class
- A class/class member with **package** scope means it is visible only inside its containing package
- A class member with **protected** scope means it is visible every where except the non-subclasses in other package.

Levels of Access Control



	public	protected	friendly (default)	private
same class	Yes	Yes	Yes	Yes
Subclass in the same package	Yes	Yes	Yes	No
Other class in the same package	Yes	Yes	Yes	No
Subclass in other packages	Yes	Yes	No	No
Non-subclass in other package	Yes	No	No	No

Method Overriding

Overriding method



- The benefit of overriding is ability to define a behavior that's specific to the subclass type, which means a subclass can implement a parent class method based on its requirement.
- In object-oriented terms, overriding means to override the functionality of an existing method.

Rules for Method Overriding



- The argument list should be exactly the same as that of the overridden method.
- The return type should be the same or a subtype of the return type declared in the original overridden method in the superclass.
- The access level cannot be more restrictive than the overridden method's access level. For example: If the superclass method is declared public then the overriding method in the sub class cannot be either private or protected.
- Instance methods can be overridden only if they are inherited by the subclass.
- A method declared final cannot be overridden.
- A method declared static cannot be overridden but can be re-declared.

Rules for Method Overriding



- If a method cannot be inherited, then it cannot be overridden.
- A subclass within the same package as the instance's superclass can override any superclass method that is not declared private or final.
- A subclass in a different package can only override the non-final methods declared public or protected.
- An overriding method can throw any unchecked exceptions, regardless of whether the overridden method throws exceptions or not. However, the overriding method should not throw checked exceptions that are new or broader than the ones declared by the overridden method. The overriding method can throw narrower or fewer exceptions than the overridden method.
- Constructors cannot be overridden.



Thank You



BITS Pilani
Pilani Campus

Object Oriented Programming and Design

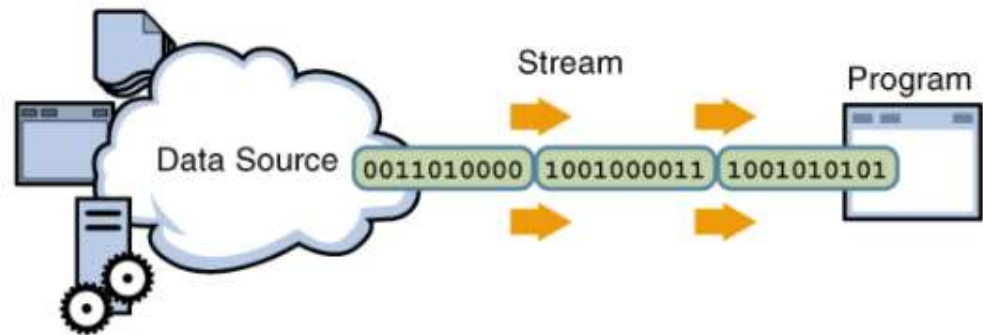
Presented by
Dr Ramya Devi M
mramyadevi@wilp.bits-pilani.ac.in
CS - 08 : 23rd Aug 2025

Streams



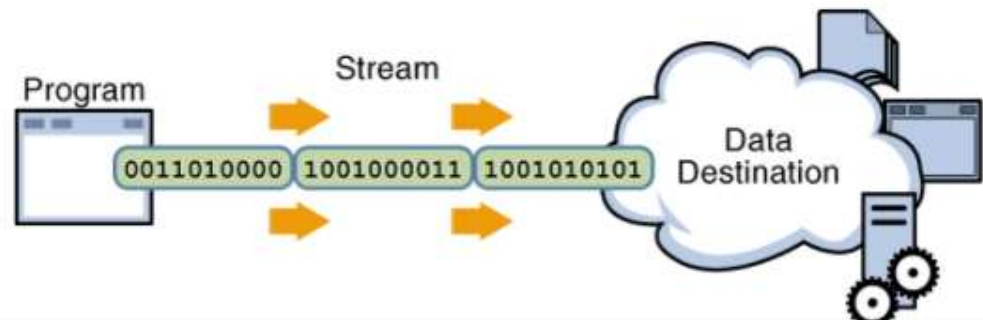
Reading information into a program.

A program uses an input stream to read data from a source, one item at a time:



Writing information from a program.

A program uses an output stream to write data to a destination, one item at a time:



Files and Streams



Java views each files as a sequential stream of bytes

Operating system provides mechanism to determine end of file

- End-of-file marker
- Count of total bytes in file

Java program processing a stream of bytes receives an indication from the operating system when program reaches end of stream

The **java.io** package defines I/O in terms of *streams*.

Streams are ordered sequences of data that have a *source* (input streams) or *destination* (output streams).

The I/O classes isolate programmers from the specific details of the underlying operating system, while enabling access to system resources through files and other means.

Streams



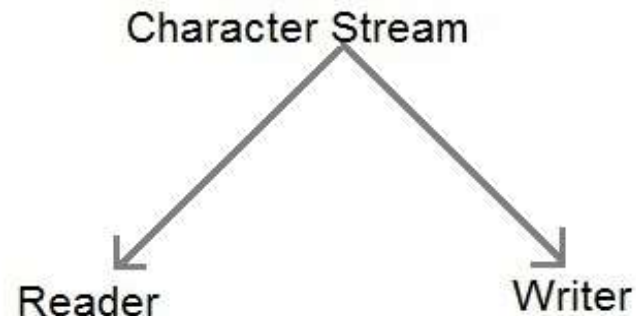
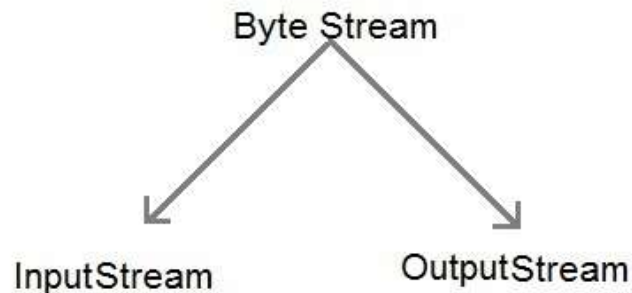
The package `java.io` has two major parts: *character streams* and *byte streams*. Characters are 16-bit **UTF**-16 characters, whereas bytes are (as always) 8 bits.

I/O is either text-based or data-based (binary).

Text-based I/O works with streams of human-readable characters, such as the source code for a program.

Data-based I/O works with streams of binary data, such as the bit pattern for an image.

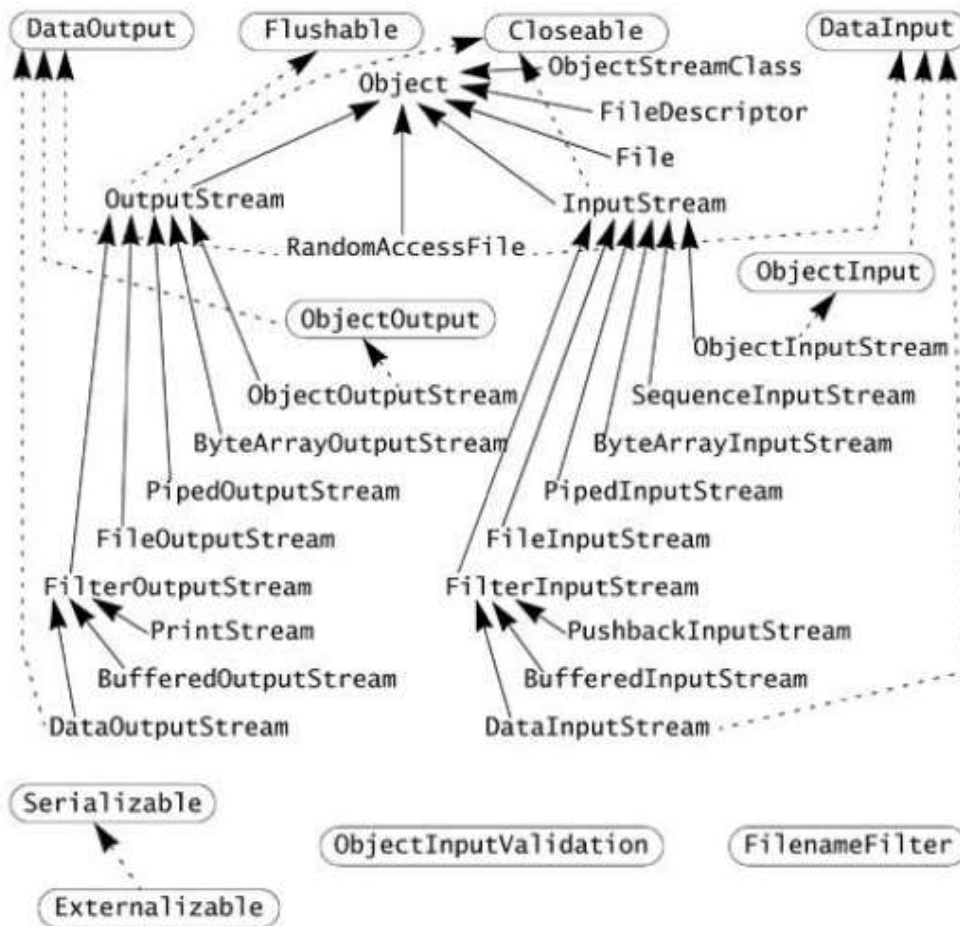
The character streams are used for text-based I/O, while byte streams are used for data-based I/O.



Byte Streams



Figure 20-1. Type Tree for Byte Streams in `java.io`



The `java.io` package defines abstract classes (`InputStream` and `OutputStream` abstract classes) for basic byte input and output streams.

These abstract classes are then extended to provide several useful stream types. Stream types are almost always paired: For example, where there is a `FileInputStream` to read from a file, there is usually a `FileOutputStream` to write to a file.

Byte Streams



InputStream

The abstract class `InputStream` declares methods to read bytes from a particular source.

`InputStream` is the superclass of most byte input streams in `java.io`, and has the following methods:

`public abstract int read() throws IOException`

- Reads a single byte of data and returns the byte that was read, as an integer in the range 0 to 255

`public int read(byte[] buf, int offset, int count) throws IOException`

- Reads into a part of a **byte** array.

The maximum number of bytes read is **count**.

The bytes are stored from **buf[offset]** up to a maximum of **buf[offset+count-1]** all other values in **buf** are left unchanged.

The number of bytes actually read is returned.

If no bytes are read because the end of the stream was found, the value -1 is returned.



STREAM CLASSES - InputStream

The following program demonstrates the use of input streams to count the total number of bytes in a file, or from System.in if no file is specified:

```
import java.io.*;

class CountBytes {
    public static void main(String[] args) throws IOException
    {
        InputStream in;
        if (args.length == 0)
            in = System.in;
        else
            in = new FileInputStream(args[0]);
        int total = 0;
        while (in.read() != -1)
            total++;
        System.out.println(total + " bytes");
    }
}
```

STREAM CLASSES - Example program for FileInputStream and FileOutputStream classes



Following is an example which makes use of these two classes to copy an input file into an output file:

```
import java.io.*;
class StreamDemo1
{
    public static void main(String aa[ ] )
    {
        FileInputStream fis=new FileInputStream("A.txt");
        FileOutputStream fos=new FileOutputStream("B.txt");
        int c;
        While( (c=fis.read()) !=-1) fos.write(c);
        fis.close();
        fos.close();
    }
}
```

STREAM CLASSES



Assume that A.txt file has some content

Java is a portable language.

As a next step, compile the above program and execute it, which will result in creating B.txt file with the same content as we have in A.txt. So let's put the above code in StreamDemo1.java file and do the following

```
>javac StreamDemo1.java [Compilation to Byte Code]
```

```
>java StreamDemo1
```



STREAM CLASSES - OutputStream

The abstract class **OutputStream** is analogous to **InputStream**; it provides an abstraction for writing bytes to a destination.

Its methods are:

public abstract void write(int b) throws IOException

Writes **b** as a byte.

The byte is passed as an **int** because it is often the result of an arithmetic operation on a byte.

public void write(byte[] buf, int offset, int count) throws IOException

Writes part of an array of bytes, starting at `buf[offset]` and writing `count` bytes.

This method blocks until the bytes have been written.

STREAM CLASSES - OutputStream



Here is a program that copies its input to its output, translating one particular byte value to a different one along the way. The **TranslateByte** program takes two parameters: a **from** byte and a **to** byte. Bytes that match the value in the string **from** are translated into the value in the string **to**.

```
import java.io.*;
class TranslateByte {
public static void main(String[] args) throws IOException
{
byte from = (byte) args[0].charAt(0);
byte to = (byte) args[1].charAt(0);
int b;
while ((b = System.in.read()) != -1)
System.out.write(b == from ? to : b);
}
}
```

For example, if we invoked the program as

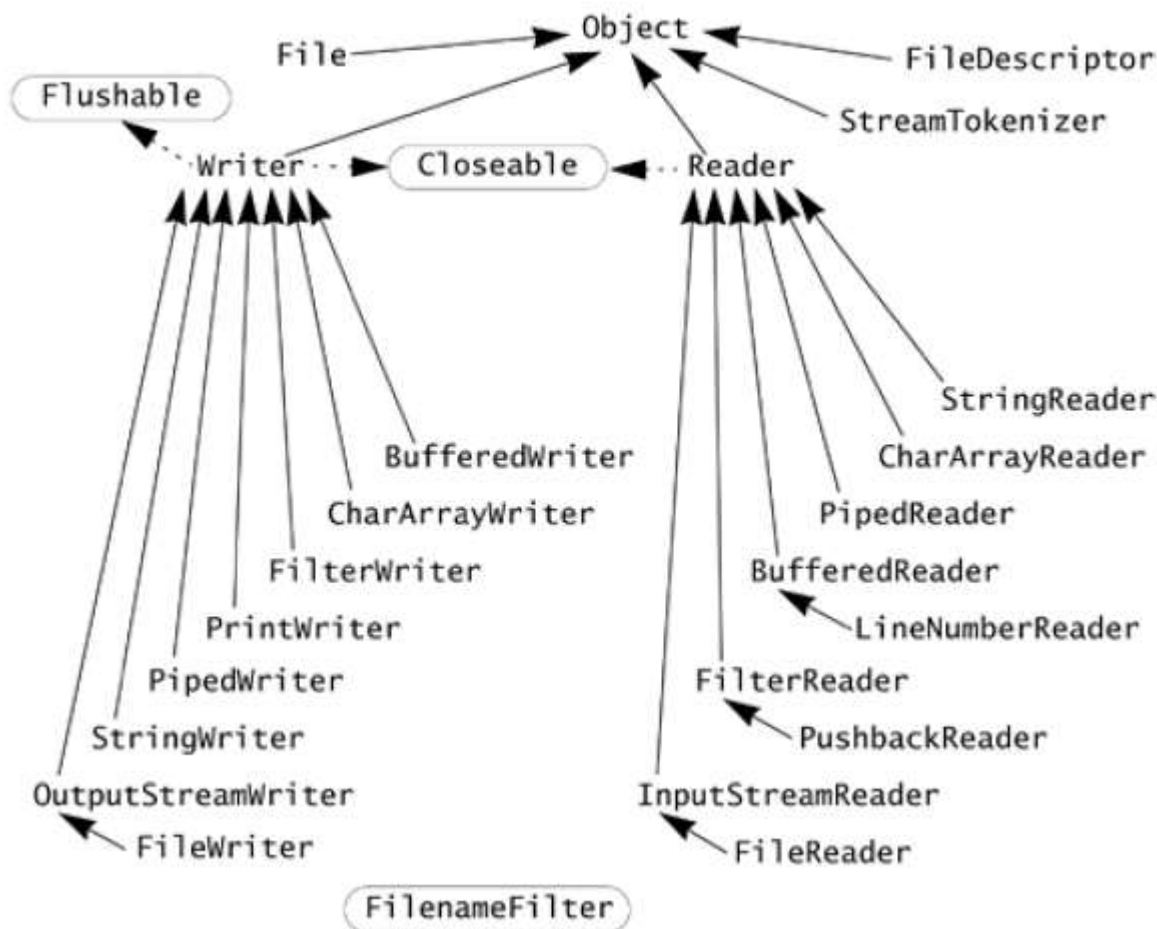
```
java TranslateByte b B
```

and entered the text **abracadabra!**, we would get the output **abracadabra!**

Character Streams



Figure 20-2. Type Tree for Character Streams in `java.io`



The abstract classes for reading and writing streams of characters are `Reader` and `Writer`.

Character Streams



Reader

The abstract class Reader provides a character stream analogous to the byte stream InputStream and the methods of Reader essentially mirror those of InputStream:

`public int read()` throws IOException

Reads a single character and returns it as an integer in the range 0 to 65535.

If no character is available because the end of the stream has been reached, the value -1 is returned.

`public abstract int read(char[] buf, int offset, int count)` throws IOException

Reads into a part of a char array.

The maximum number of characters to read is count.

The read characters are stored from buf[offset] up to a maximum of buf[offset+count-1] all other values in buf are left unchanged.

The number of characters actually read is returned.

Character Streams



```
import java.io.*;
class CountSpace {
public static void main(String[] args) throws IOException {
    Reader in;
    if (args.length == 0)
        in = new InputStreamReader(System.in);
    else
        in = new FileReader(args[0]);
    int ch;
    int total;
    int spaces = 0;
    for (total = 0; (ch = in.read()) != -1; total++) {
        if (Character.isWhitespace((char) ch))
            spaces++;
    }
    System.out.println(total + " chars, " + spaces + " spaces");
}
```

As an example, the program counts the number of whitespace characters in a character stream:

Character Streams



The abstract class `Writer` provides a stream analogous to `OutputStream` but designed for use with characters instead of bytes.

`public void write(int ch) throws IOException`

Writes `ch` as a character.

The character is passed as an `int` but only the lowest 16 bits of the integer are written.

`public abstract void write(char[] buf, int offset, int count) throws IOException`

Writes part of an array of characters, starting at `buf[offset]` and writing `count` characters.

Buffering



Not buffered: each byte is read/written from/to disk as soon as possible

- “little” delay for each byte

- A disk operation per byte---higher overhead

Buffered: reading/writing in “chunks”

- Some delay for some bytes

 - Assume 16-byte buffers

 - Reading: access the first 4 bytes, need to wait for all 16 bytes are read from disk to memory

 - Writing: save the first 4 bytes, need to wait for all 16 bytes before writing from memory to disk

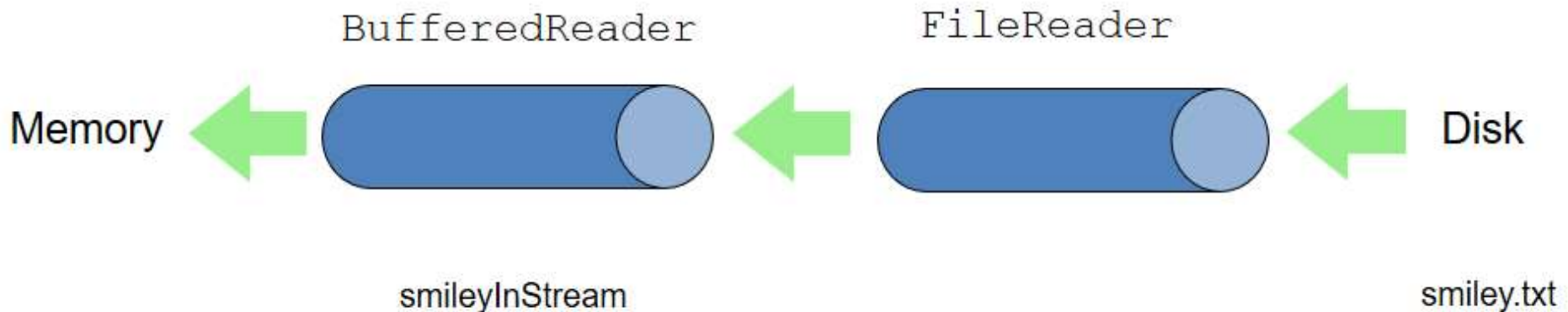
- A disk operation per a buffer of bytes---lower overhead

The Buffered stream classes `BufferedInputStream`,

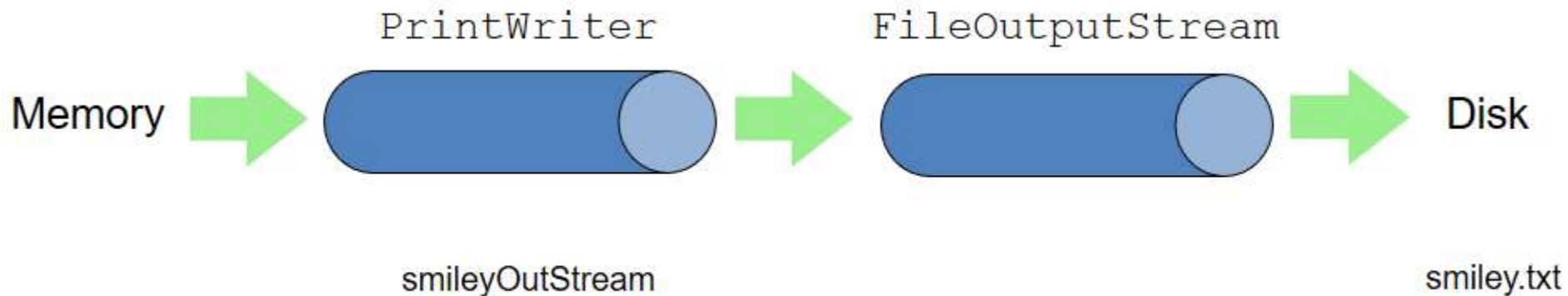
`BufferedOutputStream`, `BufferedReader`, and `BufferedWriter` buffer their data to avoid every read or write going directly to the next stream.

These classes are often used in conjunction with File stream accessing a disk file is much slower than using a memory buffer, and buffering helps reduce file accesses.

Input/Output File Streams



```
BufferedReader smileyInStream = new BufferedReader( new FileReader("smiley.txt") );
```



```
PrintWriter smileyOutputStream = new PrintWriter( new FileOutputStream("smiley.txt") );
```

STREAM CLASSES - Example program for FileReader and FileWriter Stream classes



Following is an example which makes use of these two classes to copy an input file into an output file:

```
import java.io.*;

class StreamDemo2
{
    public static void main(String aa[ ] )
    {
        FileReader fr=new FileReader("A.txt");
        FileWriter fw=new FileWriter("B.txt");
        int c;
        While( (c=fis.read()) !=-1) fos,write((char)c);
        fr.close();
        fw.close();
    }
}
```

STREAM CLASSES



Assume that A.txt file has the content

Java is an Object Oriented language.

As a next step, compile the above program and execute it, which will result in creating B.txt file with the same content as we have in A.txt. So let's put the above code in StreamDemo1.java file and do the following

```
>javac StreamDemo2.java [ Compilation to Byte  
Code ]
```

```
>java StreamDemo2
```

The Data Byte Streams



Reading and writing text characters is useful, but you also frequently need to transmit the binary data of specific types across a stream.

The `DataInput` and `DataOutput` interfaces define methods that transmit primitive types across a stream. The classes `DataInputStream` and `DataOutputStream` provide a default implementation for each interface.

Read	Write	Type
<code>readBoolean</code>	<code>writeBoolean</code>	<code>boolean</code>
<code>readChar</code>	<code>writeChar</code>	<code>char</code>
<code>readByte</code>	<code>writeByte</code>	<code>byte</code>
<code>readShort</code>	<code>writeShort</code>	<code>short</code>
<code>readInt</code>	<code>writeInt</code>	<code>int</code>
<code>readLong</code>	<code>writeLong</code>	<code>long</code>
<code>readFloat</code>	<code>writeFloat</code>	<code>float</code>
<code>readDouble</code>	<code>writeDouble</code>	<code>double</code>
<code>readUTF</code>	<code>writeUTF</code>	<code>String</code> (in UTF format)

The Data Byte Streams



```
public static void writeData(double[] data, String file)
    throws IOException
{
    OutputStream fout = new FileOutputStream(file);
    DataOutputStream out = new DataOutputStream(fout);
    out.writeInt(data.length);
    for (double d : data)
        out.writeDouble(d);
    out.close();
}

public static double[] readData(String file)
    throws IOException
{
    InputStream fin = new FileInputStream(file);
    DataInputStream in = new DataInputStream(fin);
    double[] data = new double[in.readInt()];
    for (int i = 0; i < data.length; i++)
        data[i] = in.readDouble();
    in.close();
    return data;
}
```

STREAM CLASSES - DATA INPUT STREAM



DataInputStream class is descended from the **InputStream** class so it has all behaviors of a byte input stream.

In addition, it implements the **DataInput** interface which defines the following read operations for primitive types and Strings:

readBoolean(): Reads one input byte and returns true if that byte is nonzero, false if that byte is zero.

readByte(): Reads and returns one input byte.

readChar(): Reads 2 input bytes and returns a char value.

readDouble(): Reads 8 input bytes and returns a double value.

DATA INPUT STREAM



readFloat(): Reads 4 input bytes and returns a float value.

readInt(): Reads 4 input bytes and returns an int value.

readLong(): Reads 8 input bytes and returns a long value.

readShort(): Reads 2 input bytes and returns a short value.



DATA OUTPUT STREAM

DataOutputStream class is descended from the **OutputStream** class so it inherits all behaviors of a byte output stream. In addition, it implements the **DataOutput** interface which defines the following write operations for primitive types and Strings:

writeBoolean(boolean v): Writes a boolean to the underlying output stream as a 1-byte value.

writeByte(int v): Writes out a byte to the underlying output stream as a 1-byte value.

writeInt(int v): Writes an int to the underlying output stream as four bytes, high byte first.

writeLong(long v): Writes a long to the underlying output stream as eight bytes, high byte first.

writeShort(int v): Writes a short to the underlying outputstream as two bytes, high byte first.



DATA OUTPUT STREAM

writeBytes(String s): Writes out the string to the underlying output stream as a sequence of bytes.

writeChar(int v): Writes a char to the underlying output stream as a 2-byte value, high byte first.

writeChars(String s): Writes a string to the underlying output stream as a sequence of characters.

writeDouble(double v): Converts the double argument to a long using the `doubleToLongBits` method in class `Double`, and then writes that long value to the underlying output stream as an 8-byte quantity, high byte first.

writeFloat(float v): Converts the float argument to an int using the `floatToIntBits` method in class `Float`, and then writes that int value to the underlying output stream as a 4-byte quantity, high byte first.



STREAM CLASSES - Example

Example program for DataInputStream and DataOutputStream

```
import java.io.*;

class DataOutputStreamDemo
{
    public static void main(String args[]) throws IOException
    {
        DataOutputStream dout = new DataOutputStream(new
                                                    FileOutputStream("file.dat"));

        dout.writeDouble(1.1);
        dout.writeInt(55);
        dout.writeBoolean(true);
        dout.writeChar('4');
```

STREAM CLASSES - Example



```
// reading the data back using DataInputStream
DataInputStream din = new DataInputStream(new
                                         FileInputStream("file.dat"));

    double a = din.readDouble();

    int b = din.readInt();
    boolean c = din.readBoolean();
    char d=din.readChar();
    System.out.println("Values: " + a + " " +
b + " " + c+" " + d);
    }
}
```

ObjectInputStream and ObjectOutputStream



ObjectInputStream and ObjectOutputStream

The **ObjectInputStream** is used to read the object written by using **ObjectOutputStream**.

The **ObjectOutputStream** is used to write the object that can be read by using **ObjectInputStream**.

Serialization is an important concept that deals with the conversion of objects into a byte stream to transport the java objects from one Java Virtual Machine to the other and recreate them to the original form.

ObjectInputStream and ObjectOutputStream - Example



```
import java.io.*;

class Demo implements Serializable
{
    int a;
    String b;
    public Demo(int a, String b)
    {
        this.a = a;
        this.b = b;
    }
}
```

ObjectInputStream and ObjectOutputStream - Example



```
class Test
{
    public static void main(String[] args)
    {
        Demo object = new Demo(100, "Sachin");
        FileOutputStream file = new FileOutputStream("MyFile");
        ObjectOutputStream out = new
            ObjectOutputStream(file);
        out.writeObject(object);
        out.close();
        file.close();
        System.out.println("Object has been serialized");
    }
}
```


Example - Serialize an Object



```
import java.io.*;

public class Test1
{
    public void saveObject(String outputFilename, Object anObject)
    {
        try
        {
            FileOutputStream fpout = new FileOutputStream(outputFilename);
            ObjectOutputStream obOut = new ObjectOutputStream(fpout);
            obOut.writeObject(anObject);
            obOut.flush();
            obOut.close();
        }
        catch (IOException x)
        {
            System.out.println("Error:" + x);
        }
    }
}
```

Example - Read in a Serialized Object

```
import java.io.*;

public class Test2
{
    public Object readObject(String inputFilename)
    {
        try
        {
            FileInputStream fpin = new FileInputStream(inputFilename);
            ObjectInputStream obIn = new ObjectInputStream(fpin);
            Object anObject = obIn.readObject();
            obIn.close();
            return anObject;
        }
        catch (IOException x)
        {
            System.out.println("Error:" + x);
        }
    }
}
```

The File Class



- Java IO provides a class which is an abstract representation of a file or directory within the file system.

- The File class has 2 constructors:

```
File(String pathName)
```

```
File(File parent, String child)
```

- The File class provides several query methods:

- `canRead()`, `canWrite()`, `exists()`, `getAbsolutePath()`, `getName()`, `getParent()`, `getPath()`, `isAbsolute()`, `isDirectory()`, `isHidden()`, `lastModified()`, `length()`, and `list()`

- The File class also provides several methods which act on the file system:

- `createTempFile()`, `delete()`, `deleteOnExit()`, `mkdir()`, `makedirs()`, `renameTo()`, `setLastModified()`, `setReadOnly()`

File operations



```
import java.io.File;
import java.io.FileWriter;
import java.io.BufferedReader;
import java.io.PrintWriter;
import java.io.IOException;

public class oopd7 {
    public static void main(String[] args) {
        File file = null;
        String[] paths;

        try {
            // create new file object
            file = new File("C:\\Users\\Vijay");

            // array of files and directory
            paths = file.list();

            // for each name in the path array
            for(String path:paths) {
                // prints filename and directory name
                System.out.println(path);
            }
        } catch (Exception e) {
            // if any error occurs
            e.printStackTrace();
        }
    }
}
```

File operations – Rename a file

```
package bits;
import java.io.*;

public class oopd10 {
    public static void main(String[] args)
    {
        //Old File
        File oldfile =new File("C:\\\\Users\\\\Vijay\\\\eclipse-workspace\\\\Hello\\\\ex1");
        //New File
        File newfile =new File("C:\\\\Users\\\\Vijay\\\\eclipse-workspace\\\\Hello\\\\ex2");
        /*renameTo() return boolean value
        * It return true if rename operation is
        * successful
        */
        boolean flag = oldfile.renameTo(newfile);
        if(flag){
            System.out.println("File renamed successfully");
        }else{
            System.out.println("Rename operation failed");
        }
    }
}
```

File operations – Delete a file

```
package bits;
import java.io.*;

public class oopd10 {
    public static void main(String[] args)
    {
        try{
            //Specify the file name and path
            File file = new File("C:\\\\Users\\\\Vijay\\\\eclipse-workspace\\\\Hello\\\\ex2");
            /*the delete() method returns true if the file is
             * deleted successfully else it returns false
             */
            if(file.delete()){
                System.out.println(file.getName() + " is deleted!");
            }else{
                System.out.println("Delete failed: File didn't delete");
            }
        }catch(Exception e){
            System.out.println("Exception occurred");
            e.printStackTrace();
        }
    }
}
```



Binary Versus Text Files

All data and programs are ultimately just zeros and ones
each digit can have one of two values, hence *binary*
bit is one binary digit
byte is a group of eight bits

Text files: the bits represent printable characters
one byte per character for ASCII, the most common code
for example, Java source files are text files
so is any file created with a "text editor"

Binary files: the bits represent other types of encoded information, such as
executable instructions or numeric data
these files are easily read by the computer but not humans
they are *not* "printable" files
actually, you *can* print them, but they will be unintelligible
"printable" means "easily readable by humans when printed"

Java: Text Versus Binary Files



Text files are more readable by humans

Binary files are more efficient

computers read and write binary files more easily than text

Java binary files are portable

they can be used by Java on different machines

Reading and writing binary files is normally done by a program

text files are used only to communicate with humans

Java Text Files

- Source files
- Occasionally input files
- Occasionally output files

Java Binary Files

- Executable files (created by compiling source files)
 - Usually input files
 - Usually output files
-

Example



```
package bits;

import java.io.*;

public class oopd4 {

    public static void main(String args[]) throws IOException {
        FileInputStream in = null;
        FileOutputStream out = null;

        try {
            String currentDirectory = System.getProperty("user.dir");
            System.out.println("The current working directory is " + currentDirectory);
            in = new FileInputStream("input.txt");
            out = new FileOutputStream("output.txt");

            int c;
            while ((c = in.read()) != -1) {
                out.write(c);
            }
        } finally {
            if (in != null) {
                in.close();
            }
            if (out != null) {
                out.close();
            }
        }
    }
}
```

To read a line



```
package bits;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.BufferedReader;
import java.io.PrintWriter;
import java.io.IOException;

public class oopd6 {
    public static void main(String args[]) throws IOException {
        BufferedReader inputStream = null;
        PrintWriter outputStream = null;

        try {
            inputStream =
                new BufferedReader(new FileReader("input.txt"));
            outputStream =
                new PrintWriter(new FileWriter("characteroutput.txt"));

            String l;
            while ((l = inputStream.readLine()) != null) {
                outputStream.println(l);
            }
        } finally {
            if (inputStream != null) {
                inputStream.close();
            }
            if (outputStream != null) {
                outputStream.close();
            }
        }
    }
}
```

To read from console



```
package bits;

import java.io.*;

public class oopd5 {

    public static void main(String args[]) throws IOException {
        InputStreamReader cin = null;

        try {
            cin = new InputStreamReader(System.in);
            System.out.println("Enter characters, 'q' to quit.");
            char c;
            do {
                c = (char) cin.read();
                System.out.print(c);
            } while(c != 'q');
        } finally {
            if (cin != null) {
                cin.close();
            }
        }
    }
}
```

Random Access File



```
import java.io.IOException;
import java.io.RandomAccessFile;

public class RandomAccessFileExample {
    static final String FILEPATH = "myFile.TXT";
    public static void main(String[] args) {
        try {
            System.out.println(new String(readFromFile(FILEPATH, 0, 18)));
            writeToFile(FILEPATH, "I love my country and my people", 31);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Random Access File



```
private static byte[] readFromFile(String filePath, int position, int size)
    throws IOException {
    RandomAccessFile file = new RandomAccessFile(filePath, "r");
    file.seek(position);
    byte[] bytes = new byte[size];
    file.read(bytes);
    file.close();
    return bytes;
}

private static void writeToFile(String filePath, String data, int position)
    throws IOException {
    RandomAccessFile file = new RandomAccessFile(filePath, "rw");
    file.seek(position);
    file.write(data.getBytes());
    file.close();
}
}
```

BufferedReader



```
import java.io.*;

public class BufferedReaderExample {

    public static void main(String args[]) throws Exception{
        FileReader fr=new FileReader("D:\\testout.txt");
        BufferedReader br=new BufferedReader(fr);

        int i;
        while((i=br.read())!=-1){
            System.out.print((char)i);
        }
        br.close();
        fr.close();
    }
}
```

File reader & BufferedReader

```
import java.io.FileReader;

public class FileReaderExample {
    public static void main(String args[]) throws Exception{
        FileReader fr=new FileReader("D:\\testout.txt");
        int i;
        while((i=fr.read())!=-1)
            System.out.print((char)i);
        fr.close();
    }
}
```

```
import java.io.*;

public class BufferedReaderExample {
    public static void main(String args[]) throws Exception{
        FileReader fr=new FileReader("D:\\testout.txt");
        BufferedReader br=new BufferedReader(fr);

        int i;
        while((i=br.read())!=-1){
            System.out.print((char)i);
        }
        br.close();
        fr.close();
    }
}
```



DataOutputStream

```
import java.io.*;

public class OutputExample {

    public static void main(String[] args) throws IOException {
        FileOutputStream file = new FileOutputStream(D:\\testout.txt);
        DataOutputStream data = new DataOutputStream(file);
        data.writeInt(65);
        data.flush();
        data.close();
        System.out.println("Success...");
    }
}
```


DataInputStream



```
import java.io.*;
public class DataStreamExample {
    public static void main(String[] args) throws IOException {
        InputStream input = new FileInputStream("D:\\testout.txt");
        DataInputStream inst = new DataInputStream(input);
        int count = input.available();
        byte[] ary = new byte[count];
        inst.read(ary);
        for (byte bt : ary) {
            char k = (char) bt;
            System.out.print(k+"-");
        }
    }
}
```

Here, we are assuming that you have following data in "testout.txt" file:

JAVA

Output:

J-A-V-A



Object Oriented Programming and Design



BITS Pilani
Pilani Campus

Presented by
Dr Ramya Devi M
mramyadevi@wilp.bits-pilani.ac.in
CS - 09 : 19th Sep 2025

IMP Note to Self





Object-Oriented Programming

Inheritance and Polymorphism in Java

BITS Pilani

Learning Objectives



- What is Inheritance?
- Various types of inheritance
- Super and Sub classes
- Super keyword
- Method overriding
- What is an Abstract class?
- Abstraction with abstract class

Topics



- Inheritance in Java



Inheritance Basics

- Inheritance is the process by which one class acquires the properties(data members) and functionalities(methods) of another class.
- Reusability is achieved by *Inheritance*
- Java classes can be Reused by Extending a class. Extending an existing class is nothing but reusing properties of the existing classes. The class whose properties are extended is known as *super or base or parent class*.
- The class which extends the properties of super class is known as *sub or derived or child class*
- A class can either *extends another class* or *can implement an interface*

Java's Support For Inheritance



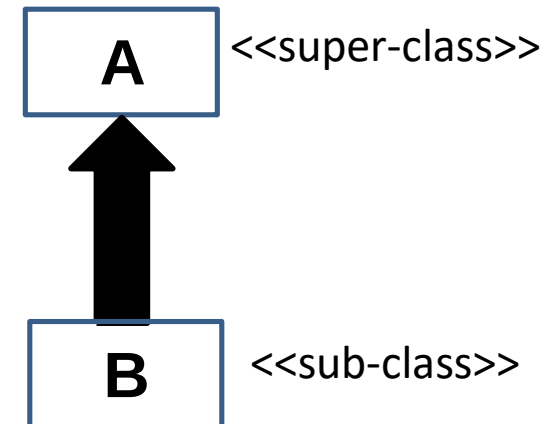
- By Extending a Class

- Syntax

```
class      class-name      extends      super-class-name
{
    .....
}
```

- Example

```
class A
{
    .....
} // End of class A
class B extends A
{
    .....
} // End of class B
```



Java's Support For Inheritance



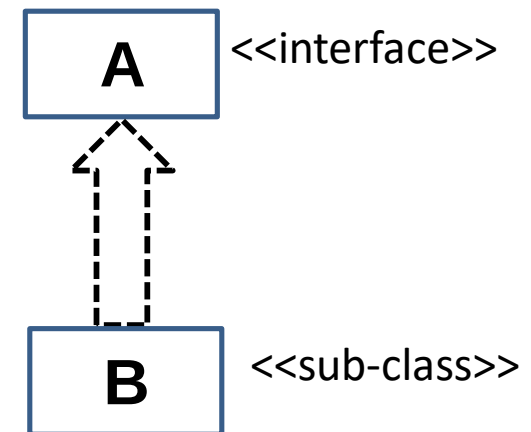
- By Implementing an Interface

- Syntax

```
class      class-name      implements      interface-name
{
    .....
}
```

- Example

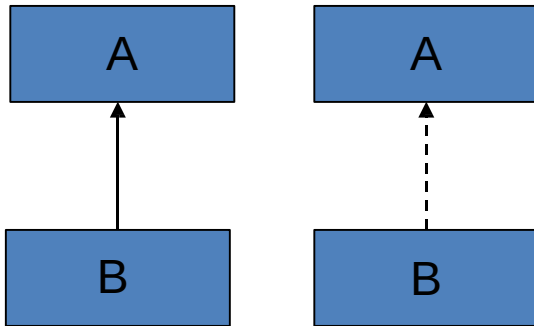
```
interface A
{
    .....
} // End of interface A
class B implements A
{
    .....
} // End of class B
```



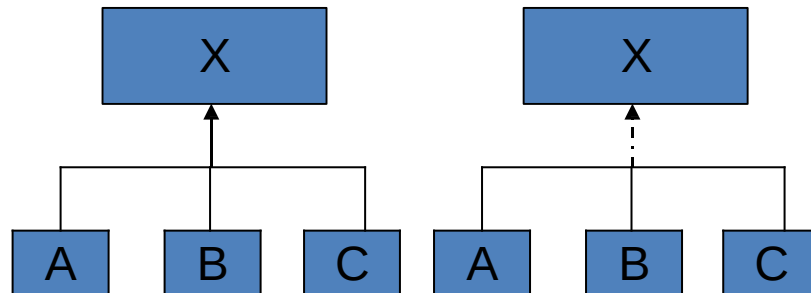
Forms of Inheritance



Single Inheritance



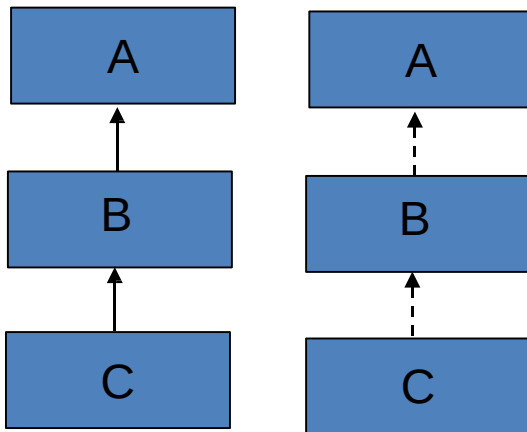
Hierarchical Inheritance



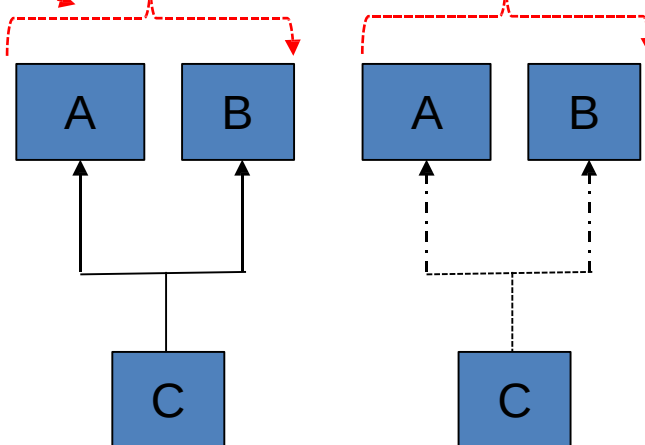
**Multiple Class Inheritance
NOT SUPPORTED BY JAVA**

**Multiple Interface Implementation
SUPPORTED BY JAVA**

Multi-Level Inheritance



Multiple Inheritance





INHERITANCE - Example

General format:

```
class ChildClass extends ParentClass
{
    Members of the child class
}
```

Example

```
class Teacher
{
    String collegeName="BITS Pilani";
    String designation = "Teacher";
    void activity()
    {
```

INHERITANCE - Example



```
        System.out.println("\n Performing Teaching..");
    }
}
class MathsTeacher extends Teacher
{
    String subject="Mathematics";
    void grade()
    {
        System.out.println("\n Secondary Teacher..");
    }
    public static void main(String aa[])
    {
```

INHERITANCE - Example



```
MathsTeacher mt=new MathsTeacher();  
System.out.println("\n College name is "+mt.collegeName);  
System.out.println("\n Designation "+mt.designation);  
mt.activity();  
mt.grade();  
}  
}
```

In the above example, we have a base class **Teacher** consisting of the data members collegeName and designation and performs() as its method.

We have a sub class **MathsTeacher** consisting of the data member subject and grade() as its method

Multiple Inheritance in Java



- Java Does Not Support Multiple Inheritance by Extending Multiple Classes
- The Following Code is Wrong

```
class A      {      } // End of class A
class B      {      } // End of class B
class C      extends A, B
{
} // End of class C
```

**Wrong Java Does not
Allow Multiple Class
Extensions**

Multiple Inheritance in Java....



- Java Supports Multiple Inheritance by Implementing Multiple Interfaces
- The Following Code is Correct

```
interface A {      } // End of interface A
interface B {      } // End of interface B
```

```
class C      implements A, B
{
} // End of class C
```

**Java Allows Multiple
Interface
Implementations**

Interface in Java....



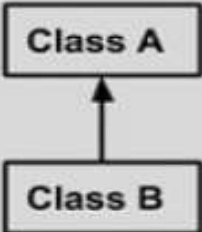
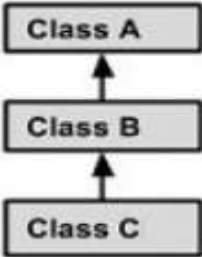
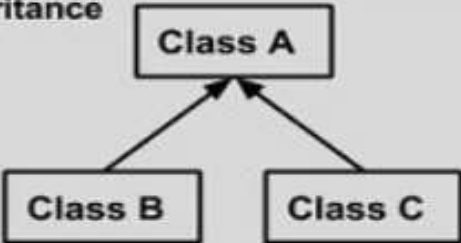
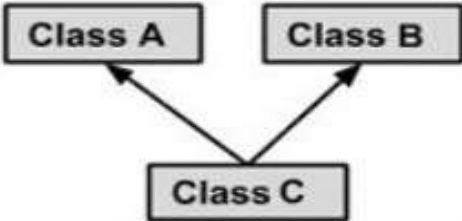
- An interface is declared by using the interface keyword. It provides total abstraction; means all the methods in an interface are declared with the empty body, and all the fields are public, static and final by default. A class that implements an interface must implement all the methods declared in the interface.

```
1.interface <interface_name>{  
2.  
3.    // declare constant fields  
4.    // declare methods that abstract  
  
5.    // by default.  
6.}
```

It has static constants and abstract methods.

Types of Inheritance

There are various types of inheritance as demonstrated below.

Single Inheritance	 <pre>graph BT; B[Class B] --> A[Class A]</pre>	<pre>public class A { } public class B extends A { }</pre>
Multi Level Inheritance	 <pre>graph BT; C[Class C] --> B[Class B]; B --> A[Class A]</pre>	<pre>public class A {} public class B extends A {.....} public class C extends B {..... }</pre>
Hierarchical Inheritance	 <pre>graph BT; B[Class B] --> A[Class A]; C[Class C] --> A</pre>	<pre>public class A {} public class B extends A {.....} public class C extends A {..... }</pre>
Multiple Inheritance	 <pre>graph BT; C[Class C] --> A[Class A]; C --> B[Class B]</pre>	<pre>public class A {} public class B {.....} public class C extends A,B { } // Java does not support mutiple Inheritance</pre>



Object-Oriented Programming

Inheritance and Polymorphism in Java

BITS Pilani

Topics



- Inheritance Basics - II

Inheritance Basics



- Suppose A is super class of B. Then class B can inherit all the features (Methods and Instance Fields) of class A except the private members.
- Super and Sub Classes May Either Belong to Same or Different Packages

		Access Modifiers			
		public	protected	package- private	private
Access Locations	With in the Same Class	Yes	Yes	Yes	Yes
	Sub-Classes in same package	Yes	Yes	Yes	No
	Other Classes in same package	Yes	Yes	Yes	No
	Subclasses in other packages	Yes	Yes	No	No
	Non-subclasses in other packages	Yes	No	No	No

Inheritance Basics



- Whenever an instance of sub-class type is created, a super-class constructor is called first.
- If a super-class constructor does not have any constructor of its own OR has an un-parametrized constructor then it is automatically called by JRE by using call **super()**
- If a super-class has a parameterized constructor then it is the responsibility of the sub-class constructor to call the super class constructor by call
super(<parameters required by super class>)
- Call to super class constructor must be the first statement in sub class constructor



Constructors in Inheritance

Constructors in Inheritance

When a super class is inherited in the sub class, its constructor can also be inherited.

It means that the base class constructor of the super class is accessible in the sub class also.

There are two methods of inheriting the super class constructor in the sub class. They are

1. Inheriting non parameterized constructor
2. Inheriting parameterized constructor

In Java, constructor of base class with no argument gets automatically called in derived class constructor.

INHERITANCE - Example:



```
class Base
{
    Base()    {
        System.out.println(" It is the Super class Constructor ");
    }
}

class Derived extends Base
{
    Derived()
    {
        System.out.println(" It is the Sub Class Constructor ");
    }
}
```

Example



```
class SuperConstDemo1
{
    public static void main(String aa[])
    {
        Derived d = new Derived ();
    }
}
```

The output of the above coding is

It is the Super class Constructor

It is the Sub Class Constructor

Note:- The super class constructor is automatically executed in the sub class. This constructor is executed before the sub class constructor

Example – Base VS Derived class constructors



But, if we want to call parameterized constructor of base class, then we can call it using the keyword **super()**. The point to note is that **base class constructor calling statement is the first statement in the derived class constructor.**

Example:

class Base

```
{  
    int data1;  
    Base(int x)  
    {  
        data1=x;  
    }  
}
```

Example



```
class Derived extends Base
{
    float data2;
    Derived(int x,float y)
    {
        super(x);
        data2=y;
    }
    void display()
    {
        System.out.println(" Base:"+data1+"Derived:"+data2);
    }
}
```

Example



```
class SuperConstDemo2
{
    public static void main(String aa[])
    {
        Derived d = new Derived (100,200.50);
        d.display();
    }
}
```

The output of the above coding is

Base:100 Derived:200.50

Note:- The super class constructor is executed in the sub class through the keyword **super**.

Extending Classes : Example 1



When super-class has an Un-parametrized Constructor

// File Name : Sample.java

```
class A
{
```

```
    A()
    {
```

```
        System.out.println("This is constructor of class A");
```

```
    } // End of class A
```

```
class B extends A
```

```
{
```

```
    B()
    {
```

```
        super();
        System.out.println("This is constructor of class B");
```

```
    } // End of class B
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
    {
```

```
        B b1 = new B();
```

```
    } // End of Method
```

```
} // End of Class Test
```

Call to a
Method of super
class

OUTPUT

This is constructor of class A
This is constructor of class B

Extending Classes : Example 2



When super-class has only a Parametrized Constructor

// File Name : Sample.java

```
class A
{
    A(int x, int y)
    {
        System.out.println("This is constructor of class A");
    }
} // End of class A
class B extends A
{
    B(int a, int b)
    {
        super(a,b);
        System.out.println("This is constructor of class B");
    }
} // End of class B
class Test
{
    public static void main(String[] args)
    {
        B b1 = new B();
    }
} // End of Method
} // End of Class Test
```

A sub-class must have a constructor parameterized constructor of super-class

Note : super() statement must be the first statement in a sub-class constructor

Extending Classes : Example 3



‘final’ classes cannot have sub-classes

// File Name : Sample.java

final class A

{

A(int x, int y)

 {

 System.out.println("This is constructor of class A");

 }

} // End of class A

class B extends A

{

B(int a, int b)

 {

 super(a,b);

 System.out.println("This is constructor of class B");

 }

} // End of class B

The Code Will Result in Compile-Time Error.

Role of superKeyword



- **'super'** Java keyword primarily used for two purposes
 1. To Call constructor method of super-class from a sub-class constructor
Syntax:
 - (i) `super()` ; // if super-class has only one un-parameterized constructor
 - (ii) `super(<parameters>)`; // if super-class has only one parameterized constructor
 2. To call a method of super-class in a sub-class method or to access an instance field of super-class in sub-class especially when their names are similar
Syntax:
 - (i) `super.<super-class-method()>` ;
 - (ii) `super.<super-class-instance-field>`;

Role of super : Example 1



// File Name : XYZ.java

```
class A
{
    private int a;           // instance-field
    // Constructor Method
    A( int a)
    {
        this.a =a;
        System.out.println("This is constructor of class A");
    } // End of Constructor
    // print Method
    void print()
    {
        System.out.println("a="+a);
    } // End of Method
    // Display Method
    void display()
    {
        System.out.println("hello This is Display in A");
    } // End of Method
} // End of class A
```

class B extends A

```
{
    private int b;           // instance field
    private double c;       // instance field
    B(int a,int b,double c)
    {
        super(a);
        this.b=b;
        this.c=c;
        System.out.println("This is constructor of class B");
    } // End of Constructor Method
    // show Method
    void show()
    {
        print(); ← - - - - -
        System.out.println("b = " + b);
        System.out.println("c = " + c);
    } // End of Method
} // End of class B
```

Call to print() method of super-class Can be Written as
super.print()

Call to super-class
Constructor

Role of super : Example 2



```
// File Name : XYZ.java
```

```
class A
```

```
{
```

protected

```
} // End of class A
```

```
class B extends A
```

```
{
```

private

```
void show()
```

```
{
```

```
    int    a = 50;  
    System.out.println(" a= " + a);  
    System.out.println(" a= "+ this.a);  
    System.out.println(" a = "+ super.a);
```

```
}// End of Method
```

```
}// End of class B
```

int **a = 20;** // instance-field



int **a = 30;**



int **a = 50;**



Role of super : Example 3



// File Name : XYZ.java

```
class A
{
    private int a;           // instance-field
    // Constructor Method
    A( int a)
    {
        this.a =a;
        System.out.println("This is constructor of class A");
    } // End of Constructor
    // print Method
    void print()
    {
        System.out.println("a="+a);
    }
    // Display Method
    void show()
    {
        System.out.println("Hello This is Display in A");
    }
} // End of class A
```

class B extends A

```
{
    private int b;           // instance field
    private double c;       // instance field
    // Constructor Method
    B(int a, int b, double c)
    {
        super(a);
        this.b=b;
        this.c=c;
        System.out.println("This is constructor of class B");
    } // End of Constructor Method
    // show Method
    void show()
    {
        super.show();
        System.out.println("b = " + b);
        System.out.println("c = " + c);
    } // End of Method
} // End of class B
```

Call to show() method of super-class

Call to super class Constructor



Object-Oriented Programming

Inheritance and Polymorphism in Java

BITS Pilani

Topics



- Method Overriding in Java
 - Method Hiding in Java
-

Method Overriding in Java



- Also known as Runtime Polymorphism, Dynamic Method Dispatch or Late Binding
- A sub-class overrides an object (instance) method of a super-class if the following three conditions are satisfied
 1. The method of a sub-class and method of a super class have same name
 2. The method of a sub-class and method of a super class have same signature
 3. The method of a sub-class and method of a super class have same return type
- The scope of a sub-class method should be either same or higher than that of a super-class method
- Note : private methods of a super-class are not visible in sub-class. Hence, a sub-class cannot override the private methods of super class.
- Call to an overridden method is decided at runtime.
- Call to a overridden method is not decided by the type of reference variable. Rather it is decided by the type of the object where reference variable is pointing.

Method Overriding : Example



class A

{

void show(int a, int b)

{

System.out.println("Hello ! This is show() in A");

// End of show() Method

} // End of class A

class B extends A

{

void show(int a, int b)

{

System.out.println("Hello This is show() in B");

// End of show() Method

} // End of class B

// Driver Class

class

Test

{

public

static

void

main(String args[])

A

a1

=

new

A();

a1.show(4,5);

a1

=

new

B();

a1.show(1,1);

// End of Method

Calls show() from B class

Calls show() from A class

Type of Variable 'a1' is A
Type of Object is 'A'

Type of Variable 'a1' is still A
But type of Object is 'B'

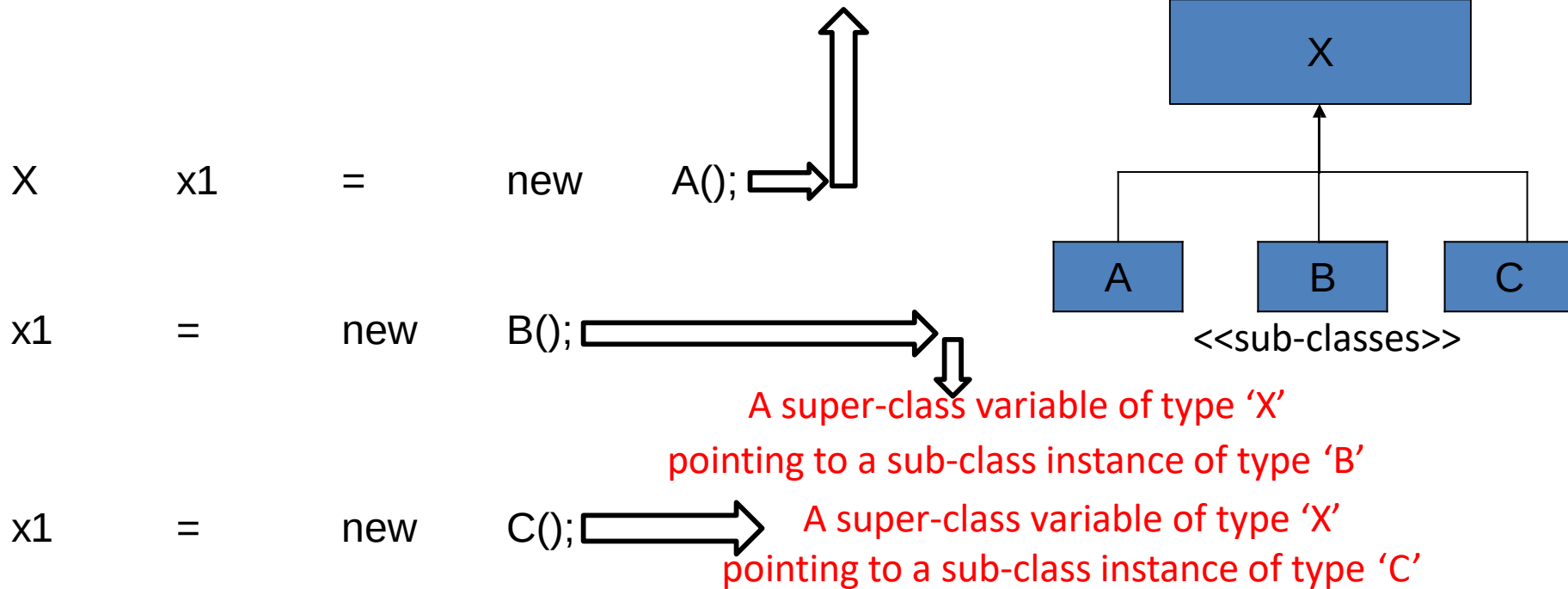
sub-class B
overrides
the show() Method
of super-class A

Basic Facts About Method Overriding : Fact I



- A super class variable can point to any of its sub-class instance
- Example : Suppose 'X' is the super-class for sub-classes 'A', 'B' and 'C'.

A super-class variable of type 'X'
pointing to a sub-class instance of type 'A'



Basic Facts About Method Overriding : Fact II



- When a super-class variable points to any of its sub-class instance, then from the sub-class only overridden methods can be invoked by using the same variable

```
class A
{
```

```
void show()
{
} // End of Method
```

```
} // End of class A
```

```
class B extends A
{
```

```
void show()
{
} // End of Method
```

```
void display()
{
} // End of Method
```

```
} // End of class B
```

B class overrides show() Method

A a1 = new B();

a1.show(); → Valid Statement

a1.display(); → ↓
Results in Compile-Time Error

Basic Facts About Method Overriding : Fact III



- Overridden Method in sub-class should have either same or higher scope than the scope of a similar method in the super class.

```
class A  
{
```

```
    void show()  
    {  
        } // End of Method
```

A's show() method has
package-private scope

```
} // End of class A
```

```
class B extends A  
{
```

```
    public void show()  
    {  
        } / End of Method
```

B's show() method has
public scope

```
    void display()  
    {  
        } / End of Method
```

```
} // End of class B
```

NO ERROR

Basic Facts About Method Overriding : Fact I I ...



- Overridden Method in sub-class should have either same or higher scope than the scope of a similar method in the super class. The scope of overridden method in sub-class should not have lesser scope

```
class A
{
```

```
    protected void show()
    {
    } // End of Method
```

A's show() method has protected scope

```
} // End of class A
```

```
class B extends A
{
```

```
    public void show()
    {
    } // End of Method
```

B's show() method has public scope

```
    void display()
    {
    } // End of Method
```

```
} // End of class B
```

NO ERROR

Basic Facts About Method Overriding : Fact I I ...



- Overridden Method in sub-class should have either same or higher scope than the scope of a similar method in the super class.

```
class A  
{
```

```
protected void show()  
{  
} // End of Method
```

A's show() method has
protected scope

```
} // End of class A
```

```
class B extends A  
{
```

```
void show()  
{  
} / End of Method
```

B's show() method has
package-private scope

```
void display()  
{  
} / End of Method
```

```
} // End of class B
```

**COMPILE-TIME
ERROR**

Basic Facts About Method Overriding : Fact IV



- 'final' Methods of super-class cannot be overridden by sub-class.

```
class A  
{
```

```
    final void show()  
    {  
        } // End of Method
```

A's show() method is final

```
} // End of class A
```

```
class B extends A  
{
```

```
    void show()  
    {  
        } / End of Method
```

B cannot override final
method from super class A

```
    void display()  
    {  
        } / End of Method
```

```
} // End of class B
```

**COMPILE-TIME
ERROR**

Find the Error?



```
class A
{
    int show(int a, int b)
    {
        }// End of Method
```

```
// End of class A
```

```
class B extends A
{
```

```
    void show(int a, int b)
    {
        } / End of Method
```

```
    void display()
    {
        } / End of Method
```

```
// End of class B
```

Compile-Time Error
(Overridden Methods Cannot Have Different Return Types)

Find the Error?



```
class A
{
    void show(int a, int b)
    {
    } // End of Method
}
```

```
// End of class A
```

```
class B extends A
{
    void show()
    {
    } / End of Method
}
```

```
void display()
{
} / End of Method
```

```
// End of class B
```

No Error

This Code Represents Method Overloading

B b1 = new B();

b1.show();

b1.show(10,8);

Calls show() from A class

Calls show() from B class

Does the Following Code Snippet Represent Method Overriding?



Method is
Private
So, Not Visible in
class B

```
class A
{
    private void show(int a, int b)
    {
        System.out.println("A : " + (a+b));
    } // End of Method

} // End of class A
class B extends A
{
    void show(int a, int b)
    {
        System.out.println("A : " + (a*b));
    } // End of Method
    void display()
    {
    } // End of Method
} // End of class B
```

NO. IT DOES NOT REPRESENT METHOD OVERRIDING

Method Hiding



- A sub-class can hide a static method (class method) of a super-class if the following conditions are satisfied for class methods
 1. Methods should have same name, same signatures and same return type
 2. Methods should be static and not instance (or object)

Method Hiding: Example 1



```
class A
{
```

```
    public static void show(int a, int b)
    {
        System.out.println(" Class A Method :" + (a + b));
    } // End of Method
```

```
} // End of class A
class B extends A
{
```

```
    public static void show(int a, int b)
    {
        System.out.println(" Class B Method :" + (a * b));
    } // End of Method
```

class B hides a
static method of
super-class A

```
} // End of class B
class Test
{
```

```
    public static void main(String[] args)
    {
```

```
        A a1 = new A();
        a1.show(5,6);
```

```
        a1 = new B();
        a1.show(5,6);
```

```
    } // End of Method
```

```
} // End of class Test
```

OUTPUT

class A Method: 11
class A Method: 11

Method Hiding: Example 2



```
class A
{
```

```
    protected static void show(int a, int b)
    {
        System.out.println(" Class A Method :" + (a + b));
    } // End of Method
```

```
} // End of class A
class B extends A
{
```

```
    public static void show(int a, int b)
    {
        System.out.println(" Class B Method :" + (a * b));
    } // End of Method
```

class B hides a
static method of
super-class A

```
} // End of class B
class Test
{
```

```
    public static void main(String[] args)
    {
```

```
        A a1 = new A();
        a1.show(5,6);
```

```
        a1 = new B();
        a1.show(5,6);
```

```
    } // End of Method
```

```
} // End of class Test
```

OUTPUT

class A Method: 11
class A Method: 11

Method Hiding: Example 3



```
class A
{
```

```
    public static void show(int a, int b)
    {
        System.out.println(" Class A Method :" + (a + b));
    } // End of Method
```

```
} // End of class A
class B extends A
```

```
{
```

```
    public void show(int a, int b)
    {
        System.out.println(" Class B Method :" + (a * b));
    } // End of Method
```

```
} // End of class B
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
    {
```

```
        A a1 = new A();
        a1.show(5,6);
```

```
        a1 = new B();
        a1.show(5,6);
```

```
    } // End of Method
```

```
} // End of class Test
```

**COMPILE-TIME
ERROR**

Method Hiding: Example 3



```
class A
{
```

```
    public static void show(int a, int b)
    {
        System.out.println(" Class A Method :" + (a + b));
    } // End of Method
```

```
} // End of class A
class B extends A
```

```
{
```

```
    protected static void show(int a, int b)
    {
        System.out.println(" Class B Method :" + (a * b));
    } // End of Method
```

```
} // End of class B
```

```
class Test
```

```
{
```

```
    public static void main(String[] args)
    {
```

```
        A a1 = new A();
        a1.show(5,6);
```

```
        a1 = new B();
        a1.show(5,6);
```

```
    } // End of Method
```

```
} // End of class Test
```

**COMPILE-TIME
ERROR**



Object-Oriented Programming

Inheritance and Polymorphism in Java

BITS Pilani

Topics

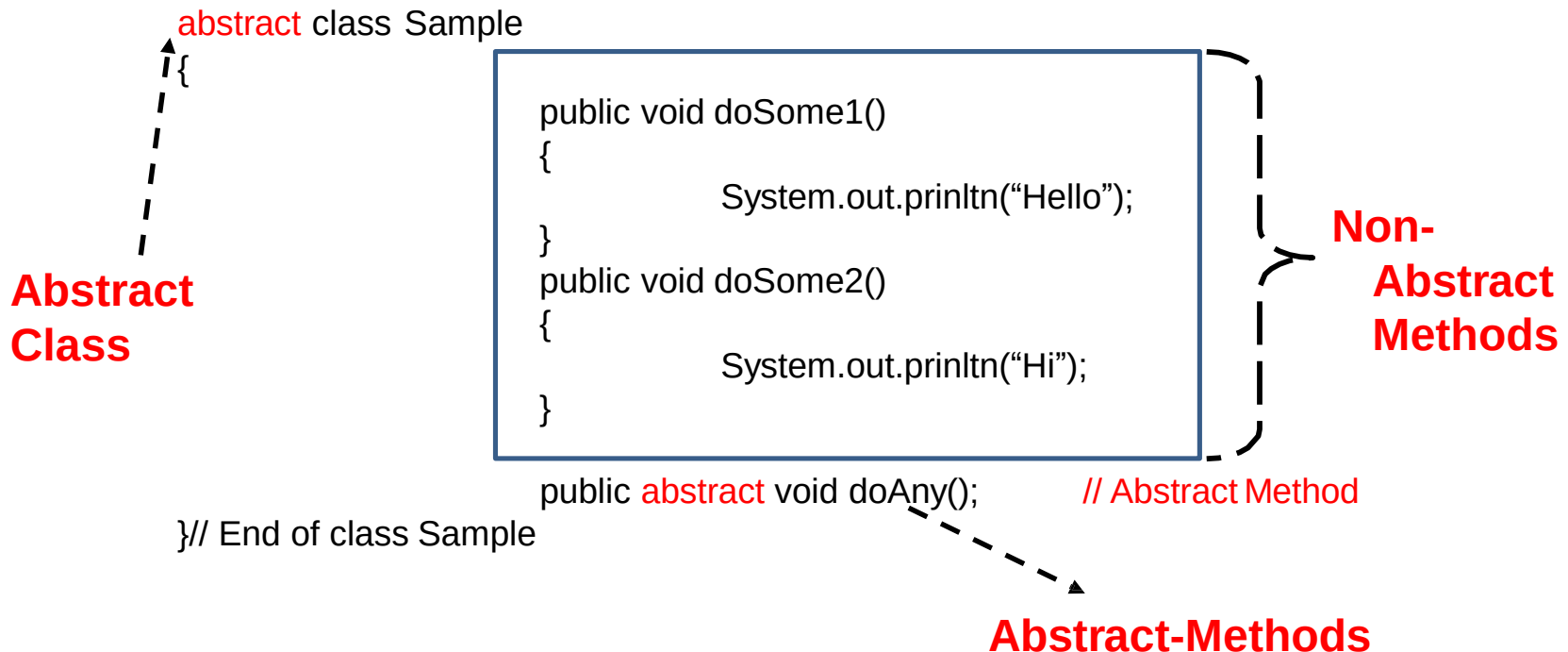


- Abstract Methods
- Abstract Classes



Abstract Methods and Abstract Classes

- Abstract Method → Method with only declaration part and without implementation **[Incomplete Method]**
- Abstract Classes → If a class has any one abstract method **[Incomplete Class]**
- Example



Abstract Classes



- An abstract class is a class which has *abstract methods* (i.e. a method with only heading with no body of executable statements)
- An object or instance of abstract classes can not be instantiated
- An abstract class needs to be extended by sub classes to provide the implementation for the abstract methods.
- Abstract classes may contain static methods also. However, abstract and static keyword combination is wrong

abstract static void print(); // wrong

- Abstract classes may extend either another abstract class or concrete (complete or non-abstract) class
- Abstract classes may include constructors, nested classes and interfaces
- Abstract classes has either public, protected, private or package accessibility

Abstract Classes : Syntax



- Syntax :**

```
<scope>      abstract      class      <class-name>      [extends      <super-class-name>]
      [implements
      interface-1, interface-2, ....., interface-n]
{
.....
<scope>      abstract      <return type>      method-name-1(<parameter List>);
<scope>      abstract      <return type>      method-name-2(<parameter List>);
.....
<scope>      abstract <return type> method-name-n(<parameter List>);
}
```

Note:

1. Abstract class can have one or more abstract methods
2. Abstract classes may extend another class , implements another interface , may have concrete methods

Abstract Classes : Fact I



- A class can be declared as abstract even if it does not have any abstract method
- Example:

```
abstract    class    A
{
    public void doS(int a, int b)
    {
        System.out.println(a+b);
    } // End of Method
} // End of class A
```

Abstract Classes : Fact II



- Only instance methods (object methods) can be declared as abstract.
- **'static'** and **'abstract'** forms illegal combination
- Example:

```
abstract    class    A
{
    public static abstract void doS(int a, int b);

} // End of class A
```

Abstract Classes : Fact III



- An abstract class may extend either another abstract class or a concrete (non-abstract) class
- Example

```
abstract class A
{
} // End of class A
abstract class B
extends A
{
} // End of class B
```

```
class A
{
} // End of class A
abstract class B
extends A
{
} // End of class B
```

Abstract Classes : Fact IV



- An instance or object cannot belong to an abstract class. However a variable can belong to an abstract class type.
- Example

```
abstract class
A
{
} // End of class
Class B extends
A
{
} // End of class
Class C
extends A
{
} // End of class
C
```

```
A    a1    =    new    A();
```

Compile-Time Error: Object Cannot belong to Abstract Class Type

```
A    a1    =    new    B();
```

Correct: Abstract Class Type Variable Can Point to Any Concrete Sub-class Instance

```
a1    =    new    C();
```

Correct: Abstract Class Type Variable Can Point to Any Concrete Sub-class

Abstract Classes : Fact V



- When any class say 'X' extends an abstract class say 'Y' then in order for class 'X' to be a complete or concrete class, the class 'X' must implement all the abstract methods of class 'Y' otherwise class 'X' has to be declared as abstract.



```
abstract class Shape {  
    String color;  
    Shape(String color) {  
        this.color = color;  
    }  
    abstract double area();  
    void display() {  
        System.out.println("Color: " + color);  
    }  
}  
class Rectangle extends Shape {  
    double width, height;  
    Rectangle(String color, double width, double height) {  
        super(color);  
        this.width = width;  
        this.height = height;  
    }  
    double area() {  
        return width * height;  
    }  
}
```

Abstract Method



Rules of Abstract Method

1. Abstract methods don't have body, they just have method signature as shown above.
 2. If a class has an abstract method it should be declared abstract, the vice versa is not true, which means an abstract class doesn't need to have an abstract method compulsory.
 3. If a regular class extends an abstract class, then the class must have to implement all the abstract methods of abstract parent class or it has to be declared abstract as well.
-



Thank You



Object Oriented Programming and Design



BITS Pilani
Pilani Campus

Presented by
Dr Ramya Devi M
mramyadevi@wilp.bits-pilani.ac.in
CS - 10 : 27th Sep 2025

IMP Note to Self





Interface



Session plan

- Designing the interfaces
 - Extending interfaces
 - Implementing interfaces
 - Using the Comparator and Comparable interfaces
 - Nested Classes and Nested Interfaces
 - Inheriting Nested Types
 - Inner Classes and Anonymous classes
-

Interfaces



A Java *interface* is a collection of abstract methods and constants

An *abstract method* is a method header without a method body

An abstract method can be declared using the modifier `abstract`, but because all methods in an interface are abstract, usually it is left off

An interface is used to establish a set of methods that a class will implement

Interfaces



interface is a reserved word



```
public interface Doable  
{
```

```
    public void doThis();
```

```
    public int doThat();
```

```
    public void doThis2 (float value, char ch);
```

```
    public boolean doTheOther (int num);
```

```
}
```

None of the methods in
an interface are given
a definition (body)



A semicolon immediately
follows each method header

Interfaces



An interface cannot be instantiated

Methods in an interface have public visibility by default

A class formally **implements** an interface by:

stating so in the class header

providing implementations for each abstract method in the interface

If a class asserts that it implements an interface, it must define all methods in the interface

Interfaces



```
public class CanDo implements Doable
```

```
{
```

```
    public void doThis ()
```

```
    {
```

```
        // whatever
```

```
    }
```

```
    public void doThat ()
```

```
    {
```

```
        // whatever
```

```
    }
```

```
    // etc.
```

```
}
```

implements is a
reserved word

Each method listed
in Doable is
given a definition

Example of an Interface in Java



This is how a class implements an interface. It has to provide the body of all the methods that are declared in interface or in other words we can say that class has to implement all the methods of interface.

interface in1

{

final float PI = 3.14f;

public void Amethod1();

public void Amethod2();

}

Example of an Interface in Java



class Sub implements inl

```
{ public void Amethod1()
{
    System.out.println("\n It is Abstract Method 1 of Interface");
}

    public void Amethod2()
    {
        System.out.println("\n It is Abstract Method 2 of Interface");
    }
}
```

Example of an Interface in Java



```
class MyDemo
{
    public static void main(String aa[])
    {
        Sub S = new S();
        S.Amethod1();
        S.Amethod2();
    }
}
```

The output of the above program is

It is Abstract Method 1 of Interface

It is Abstract Method 2 of Interface

Interfaces

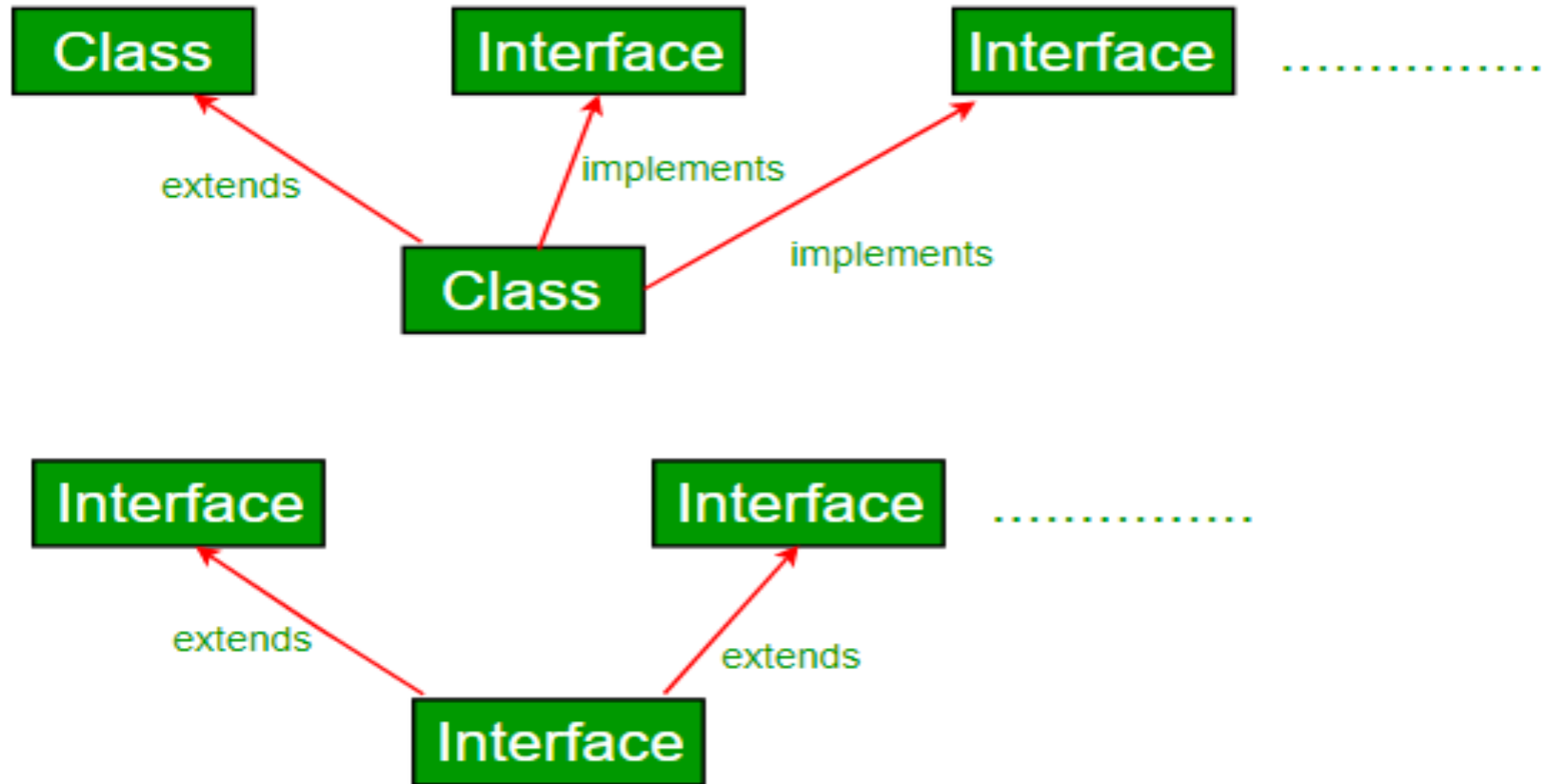


A class that implements an interface can implement other methods as well

In addition to (or instead of) abstract methods, an interface can contain constants

When a class implements an interface, it gains access to all its constants

Interfaces



Comparison with Inheritance



- Interfaces don't define any method actions...
you need to fill in all the details
 - It essentially just gives a basic collection of
method names
 - Not a strict hierarchy
 - Important: can implement several Interfaces
 - Can use this to “fake” multiple inheritance
-

Interfaces vs. Abstract Classes



Similarities

- neither can be instantiated
- both can be used as the starting point for a class

Differences

- A class can contain implementations of methods
 - A class can implement many interfaces, but only one class
-

Comparison



In order of “abstractness”:

Interface

- No method implementations

- Cannot be instantiated

Abstract class

- Some method implementations

- Cannot be instantiated

Non-abstract class

- All methods implemented can be instantiated

Interfaces



A class can implement multiple interfaces

The class must implement all methods in all interfaces listed in the header

```
class ManyThings implements  
    interface1, interface2  
{  
    // all methods of both interfaces  
}
```

Key differences between abstract class and interface.



The table below lists out the key differences between abstract class and interface.

Parameter	Abstract Class	Interface
Default Method Implementation	It can have default method implementation	Interfaces provide pure abstraction & can not have implementation at all
Variables	It may contain non-final variables.	Variables declared in an interface are by default final
Keyword Used	An abstract class can be extended using the keyword “extends	The interface should be implemented using keyword implements

Key differences between abstract class and interface.



Access Modifiers	Can have public, protected, private and default modifier	Interface methods are by default public. you can not use any other access modifier with it
Speed of Implementation	It is faster than the interface	An Interface is somewhat slower & require extra indirection
Normal Class	It can extend only one abstract class	Can implement multiple interfaces
Constructors	An abstract class can have constructors	An interface can not have constructors
Multiple Inheritance	An abstract class can extend another class and can implement multiple Java interfaces	The interface can extend another Java interface only



INTERFACE - Example

```
Import java.util.*;
interface Shape
{
    final float PI=3.1417f;
    abstract void DisplayArea();
}
class Circle implements shape
{
    float r,A;
    public void DisplayArea()
    {
        Scanner S=new Scanner(System.in);
```



INTERFACE - Example

```
r=S.nextFloat();
A=PI*r*r;
System.out.println("Area of the Circle "+A);
}
}
class Rectangle implements shape
{
float l,b,A;
public void DisplayArea()
{
    Scanner S=new Scanner(System.in);
    l=S.nextFloat();
    b=S.nextFloat();
```



INTERFACE - Example

```
A=l*r;
System.out.println("Area of the Rectangle "+A);
}
}
class Square implements shape
{
float a,A;
public void DisplayArea()
{
    Scanner S=new Scanner(System.in);
    a=S.nextFloat();
    A=a*a;
    System.out.println("Area of the Square "+A);
```

INTERFACE - Example

```
}  
}  
class DemoInterface  
{  
public static void main(String aa[])  
{  
    Circle C = new Circle();  
    C.DisplayArea();  
    Square S = new Square();  
    S.DisplayArea();  
    Rectangle R = new Rectangle();  
    R.DisplayArea();  
}
```

```
interface Drawable{  
void draw();  
}
```

//Implementation: by second user

```
class Rectangle implements Drawable{  
public void draw(){System.out.println("drawing rectangle");}  
}
```

```
class Circle implements Drawable{  
public void draw(){System.out.println("drawing circle");}  
}
```

//Using interface: by third user

class TestInterface1{

public static void main(String args[]){

**Drawable d=new Circle();//In real scenario, object is
provided by method e.g. getDrawable()**

d.draw();

}}



INTERFACE - Example

In the above example,

Shape() is an interface consisting of an abstract method **DisplayArea()** and a final data member **PI**.

The Shape interface is implemented in the classes such as **Circle**, **Rectangle** and **Square**.

The body of DisplayArea() is implemented according to the formula used for finding the area for various shapes (Circle, Rectangle and Square).

Area Circle = $PI * r * r$ [r-radius]

Area of Rectangle = $l * b$ [l-length and b-breadth]

and Area of Square = $a * a$ [a is the side]



INTERFACE-Key points

Here are the key points to remember about interfaces:

- 1) We can't instantiate an interface in java. That means we cannot create the object of an interface
- 2) Interface provides full abstraction as none of its methods have body. On the other hand abstract class provides partial abstraction as it can have abstract and concrete (methods with body) methods both.
- 3) implements keyword is used by classes to implement an interface.

INTERFACE-Key points



- 4) While providing implementation in class of any method of an interface, it needs to be mentioned as **public**.
- 5) Class that implements any interface must implement all the methods of that interface, else the class should be declared **abstract**.
- 6) All the interface methods are by default **abstract and public**.

INTERFACE - Advantages



Advantages of using interfaces are as follows:

1. Without bothering about the implementation part, we can achieve the security of implementation
2. In java, multiple inheritance is not allowed, however we can use interface to make use of it as we can implement more than one interfaces.

Nested Interfaces



An interface can be declared a member of a class or another interface.

Such an interface is called a *member interface* or a *nested interface*.

A nested interface can be declared as **public**, **private**, or **protected**.

This differs from a top-level interface, which must either be declared as **public** or use the default access level

When a nested interface is used outside of its enclosing scope, it must be qualified by the name of the class or interface of which it is a member.

Thus, outside of the class or interface in which a nested interface is declared, its name must be fully qualified.

Nested Interfaces



//This class contains a member interface.

```
class A {  
//this is a nested interface  
public interface NestedIF {  
boolean isNotNegative(int x);  
}  
}  
//B implements the nested interface.  
class B implements A.NestedIF {  
public boolean isNotNegative(int x) {  
return x < 0 ? false: true;  
}  
}
```

```
class NestedIFDemo {  
public static void main(String args[]) {  
//use a nested interface reference  
A.NestedIF nif = new B();  
if(nif.isNotNegative(10))  
System.out.println("10 is not  
negative");  
if(nif.isNotNegative(-12))  
System.out.println("this won't be  
displayed");  
}  
}
```


Extend Interfaces



One interface can inherit another by use of the keyword **extends**.

The syntax is the same as for inheriting classes.

When a class implements an interface that inherits another interface, it must provide implementations for all methods required by the interface inheritance chain.

```
//One interface can extend another.
```

```
interface A2 {  
void meth1();  
void meth2();  
}
```

```
//B now includes meth1() and meth2() -- it adds meth3().
```

```
interface B2 extends A2 {  
void meth3();  
}
```



Extend Interfaces

//This class must implement all of A
and B

```
class MyClass implements B2 {  
public void meth1() {
```

```
System.out.println("Implement  
meth1().");
```

```
}
```

```
public void meth2() {
```

```
System.out.println("Implement  
meth2().");
```

```
}
```

```
public void meth3() {
```

```
System.out.println("Implement  
meth3().");
```

```
}
```

```
}
```

```
class IFExtend {  
public static void  
main(String arg[]) {  
MyClass ob = new MyClass();
```

```
ob.meth1();
```

```
ob.meth2();
```

```
ob.meth3();
```

```
}
```

```
}
```

Default methods in interfaces



A default method lets you define a default implementation for an interface method.

In other words, by use of a default method, it is now possible for an interface method to provide a body, rather than being abstract.

A primary motivation for the default method was to provide a means by which interfaces could be expanded without breaking existing code.

Another motivation for the default method was the desire to specify methods in an interface that are, essentially, optional, depending on how the interface is used.

```
public interface MyIF {  
    // This is a "normal" interface method declaration.  
    // It does NOT define a default implementation.  
    int getNumber();  
  
    // This is a default method. Notice that it provides  
    // a default implementation.  
    default String getString() {  
        return "Default String";  
    }  
}
```

Default method



```
interface MyIF {  
int getNumber(); // This is a "normal" interface method declaration.  
default String getString() {  
return "Default String"; } // This is a default method.  
}  
  
class MyIFImp implements MyIF { // Implement MyIF.  
// Only getNumber() defined by MyIF needs to be implemented.  
// getString() can be allowed to default.  
public int getNumber() {  
return 100;  
}  
}  
  
class DefaultMethodDemo {  
public static void main(String args[]) {  
MyIFImp obj = new MyIFImp();  
System.out.println(obj.getNumber());  
// Can also call getString(), because of default  
// implementation:  
System.out.println(obj.getString());  
}}
```

A default method lets you define a default implementation for an interface method.

In other words, by use of a default method, it is now possible for an interface method to provide a body, rather than being abstract.

Multiple Inheritance Issues



Assume that two interfaces called Alpha and Beta are implemented by a class called MyClass.

What happens if both Alpha and Beta provide a method called `reset()` for which both declare a default implementation?

Is the version by Alpha or the version by Beta used by MyClass?

Or, consider a situation in which Beta extends Alpha. Which version of the default method is used?

Or, what if MyClass provides its own implementation of the method?

Multiple Inheritance Issues



A class implementation takes priority over an interface default implementation.

Second, in cases in which a class implements two interfaces that both have the same default method, but the class does not override that method, then an error will result.

In cases in which one interface inherits another, with both defining a common default method, the inheriting interface's version of the method takes precedence.

It is possible to explicitly refer to a default implementation in an inherited interface by using a new form of super.

Its general form is shown here:

InterfaceName.super.methodName()

For example, if Beta wants to refer to Alpha's default for reset(), it can use this statement: Alpha.super.reset();



Comparable Interface

- Provides an interface for comparing any two objects of same class.

- General Form :

Requires Type Casting

1. Un-Parameterized Form

```
public          interface      Comparable
{
    public          int      compareTo(Object o);
}
```

2. Parameterized Form

```
public          interface      Comparable<T>
{
    public          int      compareTo(<T> o);
}
```

- By implementing this interface , programmers can implement the logic for comparing two objects of same class for less than, greater than or equal to. Helps in Sorting.

Comparator Interface

- Also provides an interface for comparing any two objects of same class.
- But, the two objects that are to be compared have to be passed explicitly
- General Form :
 1. Un-parameterized Form (Requires Type Casting of Object Type Parameters)

```
public          interface      Comparator
{
    public          int          compare(Object first, Object second);
}
```

2. Parameterized Form

```
public          interface      Comparator<T>
{
    public          int          compare(T first, T second);
}
```

More on Comparable and Comparator interfaces in Lab session 2

Java Nested Class



It is possible to define a class within another class; such classes are known as *nested classes*.

The scope of a nested class is bounded by the scope of its enclosing class.

Thus, if class B is defined within class A, then B does not exist independently of A.

A nested class has access to the members, including private members, of the class in which it is nested.

However, the enclosing class does not have access to the members of the nested class.

A nested class that is declared directly within its enclosing class scope is a member of its enclosing class.

It is also possible to declare a nested class that is local to a block.

Java Nested Class



In Java, just like methods, variables of a class too can have another class as its member.

Writing a class within another is allowed in Java.

The class written within is called the **nested class**, and the class that holds the inner class is called the **outer class**.

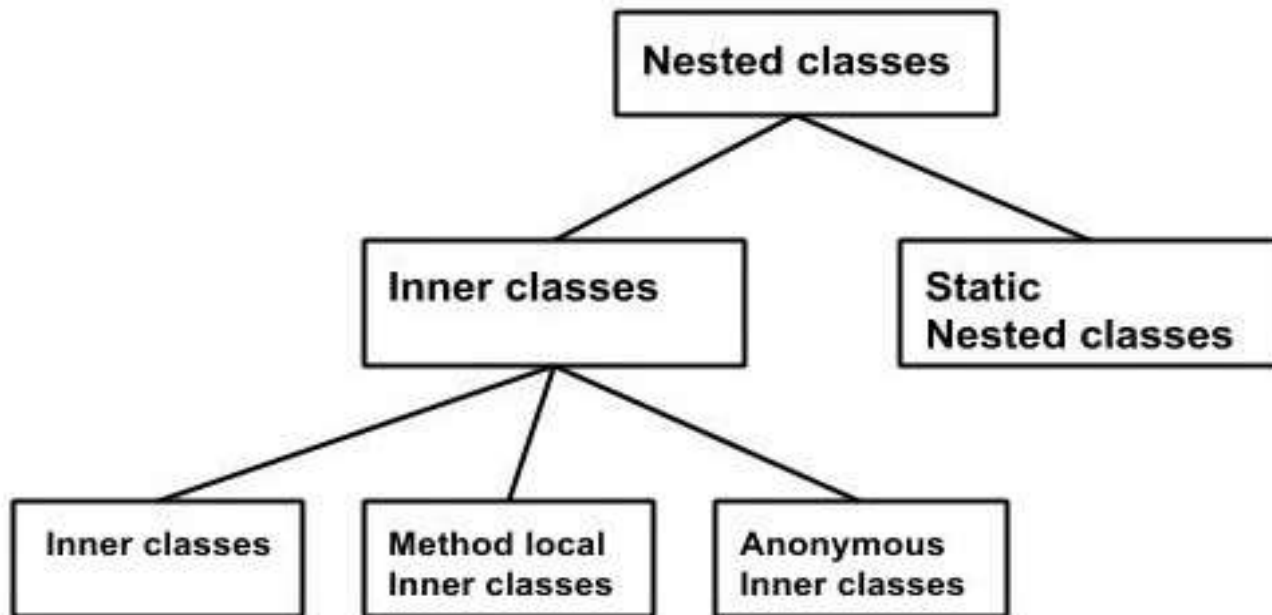
```
class Outer_Demo {  
    class Inner_Demo {  
    }  
}
```

Java Nested Class



Nested classes are divided into two types –

- **Non-static nested classes** – These are the non-static members of a class.
- **Static nested classes** – These are the static members of a class.



Nested Classes, Local Classes, Anonymous Classes



When to Use Nested Classes, Local Classes, Anonymous Classes

- **Local class:** Use it if you need to create more than one instance of a class, access its constructor, or introduce a new, named type (because, for example, you need to invoke additional methods later).
- **Anonymous class:** Use it if you need to declare fields or additional methods.
- **Nested class:** Use it if your requirements are similar to those of a local class, you want to make the type more widely available, and you don't require access to local variables or method parameters.
- Use a non-static nested class (or inner class) if you require access to an enclosing instance's non-public fields and methods. Use a static nested class if you don't require this access.

Thanks



BITS Pilani
Pilani Campus

Object Oriented Programming and Design

Presented by
Dr Ramya Devi M
mramyadevi@wilp.bits-pilani.ac.in
CS - 11 : 04th Oct 2025

IMP Note to Self





BITS Pilani
Pilani Campus



EXCEPTION HANDLING

Learning Objectives

- What is an Exception?
- Types of Exception
- Exception Handling (try and catch blocks)
- Usage of finally block
- Multiple catch blocks
- Usage of throws clause
- Creating our own exception

EXCEPTION



What is an exception?

- An Exception is an unwanted event that interrupts the normal flow of the program.
- When an exception occurs program execution gets terminated.
- In such cases we get a system generated error message.
- The good thing about Java is that exceptions can be handled.
- By handling the exceptions we can provide a meaningful message to the user about the issue rather than a system generated message, which may not be understandable to a user.

EXCEPTION



Why an exception occurs?

There can be several reasons that can cause a program to throw exception.

For example: Opening a non-existing file in our program,
Network connection problem, bad input data provided by user
etc.

Exception Handling

Exception handling is one of the most important feature of java programming that allows the users to handle runtime errors caused by exceptions.

If an exception occurs, which has not been handled by programmer then program execution gets terminated and a system generated error message is shown to the user.

Types of Errors



There are three categories of errors:

Syntax errors

- *Syntax errors* arise because the rules of the language have not been followed. They are detected by the compiler.

Runtime errors

- *Runtime errors* occur while the program is running if the environment detects an operation that is impossible to carry out.

Logic errors.

- *Logic errors* occur when a program doesn't perform the way it was intended to.

Runtime Errors



```
1      import java.util.Scanner;
2
3      public class ExceptionDemo {
4          public static void main(String[] args) {
5              Scanner scanner = new Scanner(System.in);
6              System.out.print("Enter an integer: ");
7              int number = scanner.nextInt();
8
9              // Display the result
10             System.out.println(
11                 "The number entered is " + number);
12         }
13     }
```

If an exception occurs on this line, the rest of the lines in the method are skipped and the program is terminated.

Terminated.

Exception-Handling Fundamentals



An *exception* is an abnormal condition that arises in a code sequence at run time

A Java exception is an object that describes an exceptional condition that has occurred in a piece of code

When an exceptional condition arises, an object representing that exception is created and *thrown* in the method that caused the error

An exception can be caught to handle it or pass it on

Exceptions can be generated by the Java run-time system, or they can be manually generated by your code

A program can be separated into a normal execution flow and an *exception execution flow*

Exception Handling



Java has a predefined set of exceptions and errors that can occur during execution

A program can deal with an exception in one of three ways:

- ignore it
- handle it where it occurs
- handle it in another place in the program

The manner in which an exception is processed is an important design consideration



Exception Types

All exception types are subclasses of the built-in class **Throwable**

The `java.lang.Throwable` class is the root class of Java Exception hierarchy.

`Throwable` has two subclasses, they are

- Exception (to handle exceptional conditions that user programs should catch)
 - An important subclass of Exception is **RuntimeException**, that includes division by zero and invalid array indexing
- Error (to handle exceptional conditions that are not expected to be caught under normal circumstances). i.e. stack overflow

An *error* is also represented as an object in Java, but usually represents a unrecoverable situation and should not be caught. corrective actions.

Difference between error and exception



Errors indicate that something severe enough has gone wrong, that the application should crash rather than try to handle the error.

Exceptions are events that occurs in the code.

A programmer can handle such conditions and take necessary corrective actions.

Advantages of exception handling



Exception handling ensures that the flow of the program doesn't break when an exception occurs.

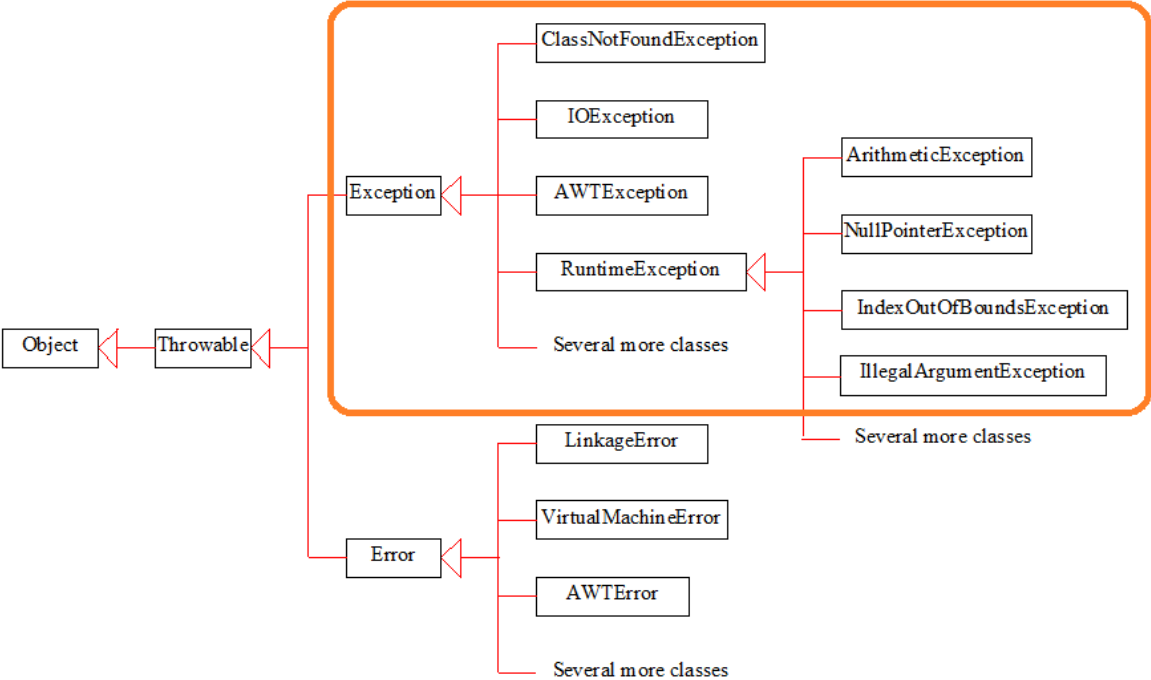
For example, if a program has bunch of statements and an exception occurs mid way after executing certain statements then the statements after the exception will not execute and the program will terminate abruptly.

By handling the exceptions, we make sure that all the statements execute and the flow of program doesn't break.

Exception Classes in Java



Exception describes errors caused by your program and external circumstances. These errors can be caught and handled by your program.



System Errors

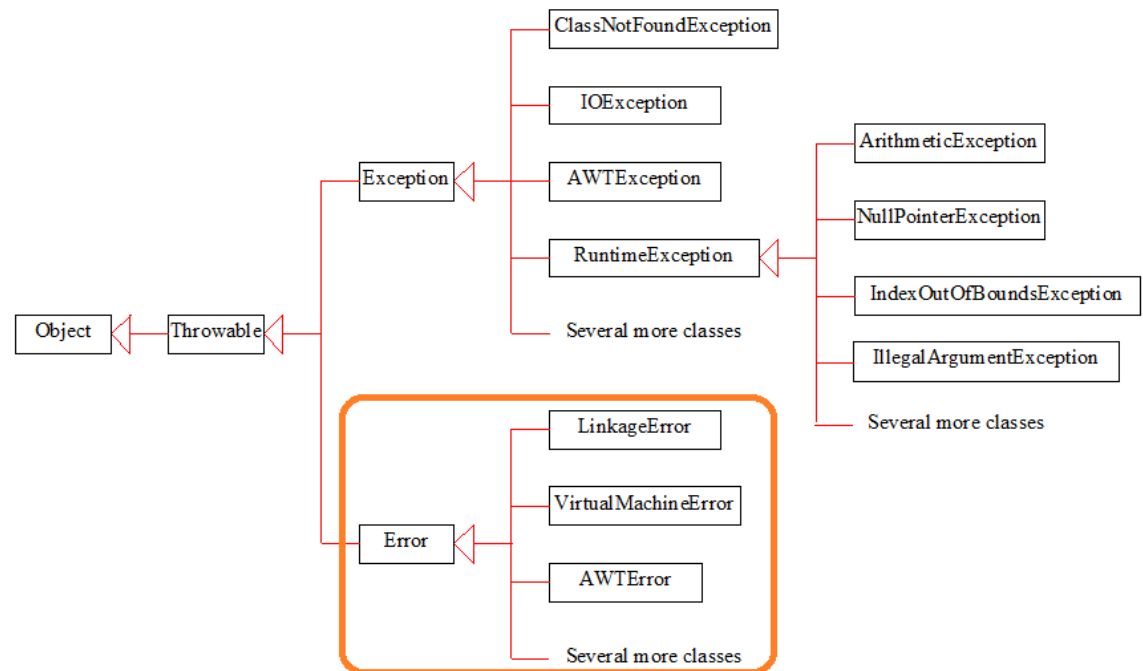


System errors are thrown by JVM and represented in the Error class.

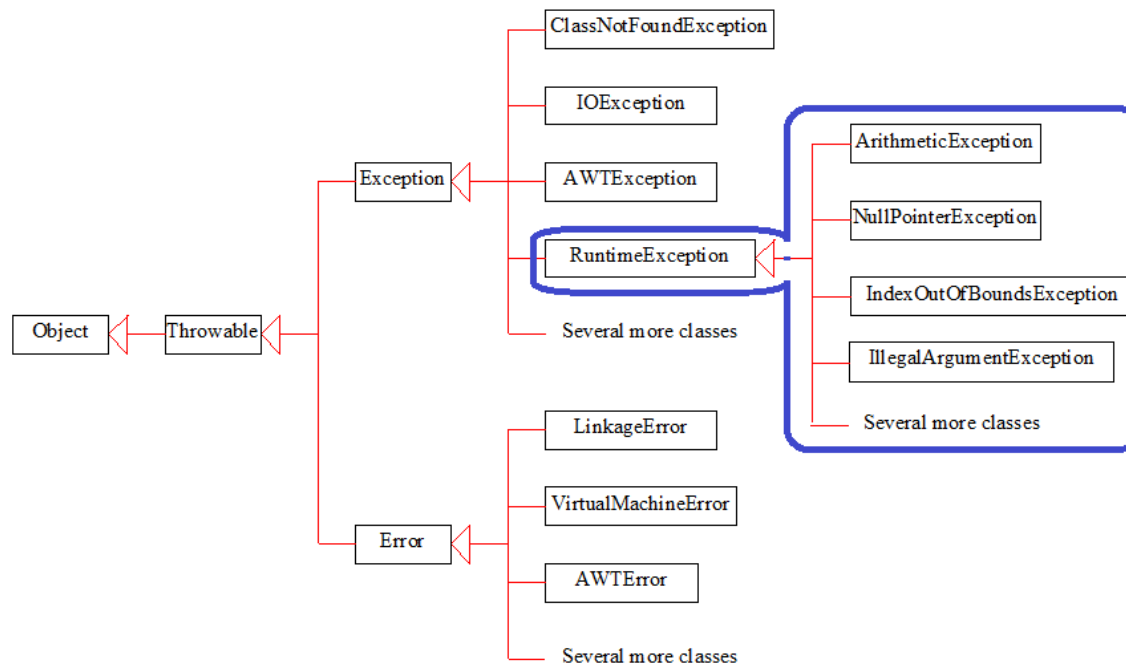
The Error class describes internal system errors.

Such errors rarely occur.

If one does, there is little you can do beyond notifying the user and trying to terminate the program gracefully.



Runtime Exceptions



RuntimeException is caused by programming errors, such as bad casting, accessing an out-of-bounds array, and numeric errors.



EXCEPTION - Examples

Few examples:

NullPointerException

- When we try to use a reference that points to null.

ArithmeticException

- When bad data is provided by user, for example, when we try to divide a number by zero this exception occurs because dividing a number by zero is undefined.

ArrayIndexOutOfBoundsException

- When we try to access the elements of an array out of its bounds, for example array size is 5 (which means it has five elements) and we are trying to access the 10th element.

Hierarchy of Java Exception classes



The `java.lang.Throwable` class is the root class of Java Exception hierarchy.

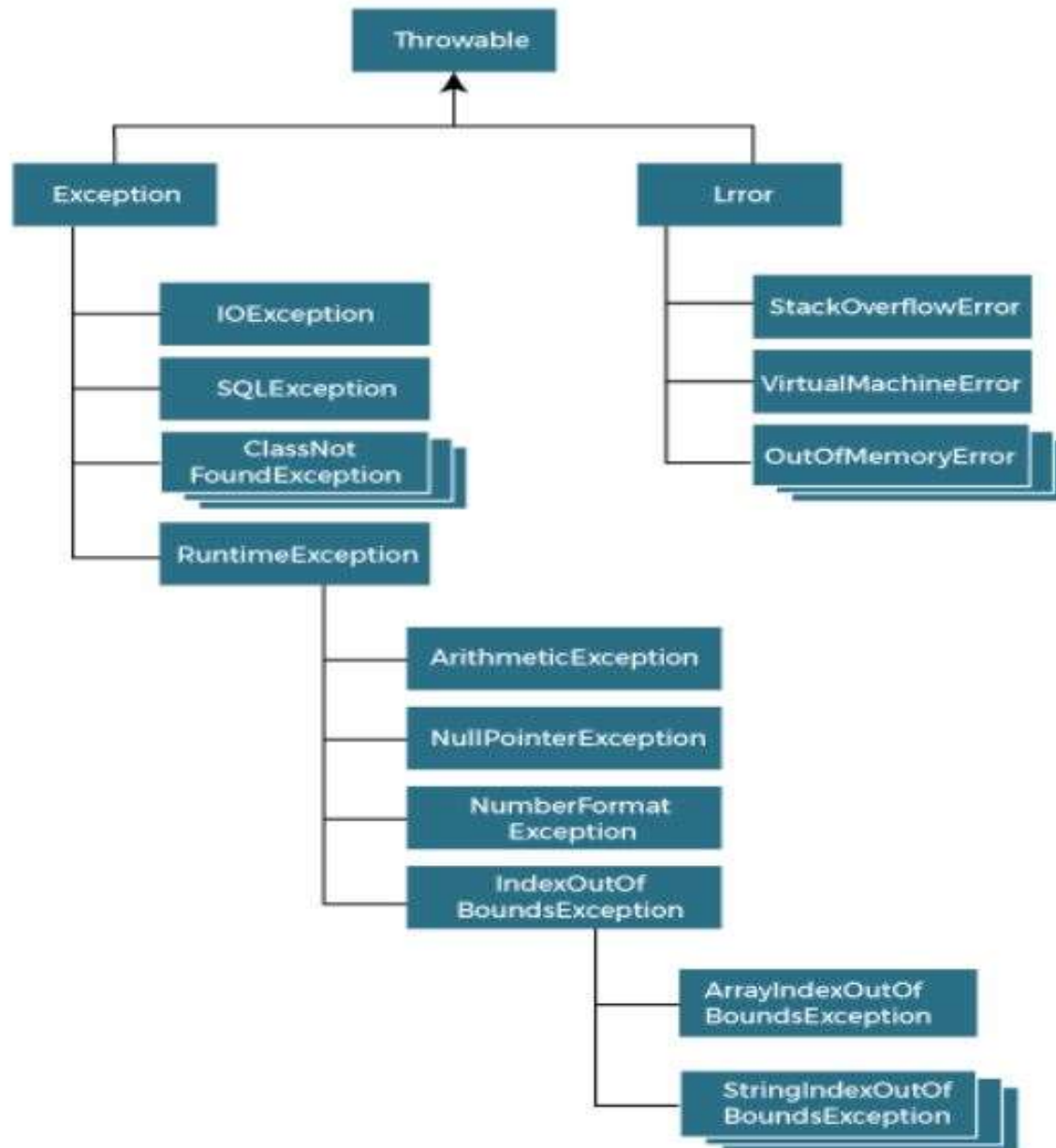
It is inherited by two subclasses:

Exception and

Error.

The hierarchy of Java Exception classes is given below:

Hierarchy of Java Exception classes





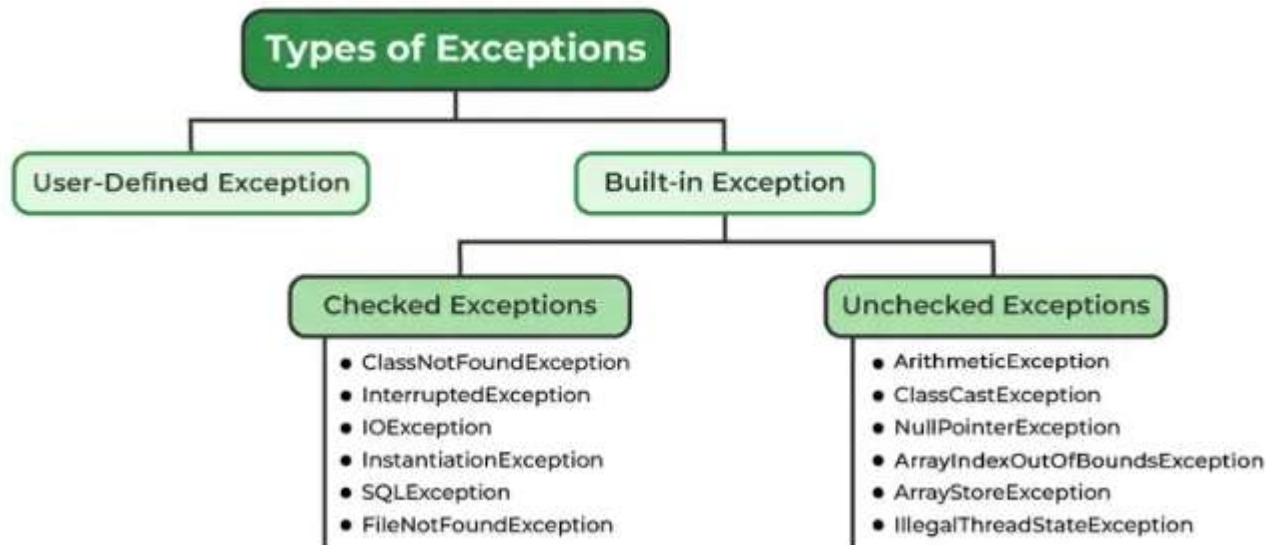
EXCEPTION - Types

Types of exceptions

There are two types of exceptions in Java:

1) **Checked exceptions** - Checked exceptions are exceptions that are checked at compile-time by the Java compiler. (FileNotFoundException)

2) **Unchecked exceptions** - Unchecked exceptions are exceptions that are not checked at compile-time by the Java compiler. (ArithmeticException)





EXCEPTION - Types

Checked exceptions

All exceptions other than Runtime Exceptions are known as Checked exceptions.

The compiler checks them during compilation to see whether the programmer has handled them or not.

If these exceptions are not handled/declared in the program, you will get compilation error.

For example, SQLException, IOException, ClassNotFoundException etc., if not handled in code generates compilation errors.

EXCEPTION



Unchecked Exceptions

Runtime Exceptions are also known as Unchecked Exceptions.

These exceptions are not checked at compile-time.

The compiler does not check whether the programmer has handled them or not.

It is the responsibility of the programmer to handle these exceptions and provide a safe exit.

For example, `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException` etc.

Compiler will never force us to catch such exception or force us to declare it in the method using `throws` keyword.



Unchecked Exceptions

In most cases, unchecked exceptions reflect programming logic errors that are not recoverable.

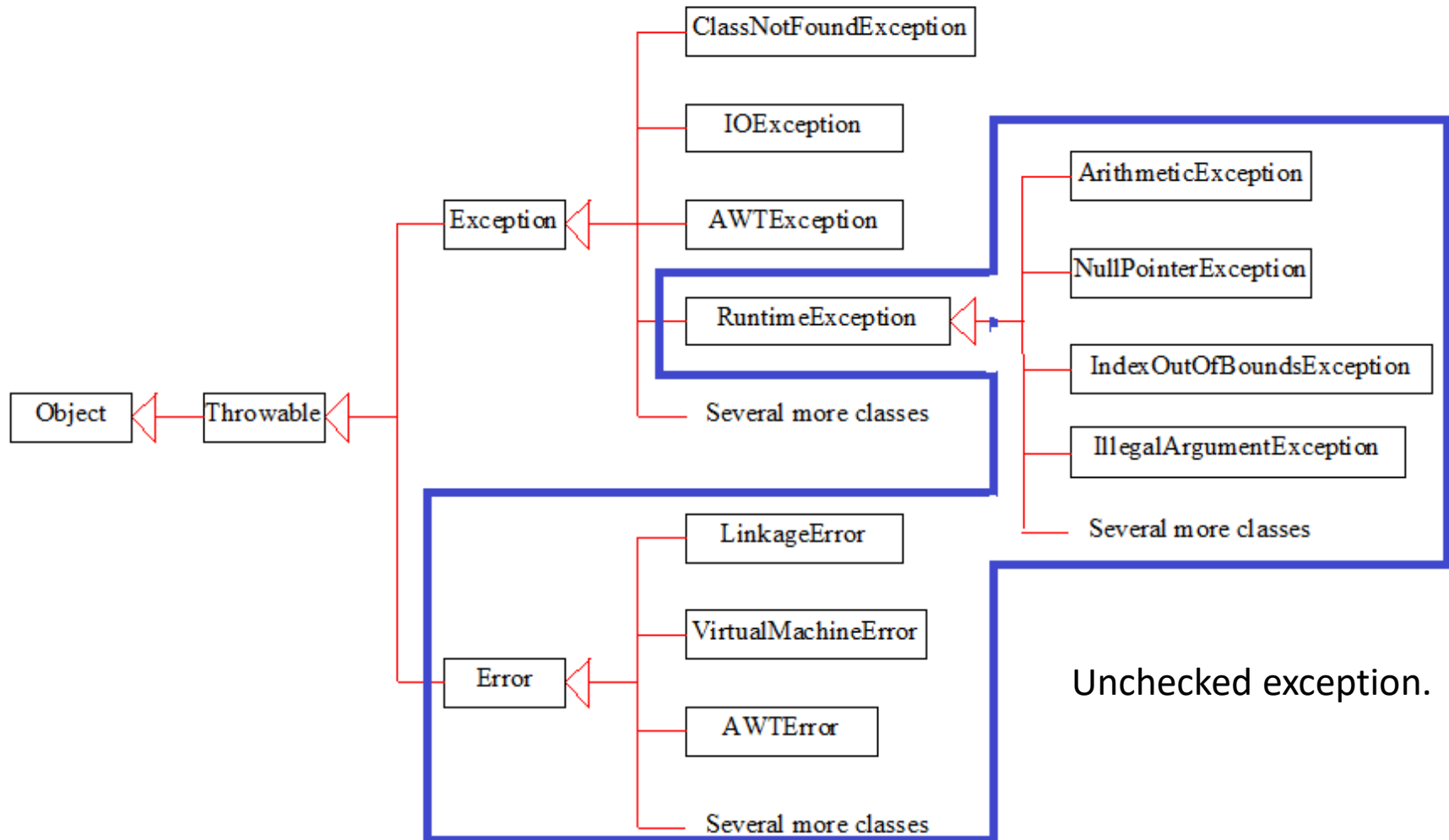
- For example, a `NullPointerException` is thrown if you access an object through a reference variable before an object is assigned to it;
- an `IndexOutOfBoundsException` is thrown if you access an element in an array outside the bounds of the array.

These are the logic errors that should be corrected in the program.

Unchecked exceptions can occur anywhere in the program.

To avoid cumbersome overuse of try-catch blocks, Java does not mandate you to write code to catch unchecked exceptions.

Checked or Unchecked Exceptions



Exception-Handling Fundamentals



Java exception handling is managed by via five keywords: **try**, **catch**, **throw**, **throws**, and **finally**

Program statements to monitor are contained within a **try** block

If an exception occurs within the **try** block, it is thrown

Code within **catch** block catch the exception and handle it

System generated exceptions are automatically thrown by the Java run-time system

To manually throw an exception, use the keyword **throw**

Any exception that is thrown out of a method must be specified as such by a **throws** clause

Two approaches for dealing with checked exceptions

Declare the exception in the method header (preferred)

- Catch the exception



Using try and catch

A **try** and **catch** statement form a unit. The scope of the **catch** clause is restricted to those statements specified by the immediately preceding **try** statement

```
import java.util.Random;

class HandleError {
    public static void main(String args[]) {
        int a=0, b=0, c=0;
        Random r = new Random();

        for(int i=0; i<10; i++) {
            try {
                b = r.nextInt();
                c = r.nextInt();
                a = 12345 / (b/c);
            } catch (ArithmeticException e) {
                System.out.println("Division by zero.");
                a = 0; // set a to zero and continue
            }
            System.out.println("a: " + a);
        }
    }
}
```

Using try and catch



Handling an exception has two benefits,

- It allows you to fix the error
- It prevents the program from automatically terminating

The **catch** clause should follow immediately the **try** block

Once an exception is thrown, program control transfer out of the **try** block into the catch block

Once the **catch** statement has executed, program control continues with the next line in the program following the entire **try/catch** mechanism

try – catch block



The try statement allows us to define a block of code to be tested for errors while it is being executed.

The catch statement allows you to define a block of code to be executed, if an error occurs in the try block.

The statements try and catch occur in pair.

The Syntax for try block is

```
try
{
    Block for try // Include the statement in which we expect the error
}
catch(Exception e)
{
    Block for catch // Remedial statements for the error
}
```

Example



Consider the following example:

```
class DemoException
{
    public static void main(String aa[] )
    {
        int [ ] a = {10,20,30};
        System.out.println(a[5]);
    }
}
```

EXCEPTION



It displays an error message as follows:

Exception in thread “main”

java.lang.ArrayIndexOutOfBoundsException : 5 at

DemoException (DemoException.java: 4)

If an error occurs, we can use **try...catch** to catch the error and execute some code to handle it.

Note : We can include a finally block after catch block.

The **finally** block lets us to execute code, after **try...catch**, regardless of the result:

Handling the exception using try – catch block



```
class DemoException
{
    public static void main(String aa[] )
    {
        int [ ] a = {10,20,30};
        try
        {
            System.out.println(a[5]);
        }
        catch(Exception e)
        {
            System.out.println(" Index out of Boundary, it must lie between 0
to 9");
        }
    }
}
```

The output will be: Index out of Boundary, it must lie between 0 to 9

Example – Using try-catch block



```
class Exc2 {  
    public static void main(String args[]) {  
        int d, a;  
  
        try { // monitor a block of code.  
            d = 0;  
            a = 42 / d;  
            System.out.println("This will not be printed.");  
        } catch (ArithmeticException e) { // catch divide-by-zero error  
            System.out.println("Division by zero.");  
        }  
        System.out.println("After catch statement.");  
    }  
}
```

Output:

Division by zero.

After catch statement.

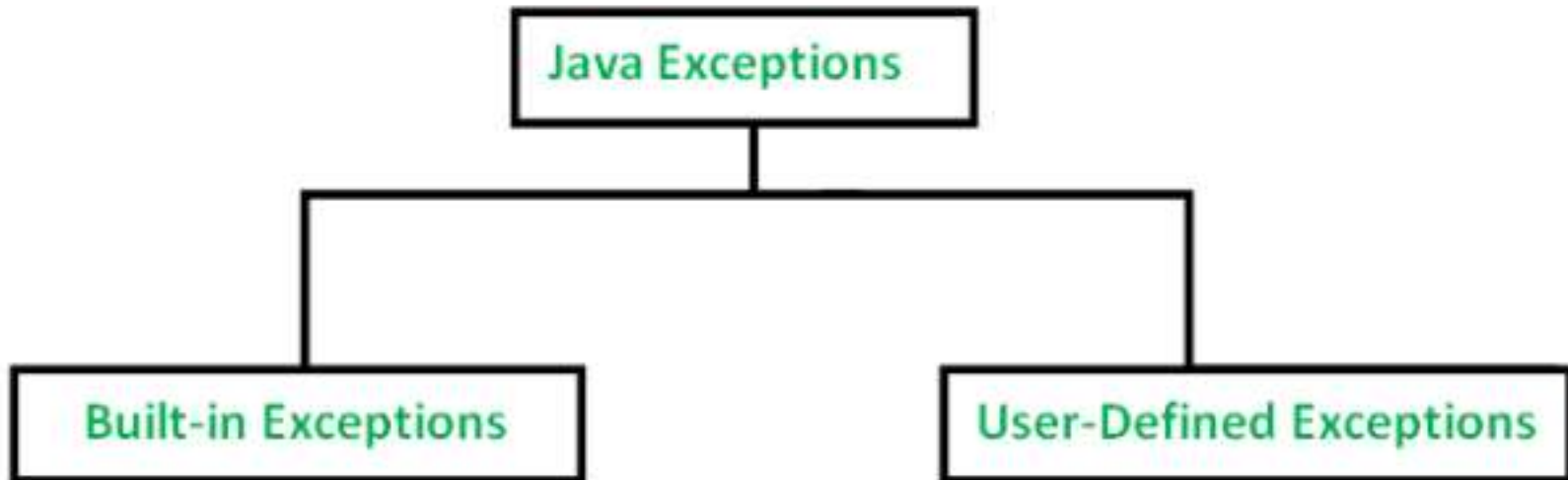
Catch Runtime Errors



```
1      import java.util.*;
2
3      public class HandleExceptionDemo {
4          public static void main(String[] args) {
5              Scanner scanner = new Scanner(System.in);
6              boolean continueInput = true;
7
8              do {
9                  try {
10                     System.out.print("Enter an integer: ");
11                     int number = scanner.nextInt();
12
13                     // Display the result
14                     System.out.println(
15                         "The number entered is " + number);
16
17                     continueInput = false;
18                 }
19                 catch (InputMismatchException ex) {
20                     System.out.println("Try again. (" +
21                         "Incorrect input: an integer is required)");
22                     scanner.nextLine(); // discard input
23                 }
24             } while (continueInput);
25         }
```

If an exception occurs on this line,
the rest of lines in the try block are
skipped and the control is
transferred to the catch block.

Types of Exception in Java with Examples



The finally block



Any code that absolutely must be executed before a method returns is put in a **finally** block

General form of an exception-handling block

```
try{
    // block of code to monitor for errors
}
catch (ExceptionType1 exOb){
    // exception handler for ExceptionType1
}
catch (ExceptionType2 exOb){
    // exception handler for ExceptionType2
}
//...
finally{
    // block of code to be executed before try block ends
}
```


The finally block



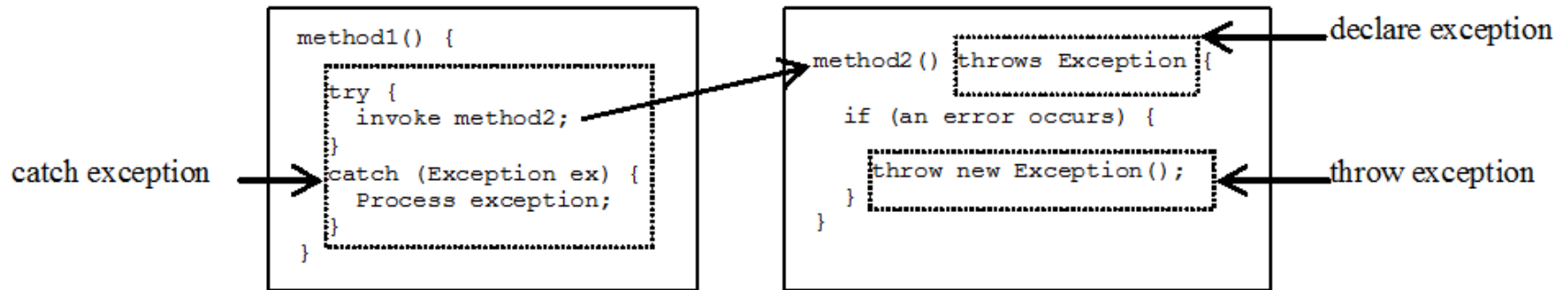
```
public class FinallyExample {  
    public static void main(String[] args) {  
        try {  
            System.out.println("Inside try block");  
            int result = 10 / 0;  
            System.out.println("This line will not execute");  
        } catch (ArithmeticException e) {  
            System.out.println("Exception caught: " + e.getMessage());  
        } finally {  
            System.out.println("Finally block executed");  
        }  
        System.out.println("Program continues...");  
    }  
}
```

Example Without Exception:



```
public class FinallyExampleNoException {  
    public static void main(String[] args) {  
        try {  
            System.out.println("Inside try block");  
            int result = 10 / 2;  
            System.out.println("Result: " + result);  
        } finally {  
            System.out.println("Finally block executed");  
        }  
  
        System.out.println("Program continues...");  
    }  
}
```

Declaring, Throwing, and Catching Exceptions



throw keyword



Usage of the throw keyword

The throw statement allows us to create a custom error.

The throw statement is used together with an **exception type**.

There are many exception types available in Java:

ArithmeticException, **FileNotFoundException**,
ArrayIndexOutOfBoundsException, **SecurityException**, etc:

Example

Throw an exception if **salary** is above 50,000 (print "No Bonus").

If salary is lesser than or equal to 50,000, print "You Will get Bonus":

throw - Example



It displays the error message as

```
Exception in thread "main" java.lang.ArithmeticException :  
No Bonus at ExceptionDemo.check (ExceptionDemo.java:4_ at  
ExceptionDemo.main (ExceptionDemo.java:12)
```

throws keyword



Use of throws Keyword

The keyword “throws” in the method header is used to specify that the method causes an exception so that the caller who calls this method can guard themselves against that exception.

The Syntax for using throws keyword is

```
Return Type method name (parameter list) throws  
                                Exception name  
{  
    Body of the method  
}
```

throws keyword



Execution of the above code

CASE 1: Let us assume that the input is given as 28,

The output is

You are allowed to work...

CASE 2: Let us assume that the input is given as 70

There is no output and it displays the error message as

Catching : `java.lang.ArithmeticException :`

Invalid Age to work ...

Multiple catch blocks



In some cases , a single code may raise more than one exceptions.

To handle this situation, we can specify two or more catch blocks, each catching a different type of exception.

The following example demonstrates the use of multiple catch blocks for handling different types of exceptions.

CUSTOMIZED EXCEPTIONS



CREATING OUR OWN EXCEPTION

Java provides rich set of built-in exception classes like:

`ArithmeticException`, `IOException`, `NullPointerException` etc.

which are available in the `java.lang` package and used in exception handling.

These exceptions are already set to trigger on pre-defined conditions such as when you divide a number by zero it triggers `ArithmeticException`.

Apart from these classes, Java allows us to create our own exception class to provide own exception implementation.

CUSTOMIZED EXCEPTIONS



These type of exceptions are called user-defined exceptions or **custom** exceptions.

We can create our own exception simply by extending java **Exception** class.

We can define a constructor for our Exception (not compulsory) and we can override the **toString()** function to display your customized message on catch.

Nested try Statements

A **try** statement can be inside the block of another try

Each time a **try** statement is entered, the context of that exception is pushed on the stack

If an inner **try** statement does not have a catch, then the next **try** statement's catch handlers are inspected for a match

If a method call within a **try** block has **try** block within it, then then it is still nested **try**

Thanks



BITS Pilani
Pilani Campus

Object Oriented Programming and Design

Presented by
Dr Ramya Devi M
mramyadevi@wilp.bits-pilani.ac.in
CS - 12 : 11th Oct 2025



BITS Pilani
Pilani Campus



COLLECTIONS

Learning Objectives

- **Collection Framework**
 - (i) Definition and Hierarchy
 - (ii) Benefits
- **Collection Interface**
- **ArrayList implementation**
- **Linked List implementation**
- **Comparison between Array List and LinkedList**
- **Stack**

Collections Framework



The java.util package contains one of the most powerful subsystems called Collections.

A **collection** is a group of objects.

It is sometimes called as a container which is used to store, retrieve, manipulate and communicate aggregated data.

A Collection Framework is an unified architecture to represent and manipulate collections and it contains the following:

(i) Interfaces – Abstract data types that allow collections to be manipulated independently of the details of their representation

(i) Implementation – These are the concrete implementation of collection interface.

They are actually reusable data structures

Collections Framework

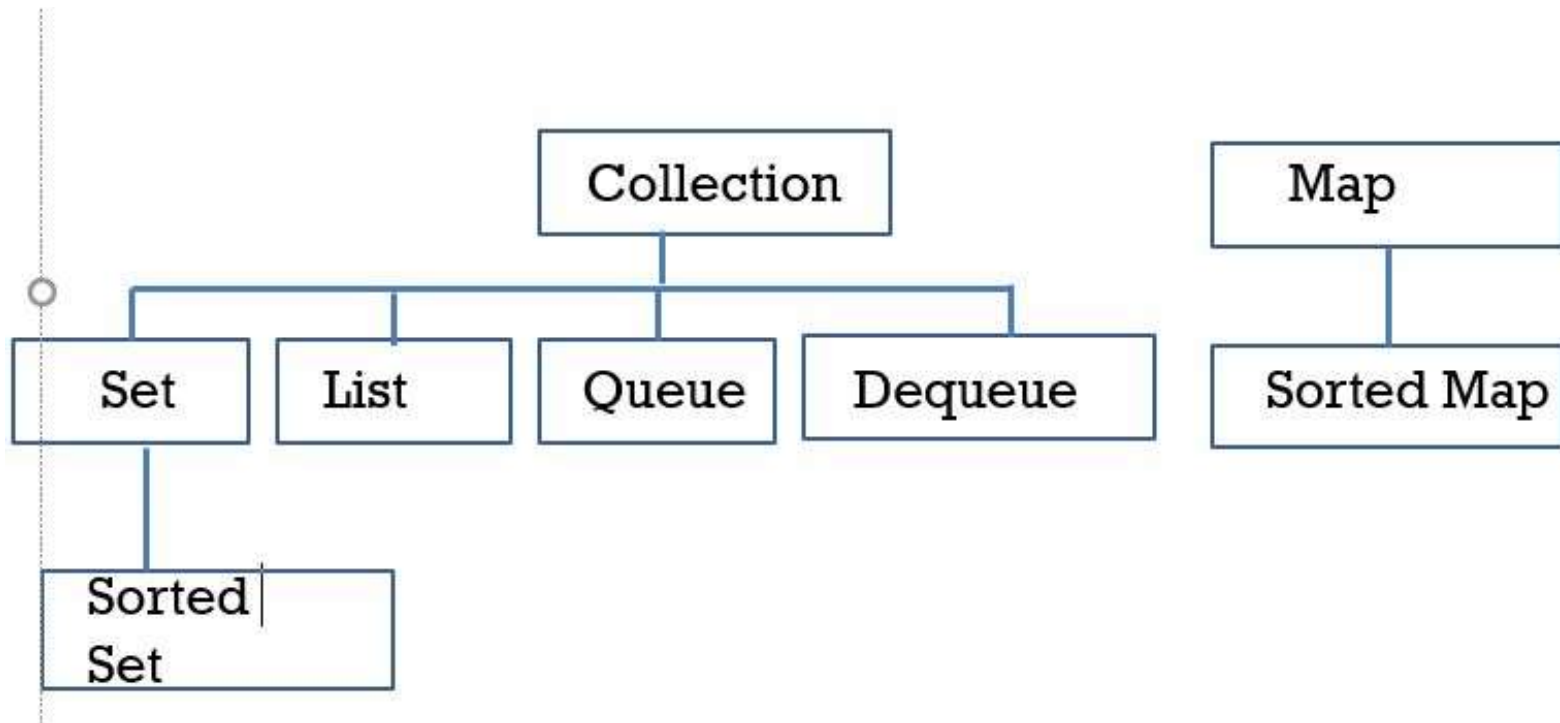


(iii) Algorithms – These are the methods that perform useful computations such as searching and sorting data in objects by implementing collection interfaces.

Benefits

- Reduces Programming - Provides useful data structures and algorithms
- Increases Programming Speed and quality – Provides high performance and high quality implementations.
- Because we are freed from writing our own data structures, we have more time to develop quality programs.
- Software Reuse – Users can include their own new algorithms in the existing data structures by means of reuse property

Collections - Interface



Collections - Interface



Collection - It is the root of the collection hierarchy

Set – It is a collection that does not contain duplicate elements

List – It can contain duplicate elements. It is an ordered collection (Sometimes called a Sequence)

Queue – It is FIFO (First in First out) data structure in which element is inserted in an end called as Rear and the element is deleted at another end called as Front.

Stack – It is LIFO (Last in First out) data structure in which element is inserted and deleted in a same end called as Top.

Deque - It can be used as both LIFO and FIFO

Map – It is an object that maps keys to values. A map can't contain duplicate keys.

Sorted Set – It is a set that maintains its elements in ascending order

Collections - Interface



Sorted Map - It is a map that maintains its mapping in ascending key order.

The List Interface

This is a child interface of the collection interface. This interface is dedicated to the data of the list type in which we can store all the ordered collection of the objects. This also allows duplicate data to be present in it. This list interface is implemented by various classes like ArrayList, Vector and Stack. etc. Since all the subclasses implement the list, we can instantiate a list object with any of these classes. For example,

```
List <T> al = new ArrayList<> ();
```

```
List <T> ll = new LinkedList<> ();
```

```
List <T> v = new Vector<> ();
```

Where T is the type of the object

The classes which implement the List interface are as follows:

Collections - ArrayList



1. ARRAYLIST

- ArrayList provides us with dynamic arrays in Java.
- Though, it may be slower than standard arrays but can be helpful in programs where lots of manipulation in the array is needed.
- The size of an ArrayList is increased automatically if the collection grows or shrinks i.e. if the objects are added or removed from the collection.
- Java ArrayList allows us to randomly access the list.
- ArrayList can not be used for primitive types, like int, char, etc.
- We will need a wrapper class for such cases.

Some of the methods in array list are listed below:

Collections - ArrayList



Method	Description
boolean add(Collection c)	Appends the specified element to the end of a list.
void add(int index, Object element)	Inserts the specified element at the specified position.
void clear()	Removes all the elements from this list.
int lastIndexOf(Object o)	Return the index in this list of the last occurrence of the specified element, or -1 if the list does not contain this element.
void trimToSize()	Trims the capacity of this ArrayList instance to be the list's current size.

Let us understand Array list with a programmatic example:



How ArrayList Works Internally

- **Initial Capacity:** When you create an ArrayList, it has an initial capacity (default is 10 if not specified). This means the internal array can hold up to 10 elements before it needs to grow.
- **Dynamic Growth:** If you add more elements than the current capacity, the ArrayList automatically increases its capacity, typically by 1.5 times the current capacity.
- **Unused Capacity:** After adding and removing elements, the list might have unused capacity—extra space in the internal array that is not used by the current elements. This extra capacity is kept to minimize resizing operations during future additions.



Why Use `trimToSize()`?

When you know that:

- No more elements will be added to the `ArrayList`.
- You want to free up memory used by the unused capacity.

You can call `trimToSize()` to shrink the internal array to match the current number of elements (`size()`).

When Should You Use It?

1. **Memory Optimization:** If your program creates large `ArrayLists` that no longer need to grow, `trimToSize()` helps reduce memory usage by eliminating unused capacity.
2. **Finalizing a List:** If an `ArrayList` is no longer going to change, and you want to optimize it before passing it to another part of your program.

Collections - ArrayList



```
import java.util.*;
class ArrayListDemo
{
    public class void main(String s[ ] )
    {
        ArrayList A = new ArrayList( ); // Creating an Array
        A.add("Sachin");
        A.add("Dhoni");
        Iterator it = A.iterator( );
        while(it.hasNext( ) )
        {
            System.out.println("\n Element "+it.next( ) );
        }
    }
}
```

Iterator Methods



hasNext():

- Returns true if there are more elements to iterate over.
- Otherwise, it returns false.

next():

- Returns the next element in the iteration.
- Throws a NoSuchElementException if no more elements exist.

remove():

- Removes the last element returned by the next() method from the collection.
- Can only be called once per call to next().
- Throws IllegalStateException if called without a preceding next().

Collections - ArrayList



The output of the above code is:

Element : Sachin

Element : Dhoni

In the code,

- *An Iterator* is an object that iterates over the elements in a collection.
- The `java.util.Iterator` is an interface specifying the capabilities of an iterator.
- If we call method of `Iterator()` object, it returns an iterator object that implements `Iterator` and knows how to step through the objects in the underlying collection.

Iterator interface specifies the following methods:

- **hasNext()** – Returns true if there are more elements in the collection. Otherwise, it returns False.

Collections – Linked List



- **next ()** - Returns the next element
- **Remove()** - Removes from the collection the last element returned by the iterator

2. LinkedList Class - LinkedList class is an implementation of the LinkedList data structure which is a linear data structure where the elements are not stored in contiguous locations and every element is a separate object with a data part and address part. The elements are linked using pointers and addresses. Each element is known as a node.

Why do we need a Linked List?

We know that **arrays have certain limitations such as:**

1) **Size of the array is fixed** which is decided when we create an array so it is hard to predict the number of elements in advance, if the declared size fall short then we

Collections – Linked List



cannot increase the size of an array and if we declare a large size array and do not need to store that many elements then it is a waste of memory.

2) Array elements **need contiguous memory locations** to store their values.

3) **Inserting an element in an array is performance wise expensive** as we have to shift several elements to make a space for the new element. For example:

Let's say we have an array that has following elements: 10, 12, 15, 20, 4, 5, 100, now if we want to insert a new element 99 after the element that has value 12 then we have to shift all the elements after 12 to their right to make space for new element. Similarly **deleting an element** from the array is also a performance wise expensive operation because all the elements after the deleted element have to be shifted left.

Collections – Linked List



These limitations are handled in the Linked List by providing following features:

1. Linked list allows **dynamic memory allocation**, which means memory allocation is done at the run time by the compiler and we do not need to mention the size of the list during linked list declaration.
2. Linked list elements **don't need contiguous memory locations** because elements are linked with each other using the reference part of the node that contains the address of the next node of the list.
3. Insert and delete operations in the Linked list are not performance wise expensive because adding and deleting an element from the linked list doesn't require element shifting, only the pointer of the previous and the next node requires change.

Collections – Linked List



Methods in LinkedList

1. Boolean **add(Object item)**: It adds the item at the end of the list
2. Void **add(int index, Object item)**: It adds an item at the given index of the the list
3. Void **addFirst(Object item)**: It adds the item (or element) at the first position in the list
4. Void **addLast(Object item)**: It inserts the specified item at the end of the list
5. Void **clear()**: It removes all the elements of a list
6. Object **get(int index)**: It returns the item of the specified index from the list
7. Object **getFirst()**: It fetches the first item from the list
8. Object **getLast()**: It fetches the last item from the list

Collections – Linked List



- 9. **Object remove()**: It removes the first element of the list.
- 10. **Object remove(int index)**: It removes the item from the list which is present at the specified index
- 11. **Object remove(Object obj)**: It removes the specified object from the list.
- 12. **Object removeFirst()**: It removes the first item from the list.
- 13. **Object removeLast()**: It removes the last item of the list.

```
import java.util.*;  
public class JavaExample  
{  
    public static void main(String args[])  
{
```


Collections – Linked List



```
LinkedList<String> list=new LinkedList<String>();  
  
//Adding elements to the Linked list  
  
list.add("Kohli");  
list.add("Laxman");  
list.add("Sachin");  
list.add("Dhoni");  
list.add("Dravid");  
//Adding an element to the first position  
list.addFirst("Ganguly");  
//Adding an element to the last position  
list.addLast("Pant");  
//Adding an element to the 3rd position  
list.add(2, "Jadeja");
```

Collections – Linked List



```
//Removing First element
```

```
//Same as list.remove(0);
```

```
list.removeFirst();
```

```
//Removing Last element
```

```
list.removeLast();
```

```
//Iterating LinkedList
```

```
Iterator<String> iterator=list.iterator();
```

```
while(iterator.hasNext()){
```

```
    System.out.println("Name: "+iterator.next()+" ");
```

```
}
```

Collections – Linked List



```
//removing 2nd element, index starts with 0
```

```
list.remove(1);
```

```
System.out.print("\nAfter removing second element: ");
```

```
//Iterating LinkedList again
```

```
Iterator<String> iterator2=list.iterator();
```

```
while(iterator2.hasNext()){
```

```
    System.out.print("Name: "+iterator2.next());
```

```
}
```

```
}
```

```
}
```

Collections – Linked List



The output of the above code is

Name : Kohli

Name : Laxman

Name : Jadeja

Name : Sachin

Name : Dhoni

Name: Dravid

After removing second element:

Name : Kohli

Name : Laxman

Name : Sachin

Name : Dhoni

Name: Dravid

Collections - Vector



Vector class

Vector is a type of data structure that maintains insertion order, that is elements are retrieved in same order as they are added into it. It is very much similar to ArrayList.

The methods in Vector Class

1. **boolean add(E e)** - This method adds element at the end of the vector
2. **void add(int index, E element)** - This method adds an element at specified index and moves the whole elements one step in forward direction
3. **void addElement(E obj)** - This method adds the specified element at the end of the vector also increasing its size by 1
4. **int capacity()** - This method gives the capacity of the vector

Collections - Vector



5. **Object elementAt(int index)** - This method gives specific element at index mentioned in the argument
6. **boolean equals(Object o)** - This method checks the equality of object with this vector
7. **Object firstElement()** - This method returns the first element at index 0
8. **Object get(int index)** - This method retrieves an element at specified index
9. **int indexOf(Object o)** - This method retrieves the index of the first occurrence of the specified element in the vector, or -1 if that element is not present
10. **void insertElementAt(Object o, int index)** - This method inserts the specified element at specified position and moves all the elements one step further

Collections - Vector



11. **Object lastElement()** - This method returns the last element of the vector
12. **Object remove(int index)** - This method removes an element at specified index
13. **Object set(int index, Object element)** - This method replaces the element with the element specified in the method at the specified index
14. **int size()** - This method gives the number of elements in the vector,
15. **List subList(int fromIndex, int toIndex)** - This method returns group of elements between fromIndex(inclusive) and toIndex(exclusive)

Collections - Vector



(iii) Example program for removing elements

```
import java.util.*;
import java.io.*;
class RemovingElementsFromVector {
    public static void main(String[] arg)
    {

        // create default vector of capacity 10
        Vector v = new Vector();
        // Add elements using add() method
        v.add(1);
        v.add(2);
        v.add("Jave");
        v.add("forAll");
        v.add(4);
```


Collections - Vector



```
// removing first occurrence element at 1
    v.remove(1);
    // checking vector
    System.out.println("After removal: " + v);
}
}
```

The output of the above code is

After removal: [1, Java, forAll, 4]

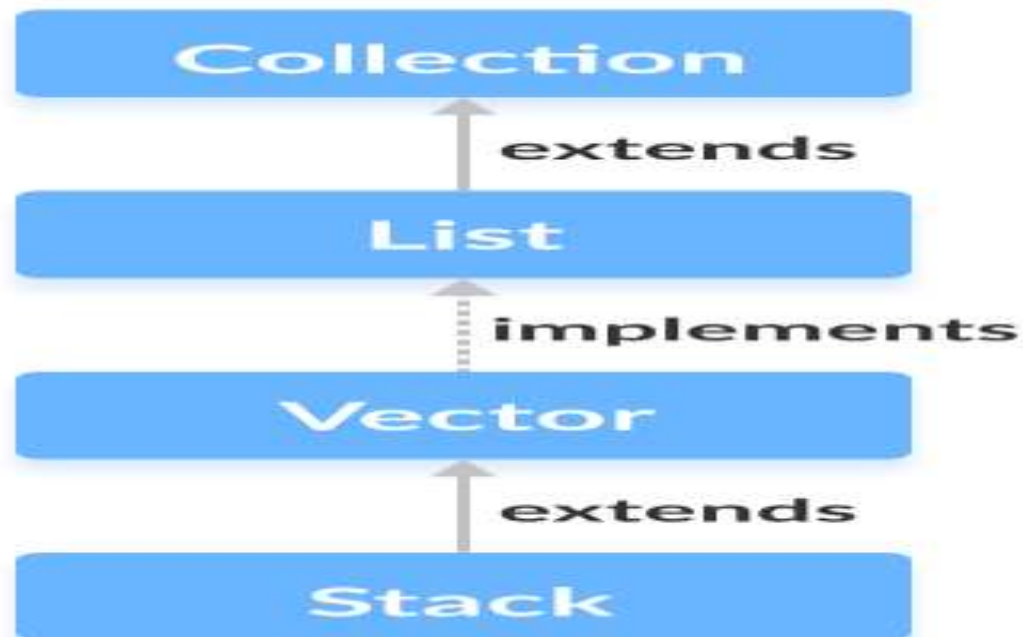
Collections - Stack



Java Stack Class

The Java collections framework has a class named Stack that provides the functionality of the stack data structure.

The Stack class extends

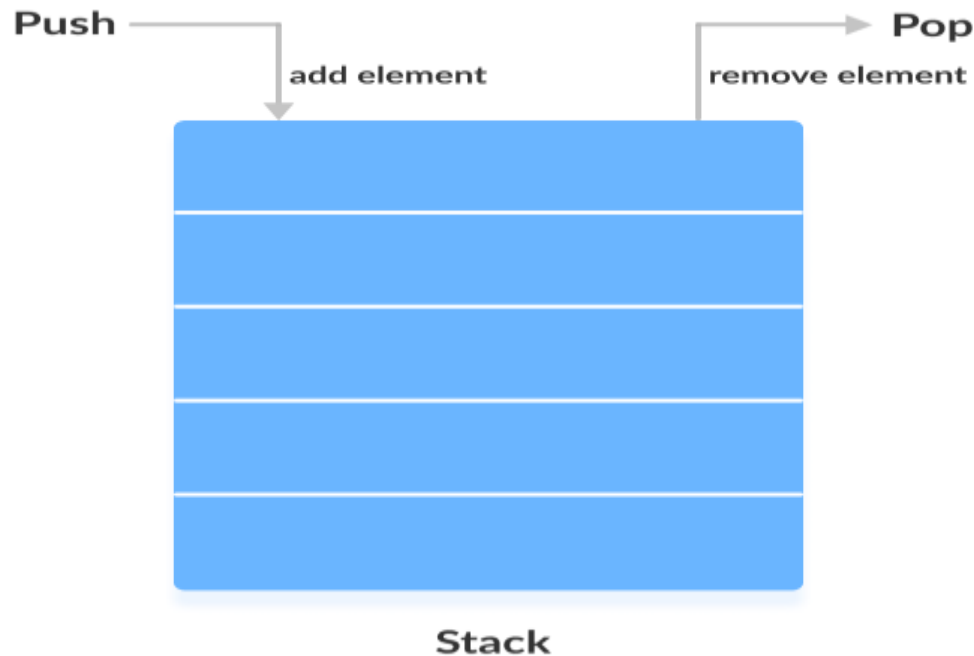


Collections - Stack



Stack Implementation

In stack, elements are stored and accessed in **Last In First Out** manner. That is, elements are added to the top of the stack and removed from the top of the stack.



Collections - Stack



Creating a Stack

In order to create a stack, we must import the `java.util.stack` package first. Once we import the package, here is how we can create a stack in Java.

```
Stack <type> S = new Stack< > ();
```

Examples:

```
Stack <Integer> S = new Stack< > (); // Creating Integer Stack
```

```
Stack <String> S = new Stack< > (); // Creating String Stack
```

Stack Methods

Since Stack extends Vector class, it inherits all the methods of Vector. Besides these methods, the Stack class includes 5 more methods that distinguish it from Vector.

Collections - Stack



Stack-Specific Methods:

- `push(E item)`
- `pop()`
- `peek()`
- `empty()`
- `search(Object o)`

Collections - Stack



Methods Inherited from Vector:

Since Stack extends Vector, it also inherits several methods from the Vector class, including:

- `add(E e)`
- `addElement(E obj)`
- `size()`
- `clear()`
- `get(int index)`
- `indexOf(Object o)`
- `remove(int index)`
- `removeElement(Object obj)`
- `toArray()`

Collections - Stack



1. **push()** Method

To add an element to the top of the stack, we use the `push()` method.

2. **pop()** Method

To remove an element from the top of the stack, we use the `pop()` method.

3. **search()** Method

To search an element in the stack, we use the `search()` method.
It returns the position of the element from the top of the stack.

empty() Method

To check whether a stack is empty or not, we use the `empty()` method.

Collections - HashMap



HashMap in Java with Example

HashMap is a Map based collection class that is used for storing Key & value pairs, it is denoted as `HashMap<Key, Value>` or `HashMap<K, V>`. This class makes no guarantees as to the order of the map. It is similar to the Hashtable class except that it is unsynchronized and permits nulls (null values and null key).

It is not an ordered collection which means it does not return the keys and values in the same order in which they have been inserted into the HashMap. It does not sort the stored keys and Values. You must need to import `java.util.HashMap` or its super class in order to use the HashMap class and methods.

Collections - HashMap



HashMap Class Methods

Here is the list of methods available in HashMap class. I have also covered examples using these methods at the end of this post.

- 1. void clear():** It removes all the key and value pairs from the specified Map.
- 2. boolean containsKey(Object key):** It is a boolean function which returns true or false based on whether the specified key is found in the map.
- 3. boolean containsValue(Object Value):** Similar to containsKey() method, however it looks for the specified value instead of key.

Collections - HashMap



4. **Value get(Object key):** It returns the value for the specified key.
5. **boolean isEmpty():** It checks whether the map is empty. If there are no key-value mapping present in the map then this function returns true else false.
6. **Set keySet():** It returns the Set of the keys fetched from the map.
7. **value put(Key k, Value v):** Inserts key value mapping into the map. Used in the above example.
8. **int size():** Returns the size of the map – Number of key-value mappings.
9. **Collection values():** It returns a collection of values of map.
10. **Value remove(Object key):** It removes the key-value pair for the specified key. Used in the above example.



Iterators and ListIterators

Iterator

Iterator interface present in java.util package. It is used to read or remove the collection elements by iterating over specified collection.

Note: Iterator interface can be applied to all collection classes but it can traverse only in single direction that is Forward direction.

The iterator() method of Collection interface is used to get the Iterator interface.

```
Iterator iterator = Collection.iterator();
```



Iterators and ListIterators

ListIterator

Iterator interface present in java.util package. It extends java.util.Iterator interface. It is used to read, remove, add, and update the collection elements by iterating over specified List type collection.

Note: Iterator interface can be applicable only on List objects like ArrayList or LinkedList but it can traverse in both directions that is Forward as well as Backward direction.

The listIterator() method of List interface is used to get the ListIterator object.

```
ListIterator listIterator = list.listIterator();
```



Iterators and ListIterators

Difference between Iterator and ListIterator in java

Both of these interfaces are used for traversing but still there are few differences in the way they can be used for traversing a collection.

Iterator vs ListIterator

1) Iterator is used for traversing List and Set both.

We can use ListIterator to traverse List only, we cannot traverse Set using ListIterator.

2) We can traverse in only forward direction using Iterator.

Using ListIterator, we can traverse a List in both the directions (forward and Backward).

3) We cannot obtain indexes while using Iterator

We can obtain indexes at any point of time while traversing a list using ListIterator. The methods `nextIndex()` and `previousIndex()` are used for this purpose.



Iterators and ListIterators

4) We cannot add element to collection while traversing it using Iterator, it throws `ConcurrentModificationException` when we try to do it.

We can add element at any point of time while traversing a list using `ListIterator`.

5) We cannot replace the existing element value when using Iterator. By using `set(E e)` method of `ListIterator` we can replace the last element returned by `next()` or `previous()` methods.

Methods of Iterator:

hasNext() - It returns true if Iterator has more element to iterate.

next() - It returns the next element in the collection until the `hasNext()` method returns true. This method throws '`NoSuchElementException`' if there is no next element.



Iterators and ListIterators

remove() - It removes the current element in the collection. This Method throws 'IllegalStateException' if this function is called before next() is invoked.

Methods of ListIterator:

add(E e) - The add() method of ListIterator interface is used to insert the given element into the specified list. The element is inserted automatically before the next element may return by next() method.

hasNext() - The hasNext() method of ListIterator interface is used to return true if the given list iterator contains more number of element during traversing the given list in the forward direction.

hasPrevious() - The hasPrevious() method of ListIterator interface is used to retrieve and remove the head of the deque.



Iterators and ListIterators

next() - The next() method of ListIterator interface is used to return the next element in the given list. This method is used to iterate through the list.

nextIndex() - The nextIndex() method of ListIterator interface is used to return the index of the element which is returned by the next() method.

previous() - The previous() method of ListIterator interface is used to return the previous element from the list and moves the cursor in a backward position. The above method can be used to iterate the list in a backward direction.

remove() - The **remove()** method of ListIterator interface is used to remove the last element from the list which is returned by next() or previous() method. The above method can only be called if the add(E) has not been called.



Iterators and ListIterators

Iterator	ListIterator
It is part of Collection framework and introduced in Java 1.2 version	It is part of Collection framework and introduced in Java 1.2 version
It can be applied to any collection classes.	Only applicable for List objects like ArrayList , LinkedList, Vector.
Can traverse only in FORWARD direction	Can traverse in both FORWARD and BACKWARD directions.
It can perform read as well as remove operation.	It can perform read, add, remove, and update operations.
The iterator() method of any collection class can be used to get Iterator object. For example, Iterator iterator= collection.iterator();	The listIterator() method of any List classes can be used to get ListIterator objectFor example, ListIterator listIterator = list.listIterator();

Thanks



BITS Pilani
Pilani Campus

Object Oriented Programming and Design

Presented by
Dr Ramya Devi M
mramyadevi@wilp.bits-pilani.ac.in
CS - 13 : 18th Oct 2025

IMP Note to Self





THREADS



Learning Objectives

- What is a Process, Process Control Block?
- What is Multitasking & What is Multithreading?
- Thread - Definition
- Methods of creating a thread
- Advantages and Disadvantages of thread
- Life Cycle of thread
- Definition, merits and demerits of Multithreading
- Demonstration program for multithreading
- Priorities of threads

THREADS



What are Java Threads?

A thread is a:

- Series of executed statements
- Mentioned as a lightweight process
- Facility that allows the users to do multiple activities within a single process
- Each thread has its own program counter, stack and local variables
- It shares memory, files and per-process state

THREADS

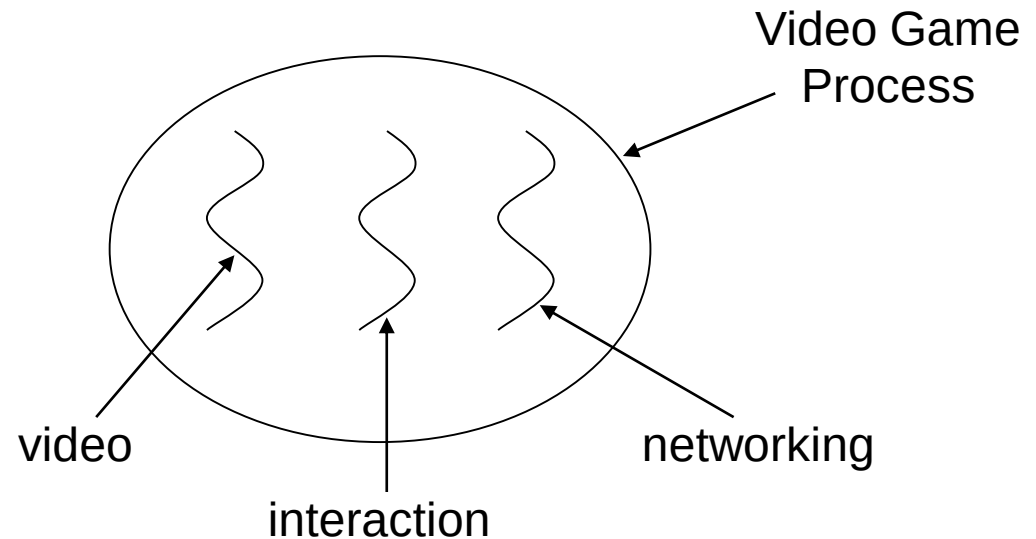


Individual and separate unit of execution that is part of a process

- multiple threads can work together to accomplish a common goal

Video Game example

- one thread for graphics
- one thread for user interaction
- one thread for networking



THREADS



Need for threads

- To perform asynchronous processing
- Increases the responsiveness of GUI applications
- Take advantage of multiprocessor systems
- Simplify program logic when there are multiple independent entities

What happens when a thread is invoked?

When a thread is invoked, there will be two paths of execution. One path will execute the thread and the other path will follow the statement after the thread invocation. There will be a separate stack and memory space for each thread.

THREADS



Risk Factor

- Proper co-ordination is required between threads accessing common variables [use of synchronized and volatile] for consistent view of data
- Overuse of java threads can be dangerous to program's performance and its maintainability.

Threads in Java

Java threads facility and API is deceptively simple:
Every java program creates at least one thread [main()
thread].

Additional threads are created through the Thread constructor or by instantiating classes that extend the Thread class.

THREADS - Creation



Thread creation in Java

Java permits us to implement thread in two ways. They are

1. Extending the `java.lang.Thread` class

2. Implementing the `java.lang.Runnable` Interface

Note: The Thread and Runnable are available in the `java.lang.*` package

1) By extending thread class

- The class should extend Java Thread class.
- The class should override the `run()` method.

THREADS – Creation by extending Thread class



- The functionality that is expected by the Thread to be executed is written in the run() method.

void start(): Creates a new thread and makes it runnable.

void run(): The new thread begins its life inside this method.

Example:

```
class MyThreadClass extends Thread
{
    public void run()
    {
        System.out.println(" Executing a thread....");
    }
}
```

THREADS – Creation by extending Thread class



```
public static void main(String aa[ ])
{
    MyThreadClass T = new MyThreadClass();
    T.start();
}
}
```

The output for the above code is

Executing a thread....

THREADS – Creation using Runnable interface



By Implementing Runnable interface

- The class should implement the Runnable interface
- The class should implement the run() method in the Runnable interface
- The functionality that is expected by the Thread to be executed is put in the run() method

Example:

```
class MyThreadClass implements Runnable
{
    public void run()
    {
```

THREADS – Creation using Runnable interface



```
System.out.println(" Executing a thread....");  
  
}  
  
public static void main(String aa[ ])  
  
{  
  
//implements Runnable  
  
//MyThreadClass T = new MyThreadClass(); extends Thread  
  
MyThreadClass() MTC= new MyThreadClass();  
  
Thread T = new Thread(MTC);  
  
T.start();  
  
}
```

Extends Thread class vs Implements Runnable Interface?



Extending the Thread class will make our class unable to extend other classes, because of the single inheritance feature in JAVA.

However, this will give us a simpler code structure.

If we implement Runnable, we can gain better object-oriented design and consistency and also avoid the single inheritance problems.

If we just want to accomplish basic functionality of a thread , we can simply implement Runnable interface and override run() method.

But if we want to do something serious with thread object as it has other methods like suspend(), resume(), ..etc which are not available in Runnable interface then we may choose to extend the Thread class.

Thread Lifecycle Management



- **start()** -> Begins the execution of the thread. The run() method is called automatically by the Java Virtual Machine (JVM).
- **run()** -> Contains the code that defines the thread's behavior. This is overridden in a subclass or by a Runnable.
- **join()** -> Waits for the thread to finish execution.join(long millis)Waits for the thread to finish execution for the specified time (in milliseconds).
- **join(long millis, int nanos)** -> Waits for the thread to finish execution for the specified time with nanosecond precision.
- **interrupt()** -> Interrupts the thread. If the thread is in a waiting or sleeping state, it will throw an InterruptedException.

Thread Lifecycle Management



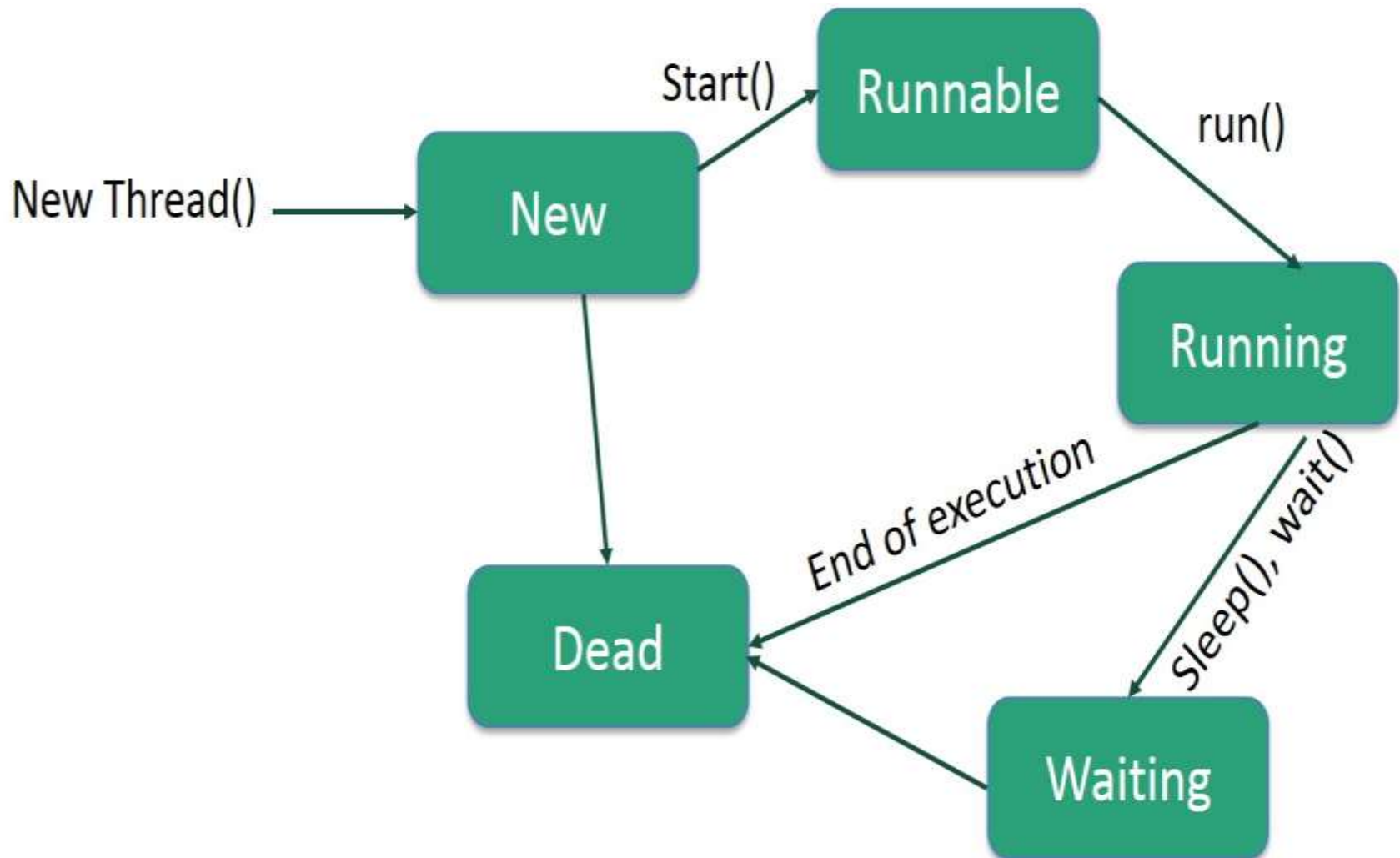
- **isInterrupted()** -> Checks if the thread has been interrupted.
- **stop() (Deprecated)** -> Forces the thread to stop execution immediately (unsafe and not recommended).
- **suspend() (Deprecated)** -> Suspends the thread execution (unsafe and not recommended).
- **resume() (Deprecated)** -> Resumes a suspended thread (unsafe and not recommended).
- In Java, deprecated means that a method, class, or field is marked as outdated or no longer recommended for use. This does not mean it is removed from the language—it still works—but its use is discouraged because there are better, safer, or more efficient alternatives available.

Thread Lifecycle Management



- **stop():** Terminates the thread immediately. Not safe as it may leave shared resources in an inconsistent state.
- **suspend() and resume():** Temporarily pauses and resumes the thread. Deprecated because they can cause deadlocks if not used carefully.
- **interrupt():** Politely signals the thread to stop or handle interruption, allowing for safer thread management.

Life Cycle of a thread



Multithreading in JAVA



Multitasking: Ability to execute more than one task at the same time is known as multitasking.

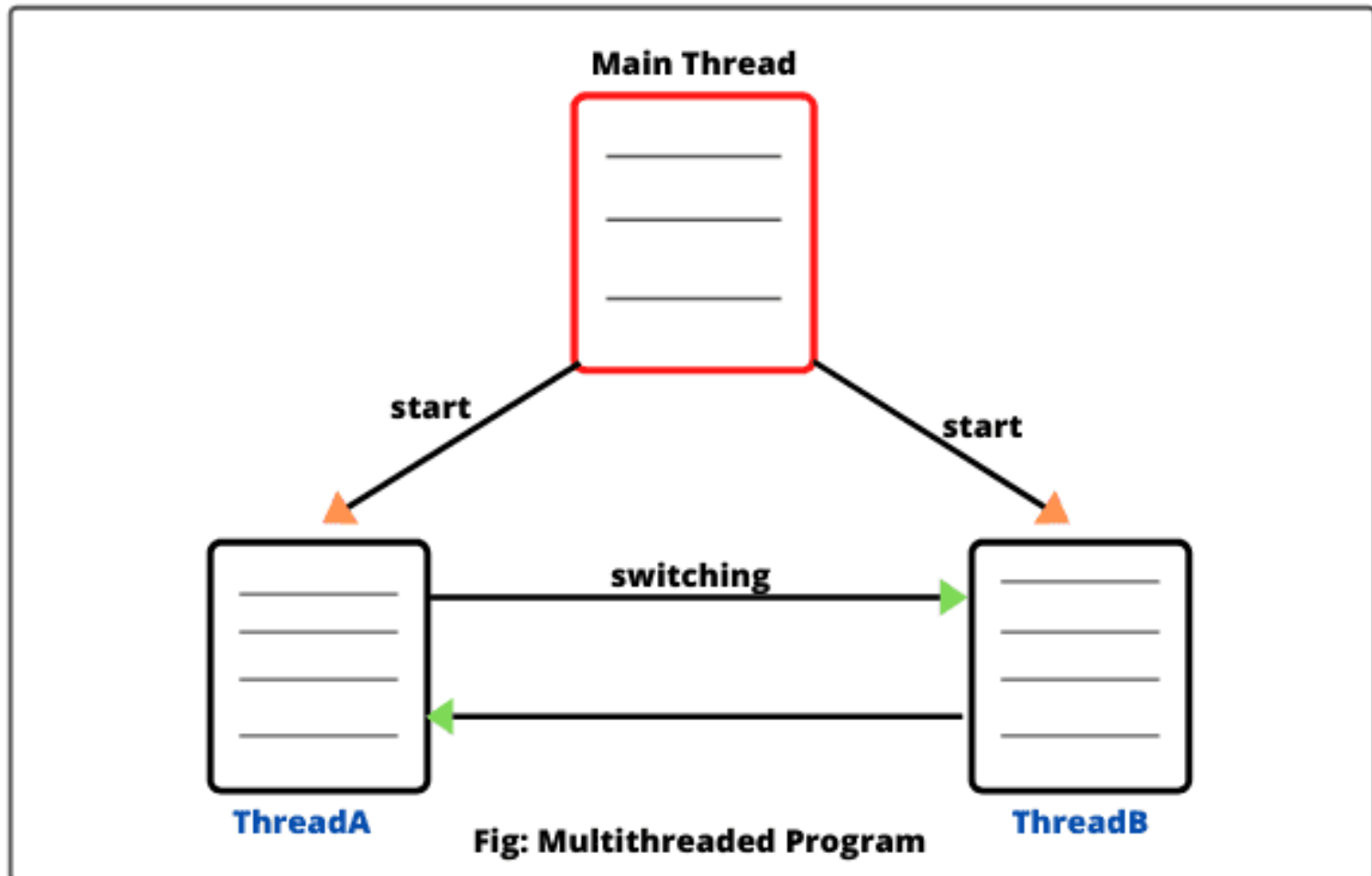
Multithreading: Multithreading is a technique in which a program (process) is divided into two or more subprograms (subprocesses), each of which can perform different tasks simultaneously (at the same time and in parallel manner). Each subprogram of a program is called thread in Java.

Look at the below figure where a Java program has three threads, one main and two others.

The other two threads ThreadA and ThreadB are created and started from the main thread.

When a program contains multiple flows of control, it is called multithreaded program.

Multithreading in JAVA



Advantages of Multithreading in Java



The advantages of using multithreading programming concept are as follows:

1. In a multithreaded application program, different parts of the application are executed by different threads concurrently. The entire application does not stop even if an exception occurs in any of the threads. It does not affect other threads during the execution of the application.
2. Different threads are allotted to different processors and each thread is executed in different processors in parallel.
3. Multithreading helps to reduce computation time.
4. Multithreading technique improves the performance of the application.

Advantages of Multithreading in Java



- 5. Threads share the same memory address space. Hence, it saves memory.
- 6. Multithreaded program makes maximum utilization of CPU and keeping the idle time of CPU to minimum.
- 7. Context switching from one thread to another thread is less expensive than between processes.

Drawbacks of Multithreading in Java

The drawbacks of multithreading are as follows:

- 1. Increased complexity.
- 2. Synchronization of shared resources.
- 3. In the multithreading programming concept, debugging is difficult. At times, result is unpredictable.
- 4. Potential deadlocks.

Thread State and Information



- **getName()** -> Returns the name of the thread.
- **setName(String name)** -> Sets the name of the thread.
- **getId()** -> Returns the unique ID of the thread.
- **getState()** -> Returns the current state of the thread (e.g., NEW, RUNNABLE, BLOCKED, WAITING, etc.).
- **isAlive()** -> Checks if the thread is still running.

Thread priorities



- Thread priorities are the integers which decide how one thread should be treated with respect to the others.
- Thread priority decides when to switch from one running thread to another, process is called context switching
- A thread can voluntarily release control and the highest priority thread that is ready to run is given the CPU.
- A thread can be preempted by a higher priority thread no matter what the lower priority thread is doing. Whenever a higher priority thread wants to run it does.
- To set the priority of the thread `setPriority()` method is used which is a method of the class Thread Class.

Thread priorities



.In place of defining the priority in integers, we can use MIN_PRIORITY, NORM_PRIORITY and MAX_PRIORITY

The values of the integers are

MIN_PRIORITY - 1

NORM_PRIORITY - 5

and

MAX_PRIORITY - 10

Methods of Thread Priority:

The following are the methods related to priorities in a thread.

1. **public final int getPriority()**

It is a method returns priority of given thread.

Thread priorities



2. **public final void setPriority(int newPriority):**

It is a method which is used to change the priority of thread to the value newPriority. This method throws `IllegalArgumentException` if value of parameter newPriority goes beyond minimum(1) and maximum(10) limit.

Thread synchronization



- Java supports multithreading: multiple threads executing simultaneously.
- Threads can share resources (like variables, objects, files).
- Problem: When multiple threads access shared resources simultaneously, data inconsistency may occur.
- Solution: Thread synchronization ensures only one thread accesses a resource at a time.

Why Synchronization is Needed



- Example of race condition:
 - Two threads update the same bank account balance.
 - Without synchronization, the final balance may be incorrect.
- Synchronization ensures atomicity of operations.
- Prevents thread interference and memory consistency errors.

Synchronization in Java



- Java provides synchronization in two ways:
 - Synchronized Methods
 - Synchronized Blocks
- Key Points: Only one thread can execute a synchronized method or block on the same object at a time.
- Other threads are blocked until the resource is free.

Synchronized Method



- Java provides synchronization in two ways:
 - Synchronized Methods
 - Synchronized Blocks
- Key Points: Only one thread can execute a synchronized method or block on the same object at a time.
- Other threads are blocked until the resource is free.

Key points



- Use synchronized to avoid race conditions.
- Prefer synchronized blocks over methods for better performance.
- Every object in Java can be used as a lock.
- Static synchronized methods lock the class, instance methods lock the object instance.

Advanced Concepts



- Reentrant Locks: `java.util.concurrent.locks`.
- ReentrantLockDeadlock: When two threads wait for each other's lock indefinitely.
- Wait & Notify: Threads can communicate while waiting for a lock.

Thread Communication in Java – `wait()` and `notify()`



- Threads sometimes need to communicate when sharing resources.
- Example: Producer-Consumer problem
- Java provides `wait()`, `notify()`, and `notifyAll()` for thread coordination.
- These methods must be called within a synchronized context.

Why wait() and notify() are needed



- A consumer thread may need to wait until the producer produces data.
- wait(): Makes the current thread release the lock and wait until notified.
- notify(): Wakes up one waiting thread.
- notifyAll(): Wakes up all waiting threads.

Syntax



```
synchronized(obj) {  
    obj.wait(); // Thread waits and releases lock  
}
```

```
synchronized(obj) {  
    obj.notify(); // Wakes up one waiting thread  
}
```

```
synchronized(obj) {  
    obj.notifyAll(); // Wakes up all waiting threads  
}
```

- Must be called inside a synchronized block/method.
- Thread must own the object's monitor to call these methods.

Thanks



BITS Pilani
Pilani Campus

Object Oriented Programming and Design

Presented by
Dr Ramya Devi M
mramyadevi@wilp.bits-pilani.ac.in
CS - 14 : 25th Oct 2025



THREADS - Continuation

Learning Objectives

- Thread synchronization
- Types of thread synchronization
 - (i) Process Synchronization
 - (ii) Thread Synchronization
 - Mutual Exclusion
 - Synchronized method, block, static synchronized
 - Inter Thread Communication
 - Wait(), notify(), notifyAll()
- join() method



What is Thread Synchronization?

- When multiple threads share a common data structure then a mechanism is required to ensure that the shared resource will be used by only one thread at a time
 - **Thread Synchronization** → Process which guarantees the shared resource will be used by only one thread at a time
 - **Key to Synchronization** → **Monitor** [An Object which is used as a mutually exclusive lock]
 - Only one thread can Own a **Monitor** at a Given Point of Time
 - In order to Access a Shared Resource, a Thread has to First Acquire the Lock over that Resource. If Lock is granted then a Thread is said have Entered the Monitor. All other Threads Attempting to Acquire the Lock over the Same Shared Resource will Enter into WAITING state.
-

Synchronization of Threads



What is Thread Synchronization?

- Thread synchronization in java is a method of programming several threads to carry out independent tasks easily.
- It is capable of controlling access to multiple threads to a particular shared resource.
- The main reason for using thread synchronization are as follows:
 - To prevent interference between threads.
 - To prevent the problem of consistency.

Types of Thread Synchronization

In Java, there are two types of thread synchronization:

1. Process synchronization
2. Thread synchronization

Synchronization of Threads



Process synchronization- The process means a program in execution and **runs independently isolated from other processes**. CPU time, memory, etc resources are allocated by the operating system.

Thread synchronization- It refers to the concept where only **one thread is executed at a time while other threads are in the waiting state**. This process is called thread synchronization. It is used because it avoids interference of thread and the problem of inconsistency.

Why Thread Synchronization?



- Without Proper Synchronization, Threads Can Corrupt Data Structures
- Example
 1. Assume we have a Queue of Greetings, with Capacity as 10.
 2. Assume there is ProducerThread which keeps on adding a particular string type greeting into the queue for a fixed number of iterations. Again assume that there are used two instances of the ProducerThread say T1 and T2.
 3. Assume there is ConsumerThread which keeps on removing greeting from the queue for a fixed number of iterations. Again assume that there is used only one instance of ConsumerThread say T3.

Why Thread Synchronization? ...



```
// Queue Class Partial Implementation
public class Queue
{
private Object[ ] elements; private int    // Elements of Queue
head;                                     // head location of queue
private int tail;                         // tail location of queue
private int size;                         // size of queue

// Constructor Method public
Queue(int capacity)                       {      }

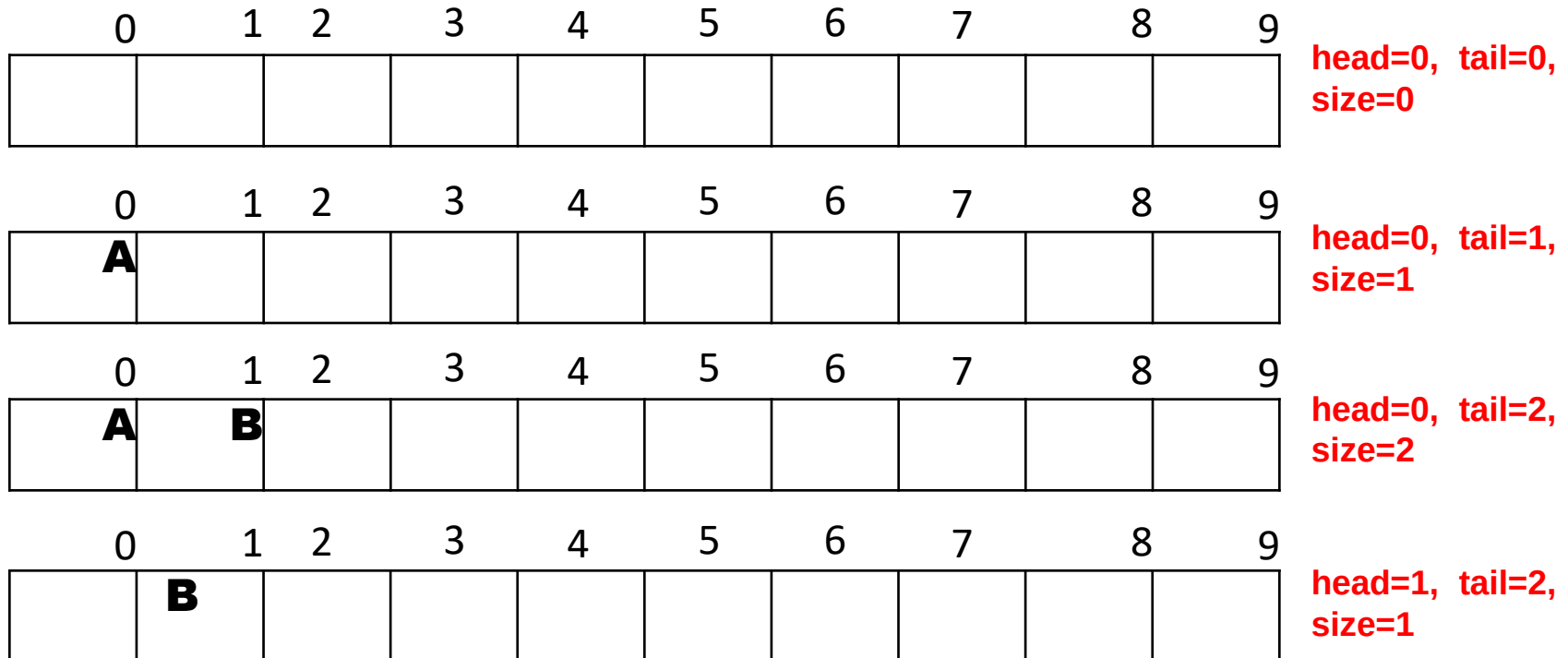
// Method to Remove
public Object removeFirst()                {      } // head++, size--

// Method to Add
    public void add(Object anotherObject)  {      } // tail++, size++
    // Methods to check if queue is full or empty
    public boolean isFull()                 {      }
    public boolean isEmpty()                {      }
} // End of class Queue
```

Why Thread Synchronization? ...



- Queue is FIFO (First-in-First-Out) Type Data Structure
- Elements are Added at one End (Tail) and Removed From Other End (Head)
- Initially Head = Tail = 0, Size = 0



Why Thread Synchronization? ...



```
class ProducerThread implements Runnable
{
private Queue queue;
private int repetitions;
private static final int DELAY =10;
public Producer(String greeting, Queue queue , int reps){} public
void run()
{
try
{
int i =1;
while(i <= repetitions)
{
if(!queue.isFull())
{
queue.add(i+": "+greeting); i++
}
Thread.sleep(DELAY);
} // End of while loop
} // End of try catch(InterruptedException e){}
} // End of Method
} // End of Class
```


Why Thread Synchronization? ...



```
class ConsumerThread implements Runnable
{
private Queue queue;
private int repetitions;
private static final int DELAY =10;  public Consumer(Queue queue , int
reps){}  public void run()
{
try
{
int i =1;
while(i <= repetitions)
{
if(!queue.isEmpty())
{
Object Obj = queue.removeFirst();  i++
}
Thread.sleep(DELAY);
} // End of while Loop
} // End of try
catch(InterruptedException e){}
} // End of Method
} // End of class
```

Why Thread Synchronization? ...



```
class Driver
{
public static void main(String args[])
{
Queue queue = new Queue(10); final int repetitions = 100;

Runnable r1 = new ProducerThread("Hello, World", queue, repetitions); Runnable r2 = new
ProducerThread("Goodbye, World", queue, repetitions); Runnable r3 = new
ConsumerThread(queue, 2*repetitions);

Thread T1 = new Thread(r1); Thread T2 = new Thread(r2); Thread T3 = new Thread(r3);

T1.start();
T2.start();
T3.start();

} //End of Method
} //End of class
```

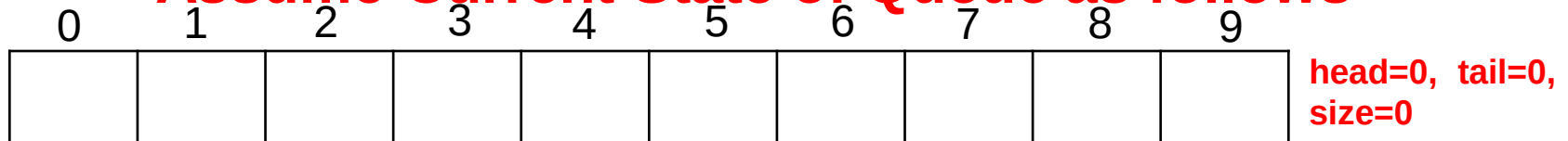
How Queue Can be Corrupted?

Why Thread Synchronization? ...



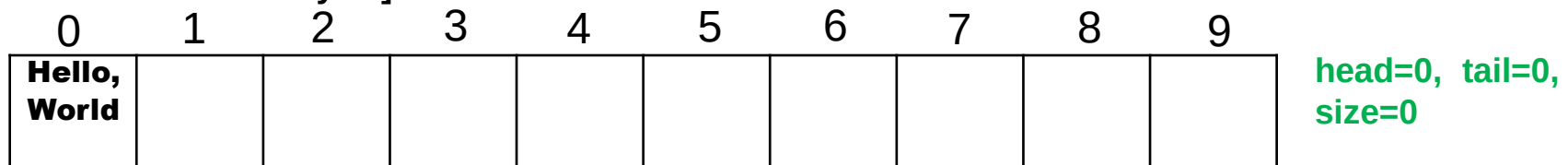
How Queue Can be Corrupted?

Assume Current State of Queue as follows

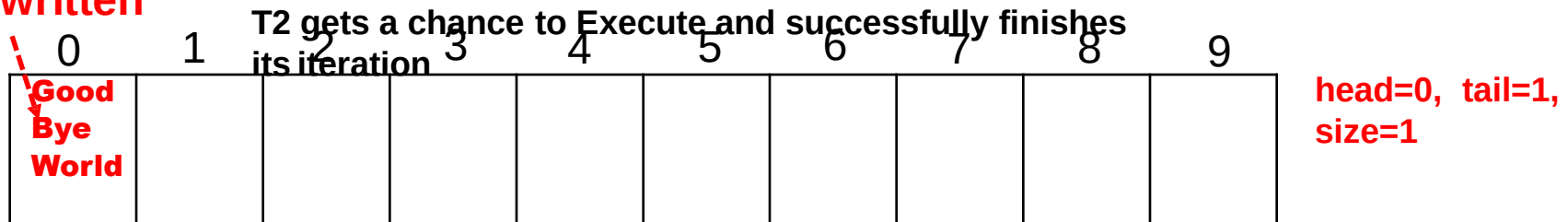


Assume Threads are Scheduled as T1 (Producer), T2 (Producer) and T1 (Producer)

T1 Executing, But just After Adding the Greeting its Allotted Time Slice is Over [T1 is not able to Update the Values of tail and size fields. So T1 will update these values in next cycle]



Overwritten



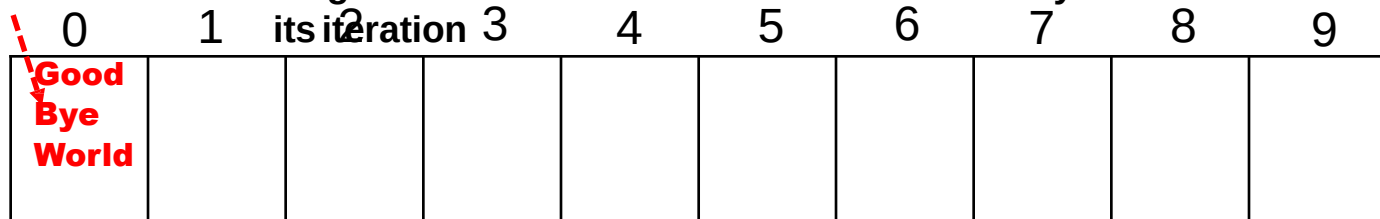
Why Thread Synchronization? ...



How Queue Can be Corrupted?

Overwritten

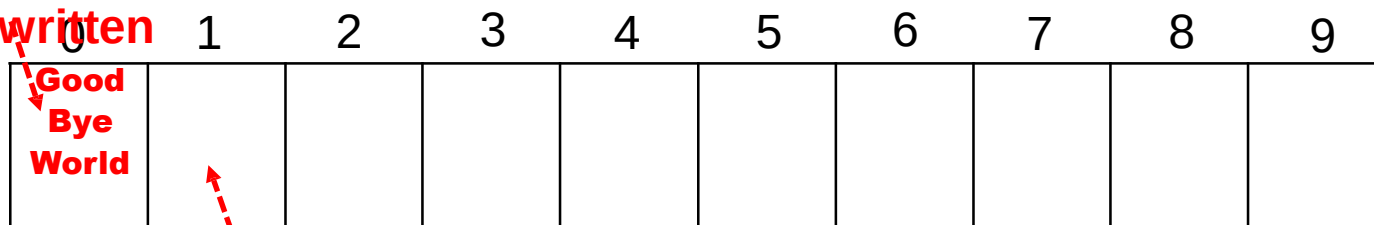
T2 gets a chance to Execute and successfully finishes its iteration



head=0,
tail=1,
size=1

T1 gets a chance to Execute and Tries Finish Its Previous Iteration

Overwritten



head=0,
tail=2,
size=2

Unwritten
Location

Synchronization Mechanism



- Using Synchronized Methods
- Using Synchronize Statement

Synchronization of Threads



In java, thread synchronization is further divided into two types:

- **Mutual exclusive-** it will keep the threads from interfering with each other while sharing any resources.
- **Inter-thread communication-** It is a mechanism in java in which a thread running in the critical section is paused and another thread is allowed to enter or lock the same critical section that is executed.

Mutual Exclusion – synchronized method



Mutual Exclusion

It is used for preventing threads from interfering with each other and to keep distance between the threads.

Mutually exclusive threads is achieved using the following:

- Synchronized Method
- Synchronized Block
- Static Synchronization

Synchronized method It is a method that can be declared synchronized using the keyword “synchronized” before the method name.

By writing this, it will make the code in a method thread-safe so that no resources are shared when the method is executed.

Mutual Exclusion – synchronized method



Syntax:

```
synchronized public void methodName() { }
```

Example:

```
public class Table
{
void printTable(int n) // non synchronized method
{
for(int i = 1; i <= 5; i++)
{
System.out.println(n * i);
}
```


Mutual Exclusion – synchronized method



```
try
{
    Thread.sleep(500);
}
catch(InterruptedException ie)
{
    System.out.println(ie);
}
}}
}

public class Thread1 extends Thread
{
    Table t; // Declaring t as class type table
```

Mutual Exclusion – synchronized method



*// Declaring parameterized constructor and passing variable t
as a parameter to the thread.*

```
Thread1(Table t)
{
    this.t = t;
}
public void run()
{
    t.printTable(2);
}
}
public class Thread2 extends Thread
{
```

Mutual Exclusion – synchronized method



```
Table t;           // Declaring t as class type table
Thread2(Table t)
{
    this.t = t;
}
public void run()
{
    t.printTable(10);
}
}
public class UnsynchronizedMethod
{
    public static void main(String[] args)
    {
```

Mutual Exclusion – synchronized method



// Creating an object of Table class.

```
Table obj = new Table();  
Thread1 t1 = new Thread1(obj);  
Thread2 t2 = new Thread2(obj);  
    t1.start();  
    t2.start();  
}  
}
```

Mutual Exclusion – synchronized block



Synchronized Block

If a block is declared as synchronized then the code which is written inside a method is only executed instead of the whole code. It is used when sequential access to code is required.

Syntax:

synchronized (object reference)

```
{  
    // Insert code here  
}
```

Example

```
class Table  
{
```

Mutual Exclusion – synchronized block



```
void printTable(int n){  
    synchronized(this){           //This is a synchronized block  
        for(int i=1;i<=5;i++){  
            System.out.println(n*i);  
            try{  
                Thread.sleep(400);  
            }catch(Exception e){System.out.println(e);}  
        }  
    }  
} //end of the synchronized block  
  
class MyThread1 extends Thread{  
    Table t;
```

Mutual Exclusion – synchronized block



```
MyThread1(Table t){  
    this.t=t;  
}  
public void run(){  
    t.printTable(5);  
}  
}  
class MyThread2 extends Thread{  
    Table t;  
    MyThread2(Table t){  
        this.t=t;  
    }  
}
```

Mutual Exclusion – synchronized block



```
public void run(){
t.printTable(100);
}
}

public class TestSynchronizedBlock1{
public static void main(String args[]){
Table obj = new Table();
MyThread1 t1=new MyThread1(obj);
MyThread2 t2=new MyThread2(obj);
t1.start();
t2.start();
}
}
```


Synchronization of Threads using static method



In this example, class `thread1` extends the class `thread` and has a method `run` that is void.

The class `thread2` extends the `thread` and has a method `run` that is void.

Both are printing tables of different numbers. In the main class, the object of threads 1 and 2 are created and the method `start` is invoked.

The difference between both examples is that this is the synchronized thread.

Static Synchronization

The method is declared static in this case. It means that lock is applied to the class instead of an object and only one thread will access that class at a time.

Syntax: `synchronized static return type class name{}`

Synchronization of Threads using static method



// Java program of multithreading with static synchronized

```
class Display
```

```
{
```

```
    public static synchronized void wish(String name)
```

```
    {
```

```
        for(int i=0;i<3;i++)
```

```
        {
```

```
            System.out.print("Good Morning: ");
```

```
            System.out.println(name);
```

```
            try{
```

```
                Thread.sleep(2000);
```

```
            }
```

Synchronization of Threads using static method



```
catch(InterruptedException e) { }  
    }  
}  
}  
class MyThread extends Thread{  
    Display d;  
    String name;  
    MyThread(Display d,String name)  
    {  
        this.d=d;  
        this.name=name;  
    }  
}
```

Synchronization of Threads using static method



```
public void run()
{
    d.wish(name);
}
}

class MainClass{
    public static void main(String arg[])
    {
        Display d1=new Display();
        Display d2=new Display();
        MyThread t1=new MyThread(d1,"Hello");
        MyThread t2=new MyThread(d2,"World");
    }
}
```

Synchronization of Threads using static method



```
t1.start();  
    t2.start();  
}  
}
```

Output:

Good Morning: World
Good Morning: World
Good Morning: World
Good Morning: Hello
Good Morning: Hello
Good Morning: Hello

Synchronization of Threads - Points to remember



1. Thread synchronization is a way of programming several threads to carry out independent tasks easily. It is capable of controlling access to multiple threads to a particular shared resource.
2. In java, there are two types of thread synchronization:
 - Process Synchronization
 - Thread Synchronization
 - Mutual exclusive
 - Inter-thread communication

Thread joining in java waits for the thread to be finished completely whereas thread synchronization is used for preventing two threads to be executed at the same time in the same piece of code.

Inter-thread Communication in Java



Inter-thread communication or Co-operation is all about allowing synchronized threads to communicate with each other.

Cooperation (Inter-thread communication) is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter (or lock) in the same critical section to be executed.

It is implemented by following methods of Object class:

`wait()`

`notify()`

`notifyAll()`

Inter-thread Communication in Java



1) **wait() method:** The wait() method causes current thread to release the lock and wait until either another thread invokes the notify() method or the notifyAll() method for this object, or a specified amount of time has elapsed.

The current thread must own this object's monitor, so it must be called from the synchronized method only otherwise it will throw exception.

Method	Description	Method
public final void wait()throws InterruptedException	It waits until object is notified.	public final void wait()throws InterruptedException
public final void wait(long timeout)throws InterruptedException	It waits for the specified amount of time.	public final void wait(long timeout)throws InterruptedException

Inter-thread Communication in Java



2) notify() method

The notify() method wakes up a single thread that is waiting on this object's monitor.

If any threads are waiting on this object, one of them is chosen to be awakened.

The choice is arbitrary and occurs at the discretion of the implementation.

Syntax:

```
public final void notify()
```

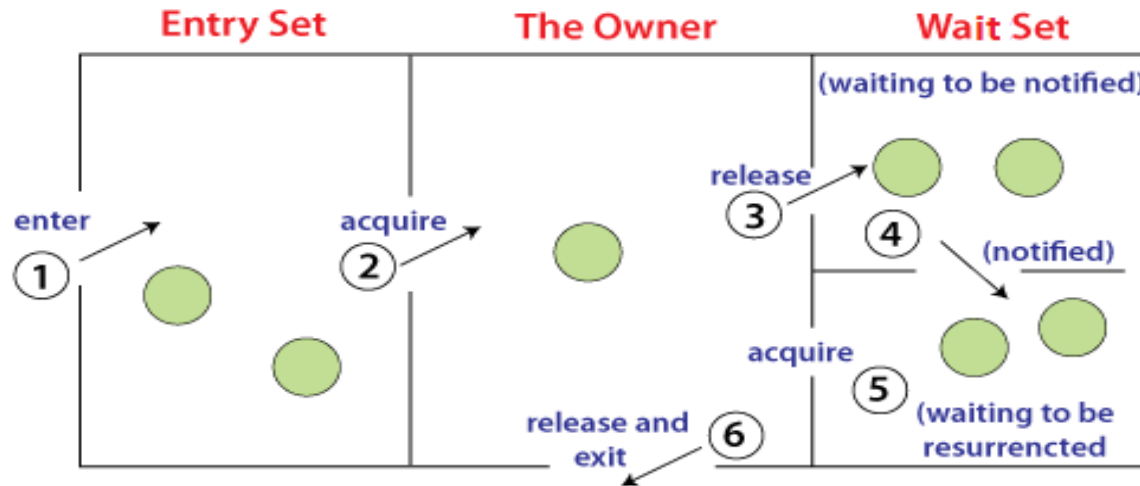
3) notifyAll() method

Wakes up all threads that are waiting on this object's monitor.

Syntax:

```
public final void notifyAll()
```

Understanding the process of inter-thread communication



1. Threads enter to acquire lock.
2. Lock is acquired by one thread.
3. Now thread goes to waiting state if you call `wait()` method on the object. Otherwise it releases the lock and exits.
4. If you call `notify()` or `notifyAll()` method, thread moves to the notified state (runnable state).
5. Now thread is available to acquire lock.
6. After completion of the task, thread releases the lock and exits the monitor state of the object.

Why `wait()`, `notify()` and `notifyAll()` methods are defined in `Object` class not `Thread` class?
It is because they are related to lock and object has a lock.

Difference between wait() and sleep()



wait()	sleep()
The wait() method releases the lock.	The sleep() method doesn't release the lock.
It is a method of Object class	It is a method of Thread class
It is a non-static method	It is a static method
It should be notified by notify() or notifyAll() methods	After the specified amount of time, sleep is completed.

Difference between wait() and join()



wait()	join()
It is a method of java.lang.Object class.	It is a method of java.lang.
wait() method can be called by a synchronized block or method.	It is used to stop the current thread.
wait() method is implemented for performing multi-thread-synchronization.	It can be called either with synchronized and without synchronized context.
Its syntax is -: public final void wait() throws InterruptedException	Its syntax is -: public final void join() throws InterruptedException
wait() method causes the thread to sleep until notify() and notifyAll() are called	It can be used to add sequence among more than one thread

Deadlock in Java Multithreading



It is important to use if our program is running in multi-threaded environment where two or more threads execute simultaneously.

But sometimes it also causes a problem which is called Deadlock. Below is a simple example of Deadlock condition.

synchronized keyword is used to make the class or method thread-safe which means only one thread can have lock of synchronized method and use it, other threads have to wait till the lock releases and anyone of them acquire that lock.

Deadlock in Java Multithreading



Figure - 1

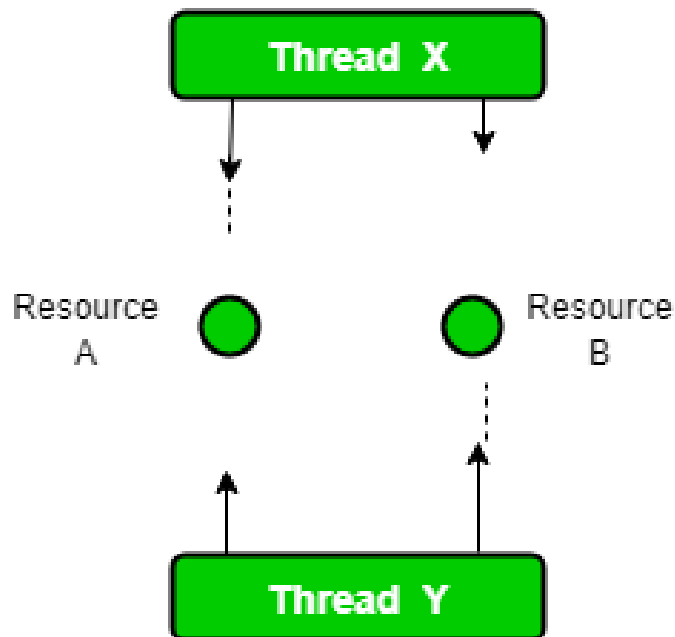
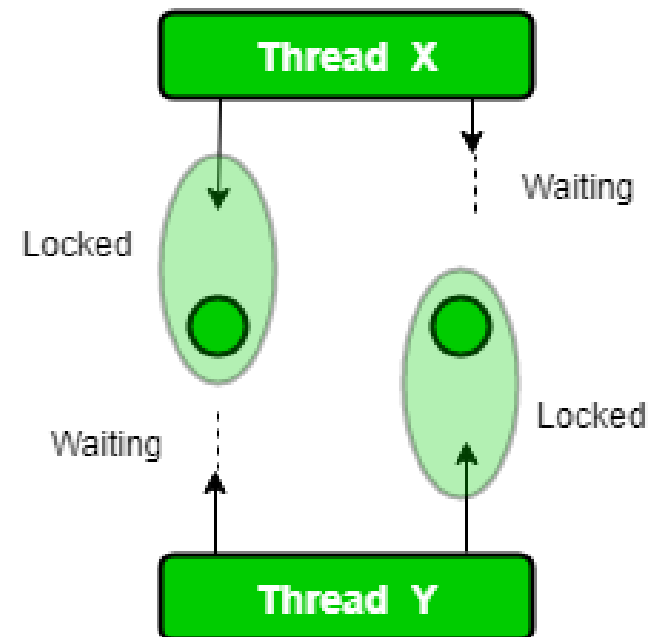


Figure - 2



Deadlock in Java Multithreading - Avoid Dead Lock condition



We can avoid dead lock condition by knowing its possibilities. It's a very complex process and not easy to catch. But still if we try, we can avoid this. There are some methods by which we can avoid this condition. We can't completely remove its possibility but we can reduce.

Avoid Nested Locks : This is the main reason for dead lock. Dead Lock mainly happens when we give locks to multiple threads. Avoid giving lock to multiple threads if we already have given to one.

Avoid Unnecessary Locks : We should have lock only those members which are required. Having lock on unnecessarily can lead to dead lock.

Using thread join : Dead lock condition appears when one thread is waiting other to finish. If this condition occurs we can use `Thread.join` with maximum time you think the execution will take.

Deadlock in Java Multithreading - Avoid Dead Lock condition



Important Points :

If threads are waiting for each other to finish, then the condition is known as Deadlock.

Deadlock condition is a complex condition which occurs only in case of multiple threads.

Deadlock condition can break our code at run time and can destroy business logic.

We should avoid this condition as much as we can.

Thanks



Object Oriented Programming Design

BITS Pilani
Pilani Campus

SESSION 15

Date : 18-01-2025



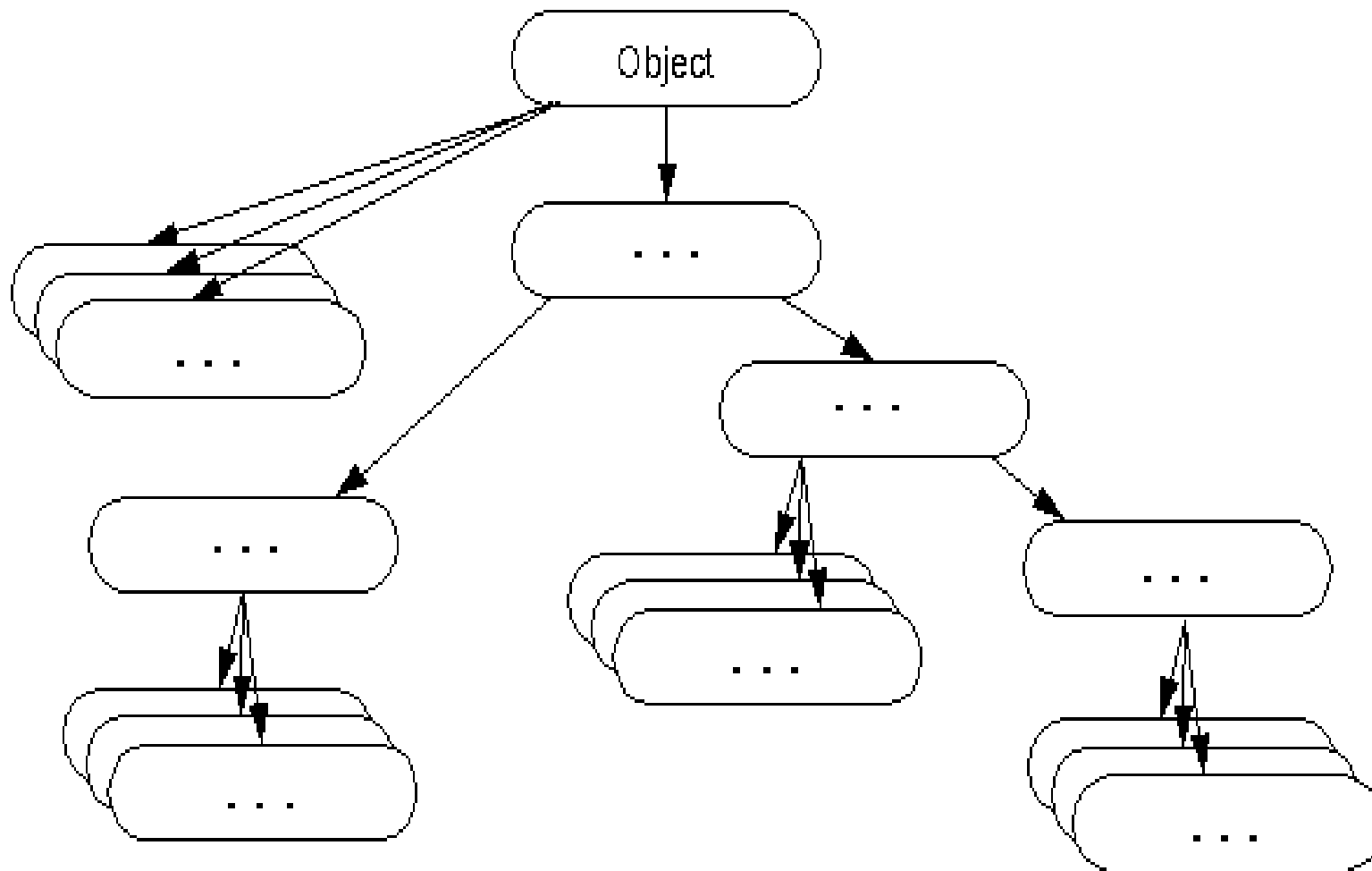
Object Class, Wrapper classes, Type inquiry, Object equality and cloning

Object Class

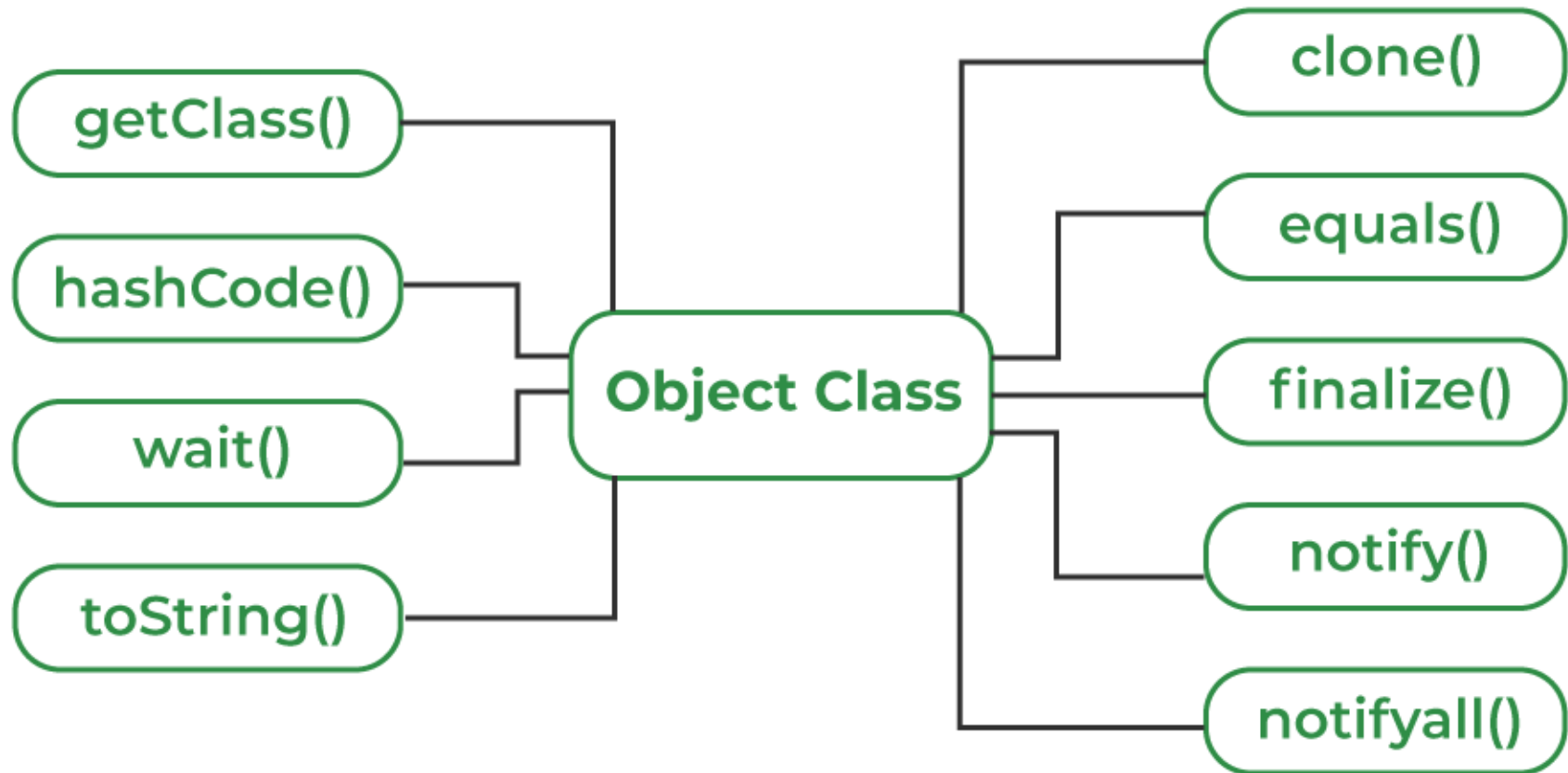


- The **Object class** is the **super class of all the classes** in java by default. This means that it is the **topmost class of java**.
- The Object class is beneficial if we want to refer any object whose type we don't know.
- The parent class reference variable can refer the child class object, known as upcasting.
- Consider an example, there is **getObject()** method that returns an object but it can be of any type like Employee, Student etc, we can use Object class reference to refer that object.
- For example:
Object obj=empl.getObject();//we don't know what object will be returned from this method
- The Object class provides some common behaviors to all the objects such as object can be compared, object can be cloned, object can be notified etc.

Object Class



Object Class methods



Object Class



Methods of Object class

Method	Description
<code>public final Class getClass()</code>	returns the Class class object of this object. The Class class can further be used to get the metadata of this class.
<code>public boolean equals(Object obj)</code>	compares the given object to this object.
<code>protected Object clone() throws CloneNotSupportedException</code>	creates and returns the exact copy (clone) of this object.
<code>public String toString()</code>	returns the string representation of this object.
<code>public final void notify()</code>	wakes up single thread, waiting on this object's monitor.
<code>public final void notifyAll()</code>	wakes up all the threads, waiting on this object's monitor.

Object Class



Method	Description
public final void wait(long timeout)throws InterruptedException	causes the current thread to wait for the specified milliseconds, until another thread notifies (invokes notify() or notifyAll() method).
public final void wait(long timeout,int nanos)throws InterruptedException	causes the current thread to wait for the specified milliseconds and nanoseconds, until another thread notifies (invokes notify() or notifyAll() method).
public final void wait()throws InterruptedException	causes the current thread to wait, until another thread notifies (invokes notify() or notifyAll() method).
protected void finalize()throws Throwable	is invoked by the garbage collector before object is being garbage collected.

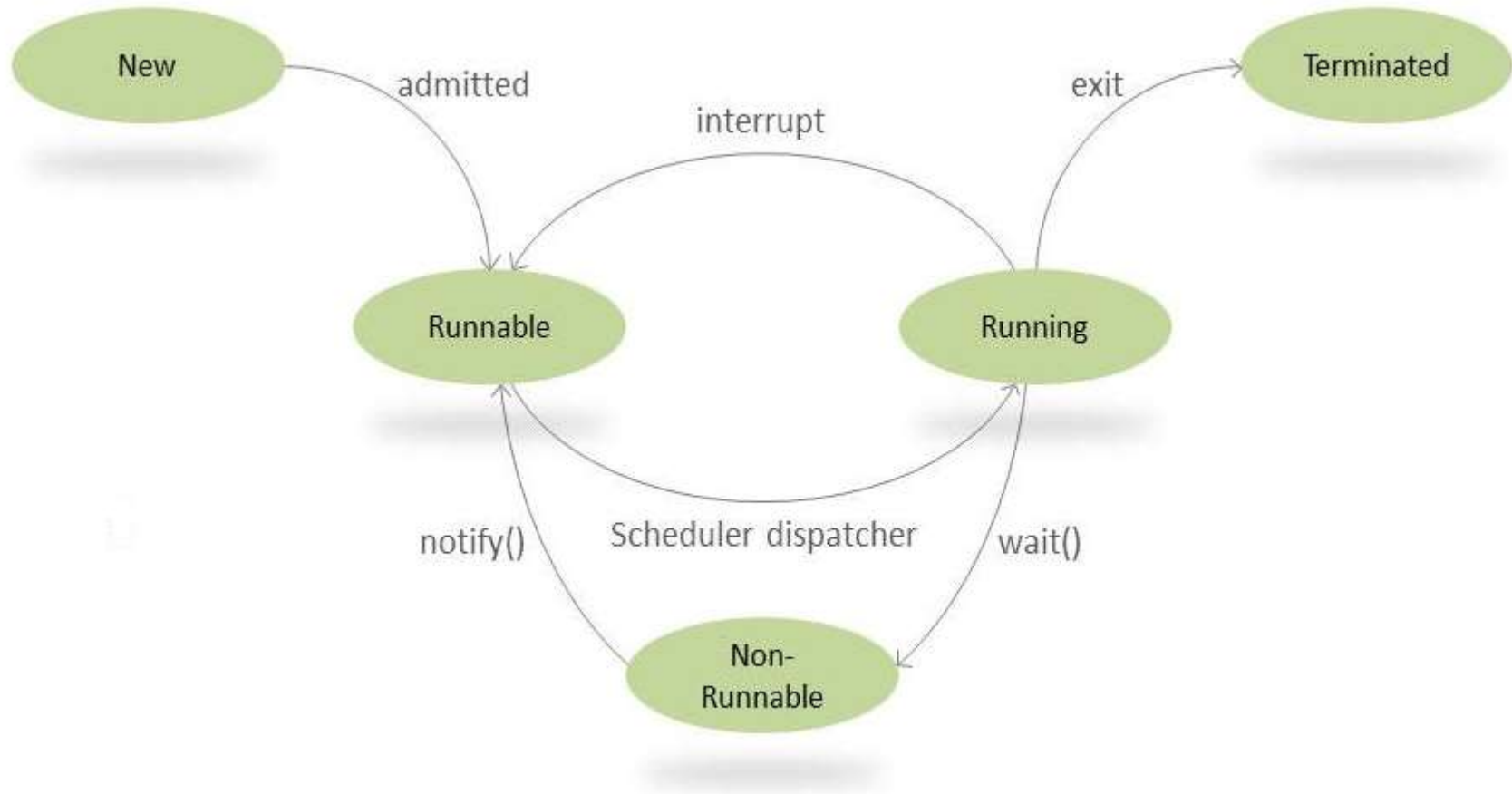
Object Class



Thread Synchronization in Java

- In a multithreaded environment, multiple threads might try to modify the same resource.
- If threads aren't managed properly, this will, of course, lead to consistency issues
- One tool we can use to coordinate actions of multiple threads in Java – is guarded blocks.
- Such blocks keep a check for a particular condition before resuming the execution.
- With that in mind, we'll make use of:
 - [Object.wait\(\)](#) – to suspend a thread
 - [Object.notify\(\)](#) – to wake a thread up
- This can be better understood from the following diagram, that depicts the lifecycle of a *Thread*:

Object Class



Object Class



There are many ways of controlling thread lifecycle;
Using `wait()` and `notify()` is one among the several ways.

1. `wait()` method

Simply put, when we call `wait()` – this forces the current thread to wait until some other thread wants to complete.

For this, the current thread must own the object's [monitor](#).

A thread can become the owner of the object in three ways:

- By executing a synchronized instance method of that object.
- By executing the body of a synchronized statement that synchronizes on the object.
- For objects of type `Class`, by executing a synchronized static method of that class.

Object Class



Note that only one active thread can own an object's monitor at a time.

This *wait()* method comes with three overloaded signatures.
Let's have a look at these.

wait()

The *wait()* method causes the current thread to wait indefinitely until another thread either invokes *notify()* for this object or *notifyAll()*.

wait(long timeout)

Using this method, we can specify a timeout after which thread will be woken up automatically.

A thread can be woken up before reaching the timeout using *notify()* or *notifyAll()*.

Note that calling *wait(0)* is the same as calling *wait()*.

Object Class



wait(long timeout, int nanos)

This is yet another signature providing the same functionality, with the only difference being that we can provide higher precision.

The total timeout period (in nanoseconds), is calculated as $1_000_000 * \text{timeout} + \text{nanos}$.

notify()* and *notifyAll()

The *notify()* method is used for waking up threads that are waiting for an access to this object's monitor.

For all threads waiting on this object's monitor (by using any one of the *wait()* method), the method *notify()* notifies any one of them to wake up arbitrarily.

The choice of exactly which thread to wake is non-deterministic and depends upon the implementation.

Object Class



Since *notify()* wakes up a single random thread it can be used to implement mutually exclusive locking where threads are doing similar tasks, but in most cases, it would be more viable to implement *notifyAll()*.

notifyAll()

This method simply wakes all threads that are waiting on this object's monitor.

The awakened threads will complete in the usual manner – like any other thread.

But before we allow their execution to continue, always **define a quick check for the condition required to proceed with the thread** – because there may be some situations where the thread got woken up without receiving a notification.

Wrapper classes



- As the name suggests, **wrapper classes are objects encapsulating primitive Java types.**
- Each Java primitive has a corresponding wrapper:
 - *boolean, byte, short, char, int, long, float, double*
 - *Boolean, Byte, Short, Character, Integer, Long, Float, Double*
- These are all defined in the *java.lang* package, hence we don't need to import them manually.
- Wrapper class provides various methods which we can use with collections like sorting, searching.

Wrapper classes



Why Wrapper classes?

- Basically, generic classes only work with objects and don't support primitives.
- As a result, if we want to work with them, we have to convert primitive values into wrapper objects.
- For example, the Java Collection Framework works with objects exclusively.
- Earlier it was not possible to simply call `add(5)` on a collection of `Integers`.
- Primitive values needed to be manually converted to corresponding wrapper classes and stored in collections.
- But now with autoboxing, we can easily do `ArrayList.add(101)` but internally Java converts the primitive value to an `Integer` before storing it in the `ArrayList` using the `valueOf()` method.

Wrapper classes



Primitive to Wrapper Class Conversion

- How do we convert a primitive value to a corresponding wrapper class.
- E.g. an int to Integer or a char to Character?
- We can either use constructor or static factory methods to convert a primitive value to an object of a wrapper class.
- Example of converting an int value to an Integer object in Java:
 - **Integer object = new Integer(1);**
 - **Integer anotherObject = Integer.valueOf(1);**
- The valueOf() method returns an instance representing the specified int value.

Wrapper classes



- Similarly, we can also convert boolean to Boolean, byte to Byte, char to Character, long to Long, float to Float, and double to Double.
- Though if we have to convert String to Integer then we need to use `parseInt()` method because String isn't a wrapper class.

Converting Wrapper object to Primitive type

- On the other hand, to convert from a wrapper object to a primitive value, we can use the corresponding method such as `intValue()`, `doubleValue()` etc:

`int val = object.intValue();`

Wrapper classes



Autoboxing and Unboxing

“**Boxing**” refers to converting a primitive value into a corresponding wrapper object.

Because this can happen automatically, it's known as autoboxing.

Similarly, when a wrapper object is unwrapped into a primitive value then this is known as “**unboxing**”.

Auto Boxing:

```
List<Integer> list = new ArrayList<>();  
list.add(1); // autoboxing
```

```
Integer val = 2; // autoboxing
```

Wrapper classes



In this example, Java will automatically convert the primitive int value to the wrapper.

Internally, it uses the `valueOf()` method to facilitate the conversion.

For example, the following lines are equivalent:

```
Integer value = 3;
```

```
Integer value = Integer.valueOf(3);
```

Though this makes conversion easy and codes more readable, there are some cases where we shouldn't use autoboxing e.g. inside a loop.

Wrapper classes



Unboxing

Similar to autoboxing, unboxing is done automatically when passing an object to a method that expects a primitive or when assigning it to a primitive variable:

```
Integer object = new Integer(1);  
int val1 = getSquareValue(object); //unboxing  
int val2 = object; //unboxing  
public static int getSquareValue(int i) {  
    return i*i; }
```

Note: if we write a method that accepts a primitive value or wrapper object, we can still pass both values to them.

Java will take care of passing the right type e.g. primitive or wrapper depending upon context.

Wrapper classes



```
class WrapperDemo {  
    public static void main(String[] args) {  
int a = 5; // create primitive types  
double b = 5.65;  
Integer aObj = Integer.valueOf(a); //converts into wrapper objects  
Double bObj = Double.valueOf(b);  
if(aObj instanceof Integer) {  
    System.out.println("An object of Integer is created.");  
}  
if(bObj instanceof Double) {  
    System.out.println("An object of Double is created.");  
}  
}  
}
```

Wrapper classes



However, the Java compiler can directly convert the primitive types into corresponding objects.

For example,

```
int a = 5;
```

```
// converts into object
```

```
Integer aObj = a;
```

```
double b = 5.6;
```

```
// converts into object
```

```
Double bObj = b;
```

Java Types and Type inquiry



Type: set of values and the operations that can be applied to the values

Strongly typed language: compiler and run-time system check that no operation can execute that violates type system rules

Compile-time check

```
Employee e = new Employee();  
e.clear(); // ERROR
```

Run-time check:

```
e = null;  
e.setSalary(20000); // ERROR
```


Java Types and Type inquiry



Java Types

Primitive types: int, short, long, byte, char, float, double, boolean

Class types, Interface types, Array types, The null type

Note: void is not a type

Example: What kind of type?

int

Rectangle

Shape

String[]

double[][]

Java Types and Type inquiry



Java Values

- value of primitive type
- reference to object of class type
- reference to array
- Null

Example: What kind of value?

13

`new Rectangle(5, 10, 20, 30);`

`"Hello"`

`new int[] { 2, 3, 5, 7, 11, 13 }`

`null`

Note: Can't have value of interface type. Why?

Java Types and Type inquiry



Subtype Relationship - Rules

S is a subtype of T if

S and T are the same type

S and T are both class types, and T is a direct or indirect superclass of S

S is a class type, T is an interface type, and S or one of its superclasses implements T

S and T are both interface types, and T is a direct or indirect superinterface of S

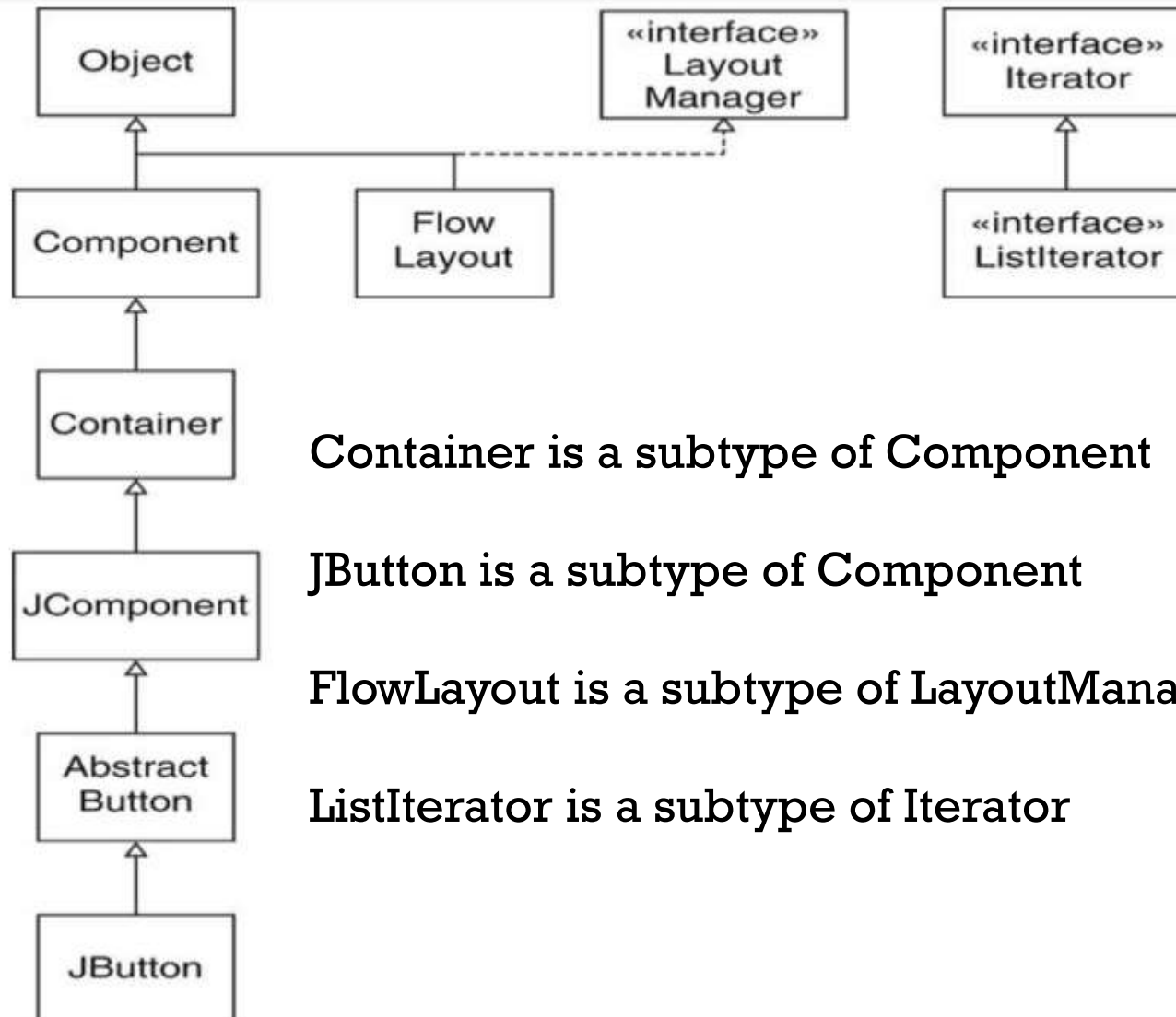
S and T are both array types, and the component type of S is a subtype of the component type of T

S is not a primitive type and T is the type Object

S is an array type and T is Cloneable or Serializable

S is the null type and T is not a primitive type

Java Types and Type inquiry



Container is a subtype of Component

JButton is a subtype of Component

FlowLayout is a subtype of LayoutManager

ListIterator is a subtype of Iterator

Java Types and Type inquiry



More Subtype Examples

Rectangle[] is a subtype of Shape[]

int[] is a subtype of Object

int is not a subtype of long

long is not a subtype of int

int[] is not a subtype of Object[]

Java Types and Type inquiry



Type Inquiry

Test whether e is a Shape:

if (e instanceof Shape) ...

Common before casts:

Shape s = (Shape) e;

Don't know exact type of e

Could be any class
implementing Shape

If e is null, test returns false (no
exception)

Type Inquiry with instanceof operator

Test whether e is a Rectangle:

if (e.getClass() ==
Rectangle.class) ...

Ok to use ==

A unique Class object for every
class

Test fails for subclasses

**Use instanceof to test for
subtypes:**

if (e instanceof Rectangle) ...

Java Types and Type inquiry



The equals Method

`equals` tests for equal contents

`==` tests for equal location

Used in many standard library methods

Example: `ArrayList.indexOf`

The following example Searches for the first occurrence of the given argument, testing for equality using the `equals` method.

`@param elem` an object.

`@return` the index of the first occurrence of the argument in this list;

returns -1 if the object is not found.

Java Types and Type inquiry



```
public int indexOf(Object elem)
{
    if (elem == null)
    {
        for (int i = 0; i < size; i++)
            if (elementData[i] == null) return i;
    }
    else
    {
        for (int i = 0; i < size; i++)
            if (elem.equals(elementData[i])) return i;
    }
    return -1;
}
```


Object equality and cloning



Object Cloning in Java

The object cloning is a way to create exact copy of an object.

The `clone()` method of `Object` class is used to clone an object.

The **`java.lang.Cloneable`** interface must be implemented by the class whose object clone we want to create.

If we don't implement `Cloneable` interface, `clone()` method generates **`CloneNotSupportedException`**.

The `clone()` method is defined in the `Object` class.

Object equality and cloning



Why use clone() method instead of new operator?

The clone() method saves the extra processing task for creating the exact copy of an object.

If we perform it by using the new keyword, it will take a lot of processing time to be performed that is why we use object cloning.

Object equality and cloning



Advantage of Object cloning

You don't need to write lengthy and repetitive codes.

Just use clone() method.

It is the easiest and most efficient way for copying objects, especially if we are applying it to an already developed or an old project.

Just define a parent class, implement Cloneable in it, provide the definition of the clone() method and the task will be done.

For example Clone() is the fastest way to copy array.

Object equality and cloning



Disadvantage of Object cloning

Following is a list of some disadvantages of clone() method:

To use the Object.clone() method, we have to change a lot of syntaxes to our code, like implementing a Cloneable interface, defining the clone() method and handling CloneNotSupportedException, and finally, calling Object.clone() etc.

We have to implement cloneable interface while it doesn't have any methods in it. We just have to use it to tell the JVM that we can perform clone() on our object.

Object equality and cloning



Disadvantage of Object cloning

`Object.clone()` is protected, so we have to provide our own `clone()` and indirectly call `Object.clone()` from it.

`Object.clone()` doesn't invoke any constructor so we don't have any control over object construction.

If you want to write a clone method in a child class then all of its superclasses should define the `clone()` method in them or inherit it from another parent class.

Otherwise, the `super.clone()` chain will fail.

`Object.clone()` supports only shallow copying but we will need to override it if we need deep cloning.

Object equality and cloning



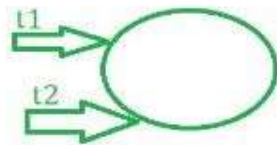
Shallow and Deep copy in cloning

Methods of Copying Objects in Java

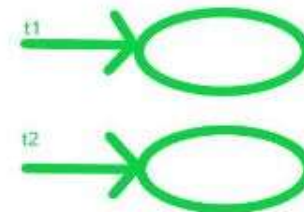
Copying can be of three types as follows:

- ❖ Shallow copy
- ❖ Deep copy
- ❖ Cloning

Where deep copy and cloning is referred to as the same deep cloning



Shallow Cloning



Deep Cloning

Object equality and cloning



1. Shallow Copy/ Cloning

Default implementation while using the clone() method a shallow copy of the object is created.

It means it creates a new instance and copies all the fields of the object to that new instance where both are referencing to the same memory in heap memory.

If the object contains primitive as well as non-primitive or reference type variable in shallow copy, the cloned object also refers to the same object to which the original object refers as only the object references get copied and not the referred objects themselves.

In simpler terms, One object is being created here while two references in the stack memory.

It returns it as an object type, we need to explicitly cast it back to our original object. This is a shallow copy of the object.

Object equality and cloning



2. Deep copy/ cloning

It is the process of creating exactly the independent duplicate objects in the heap memory and manually assigning the values of the second object where values are supposed to be copied is called deep cloning.

For the deep copy, it needs to ensure that all the member classes also implement the Cloneable interface because of overriding the clone() method of the object class.

Object equality and cloning



Difference between Deep and Shallow Copy

Shallow Copy	Deep Copy
It is fast as no new memory is allocated.	It is slow as new memory is allocated.
Changes in one entity is reflected in other entity.	Changes in one entity are not reflected in changes in another identity.
The default version of the clone() method supports shallow copy.	In order to make the clone() method support the deep copy, one has to override the clone() method.
A shallow copy is less expensive.	Deep copy is highly expensive.
Cloned object and the original object are not disjoint.	Cloned object and the original object are disjoint.

CloneDemo.java for demonstrating Deep and Shallow copy

[CloneDemo.java](#)

Thanks



Object Oriented Programming Design

BITS Pilani
Pilani Campus

SESSION 16

Date : 25-01-2025



BITS Pilani
Pilani Campus



Generics in Java

Generics



- In any nontrivial software project, bugs are simply a fact of life.
- Careful planning, programming, and testing can help reduce their pervasiveness, but somehow, somewhere, they'll always find a way to creep into your code.
- This becomes especially apparent as new features are introduced and your code base grows in size and complexity.
- Fortunately, some bugs are easier to detect than others.
- Compile-time bugs, for example, can be detected early on; you can use the compiler's error messages to figure out what the problem is and fix it, right then and there.
- Runtime bugs, however, can be much more problematic; they don't always surface immediately, and when they do, it may be at a point in the program that is far removed from the actual cause of the problem.
- Generics add stability to your code by making more of your bugs detectable at compile time. After completing this lesson, you may want to follow up with the [Generics](#) tutorial by Gilad Bracha.

Why Use Generics?



Why Use Generics?

- In a nutshell, generics enable *types* (classes and interfaces) to be parameters when defining classes, interfaces and methods.
- Much like the more familiar *formal parameters* used in method declarations, type parameters provide a way for you to re-use the same code with different inputs.
- The difference is that the inputs to formal parameters are values, while the inputs to type parameters are types.
- Code that uses generics has many benefits over non-generic code:

Stronger type checks at compile time.

A Java compiler applies strong type checking to generic code and issues errors if the code violates type safety. Fixing compile-time errors is easier than fixing runtime errors, which can be difficult to find.

Advantages of Generics



Elimination of casts.

The following code snippet without generics requires casting:

```
List list = new ArrayList();  
list.add("hello");  
String s = (String) list.get(0);
```

When re-written to use generics, the code does not require casting:

```
List<String> list = new ArrayList<String>();  
list.add("hello");  
String s = list.get(0); // no cast
```

Enabling programmers to implement generic algorithms.

By using generics, programmers can implement generic algorithms that work on collections of different types, can be customized, and are type safe and easier to read.

Generic Types



- A *generic type* is a generic class or interface that is parameterized over types.
- The following Box class will be modified to demonstrate the concept.

A Simple Box Class

Begin by examining a non-generic Box class that operates on objects of any type. It needs only to provide two methods: set, which adds an object to the box, and get, which retrieves it:

```
public class Box
{
    private Object object;
    public void set(Object object)
    {
        this.object = object;
    }
    public Object get()
    {
        return object;
    }
}
```


Generic Types



What's the problem with the code?

Since its methods accept or return an Object, you are free to pass in whatever you want, provided that it is not one of the primitive types.

There is no way to verify, at compile time, how the class is used.

One part of the code may place an Integer in the box and expect to get Integers out of it, while another part of the code may mistakenly pass in a String, resulting in a runtime error.

A Generic Version of the Box Class



A *generic class* is defined with the following format:

```
class name<T1, T2, ..., Tn> { /* Class code... */ }
```

The type parameter section, delimited by angle brackets (<>), follows the class name.

It specifies the *type parameters* (also called *type variables*) T1, T2, ..., and Tn.

To update the Box class to use generics, you create a *generic type declaration* by changing the code "public class Box" to "public class Box<T>".

This introduces the type variable, T, that can be used anywhere inside the class.

A Generic Version of the Box Class



With this change, the Box class becomes:

```
/** * Generic version of the Box class. *  
 * @param <T> the type of the value being boxed */  
public class Box<T> // T stands for "Type"  
{  
    private T t;  
    public void set(T t)  
    {  
        this.t = t;  
    }  
    public T get() .  
    {  
        return t;  
    }  
}
```

A Generic Version of the Box Class



As you can see, all occurrences of `Object` are replaced by `T`.

A type variable can be any **non-primitive** type you specify: any class type, any interface type, any array type, or even another type variable.

This same technique can be applied to create generic interfaces.

Type Parameter Naming Conventions

By convention, type parameter names are single, uppercase letters.

Without this convention, it would be difficult to tell the difference between a type variable and an ordinary class or interface name.

The most commonly used type parameter names are:

- `E` - Element (used extensively by the Java Collections Framework)
- `K` - Key
- `N` - Number
- `T` - Type
- `V` - Value
- `S`, `U`, `V` etc. - 2nd, 3rd, 4th types

You'll see these names used throughout the Java SE API.

Invoking and Instantiating a Generic Type



To reference the generic Box class from within your code, you must perform a *generic type invocation*, which replaces T with some concrete value, such as Integer:

```
Box<Integer> integerBox;
```

You can think of a generic type invocation as being similar to an ordinary method invocation, but instead of passing an argument to a method, you are passing a *type argument* — Integer in this case — to the Box class itself.

Like any other variable declaration, this code does not actually create a new Box object. It simply declares that integerBox will hold a reference to a "Box of Integer", which is how Box<Integer> is read.

An invocation of a generic type is generally known as a *parameterized type*.

To instantiate this class, use the new keyword, as usual, but place <Integer> between the class name and the parenthesis:

```
Box<Integer> integerBox = new Box<Integer>();
```

Generic Methods



You can write a single generic method declaration that can be called with arguments of different types. Based on the types of the arguments passed to the generic method, the compiler handles each method call appropriately. Following are the rules to define Generic Methods –

- All generic method declarations have a **type parameter** section delimited by angle brackets (< and >) that precedes the method's return type (< E > in the next example).
- Each type parameter section contains one or more type parameters separated by commas. **A type parameter, also known as a type variable, is an identifier that specifies a generic type name.**
- The type parameters can be used to declare the return type and act as placeholders for the types of the arguments passed to the generic method, which are known as actual type arguments.
- A generic method's body is declared like that of any other method.
- **Note that type parameters can represent only reference types, not primitive types (like int, double and char).**

Bounded Type Parameters



There may be times when you'll want to restrict the kinds of types that are allowed to be passed to a type parameter.

For example, a method that operates on numbers might only want to accept instances of `Number` or its subclasses.

This is what bounded type parameters are for.

To declare a bounded type parameter, list the type parameter's name, followed by the `extends` keyword, followed by its upper bound.

Example

Following example illustrates how `extends` is used in a general sense to mean either "extends" (as in classes) or "implements" (as in interfaces).

This example is Generic method to return the largest of three `Comparable` objects –

What are `Comparable` objects? – Left as self study exercise.

Wildcards in generics



In generic code, the question mark (?), called the wildcard, represents an unknown type.

The wildcard can be used in a variety of situations: as the type of a parameter, field, or local variable; sometimes as a return type (though it is better programming practice to be more specific).

The wildcard is never used as a type argument for a generic method invocation, a generic class instance creation, or a supertype.

The following sections discuss wildcards in more detail, including upper bounded wildcards, lower bounded wildcards, and wildcard capture.

Wildcards in generics - Upper Bounded Wildcards



You can use an upper bounded wildcard to relax the restrictions on a variable.

For example, say you want to write a method that works on `List<Integer>`, `List<Double>`, and `List<Number>`; you can achieve this by using an upper bounded wildcard.

To declare an upper-bounded wildcard, use the wildcard character ('?'), followed by the `extends` keyword, followed by its upper bound.

Note that, in this context, `extends` is used in a general sense to mean either "extends" (as in classes) or "implements" (as in interfaces).

To write the method that works on lists of `Number` and the subtypes of `Number`, such as `Integer`, `Double`, and `Float`, you would specify `List<? extends Number>`.

The term `List<Number>` is more restrictive than `List<? extends Number>` because the former matches a list of type `Number` only, whereas the latter matches a list of type `Number` or any of its subclasses.

Wildcards in generics – Un-Bounded Wildcards



The unbounded wildcard type is specified using the wildcard character (?), for example, `List<?>`.

This is called a list of unknown type.

There are two scenarios where an unbounded wildcard is a useful approach:

1. If you are writing a method that can be implemented using functionality provided in the `Object` class.
2. When the code is using methods in the generic class that don't depend on the type parameter.

For example, `List.size` or `List.clear`.

In fact, `Class<?>` is so often used because most of the methods in `Class<T>` do not depend on `T`.

Wildcards in generics – Un-Bounded Wildcards



Consider the following method, printList:

```
public static void printList(List<Object> list) {  
    for (Object elem : list)  
        System.out.println(elem + " ");  
    System.out.println();  
}
```

The goal of printList is to print a list of any type, but it fails to achieve that goal.

It prints only a list of Object instances;

It cannot print List<Integer>, List<String>, List<Double>, and so on, because they are not subtypes of List<Object>.

To write a generic printList method, use List<?>:

Because for any concrete type A, List<A> is a subtype of List<?>, you can use printList to print a list of any type.

Wildcards in generics – Lower Bounded Wildcards



The Upper Bounded Wildcards section shows that an upper bounded wildcard restricts the unknown type to be a specific type or a subtype of that type and is represented using the `extends` keyword.

In a similar way, a lower bounded wildcard restricts the unknown type to be a specific type or a **super type** of that type.

A lower bounded wildcard is expressed using the wildcard character ('?'), following by the **super keyword**, followed by its lower bound: **<? super A>.**

Note: You can specify an upper bound for a wildcard, or you can specify a lower bound, but you cannot specify both.

Say you want to write a method that puts Integer objects into a list.

To maximize flexibility, you would like the method to work on `List<Integer>`, `List<Number>`, and `List<Object>` — anything that can hold Integer values.

Wildcards in generics – Lower Bound Wildcards



To write the method that works on lists of Integer and the supertypes of Integer, such as Integer, Number, and Object, you would specify `List<? super Integer>`.

The term `List<Integer>` is more restrictive than `List<? super Integer>` because the former matches a list of type Integer only, whereas the latter matches a list of any type that is a supertype of Integer.

The following code adds the numbers 1 through 10 to the end of a list:

```
public static void addNumbers(List<? super Integer> list) {  
    for (int i = 1; i <= 10; i++) {  
        list.add(i);  
    }  
}
```



Reference

<https://docs.oracle.com/javase/tutorial/java/generics/index.html>

https://www.tutorialspoint.com/java/java_generics.htm

References for further reading

<https://docs.oracle.com/javase/tutorial/extra/generics/index.html>